classical-physics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `classical_physics_ai999x.about()` for the same answer.

Every capability classical-physics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import classical_physics_ai999x

result = classical_physics_ai999x.solve('coulomb_force', q1='2e-06 C', q2='3e-06 C', r='0.1
m')
result.show()
# F = 5.39253 N, F_radial = 5.39253 N
```

By topic

topic	capabilities
Ac circuits	Rms voltage of a sinusoidal supply, Rms current of a sinusoidal supply, Reactance of an inductor, Reactance of a capacitor, Resonant frequency of an LC circuit
AC circuits / series RLC	Series RLC circuit on an AC supply: impedance, current, phase and power
Capacitors	Charge on a capacitor, Energy stored in a capacitor, Capacitance of parallel plates
Circuits	Power from current and resistance, Power from voltage and resistance, Resistance of a uniform wire, Two resistors in series and in parallel
Circular	Centripetal acceleration, Centripetal force, Speed on a circle from angular speed, Angular frequency and frequency, Period and frequency
Collisions	Final velocity of the first body in a one-dimensional elastic collision, Final velocity of the second body in a one-dimensional elastic collision
DC circuits / emf and internal resistance	Terminal voltage of a source with internal resistance
Dynamics	Net force and acceleration, Weight in a uniform gravitational field, Net force with an opposing friction force, Sliding friction, Normal force on a level surface, Linear momentum, Impulse of a constant force, Normal force on an incline, Component of the weight along an incline, Acceleration down a frictionless incline, Tension in a rope holding a hanging body, Average net force from a change of momentum
Elasticity / bulk modulus	Bulk compression: volume change under a pressure change (bulk modulus)
Elasticity / shear modulus	Shear: shear stress, shear strain and displacement (shear modulus)

topic	capabilities
Elasticity / Young's modulus	Tension or compression: stress, strain and change of length (Young's modulus)
Electricity	Voltage across an ohmic resistance, Electric power, Charge carried by a steady current
Electromagnetic induction / Faraday's law	Faraday's law: average emf induced by a change of magnetic flux
Electromagnetic induction / transformers	Ideal transformer: secondary voltage and current
Electrostatics	Force on a charge in an electric field, Electric field of a point charge, Electric potential of a point charge, Potential difference in a uniform field
Electrostatics / Coulomb's law	Coulomb's law: force between two point charges
Electrostatics / Gauss's law	Gauss's law: electric field of a line, a plane, a charged sphere or a shell
Energy	Work by a constant force, Translational kinetic energy, Gravitational potential energy near the surface, Force of a linear spring, Energy stored in a linear spring, Average power, Power of a force on a moving body, Speed of a body launched by a spring, Work-energy theorem
Fluids	Mass density, Pressure of a force on an area, Pressure of a column of fluid, Absolute pressure at a depth in a fluid, Flow rate as volume per time, Flow rate through a cross section
Forces / static friction	Static friction: maximum friction force and whether a body slips, Static friction on an incline: critical angle and whether a body slides
Geometric optics / images	Magnification and image height of a thin lens or a spherical mirror
Geometric optics / refraction	Snell's law: angle of refraction
Geometric optics / total internal reflection	Critical angle for total internal reflection
Gravitation	Gravitational acceleration at a distance from a mass, Speed in a circular orbit, Period of a circular orbit
Gravitation / universal gravitation	Newton's law of universal gravitation: force between two masses
Induction	Time constant of an RL circuit, Emf of a rod moving through a magnetic field, Current dying away in an RL circuit
Kinematics	Motion at constant velocity, Velocity under constant acceleration, Displacement under constant acceleration, Speed after a displacement under constant acceleration, Displacement from the mean of initial and final velocity, Maximum height of a vertical throw, Time to the top of a vertical throw, Speed after falling from rest, Velocity and displacement after constant acceleration
Magnetism	Magnetic force on a moving charge, Magnetic force on a current-carrying wire, Magnetic field of a long straight wire, Magnetic field inside a long solenoid, Magnetic field at the centre of a circular loop, Radius of a charged particle's circle in a magnetic field
Optics	Thin lens equation, Wavelength of light in a medium, Index of refraction, Focal length of a spherical mirror
Orbits	Orbital speed on an elliptical orbit (vis-viva), Escape speed, Orbital period from the semi-major axis (Kepler's third law)

topic	capabilities
Oscillations	Period of a simple pendulum, Period of a mass on a spring, Maximum speed of a mass on a spring, Angular frequency of a damped oscillator, Damping ratio, Quality factor of a damped oscillator, Steady-state amplitude of a driven damped oscillator, Amplitude-resonance frequency of a driven damped oscillator, Higher normal-mode frequency of two coupled oscillators, Period of a physical pendulum, Lower normal mode of a double pendulum, Higher normal mode of a double pendulum
Proofs and derivations	Acceleration with friction level, Angular acceleration from tangential, Centripetal acceleration from period, Derive absolute pressure depth, Derive acceleration frictionless incline, Derive centripetal acceleration omega, Derive centripetal force, Derive charge current, Derive cyclotron radius, Derive electric power, Derive impulse, Derive inductive reactance, Derive lc resonance, Derive long wire field, Derive normal on level, Derive ohm power current, Derive ohm power voltage, Derive orbital period, Derive photon energy, Derive photon energy wavelength, Derive photon momentum, Derive rl time constant, Derive shm max speed, Derive solenoid field, Derive spring oscillation period, Derive surface gravity, Derive time to top vertical throw, Displacement from velocity time, Distance fallen time, Final speed from work, Max height from velocity displacement, Net work to speed, Period from wave speed, Solve for g, Speed after height, Speed after time dropped, Speed from uniform motion, Time to stop, Work to speed rest, Angular frequency period, Average speed equal instantaneous, Dielectric increases capacitance, Energy via resistance, Focal length, Temp change inverse mass, Zero initial speed
Quantum	Energy of a photon, Photon energy from wavelength, Momentum of a photon, De Broglie wavelength
Rc circuits	Time constant of an RC circuit, Voltage of a discharging capacitor, Voltage of a charging capacitor
Relativity	Rest energy
Rigid body	Moment of inertia of a solid sphere, Moment of inertia of a thin spherical shell, Moment of inertia of a solid disk about its axis, Moment of inertia of a thin rod about its center, Moment of inertia of a thin rod about one end, Acceleration of a body rolling without slipping down an incline, Acceleration of an Atwood machine with a massive pulley, Body-frame precession rate of a torque-free symmetric body
Rotating frames	Precession period of a Foucault pendulum, Off-axis equilibrium of a bead on a rotating hoop
Rotation	Torque of a force, Rotational kinetic energy, Angular momentum of a rotating body, Tangential acceleration of a point on a rotating body, Centripetal acceleration from angular velocity, Angular momentum of a point mass on a circle, Angular velocity under constant angular acceleration, Angular velocity and angle turned under constant angular acceleration
Rotational dynamics / conservation of angular momentum	Conservation of angular momentum: change of spin rate
Rotational dynamics / torque and angular acceleration	Newton's second law for rotation: angular acceleration from torque
Several bodies, states or stages	Two masses over a pulley, Ideal banking of a curve, Mixing to thermal equilibrium, Collisions along a line, Doppler effect for sound, Apparent weight in an elevator, Maximum speed on a flat curve, A gas between two states, Radioactive decay, Series and parallel resistor and capacitor networks, Projectile motion, Block on a surface pulled over a pulley, RC circuits, Recoil from rest, Rolling from a height, Motion in stages
Sound / intensity level	Sound intensity level in decibels, Change of sound level and the ratio of two intensities
Statics / equilibrium of a beam	Static equilibrium of a beam on two supports: support reactions
Systems of particles / centre of	Centre of mass of a set of point masses

topic	capabilities
mass	
Thermal	Heat for a temperature change, Ideal gas law, Heat to melt or freeze, Heat to boil or condense, Linear thermal expansion, Thermal expansion of volume, Efficiency of a heat engine, Carnot (maximum) efficiency, Energy balance of a cyclic heat engine, Coefficient of performance of a refrigerator, Coefficient of performance of a heat pump, Thermal expansion of an area
Variable mass	Velocity change of a rocket (Tsiolkovsky)
Vectors / two-dimensional vectors	Vector components and the resultant of two-dimensional vectors
Wave optics / diffraction	Single-slit diffraction: dark fringes and the width of the central maximum
Wave optics / diffraction grating	Diffraction grating: angle of an order and the highest visible order
Wave optics / interference	Double-slit interference: angle and position of a bright fringe
Wave optics / polarization	Malus's law: intensity through a polarizer
Waves	Speed of a periodic wave, Intensity as power per area
Waves / standing waves and harmonics	Standing waves: harmonics of a string and of open or closed pipes
Waves / wave speed on a string	Wave speed on a stretched string

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Giving a unit of the wrong kind (a mass in m): INVALID_INPUT names the expected unit.
- Leaving out an input: NEEDS_INPUT names it; declared constants (g, k, h ...) are filled and stated.
- Giving every quantity of an equation: say which one to compute with target=.
- Asking several quantities in one question: ask one at a time (or call solve() once per quantity).
- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Catalogue

id	operation	family	inputs
uniform_motion	Motion at constant velocity	kinematics	d, v, t
velocity_time	Velocity under constant acceleration	kinematics	v, v0, a, t
displacement_time	Displacement under constant acceleration	kinematics	x, v0, a, t
velocity_displacement	Speed after a displacement under constant acceleration	kinematics	v, v0, a, x
average_velocity_displacement	Displacement from the mean of initial and final velocity	kinematics	x, v0, v, t
newton_second	Net force and acceleration	dynamics	F, m, a
weight	Weight in a uniform gravitational field	dynamics	W, m, g
net_force_friction	Net force with an opposing friction force	dynamics	F, P, f
kinetic_friction	Sliding friction	dynamics	f, mu, N
normal_on_level	Normal force on a level surface	dynamics	N, W
momentum	Linear momentum	dynamics	p, m, v
impulse	Impulse of a constant force	dynamics	J, F, t
work	Work by a constant force	energy	W, F, d, theta

id	operation	family	inputs
kinetic_energy	Translational kinetic energy	energy	K, m, v
gravitational_pe	Gravitational potential energy near the surface	energy	U, m, g, h
spring_force	Force of a linear spring	energy	F, k, x
spring_energy	Energy stored in a linear spring	energy	U, k, x
power_energy	Average power	energy	P, E, t
power_force_speed	Power of a force on a moving body	energy	P, F, v
centripetal_acceleration	Centripetal acceleration	circular	a, v, r
centripetal_force	Centripetal force	circular	F, m, v, r
tangential_speed	Speed on a circle from angular speed	circular	v, omega, r
angular_frequency	Angular frequency and frequency	circular	omega, f
period_frequency	Period and frequency	circular	T, f
density	Mass density	fluids	rho, m, V
pressure_force	Pressure of a force on an area	fluids	P, F, A
hydrostatic_pressure	Pressure of a column of fluid	fluids	P, rho, g, h
ohm	Voltage across an ohmic resistance	electricity	V, I, R
electric_power	Electric power	electricity	P, V, I
charge_current	Charge carried by a steady current	electricity	Q, I, t
wave_speed	Speed of a periodic wave	waves	v, f, lam
heat	Heat for a temperature change	thermal	Q, m, c, dT
max_height_vertical_throw	Maximum height of a vertical throw	kinematics	h, v0, g
time_to_top_vertical_throw	Time to the top of a vertical throw	kinematics	t, v0, g
normal_force_incline	Normal force on an incline	dynamics	N, m, g, theta
weight_component_along_incline	Component of the weight along an incline	dynamics	F, m, g, theta
acceleration_frictionless_incline	Acceleration down a frictionless incline	dynamics	a, g, theta
spring_launch_speed	Speed of a body launched by a spring	energy	k, x, m, v
net_work_kinetic_energy	Work-energy theorem	energy	W, m, v, v0
tension_hanging_at_rest	Tension in a rope holding a hanging body	dynamics	T, m, g
average_force_from_momentum_change	Average net force from a change of momentum	dynamics	F, p2, p1, t
speed_after_fall_from_rest	Speed after falling from rest	kinematics	v, g, h
electric_force_in_field	Force on a charge in an electric field	electrostatics	F, q, Ef
point_charge_field	Electric field of a point charge	electrostatics	Ef, Q, r, ke
point_charge_potential	Electric potential of a point charge	electrostatics	V, Q, r, ke
uniform_field_potential	Potential difference in a uniform field	electrostatics	V, Ef, d
capacitance_definition	Charge on a capacitor	capacitors	Q, C, V
capacitor_energy	Energy stored in a capacitor	capacitors	Ecap, C, V
parallel_plate_capacitor	Capacitance of parallel plates	capacitors	C, A, d, eps0
ohm_power_current	Power from current and resistance	circuits	P, I, R
ohm_power_voltage	Power from voltage and resistance	circuits	P, V, R

id	operation	family	inputs
resistivity	Resistance of a uniform wire	circuits	R, rho_e, L, A
magnetic_force_charge	Magnetic force on a moving charge	magnetism	F, q, v, B, theta
magnetic_force_wire	Magnetic force on a current-carrying wire	magnetism	F, I, L, B, theta
long_wire_field	Magnetic field of a long straight wire	magnetism	B, I, r, mu0
solenoid_field	Magnetic field inside a long solenoid	magnetism	B, nt, I, mu0
surface_gravity	Gravitational acceleration at a distance from a mass	gravitation	g, M, r, G
orbital_speed	Speed in a circular orbit	gravitation	v, M, r, G
orbital_period	Period of a circular orbit	gravitation	T, r, M, G
ideal_gas_moles	Ideal gas law	thermal	P, V, n, T, R
latent_heat_fusion	Heat to melt or freeze	thermal	Q, m, Lf
latent_heat_vaporization	Heat to boil or condense	thermal	Q, m, Lv
linear_expansion	Linear thermal expansion	thermal	dL, alpha_t, L, dT
thin_lens	Thin lens equation	optics	d_o, d_i, f
photon_energy	Energy of a photon	quantum	E, f, h
photon_energy_wavelength	Photon energy from wavelength	quantum	E, lam, h, c
photon_momentum	Momentum of a photon	quantum	p, lam, h
de_broglie	De Broglie wavelength	quantum	lam, p, h
rest_energy	Rest energy	relativity	E0, m, c
pendulum_period	Period of a simple pendulum	oscillations	T, Lp, g
spring_oscillation_period	Period of a mass on a spring	oscillations	T, m, k
torque_lever	Torque of a force	rotation	tau, r, F, theta
rotational_kinetic_energy	Rotational kinetic energy	rotation	KR, Ir, omega
angular_momentum	Angular momentum of a rotating body	rotation	L, Ir, omega
tangential_acceleration	Tangential acceleration of a point on a rotating body	rotation	a, r, alpha
centripetal_acceleration_omega	Centripetal acceleration from angular velocity	rotation	a, r, omega
angular_momentum_point_mass	Angular momentum of a point mass on a circle	rotation	L, m, v, r
absolute_pressure_depth	Absolute pressure at a depth in a fluid	fluids	P, P0, rho, g, h
flow_rate_volume_time	Flow rate as volume per time	fluids	Q, V, t
flow_rate_area_speed	Flow rate through a cross section	fluids	Q, A, v
volume_expansion	Thermal expansion of volume	thermal	dV, beta, V0, dT
engine_efficiency	Efficiency of a heat engine	thermal	eff, W, Qh
carnot_efficiency	Carnot (maximum) efficiency	thermal	eff, Tc, Th
shm_max_speed	Maximum speed of a mass on a spring	oscillations	v, X, k, m
intensity_power_area	Intensity as power per area	waves	I, P, A
wavelength_in_medium	Wavelength of light in a medium	optics	lam_n, lam, n
index_of_refraction	Index of refraction	optics	n, c, v
mirror_focal_length	Focal length of a spherical mirror	optics	f, R

id	operation	family	inputs
rms_voltage	Rms voltage of a sinusoidal supply	ac_circuits	Vrms, V0
rms_current	Rms current of a sinusoidal supply	ac_circuits	Irms, I0
loop_center_field	Magnetic field at the centre of a circular loop	magnetism	B, I, r, mu0
rl_time_constant	Time constant of an RL circuit	induction	tau, L, R
inductive_reactance	Reactance of an inductor	ac_circuits	X, f, L
capacitive_reactance	Reactance of a capacitor	ac_circuits	X, f, C
lc_resonance	Resonant frequency of an LC circuit	ac_circuits	f, L, C
angular_velocity_time	Angular velocity under constant angular acceleration	rotation	omega, omega0, alpha, t
angular_velocity_angle	Angular velocity and angle turned under constant angular acceleration	rotation	omega, omega0, alpha, theta
engine_energy_balance	Energy balance of a cyclic heat engine	thermal	W, Qh, Qc
cop_refrigerator	Coefficient of performance of a refrigerator	thermal	cop, Qc, W
cop_heat_pump	Coefficient of performance of a heat pump	thermal	cop, Qh, W
cyclotron_radius	Radius of a charged particle's circle in a magnetic field	magnetism	r, m, v, q, B, theta
rc_time_constant	Time constant of an RC circuit	rc_circuits	tau, R, C
rc_discharging	Voltage of a discharging capacitor	rc_circuits	V, V0, t, R, C
rc_charging	Voltage of a charging capacitor	rc_circuits	V, emf, t, R, C
area_expansion	Thermal expansion of an area	thermal	dA, alpha_t, A, dT
motional_emf	Emf of a rod moving through a magnetic field	induction	emf, B, l, v
rl_current_decay	Current dying away in an RL circuit	induction	I, I0, t, tau
damped_angular_frequency	Angular frequency of a damped oscillator	oscillations	wd, k, m, b
damping_ratio	Damping ratio	oscillations	zeta, b, k, m
quality_factor	Quality factor of a damped oscillator	oscillations	Qf, k, m, b
driven_amplitude	Steady-state amplitude of a driven damped oscillator	oscillations	A, F0, k, m, b, w
amplitude_resonance_frequency	Amplitude-resonance frequency of a driven damped oscillator	oscillations	wr, k, m, b
coupled_fast_mode	Higher normal-mode frequency of two coupled oscillators	oscillations	w2, k, kc, m
physical_pendulum_period	Period of a physical pendulum	oscillations	T, Ir, m, g, d
double_pendulum_slow_mode	Lower normal mode of a double pendulum	oscillations	w, g, L
double_pendulum_fast_mode	Higher normal mode of a double pendulum	oscillations	w, g, L
vis_viva	Orbital speed on an elliptical orbit (vis-viva)	orbits	v, G, M, r, a
escape_speed	Escape speed	orbits	v, G, M, r

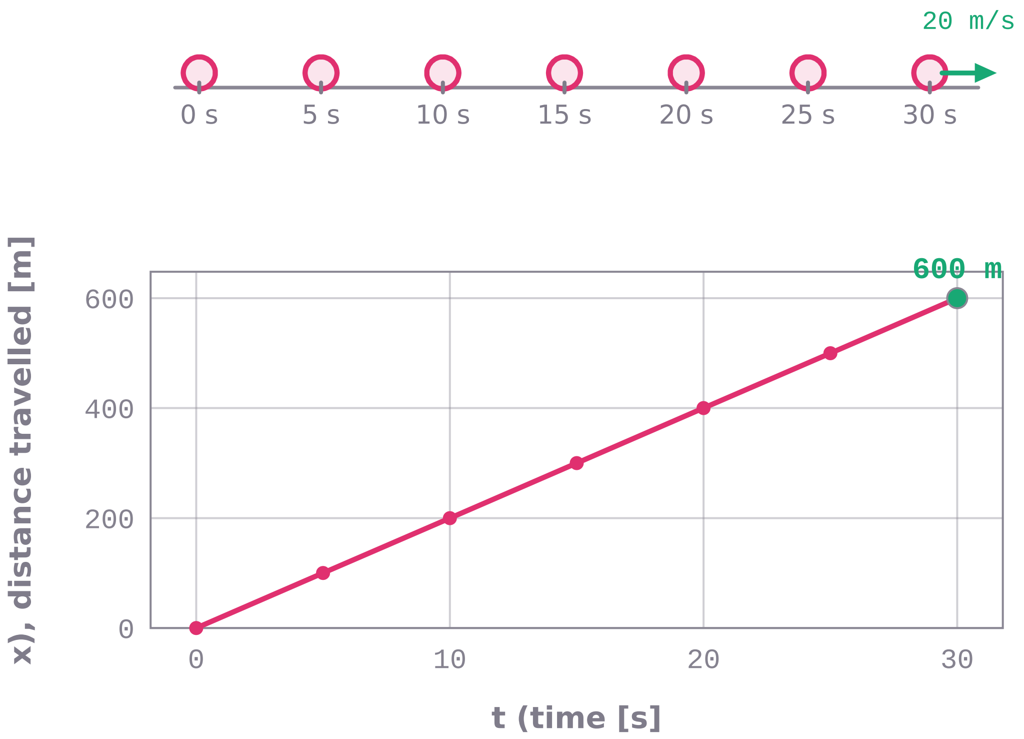
id	operation	family	inputs
kepler_period_semi_major	Orbital period from the semi-major axis (Kepler's third law)	orbits	T, a, G, M
inertia_solid_sphere	Moment of inertia of a solid sphere	rigid_body	Ir, m, R
inertia_hollow_sphere	Moment of inertia of a thin spherical shell	rigid_body	Ir, m, R
inertia_solid_disk	Moment of inertia of a solid disk about its axis	rigid_body	Ir, m, R
inertia_rod_center	Moment of inertia of a thin rod about its center	rigid_body	Ir, m, Lr
inertia_rod_end	Moment of inertia of a thin rod about one end	rigid_body	Ir, m, Lr
rolling_incline_acceleration	Acceleration of a body rolling without slipping down an incline	rigid_body	a, g, theta, Ir, m, R
atwood_massive_pulley	Acceleration of an Atwood machine with a massive pulley	rigid_body	a, m1, m2, g, Ir, R
torque_free_precession	Body-frame precession rate of a torque-free symmetric body	rigid_body	Wp, w3, I1, I3
foucault_precession_period	Precession period of a Foucault pendulum	rotating_frames	T, W, phi
bead_hoop_equilibrium	Off-axis equilibrium of a bead on a rotating hoop	rotating_frames	theta, g, w, R
elastic_collision_first_final	Final velocity of the first body in a one-dimensional elastic collision	collisions	v1f, m1, m2, v1, v2
elastic_collision_second_final	Final velocity of the second body in a one-dimensional elastic collision	collisions	v2f, m1, m2, v1, v2
rocket_delta_v	Velocity change of a rocket (Tsiolkovsky)	variable_mass	dv, ve, m0, mf

Motion at constant velocity, `uniform_motion`

$$d = vt$$

input	meaning	kind / role	unit	default
`d`	distance travelled	length / distance	`m`	
`v`	constant speed	speed / plain	`m/s`	
`t`	elapsed time	time / plain	`s`	

- **Not when:** accelerated, pendulum, shm
- **Assumes:** constant velocity
- **Evidence:** hand-checked pack



A body moving at a constant 20 m/s, shown every 5 s along its path (equal steps: equal distances) and as a distance–time line; each position from the package. Marked: 600 m after 30 s.

```
classical_physics_ai999x.solve('uniform_motion', v='20 m/s', t='30 s', target='d')
# -> 600 m   (SOLVED)
```

$$d = vt$$
$$d = tv = 600\ \mathrm{m}$$

invalid call	outcome	message
without `v`	NEEDS_INPUT	uniform_motion needs v (constant speed, m/s). Example: solve('uniform_motion', v='... m/s', t='... s').
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

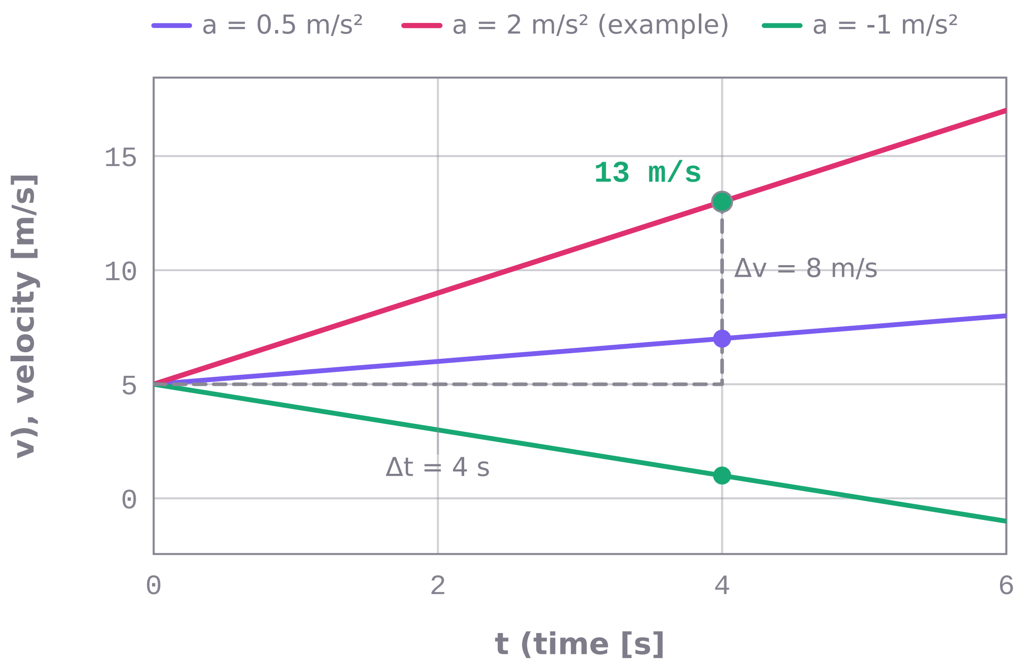
Velocity under constant acceleration, `velocity_time`

$$v = v_0 + at$$

input	meaning	kind / role	unit	default
`v`	final velocity	speed / final	`m/s`	
`v0`	initial velocity	speed / initial	`m/s`	
`a`	acceleration	acceleration / plain	`m/s^2`	
`t`	elapsed time	time / plain	`s`	

- **Not when:** pendulum, shm
- **Assumes:** constant acceleration along a line

- **Evidence:** hand-checked pack



Velocity against time from v_0 = 5 m/s for three constant accelerations, 12 times each from the package; the slope of each line is its acceleration (the dashed triangle on the example line: Δv over Δt, Δv derived from the returned velocities). Marked at t = 4 s: a = 0.5 m/s²: v = 7 m/s; a = 2 m/s²: v = 13 m/s; a = -1 m/s²: v = 1 m/s.

```
classical_physics_ai999x.solve('velocity_time', v0='5 m/s', a='2 m/s^2', t='4 s',
target='v')
# -> 13 m/s   (SOLVED)
```

$$v = v_0 + at$$
$$v = at + v_0 = 13\ \mathrm{m/s}$$

invalid call	outcome	message
without `v0`	NEEDS_INPUT	velocity_time needs v0 (initial velocity, m/s). Example: solve('velocity_time', v0='... m/s', a='... m/s^2', t='... s').
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

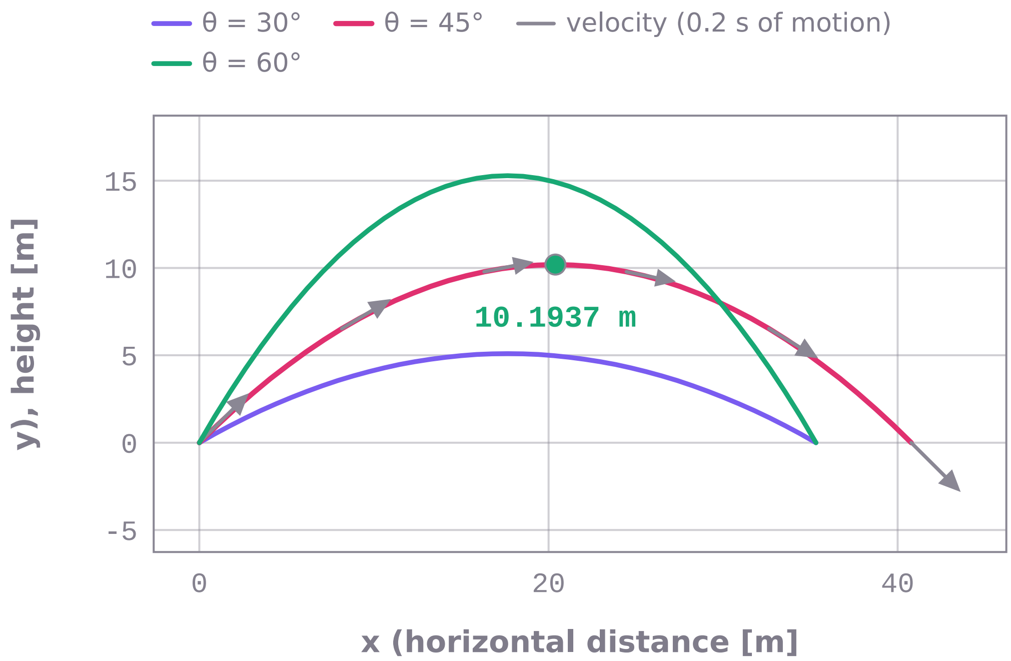
Displacement under constant acceleration, `displacement_time`

$$x = v_0 t + \frac{at^2}{2}$$

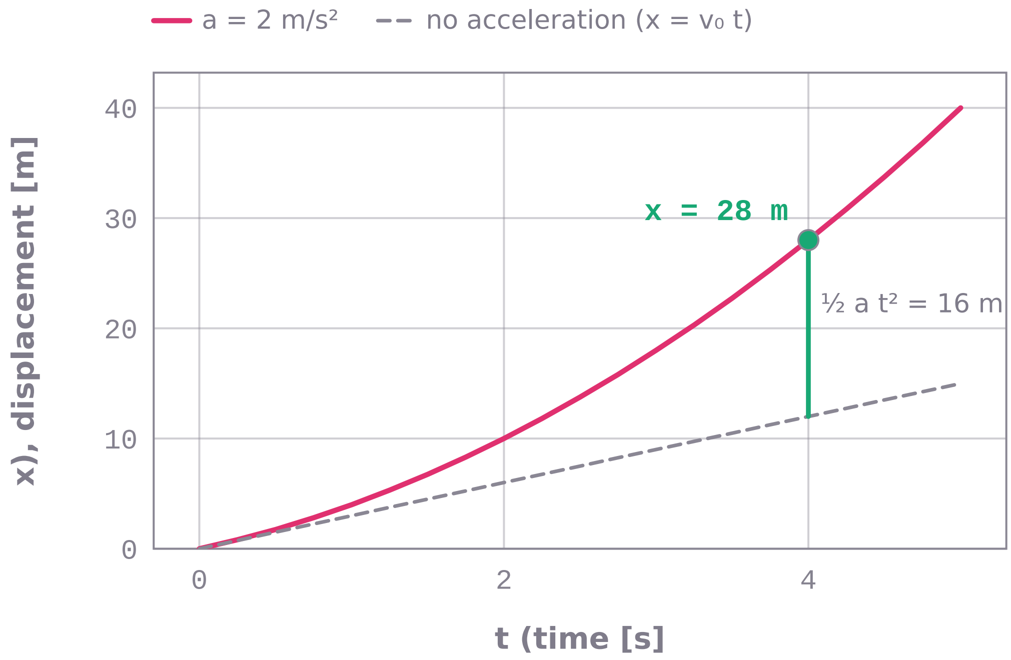
input	meaning	kind / role	unit	default
`x`	displacement	length / distance	`m`	
`v0`	initial velocity	speed / initial	`m/s`	
`a`	acceleration	acceleration / plain	`m/s^2`	
`t`	elapsed time	time / plain	`s`	

- **Not when:** pendulum, shm

- **Assumes:** constant acceleration along a line
- **Evidence:** hand-checked pack



Trajectories of a projectile launched at 20 m/s ($g = 9.81$ m/s², no air resistance) at three angles: each point's horizontal distance and height computed by the package (uniform motion and constant acceleration), the top from its maximum-height formula, and on the 45° path the velocity vectors (horizontal part constant, vertical part from $v = v_0 + a t$), $\theta = 30°$: range 35.3119 m, top 5.09684 m; $\theta = 45°$: range 40.7747 m, top 10.1937 m; $\theta = 60°$: range 35.3119 m, top 15.2905 m.



Displacement against time for $v_0 = 3$ m/s and $a = 2$ m/s² (20 times from the package), beside the straight line the same start speed gives without acceleration (uniform motion, also from the package); the gap between them at $t = 4$ s is the part due to the acceleration (derived as the difference). Marked: $x = 28$ m at $t = 4$ s, 12 m without acceleration.

```
classical_physics_ai999x.solve('displacement_time', v0='3 m/s', a='2 m/s^2', t='4 s',
target='x')
# -> 28 m   (SOLVED)
```

$$x = v_0 t + \frac{a t^2}{2}$$
$$x = \frac{t\left(a t + 2 v_0\right)}{2} = 28\ \mathrm{m}$$

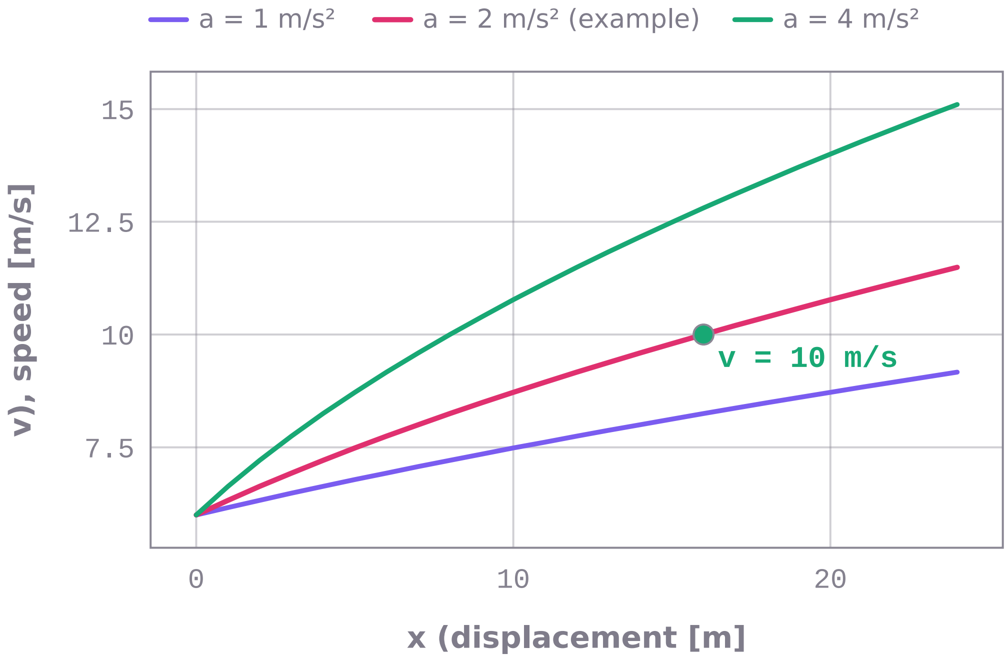
invalid call	outcome	message
without `v0`	NEEDS_INPUT	displacement_time needs v0 (initial velocity, m/s). Example: solve('displacement_time', v0='... m/s', a='... m/s^2', t='... s').
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

Speed after a displacement under constant acceleration, `velocity_displacement`

$$v^2 = v_0^2 + 2 a x$$

input	meaning	kind / role	unit	default
`v`	final speed	speed / final	`m/s`	
`v0`	initial speed	speed / initial	`m/s`	
`a`	acceleration	acceleration / plain	`m/s^2`	
`x`	displacement	length / distance	`m`	

- **Not when:** pendulum, shm
- **Assumes:** constant acceleration along a line; motion does not reverse
- **Evidence:** hand-checked pack



Speed reached after a displacement x from $v_0 = 6$ m/s, for three constant accelerations (24 displacements each from the package): the speed grows as the square root of the distance, not linearly. Marked at x = 16 m: a = 1: 8.24621 m/s; a = 2: 10 m/s; a = 4: 12.8062 m/s.

```
classical_physics_ai999x.solve('velocity_displacement', v0='6 m/s', a='2 m/s^2', x='16 m',
target='v')
# -> 10 m/s   (SOLVED)
```

$$v^2 = v_0^2 + 2ax$$
$$v = -\sqrt{2ax + v_0^2} = 10\ \mathrm{m/s}$$

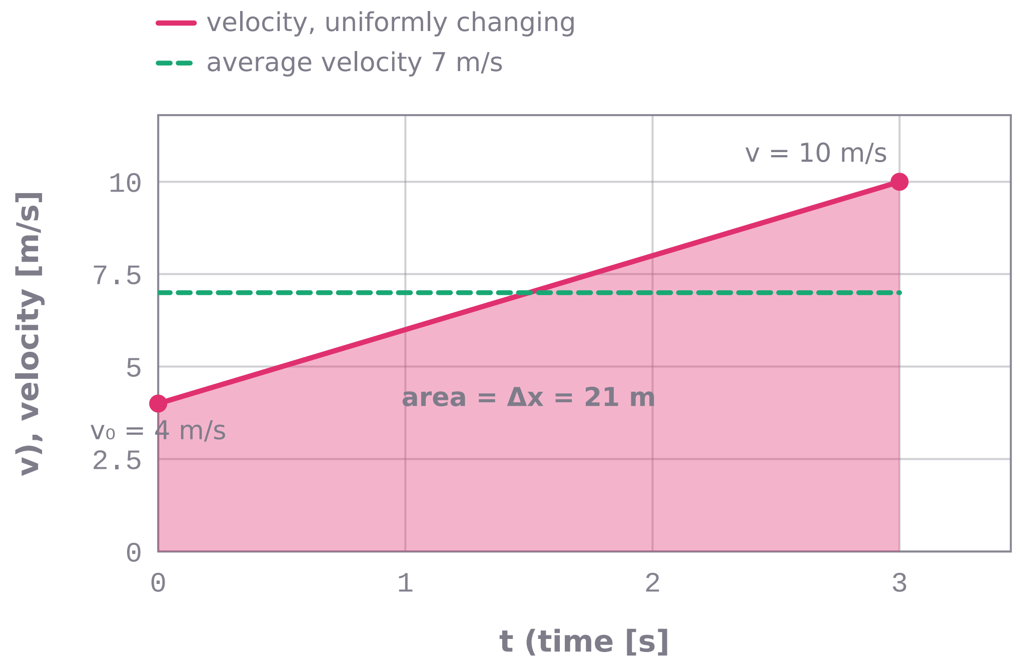
invalid call	outcome	message
without `v0`	NEEDS_INPUT	velocity_displacement needs v0 (initial speed, m/s). Example: solve('velocity_displacement', v0='... m/s', a='... m/s^2', x='... m').
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

Displacement from the mean of initial and final velocity, `average_velocity_displacement`

$$x = \frac{(v_0 + v)t}{2}$$

input	meaning	kind / role	unit	default
`x`	displacement	length / distance	`m`	
`v0`	initial velocity	speed / initial	`m/s`	
`v`	final velocity	speed / final	`m/s`	
`t`	elapsed time	time / plain	`s`	

- **Assumes:** constant acceleration along a line
- **Evidence:** hand-checked pack



Velocity changing uniformly from v₀ = 4 m/s to v = 10 m/s in t = 3 s: the trapezoid under the line is the displacement the package returns, Δx = 21 m; the dashed line is the average velocity Δx / t = 7 m/s (derived), which encloses the same area. Marked: Δx = 21 m.

```
classical_physics_ai999x.solve('average_velocity_displacement', v0='4 m/s', v='10 m/s',
t='3 s', target='x')
# -> 21 m   (SOLVED)
```

$$x = \frac{(v_0 + v)t}{2}$$

$$x = \frac{t\,(v + v_0)}{2} = 21\ \mathrm{m}$$

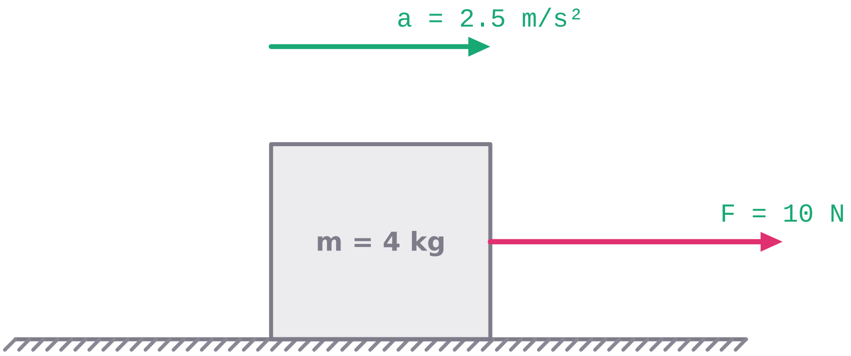
invalid call	outcome	message
without `v0`	NEEDS_INPUT	average_velocity_displacement needs v0 (initial velocity, m/s). Example: solve('average_velocity_displacement', v0='... m/s', v='... m/s', t
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

Net force and acceleration, `newton_second`

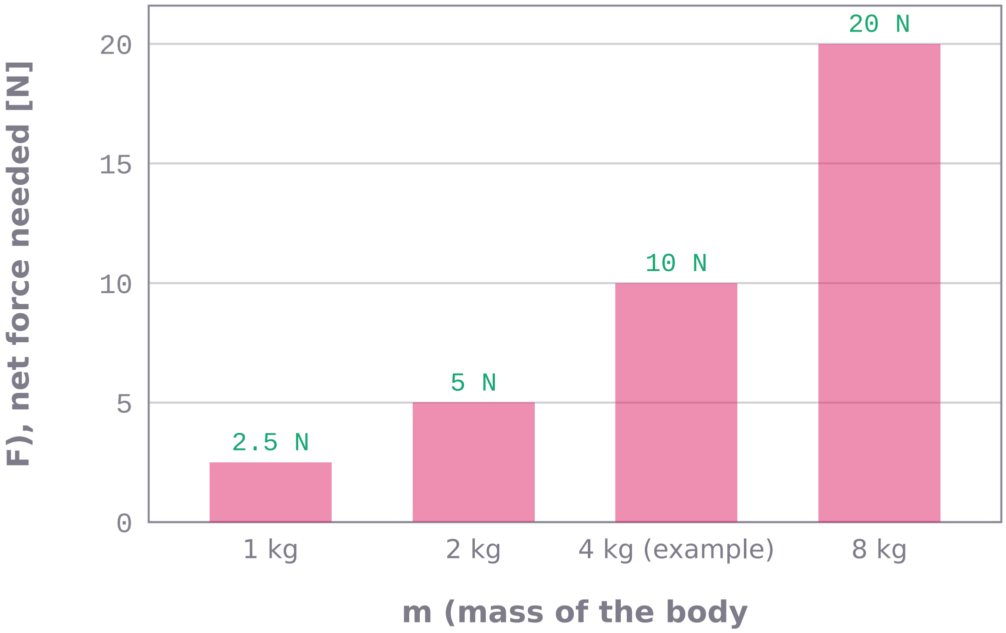
$$F = ma$$

input	meaning	kind / role	unit	default
`F`	net force	force / net	`N`	
`m`	mass	mass / plain	`kg`	
`a`	acceleration	acceleration / plain	`m/s^2`	

- **Assumes:** constant mass; inertial frame
- **Evidence:** hand-checked pack



Free-body view of Newton's second law: a body of m = 4 kg accelerating at a = 2.5 m/s² (the inputs) needs the net force the package returns, in the direction of the acceleration. Marked: F = 10 N.



Net force needed to give bodies of 1, 2, 4 and 8 kg the same acceleration a = 2.5 m/s², each from the package: twice the mass needs twice the force. Marked: the force for each mass.

```
classical_physics_ai999x.solve('newton_second', m='4 kg', a='2.5 m/s^2', target='F')
# -> 10 N   (SOLVED)
```

$$F = ma$$
$$F = am = 10\,\mathrm{N}$$

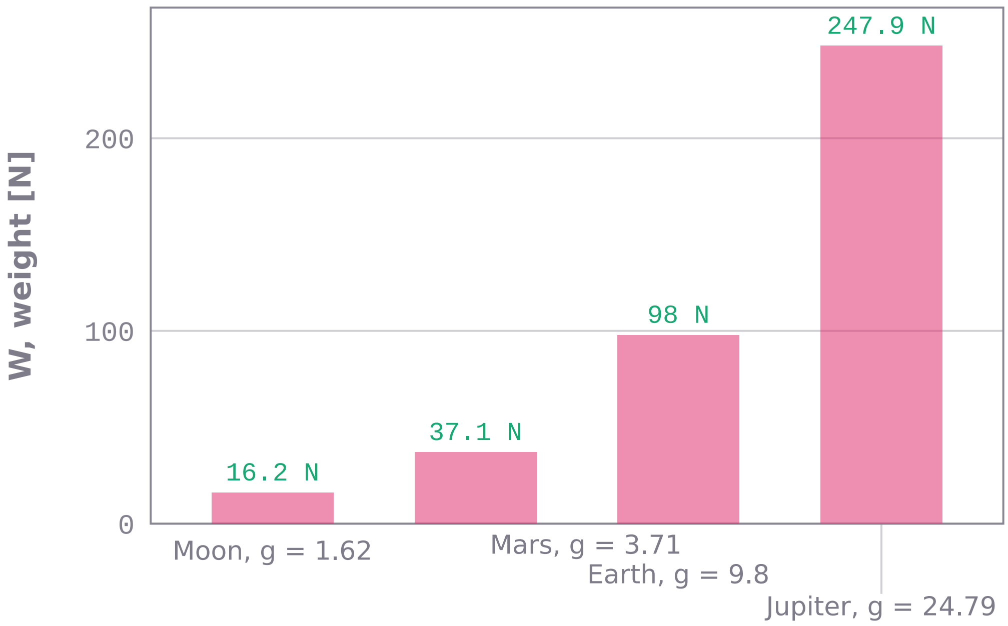
invalid call	outcome	message
without `m`	NEEDS_INPUT	newton_second needs m (mass, kg). Example: solve('newton_second', m='... kg', a='... m/s^2').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Weight in a uniform gravitational field, `weight`

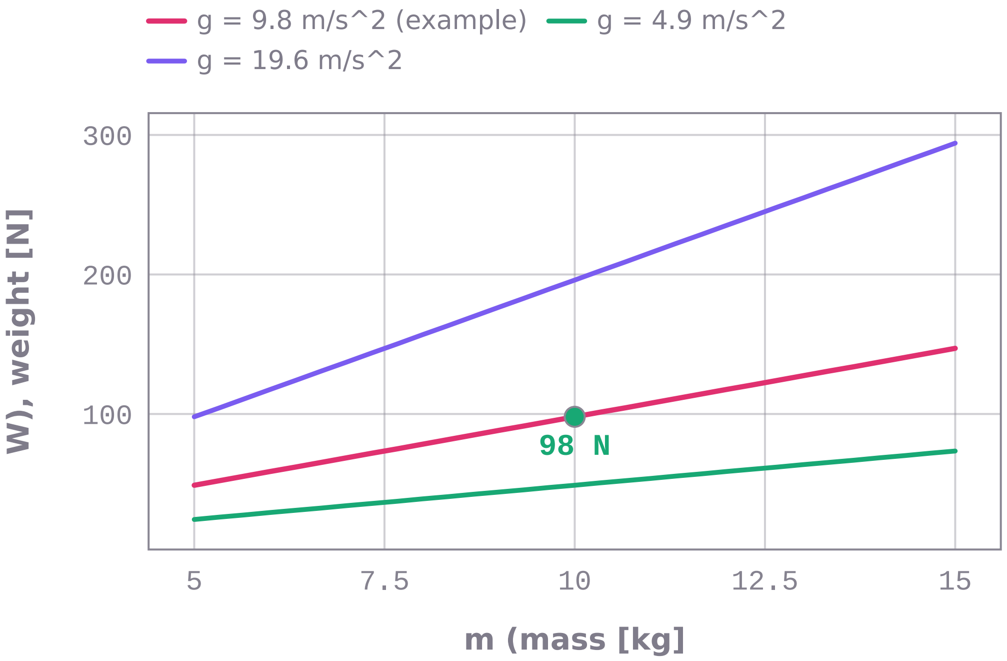
$$W = mg$$

input	meaning	kind / role	unit	default
`W`	weight	force / weight	`N`	
`m`	mass	mass / plain	`kg`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Assumes:** uniform gravitational field
- **Evidence:** hand-checked pack



Weight of the same 10 kg mass where the gravitational acceleration is 1.62 m/s² (Moon), 3.71 m/s² (Mars), 9.8 m/s² (Earth), 24.79 m/s² (Jupiter), each from the package: the mass is the same, the weight scales with g. Marked: each weight.



W (weight) against m (mass) from 5 kg to 15 kg. One curve for each value of g (gravitational acceleration): 9.8 m/s^2, 4.9 m/s^2, 19.6 m/s^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 10 kg, W = 98 N.

```
classical_physics_ai999x.solve('weight', m='10 kg', g='9.8 m/s^2', target='W')
# -> 98 N   (SOLVED)
```

$$W = mg$$
$$W = gm = 98\,\mathrm{N}$$

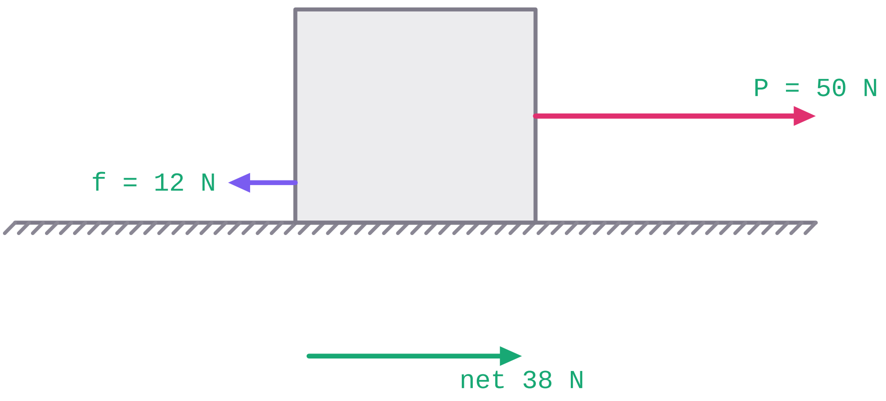
invalid call	outcome	message
without `m`	NEEDS_INPUT	weight needs m (mass, kg). Example: solve('weight', m='... kg', g='... m/s^2').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Net force with an opposing friction force, `net_force_friction`

$$F = P - f$$

input	meaning	kind / role	unit	default
`F`	net force	force / net	`N`	
`P`	applied force	force / applied	`N`	
`f`	friction force	force / friction	`N`	

- **Assumes:** applied force and friction act along the same line, in opposite directions
- **Evidence:** hand-checked pack



A block pushed by an applied force P = 50 N against friction f = 12 N (the inputs), arrows to scale; the package returns the net horizontal force, drawn below. Marked: net force 38 N.

```
classical_physics_ai999x.solve('net_force_friction', P='50 N', f='12 N', target='F')
# -> 38 N   (SOLVED)
```

$$F = P - f$$
$$F = P - f = 38\,\mathrm{N}$$

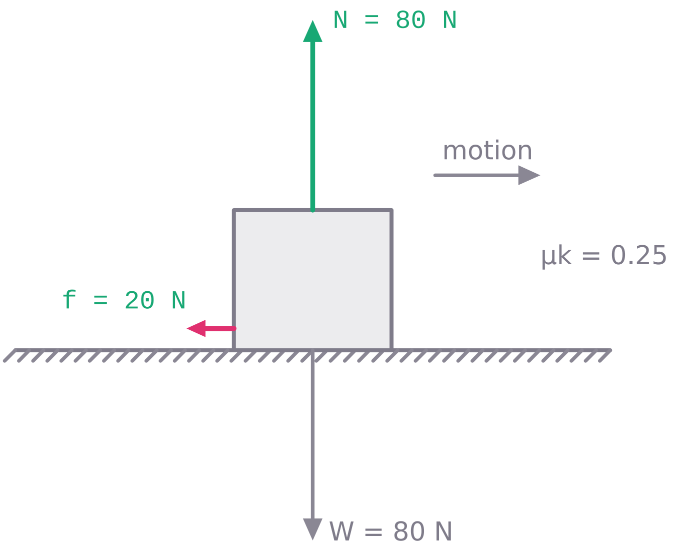
invalid call	outcome	message
without `P`	NEEDS_INPUT	net_force_friction needs P (applied force, N). Example: solve('net_force_friction', P='... N', f='... N').
`P` in a wrong unit	INVALID_INPUT	P is a force: its unit must be like N, not 'kg'.

Sliding friction, `kinetic_friction`

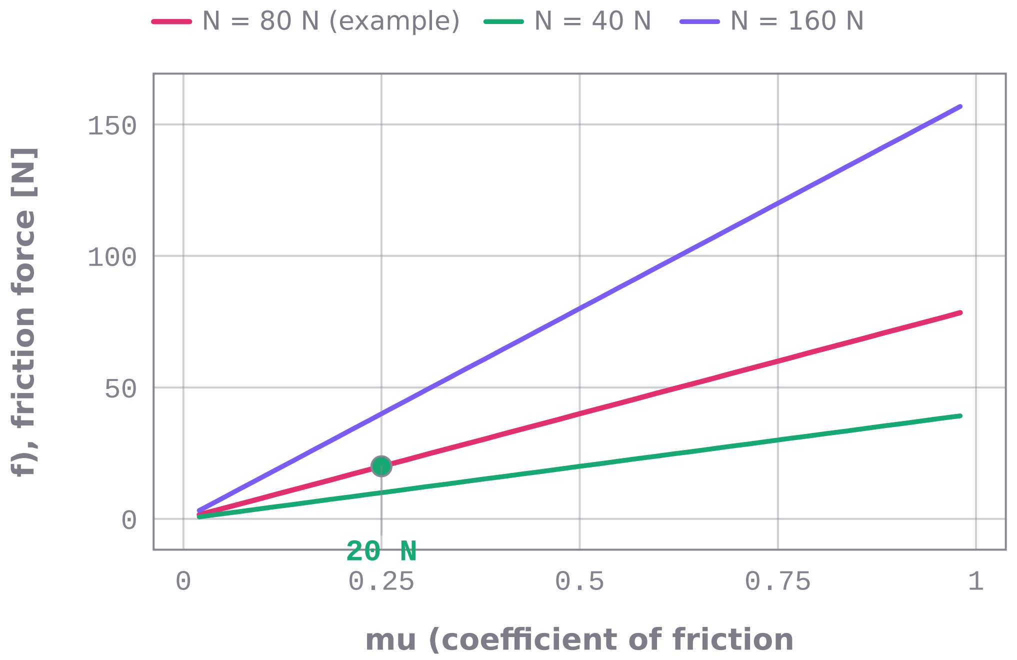
$$f = \mu N$$

input	meaning	kind / role	unit	default
`f`	friction force	force / friction	`N`	
`mu`	coefficient of friction	friction_coefficient / plain	`1`	
`N`	normal force	force / normal	`N`	

- **Assumes:** the surfaces slide over each other
- **Evidence:** hand-checked pack



A block sliding on a level floor: normal force N = 80 N (input, balanced by the weight), coefficient of kinetic friction μk = 0.25; the friction force the package returns acts against the motion. Arrows to scale. Marked: f = 20 N.



f (friction force) against mu (coefficient of friction) from 0.02 to 0.98. One curve for each value of N (normal force): 80 N, 40 N, 160 N. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, mu = 0.25, f = 20 N.

```
classical_physics_ai999x.solve('kinetic_friction', mu='0.25 1', N='80 N', target='f')
# -> 20 N   (SOLVED)
```

$$f = \mu N$$
$$f = N\mu = 20\ \mathrm{N}$$

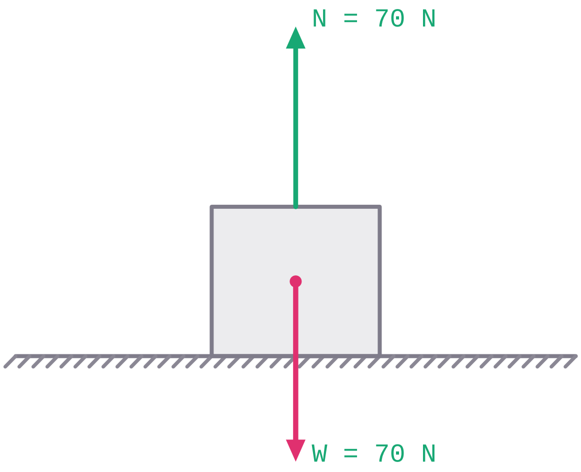
invalid call	outcome	message
without `mu`	NEEDS_INPUT	kinetic_friction needs mu (coefficient of friction, 1). Example: solve('kinetic_friction', mu='... 1', N='... N').
`mu` in a wrong unit	INVALID_INPUT	mu is a friction coefficient: its unit must be like 1, not 'kg'.

Normal force on a level surface, `normal_on_level`

$$N = W$$

input	meaning	kind / role	unit	default
`N`	normal force	force / normal	`N`	
`W`	weight	force / weight	`N`	

- **Not when:** inclined
- **Assumes:** level surface; no other vertical forces
- **Evidence:** hand-checked pack



A body at rest on a level surface with weight W = 70 N (input): the surface pushes back with the normal force the package returns, equal and opposite. Arrows to scale. Marked: N = 70 N.

```
classical_physics_ai999x.solve('normal_on_level', W='70 N', target='N')
# -> 70 N   (SOLVED)
```

$$N = W$$
$$N = W = 70\,\mathrm{N}$$

invalid call	outcome	message
without `W`	NEEDS_INPUT	normal_on_level needs W (weight, N). Example: solve('normal_on_level', W='... N').
`W` in a wrong unit	INVALID_INPUT	W is a force: its unit must be like N, not 'kg'.

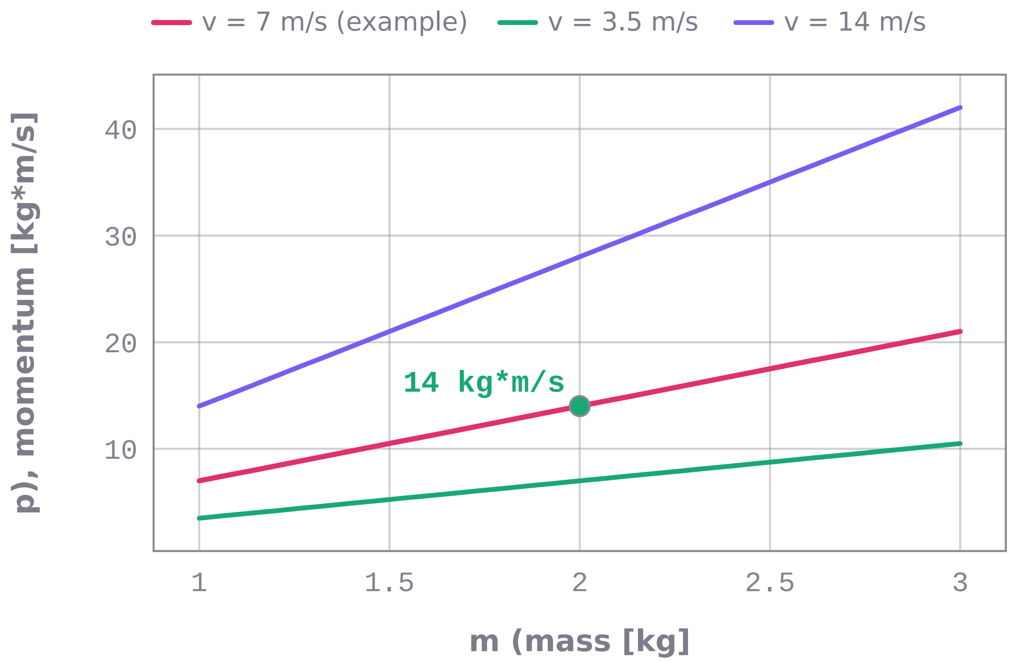
Linear momentum, `momentum`

$$p = mv$$

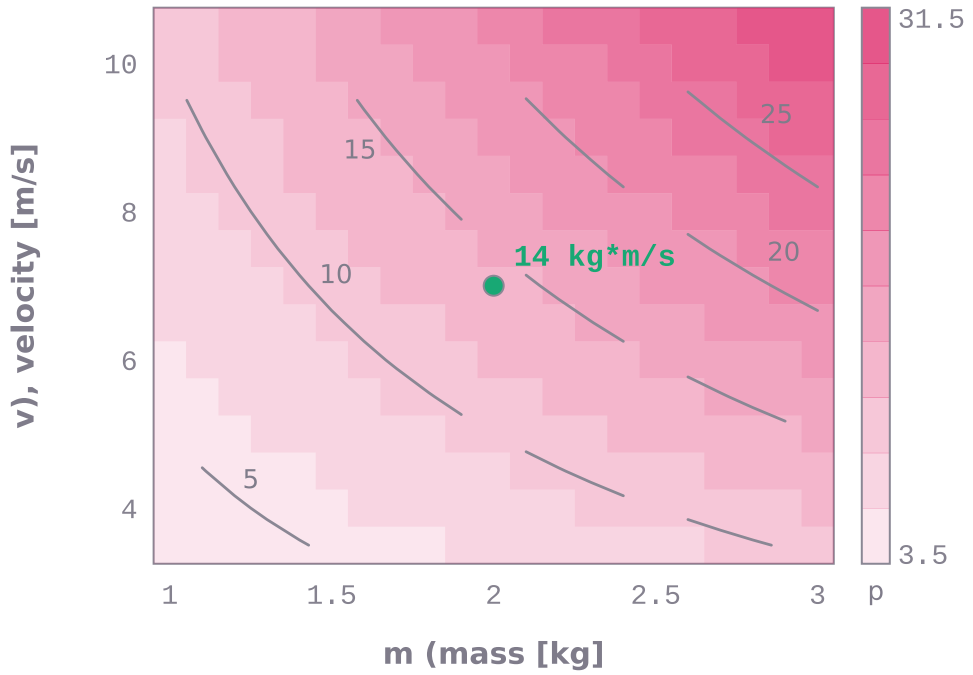
input	meaning	kind / role	unit	default
`p`	momentum	momentum / plain	`kg*m/s`	

input	meaning	kind / role	unit	default
`m`	mass	mass / plain	`kg`	
`v`	velocity	speed / plain	`m/s`	

- **Assumes:** speeds far below the speed of light
- **Evidence:** hand-checked pack



p (momentum) against m (mass) from 1 kg to 3 kg. One curve for each value of v (velocity): 7 m/s, 3.5 m/s, 14 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, p = 14 kg*m/s.



p (momentum) over m (mass) from 1 kg to 3 kg and v (velocity) from 3.5 m/s to 10.5 m/s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 3.5 kg*m/s to dark 31.5 kg*m/s); iso-lines at p = 5, 10, 15, 20, 25 kg*m/s. Marked: the worked example, m = 2 kg, v = 7 m/s, p = 14 kg*m/s.

```
classical_physics_ai999x.solve('momentum', m='2 kg', v='7 m/s', target='p')
# -> 14 kg*m/s   (SOLVED)
```

$$p = mv$$
$$p = mv = 14\,\mathrm{kg\,m/s}$$

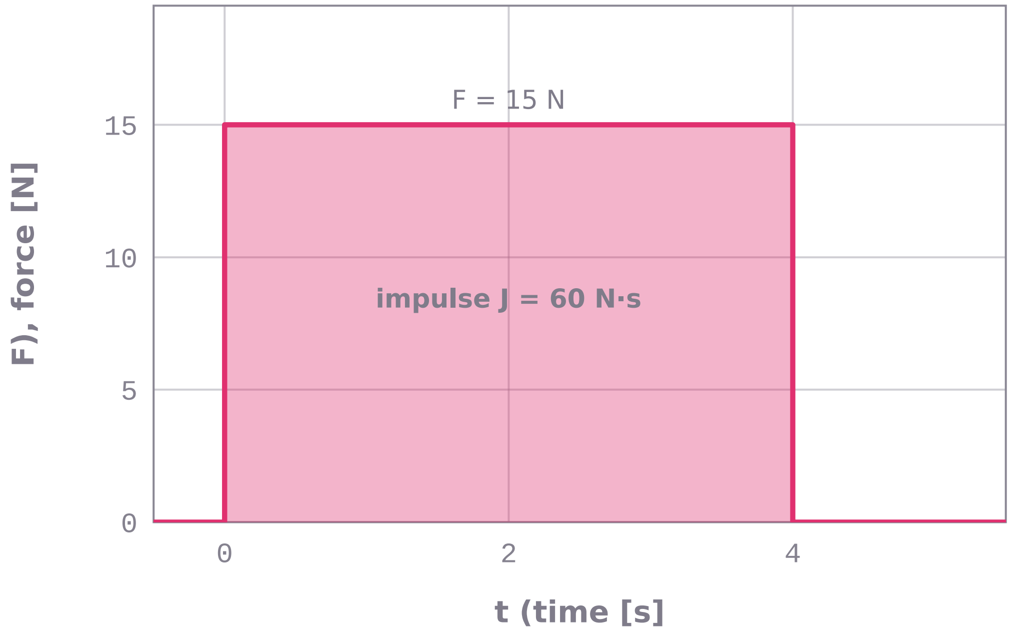
invalid call	outcome	message
without `m`	NEEDS_INPUT	momentum needs m (mass, kg). Example: solve('momentum', m='... kg', v='... m/s').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Impulse of a constant force, `impulse`

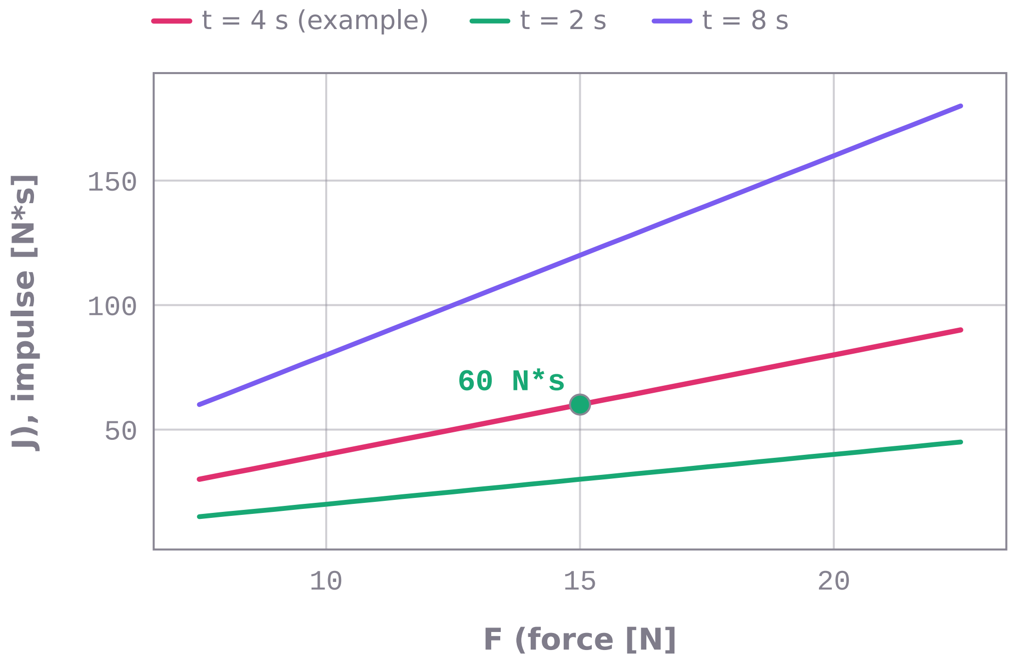
$$J = Ft$$

input	meaning	kind / role	unit	default
`J`	impulse	impulse / plain	`N*s`	
`F`	force	force / plain	`N`	
`t`	time the force acts	time / plain	`s`	

- **Assumes:** constant force
- **Evidence:** hand-checked pack



A constant force F = 15 N acting for t = 4 s: the impulse is the area under the force–time graph. Marked: the impulse the package returns, J = 60 N·s (the change of momentum it causes).



J (impulse) against F (force) from 7.5 N to 22.5 N. One curve for each value of t (time the force acts): 4 s, 2 s, 8 s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, F = 15 N, J = 60 N*s.

```
classical_physics_ai999x.solve('impulse', F='15 N', t='4 s', target='J')
# -> 60 N*s   (SOLVED)
```

$$J = F t$$
$$J = F t = 60\,\mathrm{N\,s}$$

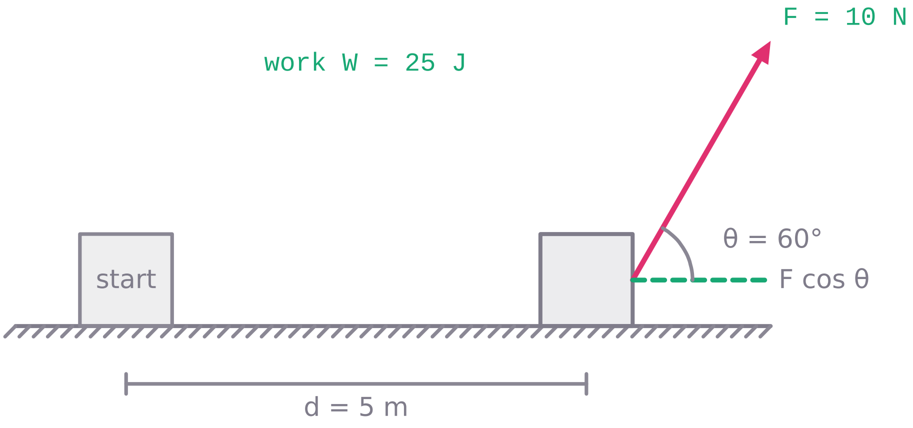
invalid call	outcome	message
without `F`	NEEDS_INPUT	impulse needs F (force, N). Example: solve('impulse', F='... N', t='... s').
`F` in a wrong unit	INVALID_INPUT	F is a force: its unit must be like N, not 'kg'.

Work by a constant force, `work`

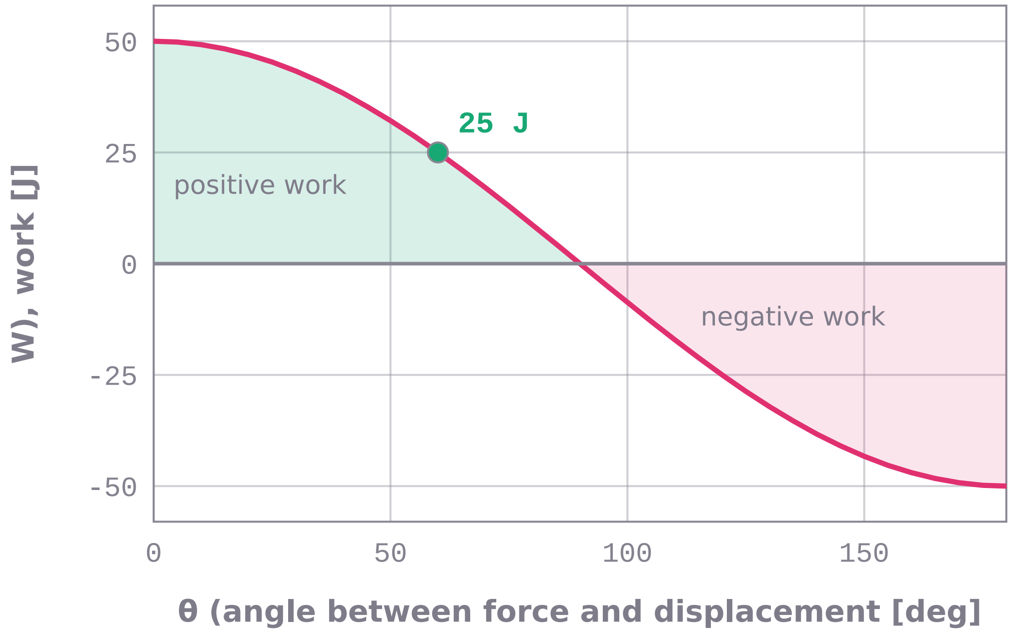
$$W = F d \cos\left(\theta\right)$$

input	meaning	kind / role	unit	default
`W`	work done	energy / work	`J`	
`F`	force	force / plain	`N`	
`d`	displacement	length / distance	`m`	
`theta`	angle between force and displacement	angle / plain	`rad`	0 deg

- **Fails when:** the force is friction and no angle is stated: friction opposes the motion (180 degrees), which must be said, not assumed
- **Assumes:** constant force
- **Evidence:** hand-checked pack



A force F = 10 N pulling at θ = 60° above the direction of a displacement d = 5 m (the inputs): only its component along the motion, F cos θ, does work. Marked: the work the package returns, W = 25 J.



Work of a force F = 10 N over d = 5 m against the angle between them (37 angles from the package): positive below 90°, zero for a perpendicular force, negative when the force opposes the motion. Marked: 25 J at 60°.

```
classical_physics_ai999x.solve('work', F='10 N', d='5 m', theta='60 deg', target='W')
# -> 25 J   (SOLVED)
```

$$W = F d \cos\left(\theta\right)$$

$$W = F d \cos\left(\theta\right) = 25\ \mathrm{J}$$

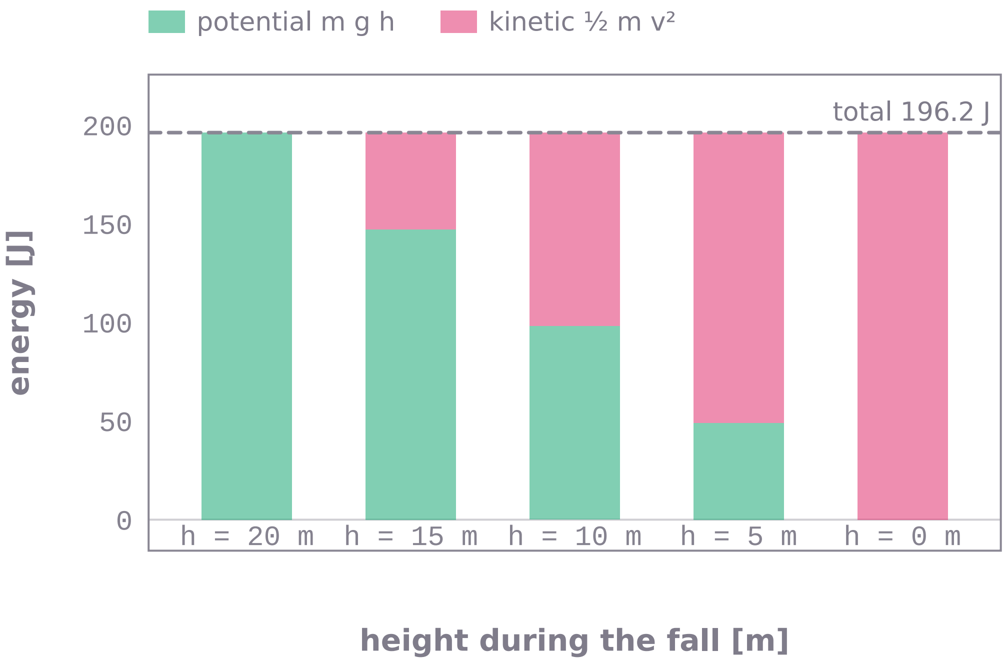
invalid call	outcome	message
without `F`	NEEDS_INPUT	work needs F (force, N). Example: solve('work', F='... N', d='... m', theta='... rad').
`F` in a wrong unit	INVALID_INPUT	F is a force: its unit must be like N, not 'kg'.

Translational kinetic energy, `kinetic_energy`

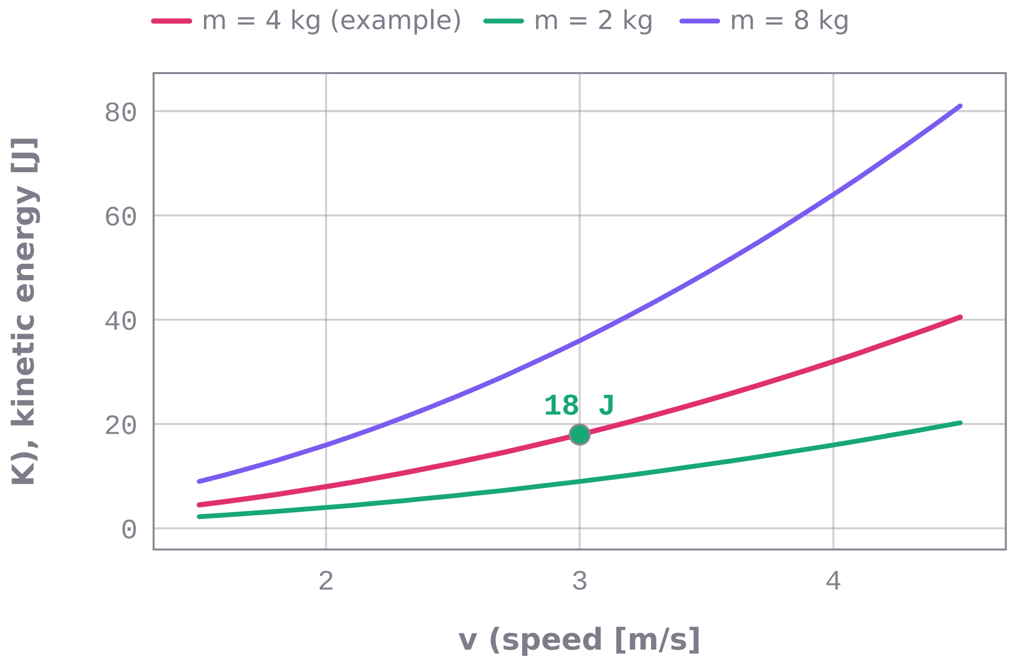
$$K = \frac{mv^2}{2}$$

input	meaning	kind / role	unit	default
`K`	kinetic energy	energy / kinetic	`J`	
`m`	mass	mass / plain	`kg`	
`v`	speed	speed / plain	`m/s`	

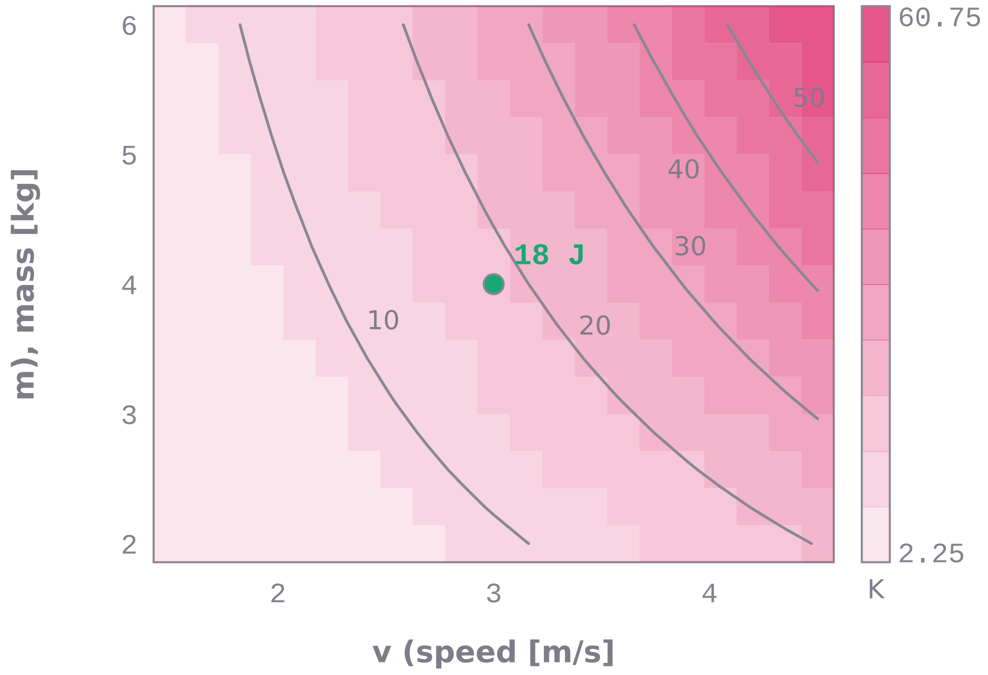
- **Assumes:** speeds far below the speed of light
- **Evidence:** hand-checked pack



Energy of a 1 kg body falling from rest from 20 m (g = 9.81 m/s², no air resistance): potential energy and kinetic energy (from the speed after the fall), each computed by the package, stacked, their sum stays 196.2 J: h = 20 m: U = 196.2 J, K = 0 J (v = 0 m/s); h = 15 m: U = 147.15 J, K = 49.05 J (v = 9.90454 m/s); h = 10 m: U = 98.1 J, K = 98.1 J (v = 14.0071 m/s); h = 5 m: U = 49.05 J, K = 147.15 J (v = 17.1552 m/s); h = 0 m: U = 0 J, K = 196.2 J (v = 19.8091 m/s).



K (kinetic energy) against v (speed) from 1.5 m/s to 4.5 m/s. One curve for each value of m (mass): 4 kg, 2 kg, 8 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, v = 3 m/s, K = 18 J.



K (kinetic energy) over v (speed) from 1.5 m/s to 4.5 m/s and m (mass) from 2 kg to 6 kg: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 2.25 J to dark 60.75 J); iso-lines at K = 10, 20, 30, 40, 50 J. Marked: the worked example, v = 3 m/s, m = 4 kg, K = 18 J.

```
classical_physics_ai999x.solve('kinetic_energy', m='4 kg', v='3 m/s', target='K')
# -> 18 J   (SOLVED)
```

$$K = \frac{mv^2}{2}$$

$$K = \frac{mv^2}{2} = 18\ \mathrm{J}$$

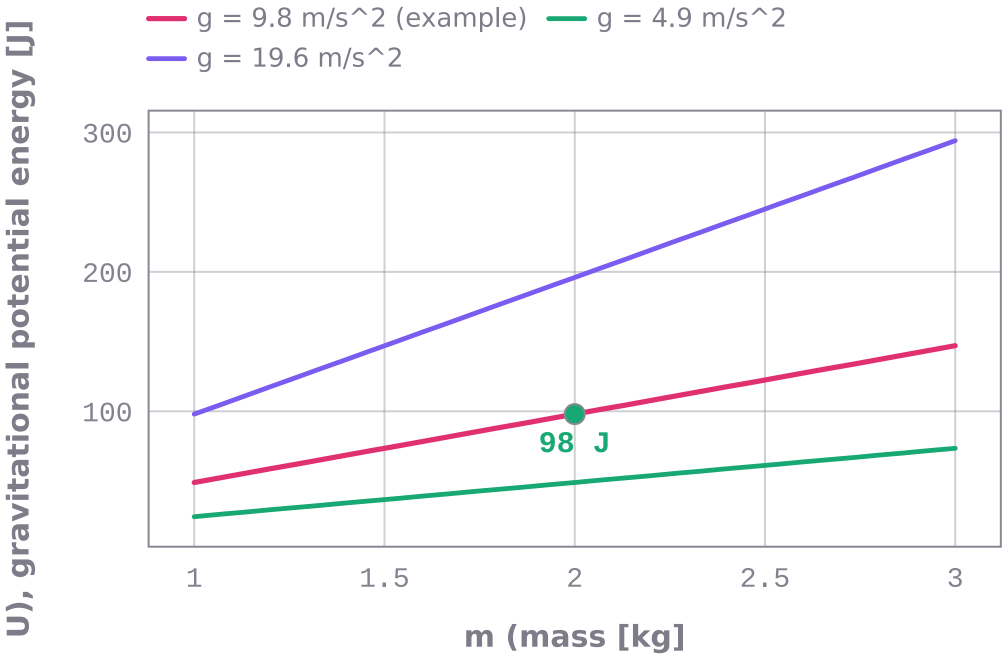
invalid call	outcome	message
without `m`	NEEDS_INPUT	kinetic_energy needs m (mass, kg). Example: solve('kinetic_energy', m='... kg', v='... m/s').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Gravitational potential energy near the surface, `gravitational_pe`

$$U = mgh$$

input	meaning	kind / role	unit	default
`U`	gravitational potential energy	energy / potential	`J`	
`m`	mass	mass / plain	`kg`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	height above the reference level	length / height	`m`	

- **Assumes:** uniform gravitational field; zero of potential energy at $h = 0$
- **Evidence:** hand-checked pack



U (gravitational potential energy) against m (mass) from 1 kg to 3 kg. One curve for each value of g (gravitational acceleration): 9.8 m/s^2, 4.9 m/s^2, 19.6 m/s^2. Other inputs as in the worked example: h = 5 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, U = 98 J.

```
classical_physics_ai999x.solve('gravitational_pe', m='2 kg', g='9.8 m/s^2', h='5 m',
target='U')
# -> 98 J   (SOLVED)
```

$$U = mgh$$
$$U = ghm = 98\ \mathrm{J}$$

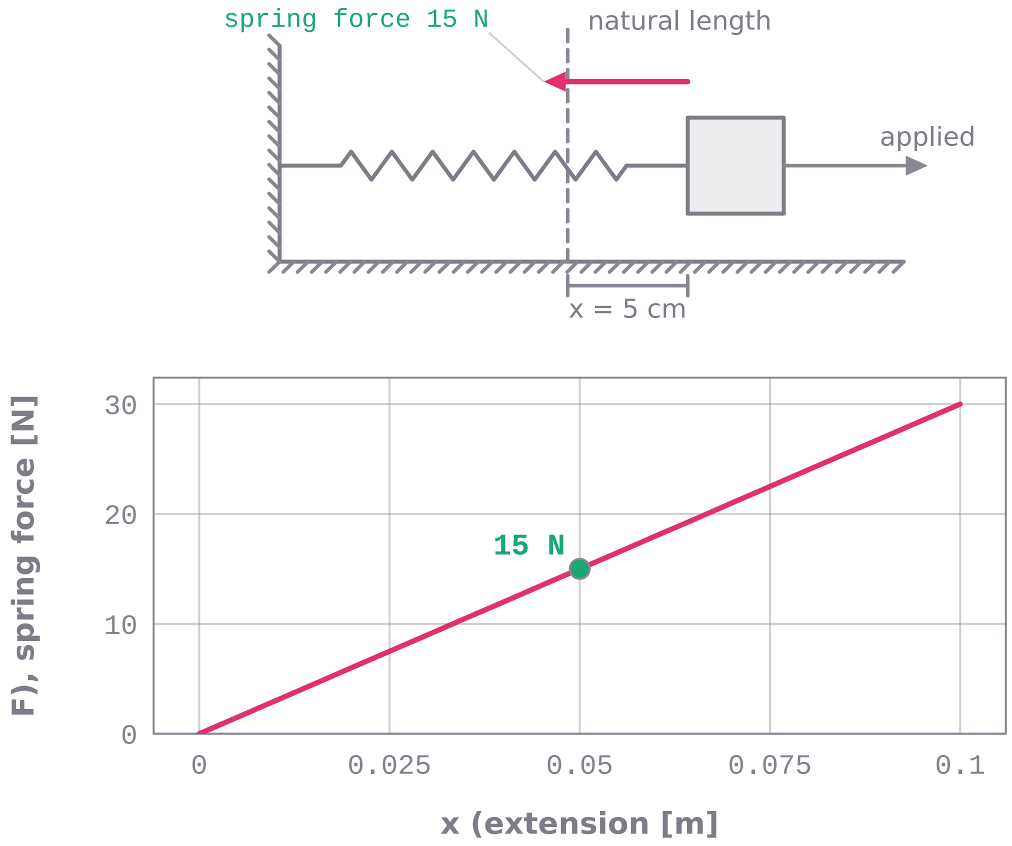
invalid call	outcome	message
without `m`	NEEDS_INPUT	gravitational_pe needs m (mass, kg). Example: solve('gravitational_pe', m='... kg', g='... m/s^2', h='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Force of a linear spring, `spring_force`

$$F = kx$$

input	meaning	kind / role	unit	default
`F`	spring force (magnitude)	force / spring	`N`	
`k`	spring constant	spring_constant / plain	`N/m`	
`x`	extension or compression	length / extension	`m`	

- **Assumes:** linear elastic spring within its elastic limit
- **Evidence:** hand-checked pack



A spring of stiffness k = 300 N/m stretched x = 5 cm beyond its natural length: it pulls back with the force the package returns (Hooke's law), and the force grows in proportion to the extension (20 extensions below). Marked: F = 15 N at x = 0.05 m.

```
classical_physics_ai999x.solve('spring_force', k='300 N/m', x='5 cm', target='F')
# -> 15 N   (SOLVED)
```

$$F = kx$$
$$F = kx = 15\ \mathrm{N}$$

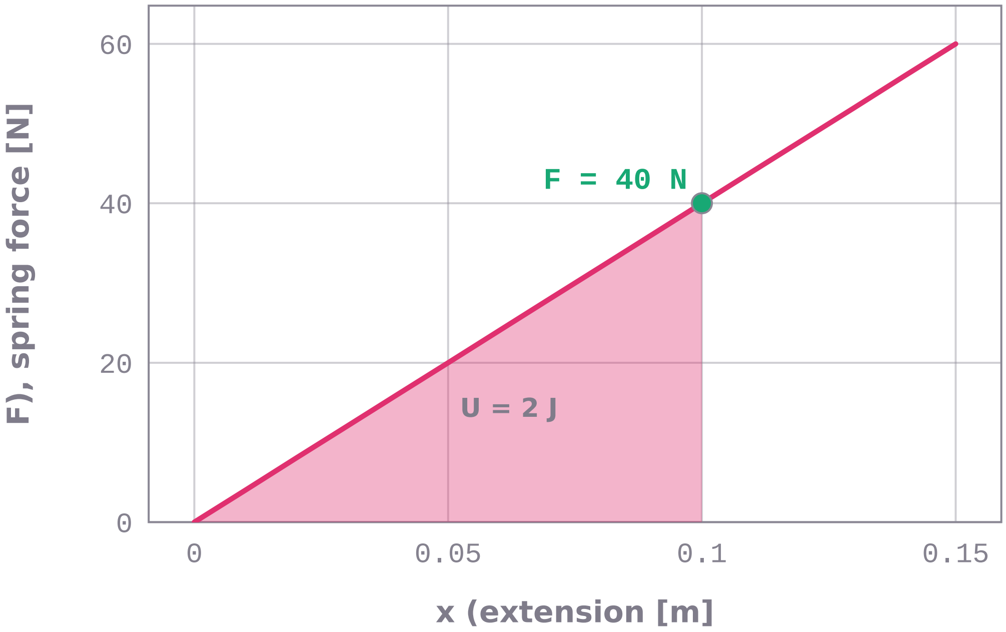
invalid call	outcome	message
without `k`	NEEDS_INPUT	spring_force needs k (spring constant, N/m). Example: solve('spring_force', k='... N/m', x='... m').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Energy stored in a linear spring, `spring_energy`

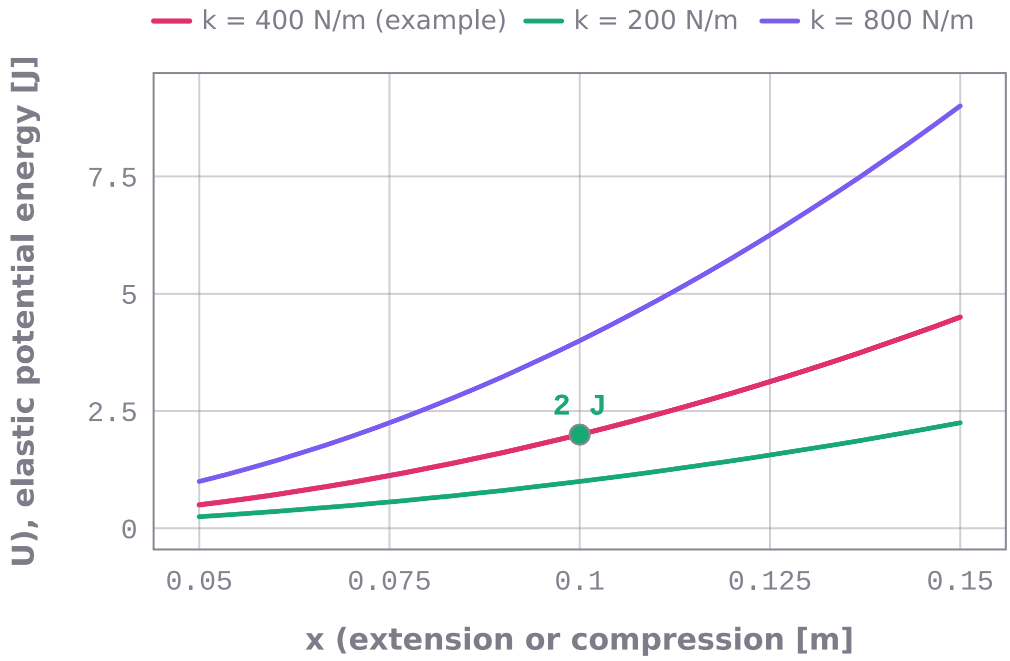
$$U = \frac{kx^2}{2}$$

input	meaning	kind / role	unit	default
`U`	elastic potential energy	energy / elastic	`J`	
`k`	spring constant	spring_constant / plain	`N/m`	
`x`	extension or compression	length / extension	`m`	

- **Assumes:** linear elastic spring within its elastic limit
- **Evidence:** hand-checked pack



Force of a spring with k = 400 N/m against its extension (from the package's Hooke's-law formula): the energy stored when it is stretched x = 0.1 m is the triangle under the line up to x. Marked: stored energy U = 2 J (returned) and F = 40 N at x.



U (elastic potential energy) against x (extension or compression) from 0.05 m to 0.15 m. One curve for each value of k (spring constant): 400 N/m, 200 N/m, 800 N/m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, x = 0.1 m, U = 2 J.

```
classical_physics_ai999x.solve('spring_energy', k='400 N/m', x='0.1 m', target='U')
# -> 2 J   (SOLVED)
```

$$U = \frac{kx^2}{2}$$

$$U = \frac{kx^2}{2} = 2\,\mathrm{J}$$

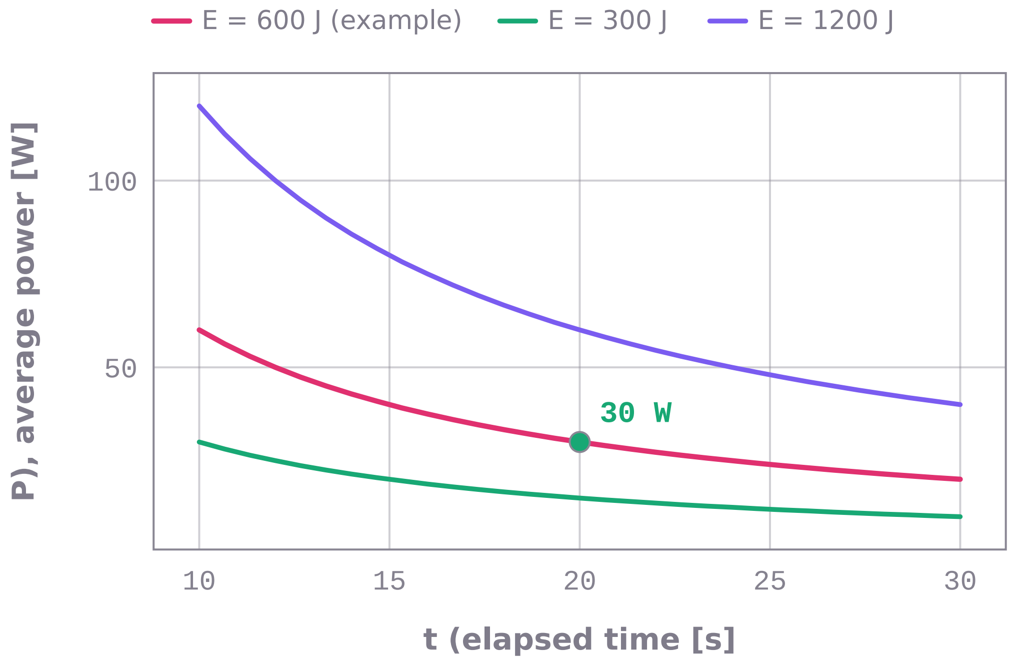
invalid call	outcome	message
without `k`	NEEDS_INPUT	spring_energy needs k (spring constant, N/m). Example: solve('spring_energy', k='... N/m', x='... m').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Average power, `power_energy`

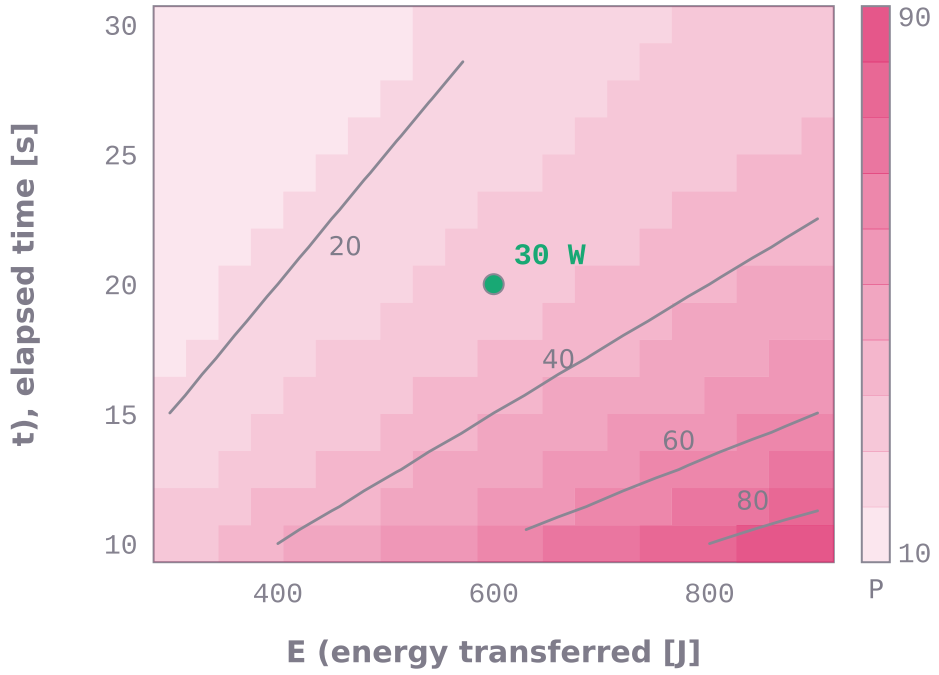
$$P = \frac{E}{t}$$

input	meaning	kind / role	unit	default
`P`	average power	power / plain	`W`	
`E`	energy transferred	energy / plain	`J`	
`t`	elapsed time	time / plain	`s`	

- **Assumes:** power averaged over the interval
- **Evidence:** hand-checked pack



P (average power) against t (elapsed time) from 10 s to 30 s. One curve for each value of E (energy transferred): 600 J, 300 J, 1200 J. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, t = 20 s, P = 30 W.



P (average power) over E (energy transferred) from 300 J to 900 J and t (elapsed time) from 10 s to 30 s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 10 W to dark 90 W); iso-lines at P = 20, 40, 60, 80 W. Marked: the worked example, E = 600 J, t = 20 s, P = 30 W.

```
classical_physics_ai999x.solve('power_energy', E='600 J', t='20 s', target='P')
# -> 30 W   (SOLVED)
```

$$P = \frac{E}{t}$$

$$P = \frac{E}{t} = 30\ \mathrm{W}$$

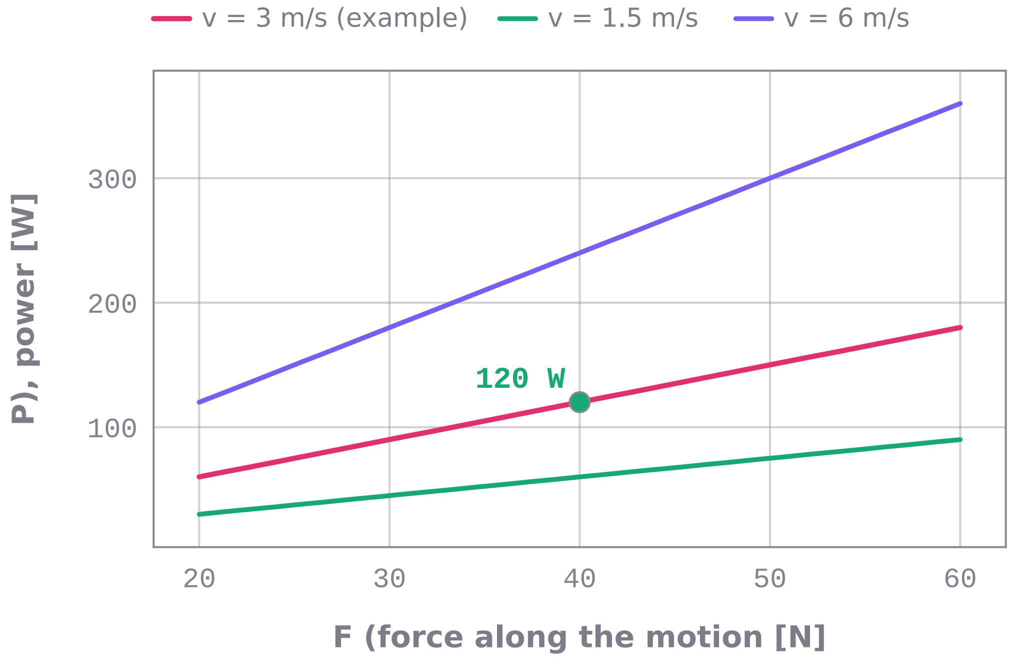
invalid call	outcome	message
without `E`	NEEDS_INPUT	power_energy needs E (energy transferred, J). Example: solve('power_energy', E='... J', t='... s').
`E` in a wrong unit	INVALID_INPUT	E is a energy: its unit must be like J, not 'kg'.

Power of a force on a moving body, `power_force_speed`

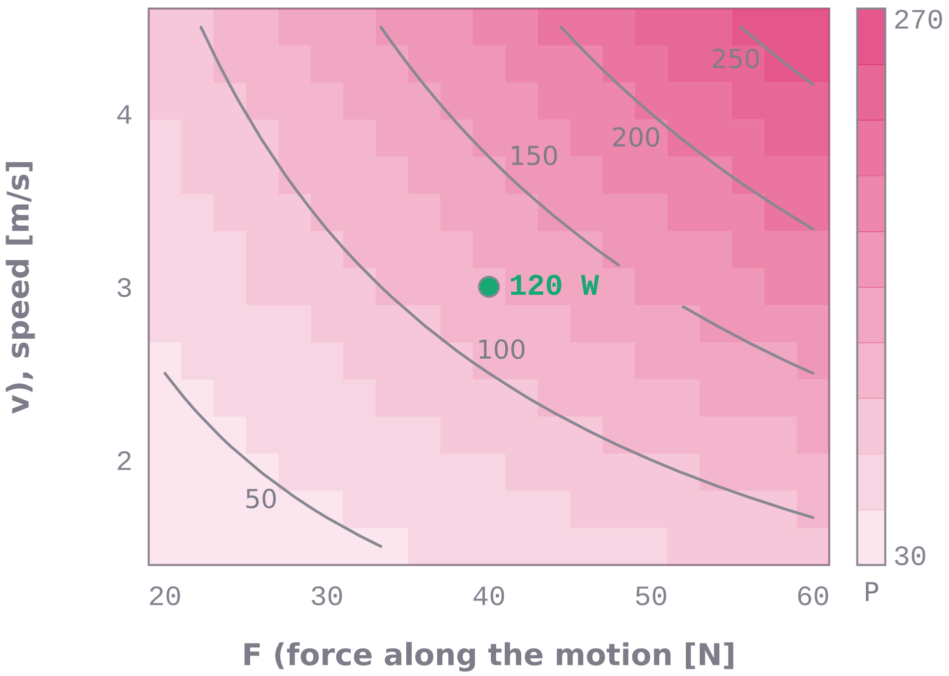
$$P = Fv$$

input	meaning	kind / role	unit	default
`P`	power	power / plain	`W`	
`F`	force along the motion	force / plain	`N`	
`v`	speed	speed / plain	`m/s`	

- **Not when:** accelerated, oblique_force
- **Assumes:** force parallel to the velocity; constant speed
- **Evidence:** hand-checked pack



P (power) against F (force along the motion) from 20 N to 60 N. One curve for each value of v (speed): 3 m/s, 1.5 m/s, 6 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, F = 40 N, P = 120 W.



P (power) over F (force along the motion) from 20 N to 60 N and v (speed) from 1.5 m/s to 4.5 m/s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 30 W to dark 270 W); iso-lines at P = 50, 100, 150, 200, 250 W. Marked: the worked example, F = 40 N, v = 3 m/s, P = 120 W.

```
classical_physics_ai999x.solve('power_force_speed', F='40 N', v='3 m/s', target='P')
# -> 120 W   (SOLVED)
```

$$P = Fv$$
$$P = Fv = 120\ \mathrm{W}$$

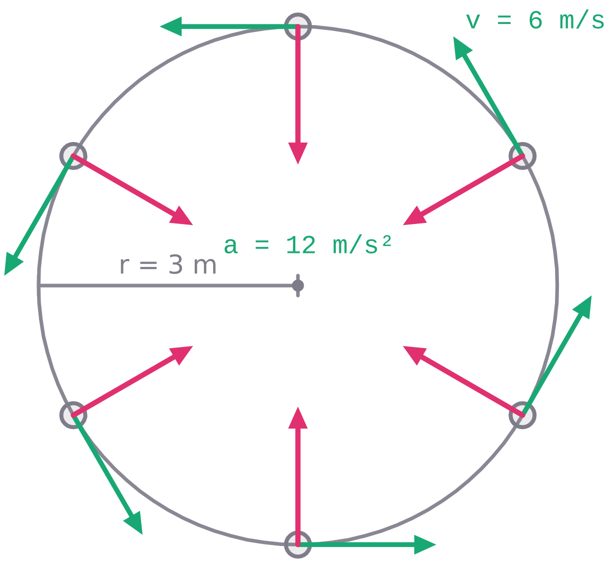
invalid call	outcome	message
without `F`	NEEDS_INPUT	power_force_speed needs F (force along the motion, N). Example: solve('power_force_speed', F='... N', v='... m/s').
`F` in a wrong unit	INVALID_INPUT	F is a force: its unit must be like N, not 'kg'.

Centripetal acceleration, `centripetal_acceleration`

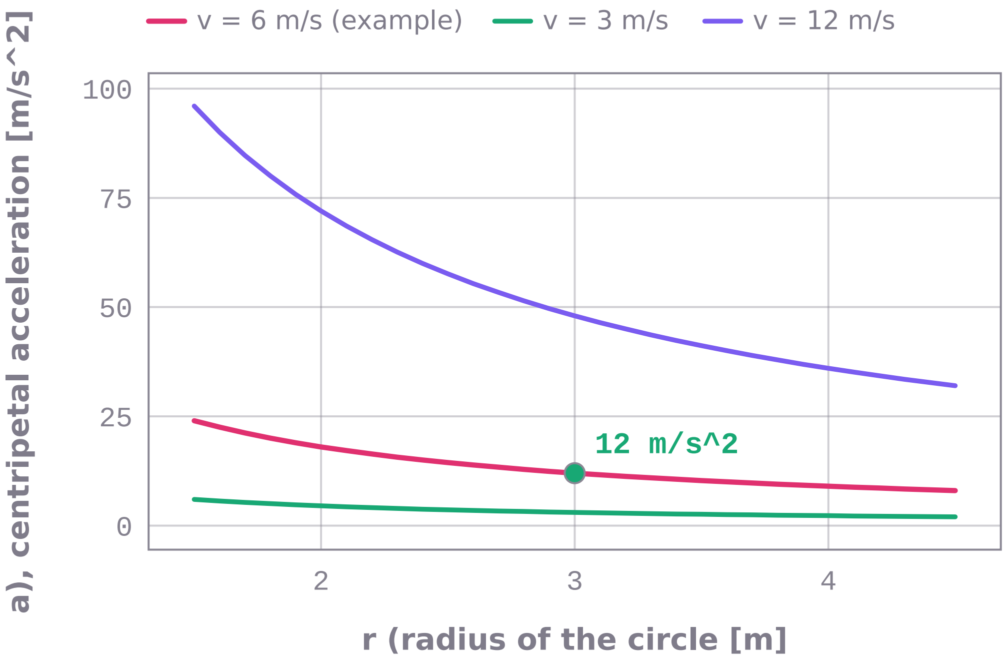
$$a = \frac{v^2}{r}$$

input	meaning	kind / role	unit	default
`a`	centripetal acceleration	acceleration / centripetal	`m/s^2`	
`v`	speed	speed / plain	`m/s`	
`r`	radius of the circle	length / radius	`m`	

- **Assumes:** uniform circular motion
- **Evidence:** hand-checked pack



A body moving round a circle of radius r = 3 m at a constant speed v = 6 m/s, at six positions: its velocity (green) is always along the circle, its acceleration (pink) always points to the centre. Marked: the centripetal acceleration the package returns, a = v²/r = 12 m/s².



a (centripetal acceleration) against r (radius of the circle) from 1.5 m to 4.5 m. One curve for each value of v (speed): 6 m/s, 3 m/s, 12 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 3 m, a = 12 m/s^2.

```
classical_physics_ai999x.solve('centripetal_acceleration', v='6 m/s', r='3 m', target='a')
# -> 12 m/s^2   (SOLVED)
```

$$a = \frac{v^2}{r}$$

$$a = \frac{v^2}{r} = 12\ \mathrm{m/s^2}$$

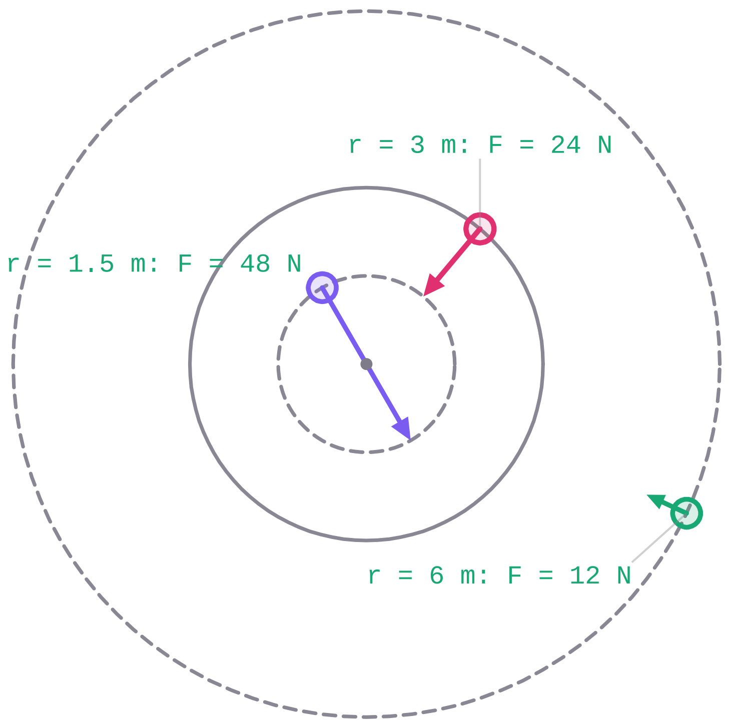
invalid call	outcome	message
without `v`	NEEDS_INPUT	centripetal_acceleration needs v (speed, m/s). Example: solve('centripetal_acceleration', v='... m/s', r='... m').
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

Centripetal force, `centripetal_force`

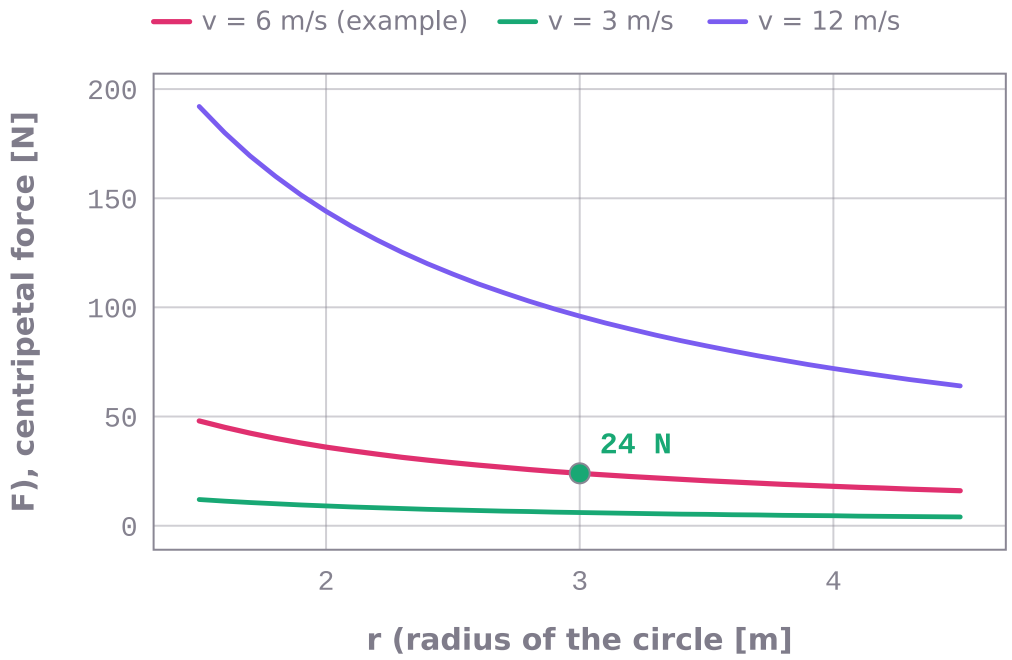
$$F = \frac{mv^2}{r}$$

input	meaning	kind / role	unit	default
`F`	centripetal force	force / centripetal	`N`	
`m`	mass	mass / plain	`kg`	
`v`	speed	speed / plain	`m/s`	
`r`	radius of the circle	length / radius	`m`	

- **Assumes:** uniform circular motion
- **Evidence:** hand-checked pack



Inward (centripetal) force needed to keep a 2 kg body moving at 6 m/s on circles of radius 1.5, 3 and 6 m (the worked example is r = 3 m), each from the package and drawn to scale towards the centre: halving the radius doubles the force. Marked: r = 1.5 m, F = 48 N; r = 3 m, F = 24 N; r = 6 m, F = 12 N.



F (centripetal force) against r (radius of the circle) from 1.5 m to 4.5 m. One curve for each value of v (speed): 6 m/s, 3 m/s, 12 m/s. Other inputs as in the worked example: m = 2 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 3 m, F = 24 N.

```
classical_physics_ai999x.solve('centripetal_force', m='2 kg', v='6 m/s', r='3 m',
target='F')
# -> 24 N   (SOLVED)
```

$$F = \frac{mv^2}{r}$$

$$F = \frac{mv^2}{r} = 24\ \mathrm{N}$$

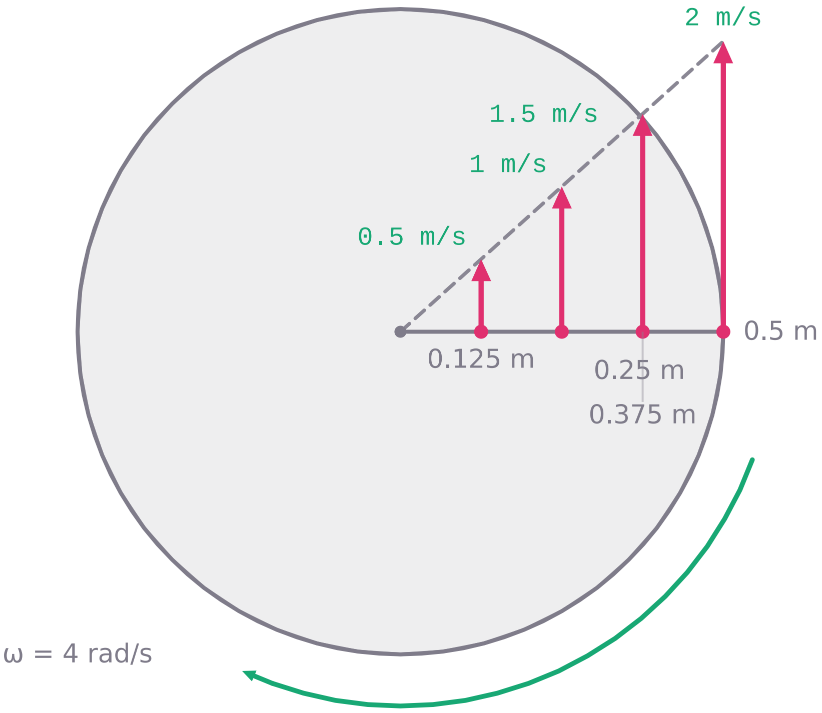
invalid call	outcome	message
without `m`	NEEDS_INPUT	centripetal_force needs m (mass, kg). Example: solve('centripetal_force', m='... kg', v='... m/s', r='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Speed on a circle from angular speed, `tangential_speed`

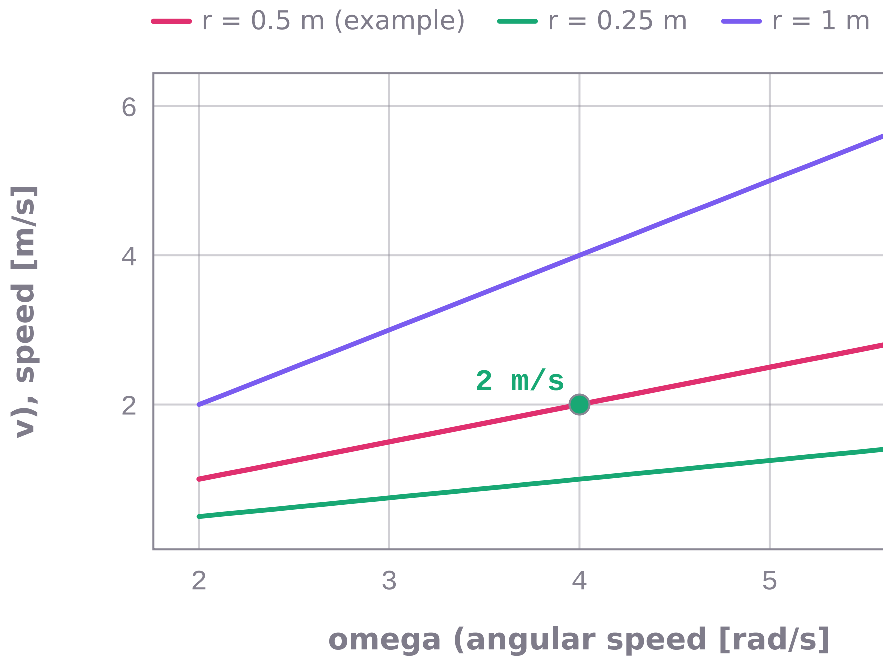
$$v = \omega r$$

input	meaning	kind / role	unit	default
`v`	speed	speed / plain	`m/s`	
`omega`	angular speed	angular_frequency / plain	`rad/s`	
`r`	radius of the circle	length / radius	`m`	

- **Assumes:** rotation about a fixed axis
- **Evidence:** hand-checked pack



A disc of radius 0.5 m turning at ω = 4 rad/s: the speed of a point grows in proportion to its distance from the axis (arrows to scale; the dashed line joins their tips). Marked: the tangential speed the package returns at r = 0.125, 0.25, 0.375, 0.5 m, 0.5 m/s, 1 m/s, 1.5 m/s, 2 m/s.



v (speed) against omega (angular speed) from 2 rad/s to 6 rad/s. One curve for each value of r (radius of the circle): 0.5 m, 0.25 m, 1 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, omega = 4 rad/s, v = 2 m/s.

```
classical_physics_ai999x.solve('tangential_speed', omega='4 rad/s', r='0.5 m', target='v')
# -> 2 m/s   (SOLVED)
```

$$v = \omega r$$

$$v = \omega r = 2\ \mathrm{m/s}$$

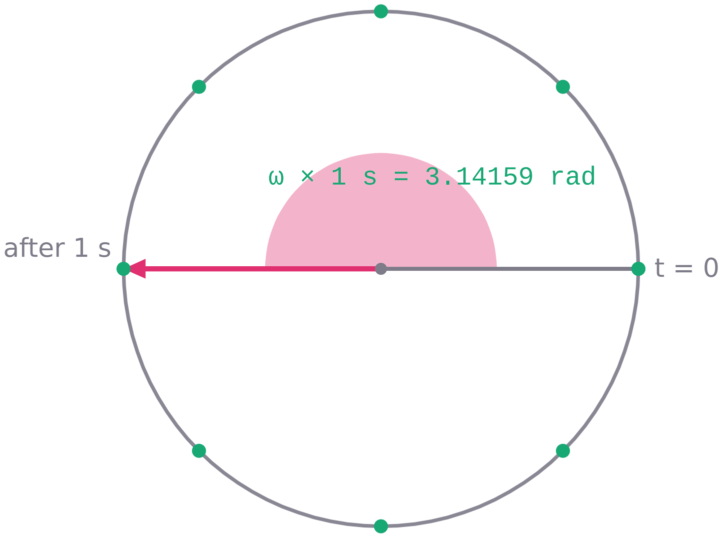
invalid call	outcome	message
without `omega`	NEEDS_INPUT	tangential_speed needs omega (angular speed, rad/s). Example: solve('tangential_speed', omega='... rad/s', r='... m').
`omega` in a wrong unit	INVALID_INPUT	omega is a angular frequency: its unit must be like rad/s, not 'kg'.

Angular frequency and frequency, `angular_frequency`

$$\omega = 2\pi f$$

input	meaning	kind / role	unit	default
`omega`	angular frequency	angular_frequency / plain	`rad/s`	
`f`	frequency	frequency / plain	`Hz`	

- **Evidence:** hand-checked pack



A point turning round a circle at f = 0.5 revolutions per second: in 1 s it sweeps the angle ω × 1 s (shaded), with ω = 2π f the angular frequency the package returns; dots every 0.25 s for one period T = 2 s (from its period formula). Marked: ω = 3.14159 rad/s.

```
classical_physics_ai999x.solve('angular_frequency', f='0.5 Hz', target='omega')
# -> 3.14159 rad/s   (SOLVED)
```

$$\omega = 2\pi f$$
$$\omega = 2\pi f = 3.14159\ \mathrm{rad/s}$$

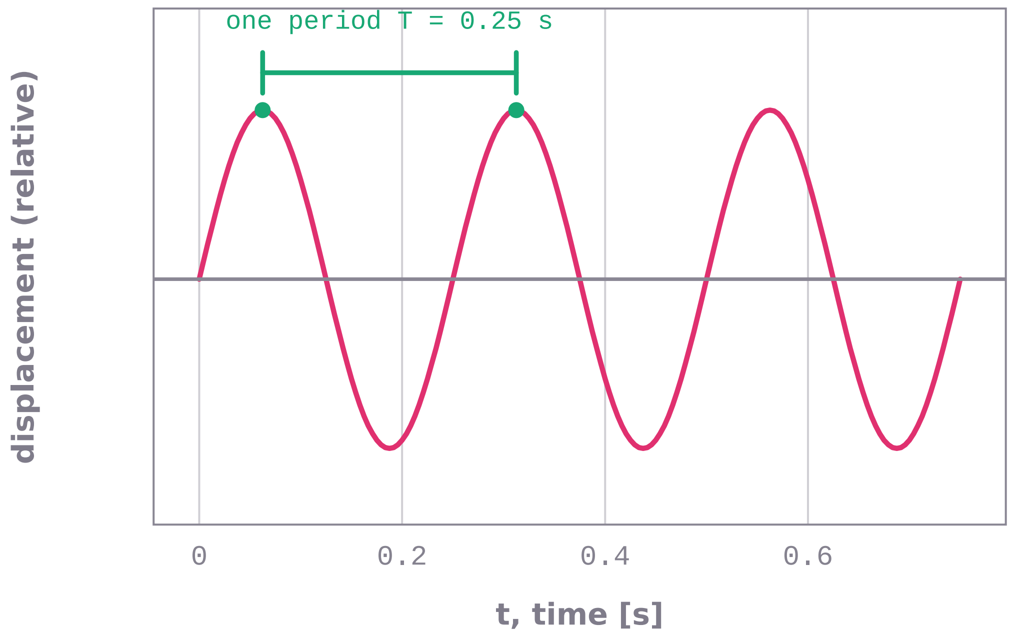
invalid call	outcome	message
without `f`	NEEDS_INPUT	angular_frequency needs f (frequency, Hz). Example: solve('angular_frequency', f='... Hz').
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Period and frequency, `period_frequency`

$$T = \frac{1}{f}$$

input	meaning	kind / role	unit	default
`T`	period	time / period	`s`	
`f`	frequency	frequency / plain	`Hz`	

- **Assumes:** periodic motion
- **Evidence:** hand-checked pack



An oscillation of frequency f = 4 Hz drawn as sin(ω t) over three cycles, with ω = 25.1327 rad/s from the package's angular-frequency formula; the time between two equal crests is the period the package returns. Marked: T = 1/f = 0.25 s.

```
classical_physics_ai999x.solve('period_frequency', f='4 Hz', target='T')
# -> 0.25 s   (SOLVED)
```

$$T = \frac{1}{f}$$

$$T = \frac{1}{f} = 0.25\,\mathrm{s}$$

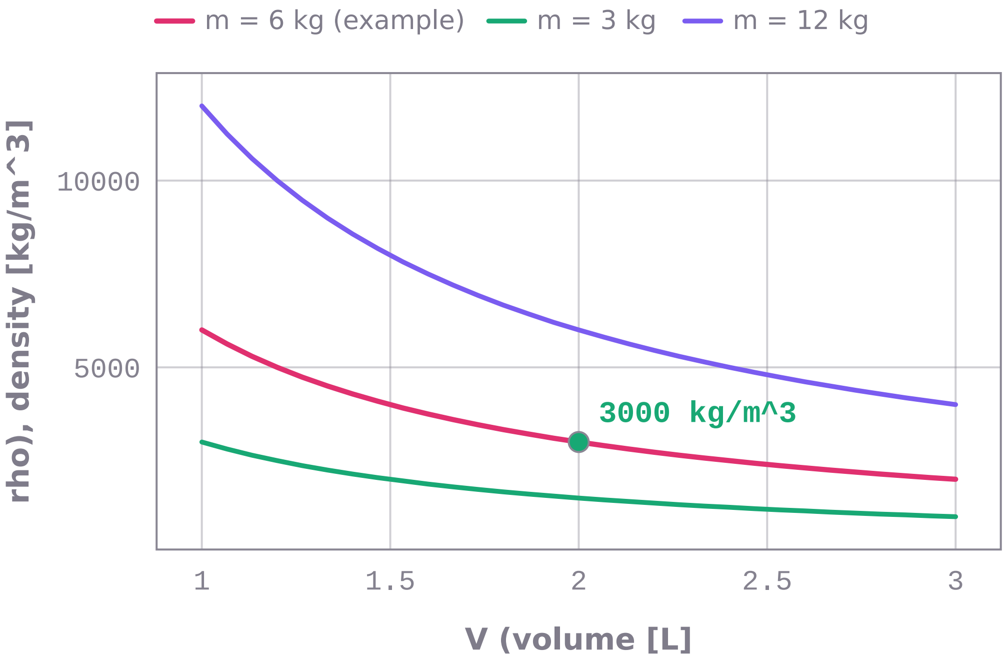
invalid call	outcome	message
without `f`	NEEDS_INPUT	period_frequency needs f (frequency, Hz). Example: solve('period_frequency', f='... Hz').
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Mass density, `density`

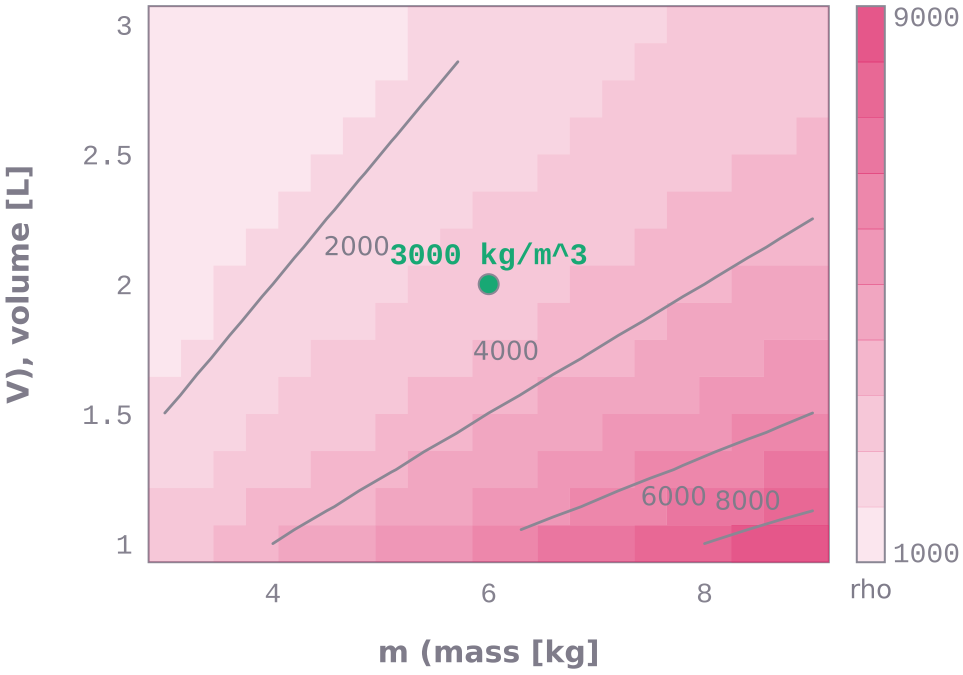
$$\rho = \frac{m}{V}$$

input	meaning	kind / role	unit	default
`rho`	density	density / plain	`kg/m^3`	
`m`	mass	mass / plain	`kg`	
`V`	volume	volume / plain	`m^3`	

- **Not when:** container_contents
- **Assumes:** uniform material, or mean density
- **Evidence:** hand-checked pack



rho (density) against V (volume) from 1 L to 3 L. One curve for each value of m (mass): 6 kg, 3 kg, 12 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, V = 2 L, rho = 3000 kg/m^3.



rho (density) over m (mass) from 3 kg to 9 kg and V (volume) from 1 L to 3 L: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1000 kg/m^3 to dark 9000 kg/m^3); iso-lines at rho = 2000, 4000, 6000, 8000 kg/m^3. Marked: the worked example, m = 6 kg, V = 2 L, rho = 3000 kg/m^3.

```
classical_physics_ai999x.solve('density', m='6 kg', V='2 L', target='rho')
# -> 3000 kg/m^3   (SOLVED)
```

$$\rho = \frac{m}{V}$$

$$\rho = \frac{m}{V} = 3000\ \mathrm{kg/m^3}$$

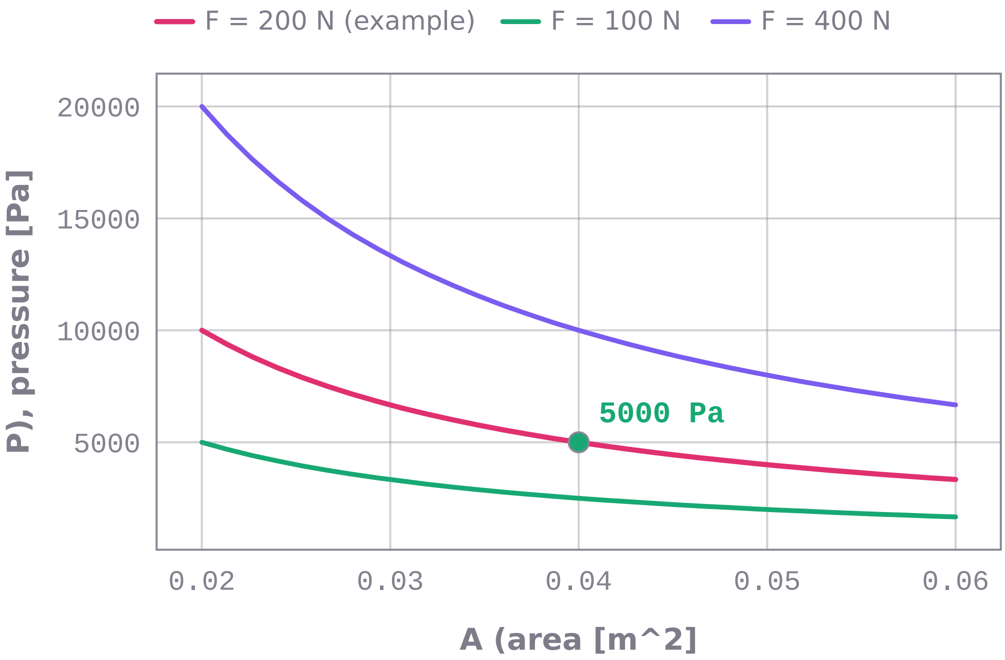
invalid call	outcome	message
without `m`	NEEDS_INPUT	density needs m (mass, kg). Example: solve('density', m='... kg', V='... m^3').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Pressure of a force on an area, `pressure_force`

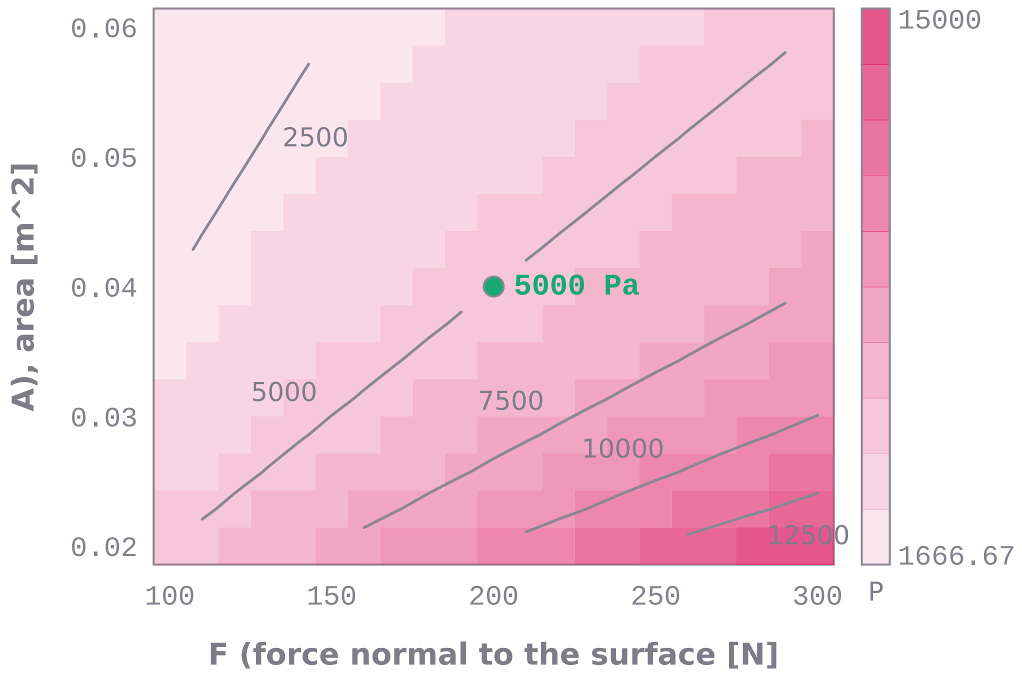
$$P = \frac{F}{A}$$

input	meaning	kind / role	unit	default
`P`	pressure	pressure / plain	`Pa`	
`F`	force normal to the surface	force / plain	`N`	
`A`	area	area / plain	`m^2`	

- **Assumes:** force spread evenly and normal to the surface
- **Evidence:** hand-checked pack



P (pressure) against A (area) from 0.02 m^2 to 0.06 m^2. One curve for each value of F (force normal to the surface): 200 N, 100 N, 400 N. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 0.04 m^2, P = 5000 Pa.



P (pressure) over F (force normal to the surface) from 100 N to 300 N and A (area) from 0.02 m^2 to 0.06 m^2: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1666.67 Pa to dark 15000 Pa); iso-lines at P = 2500, 5000, 7500, 10000, 12500 Pa. Marked: the worked example, F = 200 N, A = 0.04 m^2, P = 5000 Pa.

```
classical_physics_ai999x.solve('pressure_force', F='200 N', A='0.04 m^2', target='P')
# -> 5000 Pa   (SOLVED)
```

$$P = \frac{F}{A}$$

$$P = \frac{F}{A} = 5000\ \mathrm{Pa}$$

invalid call	outcome	message
without `F`	NEEDS_INPUT	pressure_force needs F (force normal to the surface, N). Example: solve('pressure_force', F='... N', A='... m^2').
`F` in a wrong unit	INVALID_INPUT	F is a force: its unit must be like N, not 'kg'.

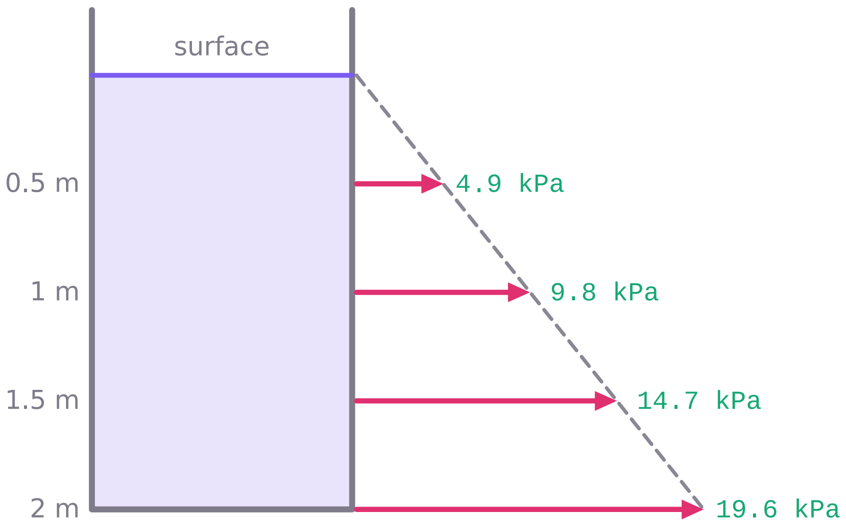
Pressure of a column of fluid, `hydrostatic_pressure`

$$P = \rho g h$$

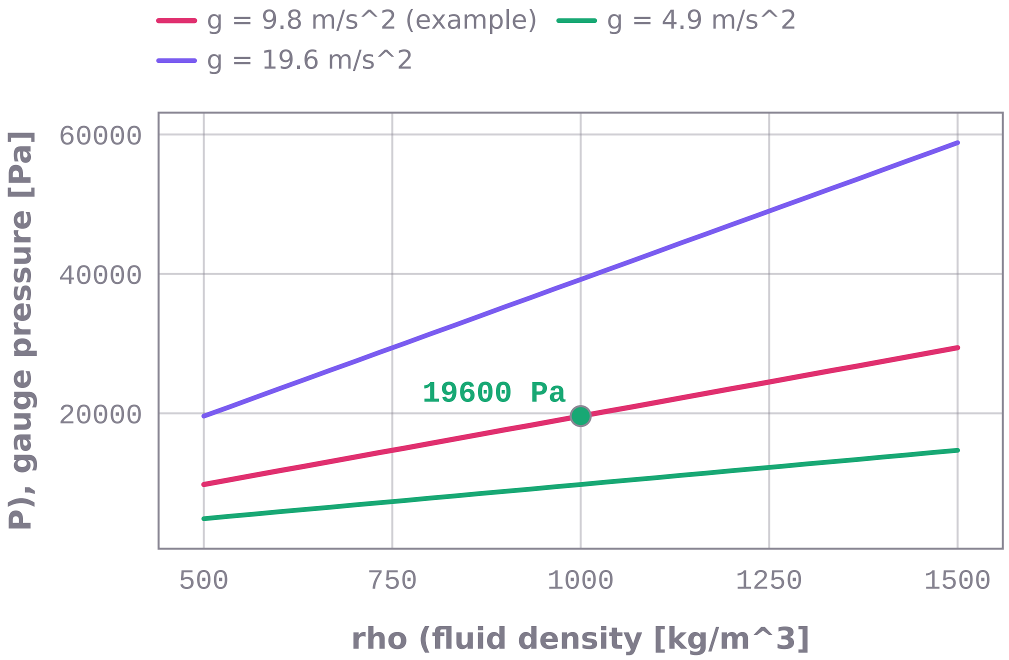
input	meaning	kind / role	unit	default
`P`	gauge pressure	pressure / plain	`Pa`	
`rho`	fluid density	density / plain	`kg/m^3`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	depth below the surface	length / depth	`m`	

- **Assumes:** fluid at rest with uniform density; pressure measured relative to the surface

- **Evidence:** hand-checked pack



Gauge pressure on the wall of a tank of liquid (ρ = 1000 kg/m³, g = 9.8 m/s²) at depths 0.5, 1, 1.5 and 2 m, arrows to scale: the pressure grows linearly with depth (dashed line). Marked: the pressure the package returns at each depth, 4900 Pa, 9800 Pa, 14700 Pa, 19600 Pa.



P (gauge pressure) against rho (fluid density) from 500 kg/m^3 to 1500 kg/m^3. One curve for each value of g (gravitational acceleration): 9.8 m/s^2, 4.9 m/s^2, 19.6 m/s^2. Other inputs as in the worked example: h = 2 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, rho = 1000 kg/m^3, P = 19600 Pa.

```
classical_physics_ai999x.solve('hydrostatic_pressure', rho='1000 kg/m^3', g='9.8 m/s^2',
h='2 m', target='P')
# -> 19600 Pa   (SOLVED)
```

$$P = \rho g h$$
$$P = g h \rho = 19600\ \mathrm{Pa}$$

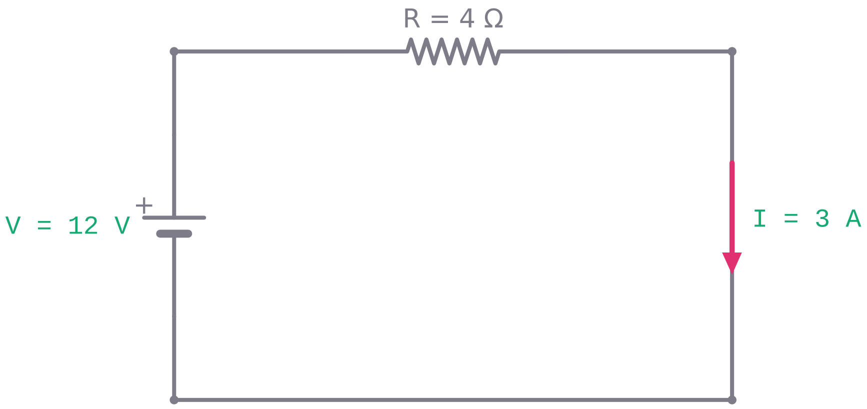
invalid call	outcome	message
without `rho`	NEEDS_INPUT	hydrostatic_pressure needs rho (fluid density, kg/m^3). Example: solve('hydrostatic_pressure', rho='... kg/m^3', g='... m/s^2', h='... m').
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Voltage across an ohmic resistance, `ohm`

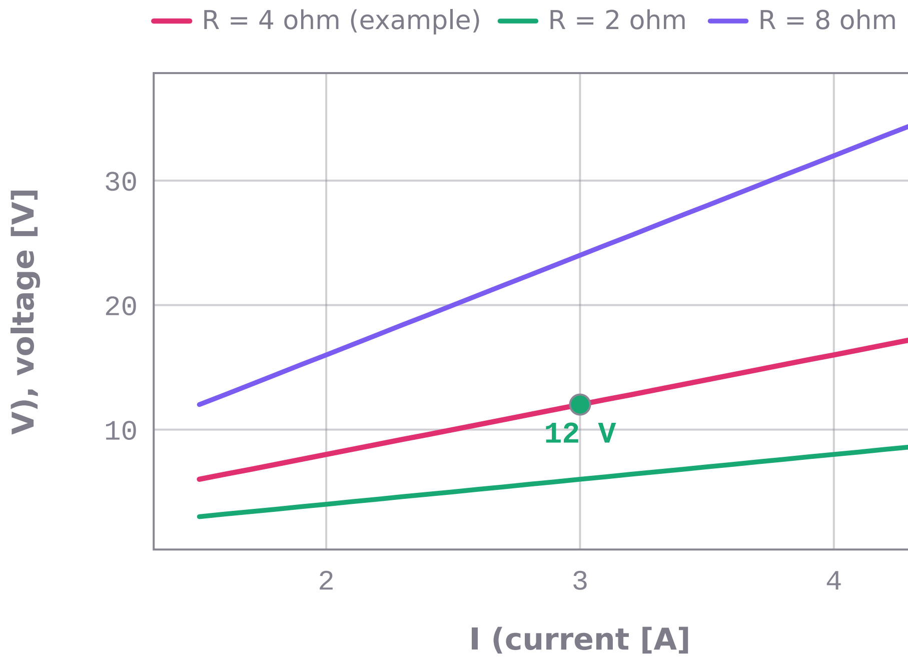
$$V = IR$$

input	meaning	kind / role	unit	default
`V`	voltage	voltage / plain	`V`	
`I`	current	current / plain	`A`	
`R`	resistance	resistance / plain	`ohm`	

- **Assumes:** ohmic element at a fixed temperature
- **Evidence:** hand-checked pack



A resistor R = 4 Ω across a source, carrying a current I = 3 A (the inputs): Ohm's law gives the voltage across it. Marked: the voltage the package returns, V = I R = 12 V.



V (voltage) against I (current) from 1.5 A to 4.5 A. One curve for each value of R (resistance): 4 ohm, 2 ohm, 8 ohm. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I = 3 A, V = 12 V.

```
classical_physics_ai999x.solve('ohm', I='3 A', R='4 ohm', target='V')
# -> 12 V   (SOLVED)
```

$$V = IR$$
$$V = IR = 12\ \mathrm{V}$$

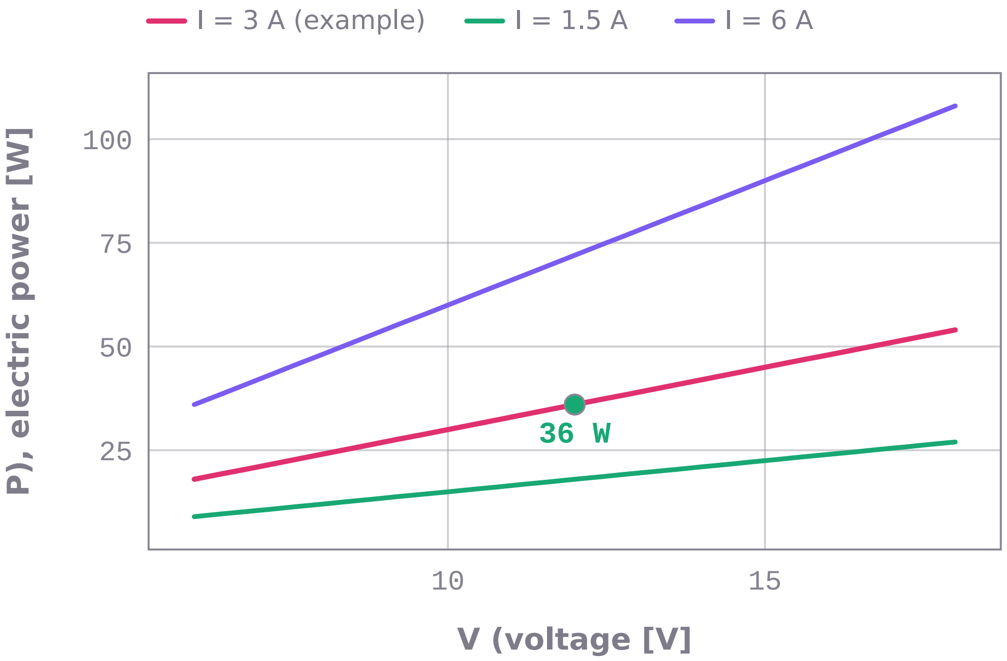
invalid call	outcome	message
without `I`	NEEDS_INPUT	ohm needs I (current, A). Example: solve('ohm', I='... A', R='... ohm').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Electric power, `electric_power`

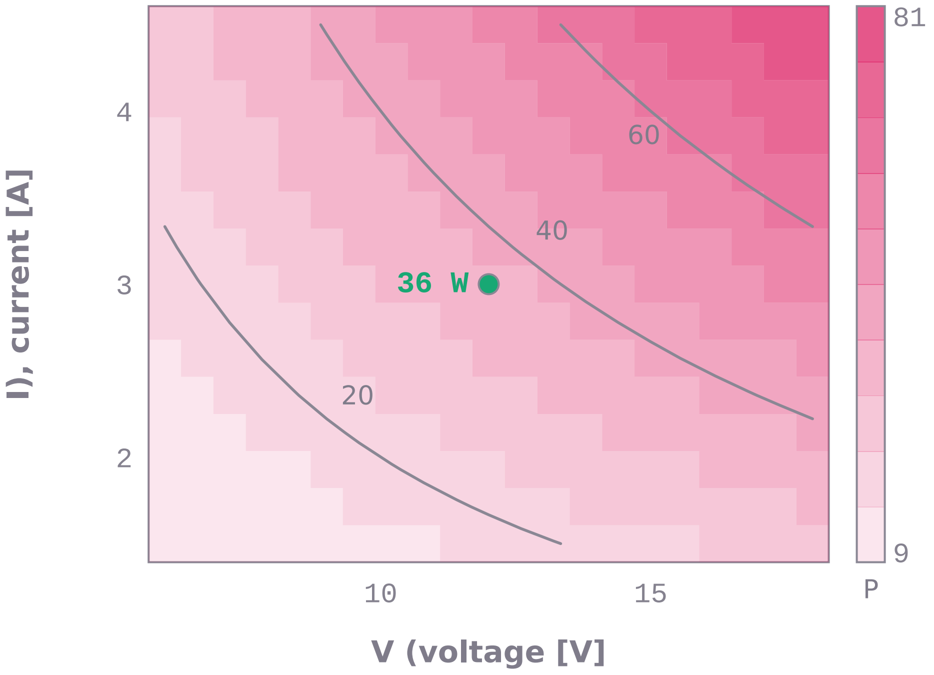
$$P = VI$$

input	meaning	kind / role	unit	default
`P`	electric power	power / plain	`W`	
`V`	voltage	voltage / plain	`V`	
`I`	current	current / plain	`A`	

- **Not when:** alternating_current
- **Assumes:** steady direct current
- **Evidence:** hand-checked pack



P (electric power) against V (voltage) from 6 V to 18 V. One curve for each value of I (current): 3 A, 1.5 A, 6 A. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, V = 12 V, P = 36 W.



P (electric power) over V (voltage) from 6 V to 18 V and I (current) from 1.5 A to 4.5 A: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 9 W to dark 81 W); iso-lines at P = 20, 40, 60 W. Marked: the worked example, V = 12 V, I = 3 A, P = 36 W.

```
classical_physics_ai999x.solve('electric_power', V='12 V', I='3 A', target='P')
# -> 36 W   (SOLVED)
```

$$P = VI$$
$$P = IV = 36\ \mathrm{W}$$

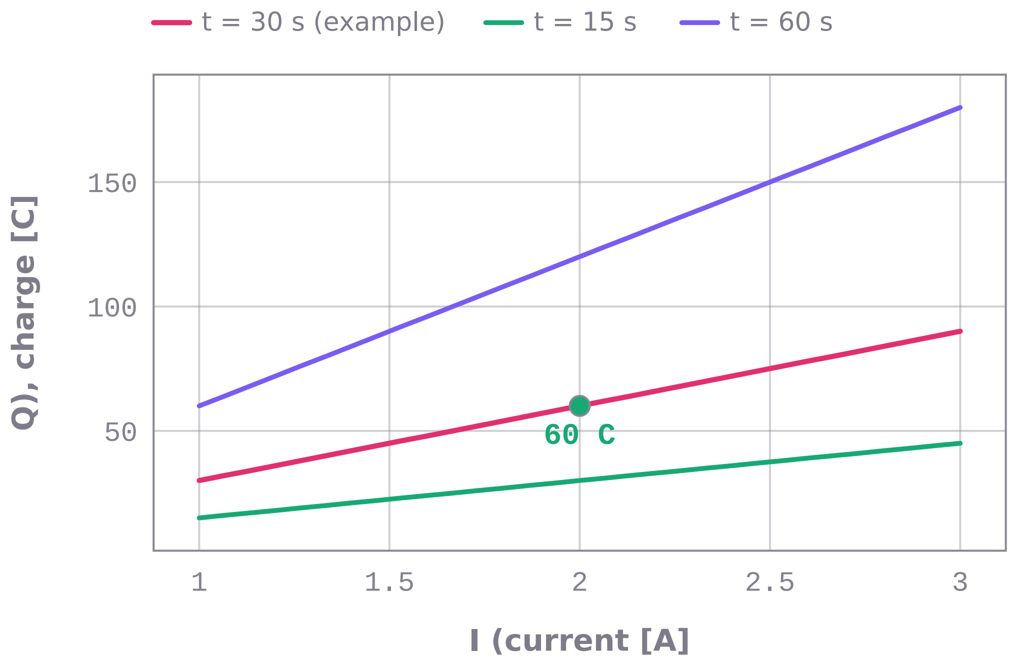
invalid call	outcome	message
without `V`	NEEDS_INPUT	electric_power needs V (voltage, V). Example: solve('electric_power', V='... V', I='... A').
`V` in a wrong unit	INVALID_INPUT	V is a voltage: its unit must be like V, not 'kg'.

Charge carried by a steady current, `charge_current`

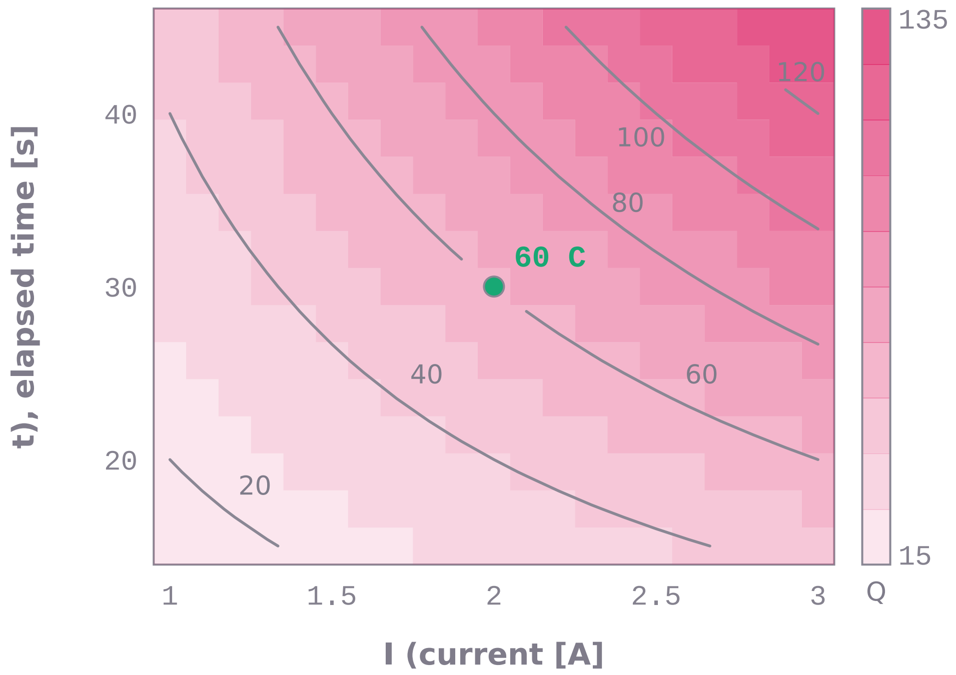
$$Q = It$$

input	meaning	kind / role	unit	default
`Q`	charge	charge / plain	`C`	
`I`	current	current / plain	`A`	
`t`	elapsed time	time / plain	`s`	

- **Assumes:** steady current
- **Evidence:** hand-checked pack



Q (charge) against I (current) from 1 A to 3 A. One curve for each value of t (elapsed time): 30 s, 15 s, 60 s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I = 2 A, Q = 60 C.



Q (charge) over I (current) from 1 A to 3 A and t (elapsed time) from 15 s to 45 s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 15 C to dark 135 C); iso-lines at Q = 20, 40, 60, 80, 100, 120 C. Marked: the worked example, I = 2 A, t = 30 s, Q = 60 C.

```
classical_physics_ai999x.solve('charge_current', I='2 A', t='30 s', target='Q')
# -> 60 C   (SOLVED)
```

$$Q = It$$
$$Q = It = 60\ \mathrm{C}$$

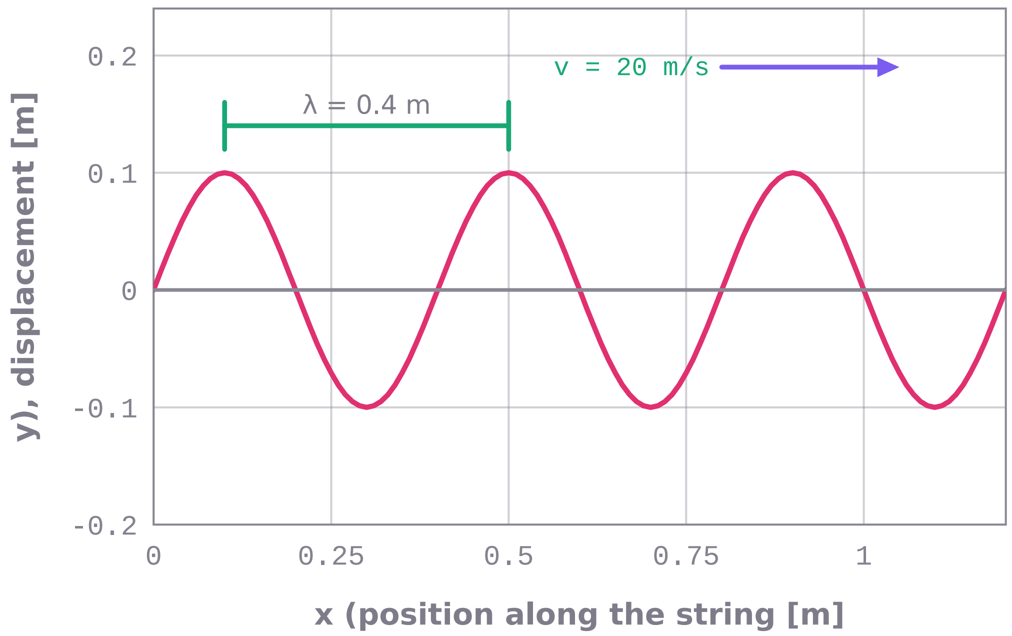
invalid call	outcome	message
without `I`	NEEDS_INPUT	charge_current needs I (current, A). Example: solve('charge_current', I='... A', t='... s').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Speed of a periodic wave, `wave_speed`

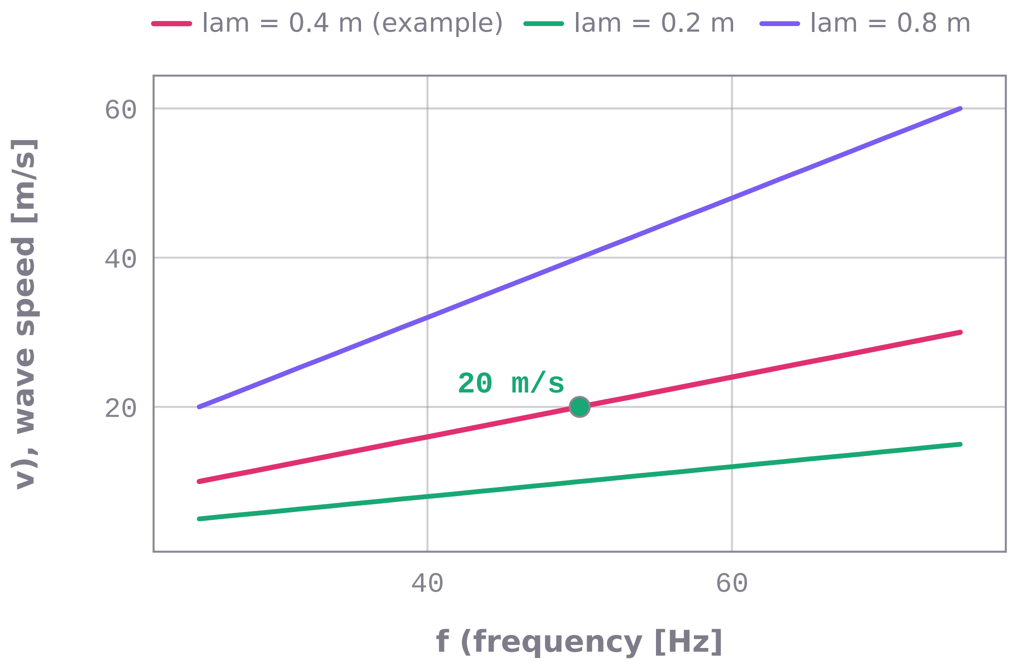
$$v = f\lambda$$

input	meaning	kind / role	unit	default
`v`	wave speed	speed / wave	`m/s`	
`f`	frequency	frequency / plain	`Hz`	
`lam`	wavelength	length / wavelength	`m`	

- **Assumes:** periodic wave in a uniform medium
- **Evidence:** hand-checked pack



A snapshot of a travelling wave of frequency f = 50 Hz and wavelength λ = 0.4 m (drawn with a stated amplitude of 0.1 m): each crest moves one wavelength forward every period. Marked: the wave speed the package returns, v = f λ = 20 m/s.



v (wave speed) against f (frequency) from 25 Hz to 75 Hz. One curve for each value of lam (wavelength): 0.4 m, 0.2 m, 0.8 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, f = 50 Hz, v = 20 m/s.

```
classical_physics_ai999x.solve('wave_speed', f='50 Hz', lam='0.4 m', target='v')
# -> 20 m/s   (SOLVED)
```

$$v = f\lambda$$
$$v = f\lambda = 20\ \mathrm{m/s}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	wave_speed needs f (frequency, Hz). Example: solve('wave_speed', f='... Hz', lam='... m').

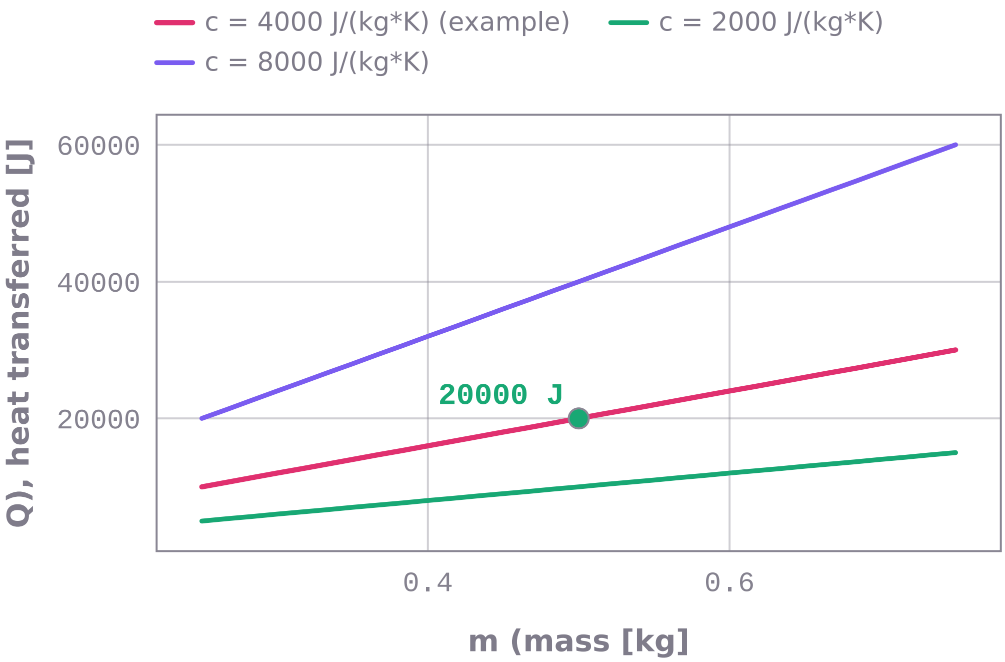
invalid call	outcome	message
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Heat for a temperature change, `heat`

$$Q = mc\Delta T$$

input	meaning	kind / role	unit	default
`Q`	heat transferred	energy / heat	`J`	
`m`	mass	mass / plain	`kg`	
`c`	specific heat capacity	specific_heat / plain	`J/(kg*K)`	
`dT`	temperature change	temperature_difference / plain	`K`	

- **Assumes:** no change of phase; specific heat constant over the range
- **Evidence:** hand-checked pack



Q (heat transferred) against m (mass) from 0.25 kg to 0.75 kg. One curve for each value of c (specific heat capacity): 4000 J/(kg*K), 2000 J/(kg*K), 8000 J/(kg*K). Other inputs as in the worked example: dT = 10 K. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 0.5 kg, Q = 20000 J.

```
classical_physics_ai999x.solve('heat', m='0.5 kg', c='4000 J/(kg*K)', dT='10 K',
target='Q')
# -> 20000 J   (SOLVED)
```

$$Q = mc\Delta T$$
$$Q = c\Delta T m = 20000 \text{ J}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	heat needs m (mass, kg). Example: solve('heat', m='... kg', c='... J/(kg*K)', dT='... K').

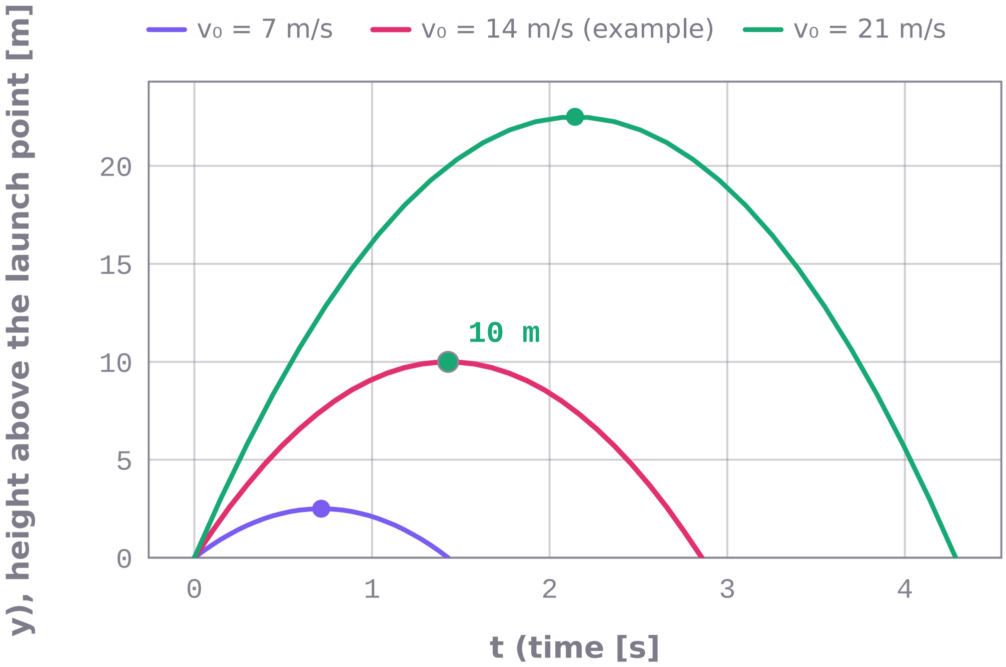
invalid call	outcome	message
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Maximum height of a vertical throw, `max_height_vertical_throw`

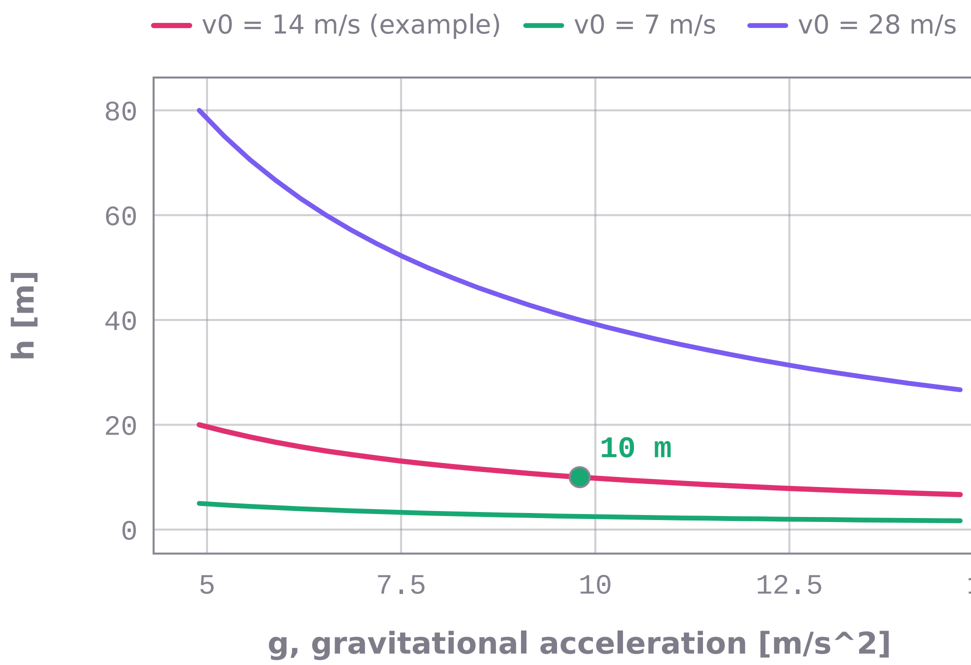
$$h = \frac{v_0^2}{2g}$$

input	meaning	kind / role	unit	default
`h`	maximum height above the launch point	length / max_height	`m`	
`v0`	launch speed	speed / initial	`m/s`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Conditions:** thrown_up
- **Applies when:** the launch is vertical; the height is measured from the launch point
- **Fails when:** air resistance matters; the launch is at an angle
- **Assumes:** thrown straight up; no air resistance; uniform gravitational field
- **Evidence:** hand-checked pack



Height against time of a body thrown straight up (g = 9.8 m/s², no air resistance) at three launch speeds: each point from the package's constant-acceleration displacement, the tops (dots) from its maximum-height and time-to-top formulas, doubling v_0 quadruples the height. Marked: v_0 = 7 m/s: top 2.5 m at 0.714286 s; v_0 = 14 m/s: top 10 m at 1.42857 s; v_0 = 21 m/s: top 22.5 m at 2.14286 s.



h (maximum height above the launch point) against g (gravitational acceleration) from 4.9 m/s^2 to 14.7 m/s^2. One curve for each value of v0 (launch speed): 14 m/s, 7 m/s, 28 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, g = 9.8 m/s^2, h = 10 m.

```
classical_physics_ai999x.solve('max_height_vertical_throw', v0='14 m/s', g='9.8 m/s^2',
target='h')
# -> 10 m   (SOLVED)
```

$$h = \frac{v_0^2}{2g}$$

$$h = \frac{v_0^2}{2g} = 10\ \mathrm{m}$$

invalid call	outcome	message
without `v0`	NEEDS_INPUT	max_height_vertical_throw needs v0 (launch speed, m/s). Example: solve('max_height_vertical_throw', v0='... m/s', g='... m/s^2').
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

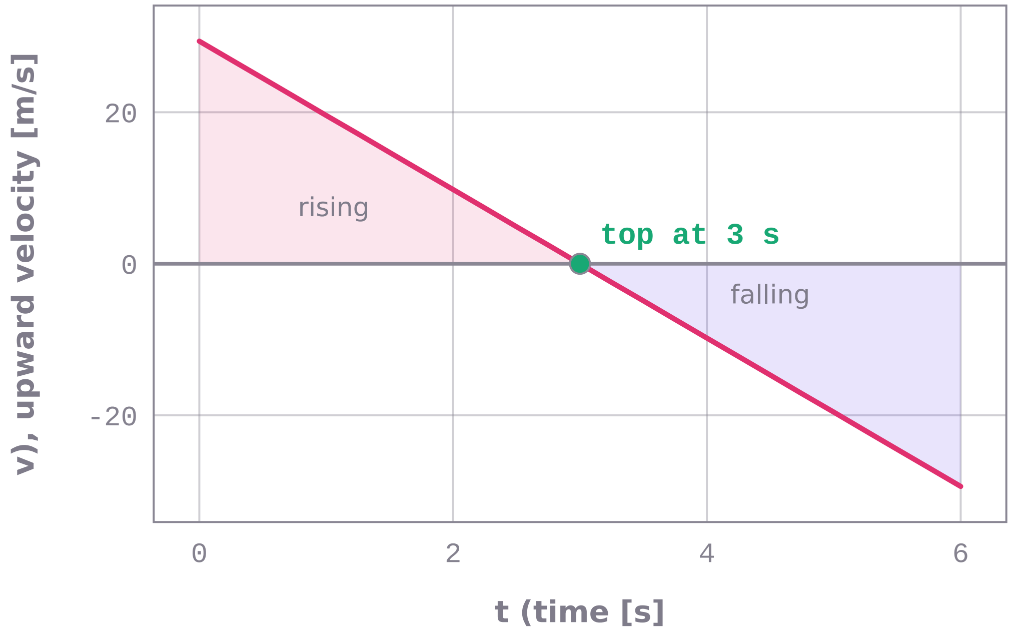
Time to the top of a vertical throw, `time_to_top_vertical_throw`

$$t = \frac{v_0}{g}$$

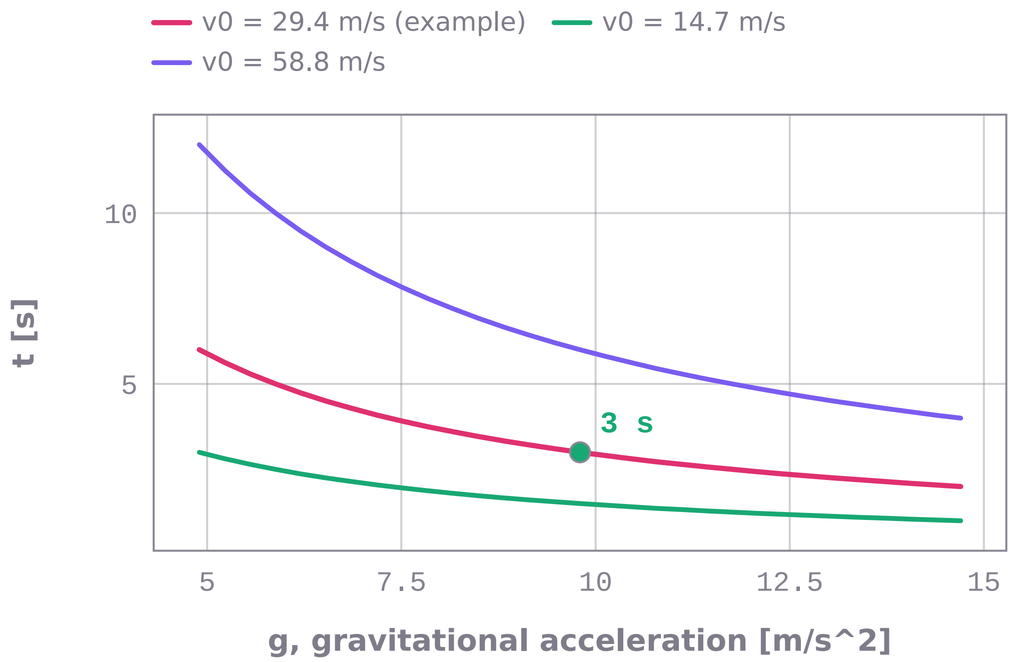
input	meaning	kind / role	unit	default
`t`	time from launch to the highest point	time / rise	`s`	
`v0`	launch speed	speed / initial	`m/s`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Conditions:** thrown_up

- **Applies when:** the launch is vertical
- **Fails when:** air resistance matters; the time asked for is the whole flight
- **Assumes:** thrown straight up; no air resistance; uniform gravitational field
- **Evidence:** hand-checked pack



Upward velocity of a body thrown up at v_0 = 29.4 m/s (g = 9.8 m/s²), 25 times from the package: it falls by g every second, is zero at the top and returns with the launch speed downwards. Marked: time to the top 3 s.



t (time from launch to the highest point) against g (gravitational acceleration) from 4.9 m/s^2 to 14.7 m/s^2. One curve for each value of v0 (launch speed): 29.4 m/s, 14.7 m/s, 58.8 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, g = 9.8 m/s^2, t = 3 s.

```
classical_physics_ai999x.solve('time_to_top_vertical_throw', v0='29.4 m/s', g='9.8 m/s^2',
target='t')
# -> 3 s   (SOLVED)
```

$$t = \frac{v_0}{g}$$

$$t = \frac{v_0}{g} = 3\ \mathrm{s}$$

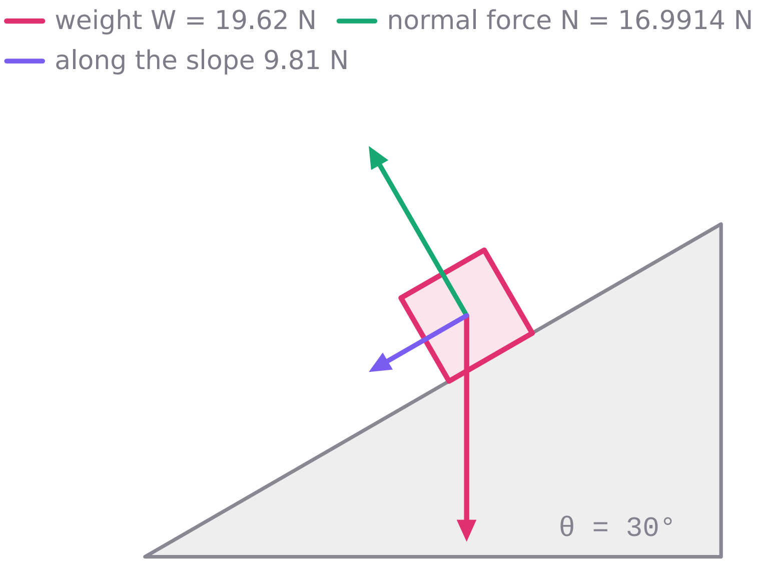
invalid call	outcome	message
without `v0`	NEEDS_INPUT	time_to_top_vertical_throw needs v0 (launch speed, m/s). Example: solve('time_to_top_vertical_throw', v0='... m/s', g='... m/s^2').
`v0` in a wrong unit	INVALID_INPUT	v0 is a speed: its unit must be like m/s, not 'kg'.

Normal force on an incline, `normal_force_incline`

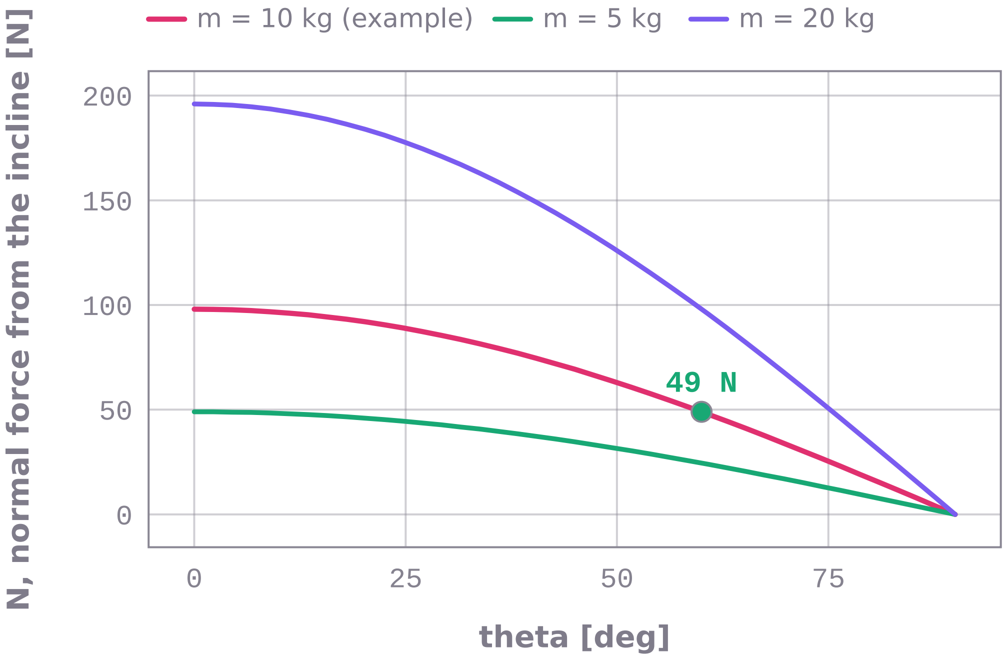
$$N = mg\cos\left(\theta\right)$$

input	meaning	kind / role	unit	default
`N`	normal force from the incline	force / normal	`N`	
`m`	mass	mass / plain	`kg`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`theta`	angle of the incline to the horizontal	angle / incline	`rad`	

- **Conditions:** inclined
- **Applies when:** the body rests or slides on a straight incline
- **Fails when:** another force pushes into or pulls away from the surface; the incline itself accelerates
- **Assumes:** no acceleration perpendicular to the incline; no other force with a component normal to the surface
- **Evidence:** hand-checked pack



Forces on a 2 kg block on a frictionless 30° incline (g = 9.81 m/s²), each computed by the package and drawn to scale (0.12 length units per newton): weight 19.62 N, normal force 16.9914 N = m g cos θ, component along the slope 9.81 N = m g sin θ, which accelerates the block at 4.905 m/s².



N (normal force from the incline) against theta (angle of the incline to the horizontal) from 0 deg to 90 deg. One curve for each value of m (mass): 10 kg, 5 kg, 20 kg. Other inputs as in the worked example: g = 9.8 m/s^2. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 60 deg, N = 49 N.

```
classical_physics_ai999x.solve('normal_force_incline', m='10 kg', g='9.8 m/s^2', theta='60
deg', target='N')
# -> 49 N   (SOLVED)
```

$$N = mg\cos(\theta)$$
$$N = gm\cos(\theta) = 49\ \mathrm{N}$$

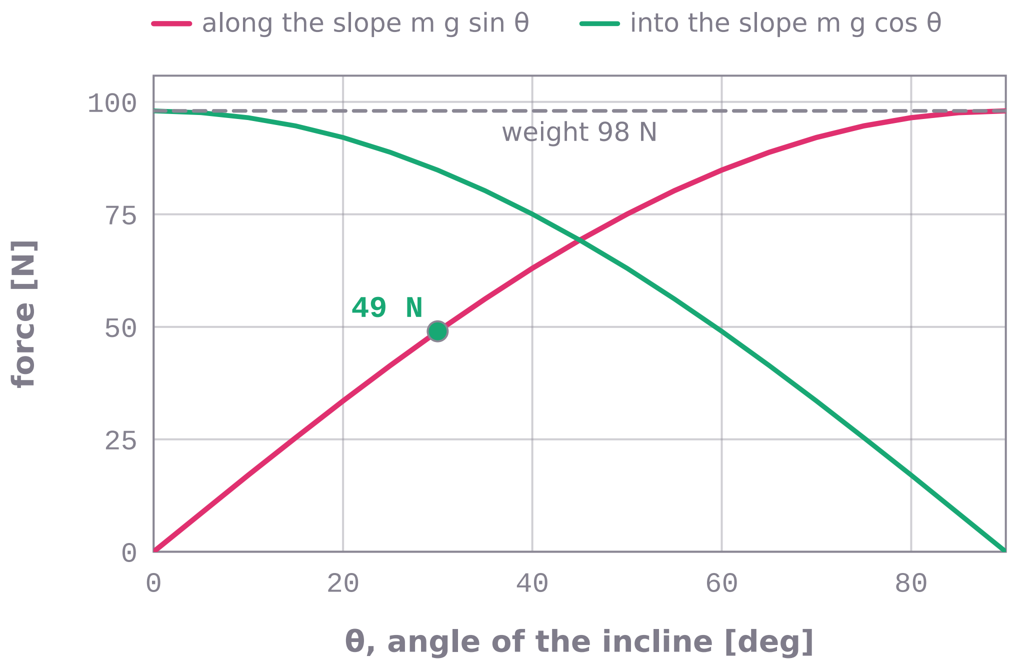
invalid call	outcome	message
without `m`	NEEDS_INPUT	normal_force_incline needs m (mass, kg). Example: solve('normal_force_incline', m='... kg', g='... m/s^2', theta='... rad').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Component of the weight along an incline, `weight_component_along_incline`

$$F = mg\sin(\theta)$$

input	meaning	kind / role	unit	default
`F`	component of the weight along the incline	force / weight_along_slope	`N`	
`m`	mass	mass / plain	`kg`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`theta`	angle of the incline to the horizontal	angle / incline	`rad`	

- **Conditions:** inclined
- **Applies when:** a body is on a straight incline
- **Fails when:** the surface is curved
- **Assumes:** uniform gravitational field
- **Evidence:** hand-checked pack



The two components of the weight of a 10 kg body (g = 9.8 m/s²) on an incline, against the incline's angle (19 angles each from the package): the part pulling it down the slope grows from zero, the part pressing it into the surface (the normal force) falls to zero at 90°. Marked: 49 N along the slope at θ = 30°; weight 98 N.

```
classical_physics_ai999x.solve('weight_component_along_incline', m='10 kg', g='9.8 m/s^2',
theta='30 deg', target='F')
# -> 49 N   (SOLVED)
```

$$F = m g \sin(\theta)$$
$$F = g m \sin(\theta) = 49\ \mathrm{N}$$

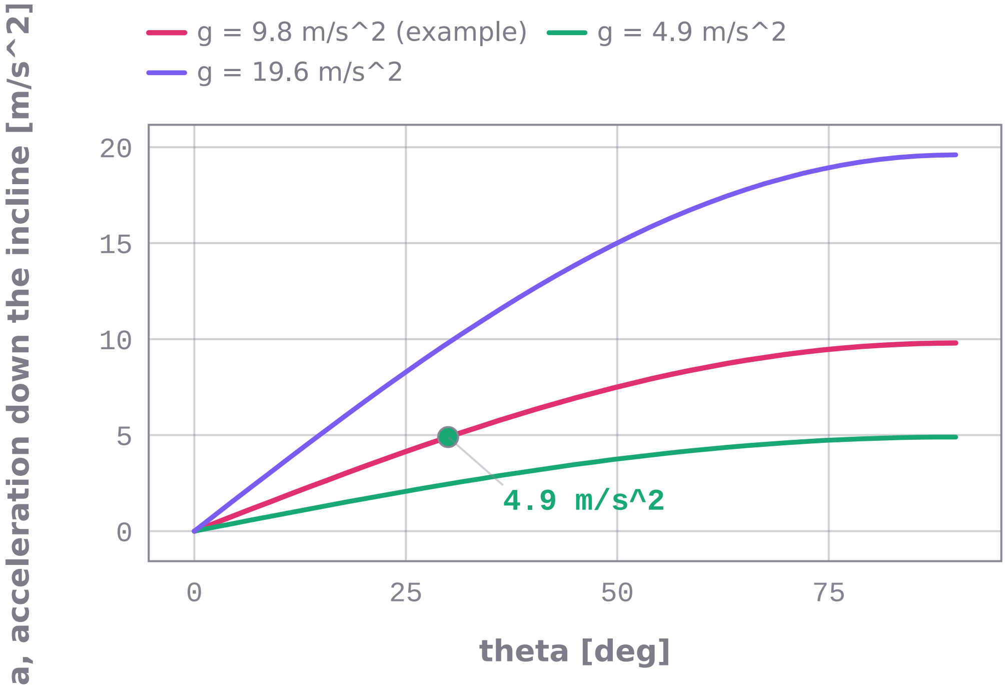
invalid call	outcome	message
without `m`	NEEDS_INPUT	weight_component_along_incline needs m (mass, kg). Example: solve('weight_component_along_incline', m='... kg', g='... m/s^2', theta='... ra
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Acceleration down a frictionless incline, `acceleration_frictionless_incline`

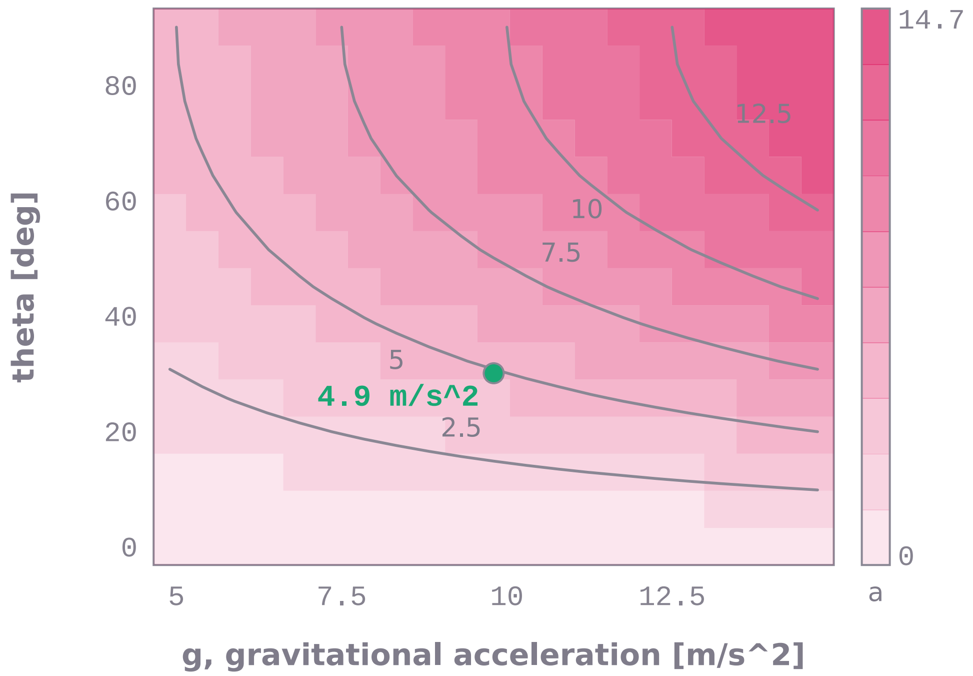
$$a = g \sin(\theta)$$

input	meaning	kind / role	unit	default
`a`	acceleration down the incline	acceleration / plain	`m/s^2`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`theta`	angle of the incline to the horizontal	angle / incline	`rad`	

- **Conditions:** inclined, frictionless
- **Not when:** friction_present, force_stated
- **Applies when:** the surface is stated to be frictionless
- **Fails when:** friction acts; another force acts along the incline; any other force is stated (a push, a pull, a rope)
- **Assumes:** no friction; the only forces are gravity and the normal force
- **Evidence:** hand-checked pack



a (acceleration down the incline) against theta (angle of the incline to the horizontal) from 0 deg to 90 deg. One curve for each value of g (gravitational acceleration): 9.8 m/s^2, 4.9 m/s^2, 19.6 m/s^2. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 30 deg, a = 4.9 m/s^2.



a (acceleration down the incline) over g (gravitational acceleration) from 4.9 m/s^2 to 14.7 m/s^2 and theta (angle of the incline to the horizontal) from 0 deg to 90 deg: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0 m/s^2 to dark 14.7 m/s^2); iso-lines at a = 2.5, 5, 7.5, 10, 12.5 m/s^2. Marked: the worked example, g = 9.8 m/s^2, theta = 30 deg, a = 4.9 m/s^2.

```
classical_physics_ai999x.solve('acceleration_frictionless_incline', g='9.8 m/s^2',
theta='30 deg', target='a')
# -> 4.9 m/s^2   (SOLVED)
```

$$a = g \sin\left(\theta\right)$$
$$a = g \sin\left(\theta\right) = 4.9\ \mathrm{m}/\mathrm{s}^{2}$$

invalid call	outcome	message
without `g`	SOLVED	
`g` in a wrong unit	INVALID_INPUT	g is a acceleration: its unit must be like m/s^2, not 'kg'.

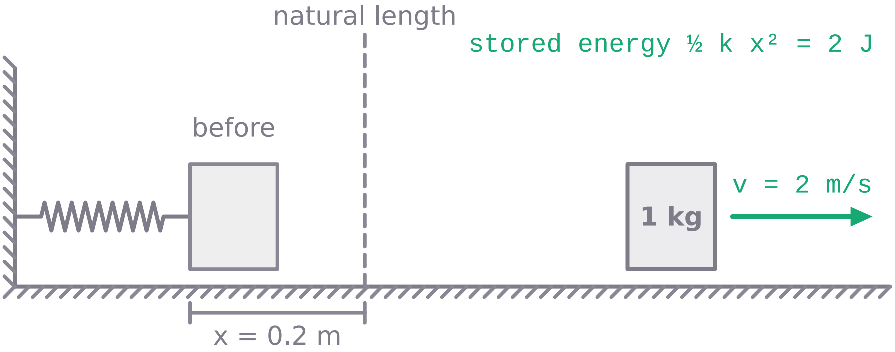
Speed of a body launched by a spring, `spring_launch_speed`

$$\frac{k x^{2}}{2} = \frac{m v^{2}}{2}$$

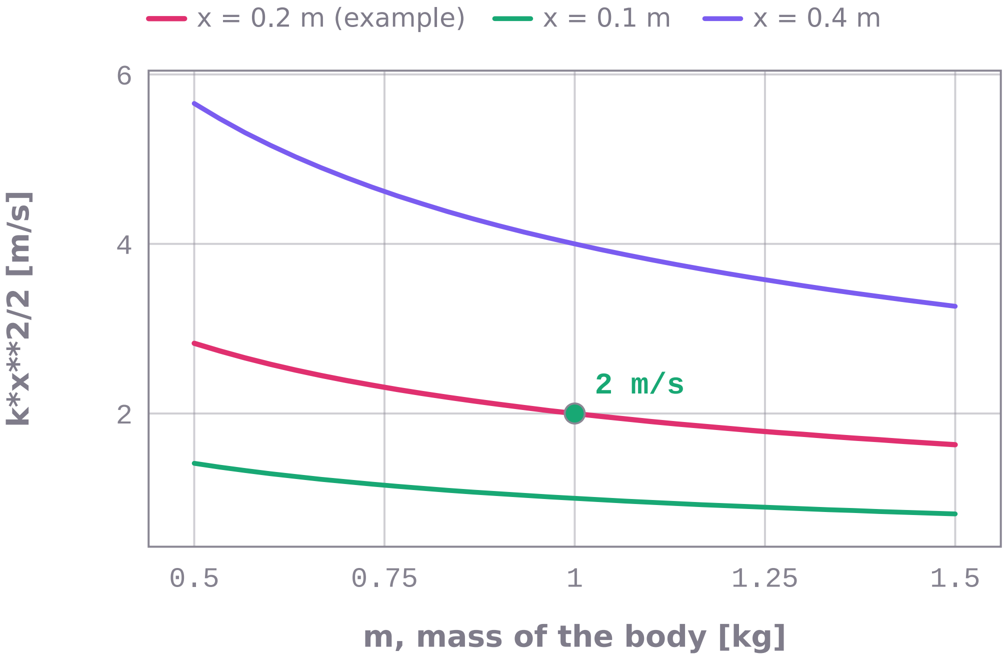
input	meaning	kind / role	unit	default
`k`	spring constant	spring_constant / plain	`N/m`	
`x`	compression of the spring before release	length / extension	`m`	
`m`	mass of the body	mass / plain	`kg`	
`v`	speed when the spring has returned to its natural length	speed / final	`m/s`	

- **Conditions:** spring_launch, frictionless
- **Not when:** friction_present, inclined

- **Applies when:** the body starts at rest against the compressed spring and is released; the surface is frictionless and level
- **Fails when:** friction acts; the body is still attached and the spring is not at its natural length; the motion is vertical or on a slope
- **Assumes:** all the energy stored in the spring becomes kinetic energy of the body; no friction; the spring is massless
- **Evidence:** hand-checked pack



A spring of k = 100 N/m compressed x = 0.2 m launches a 1 kg block on a frictionless surface: all the stored energy (from the package's elastic-energy formula) becomes kinetic energy. Marked: launch speed v = 2 m/s (returned), stored energy 2 J.



k*x**2/2 (Speed of a body launched by a spring) against m (mass of the body) from 0.5 kg to 1.5 kg. One curve for each value of x (compression of the spring before release): 0.2 m, 0.1 m, 0.4 m. Other inputs as in the worked example: k = 100 N/m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 1 kg, k*x**2/2 = 2 m/s.

```
classical_physics_ai999x.solve('spring_launch_speed', k='100 N/m', x='0.2 m', m='1 kg',
target='v')
# -> 2 m/s   (SOLVED)
```

$$\frac{kx^2}{2} = \frac{mv^2}{2}$$

$$v = -\frac{\sqrt{kx}}{\sqrt{m}} = 2\,\mathrm{m}/\mathrm{s}$$

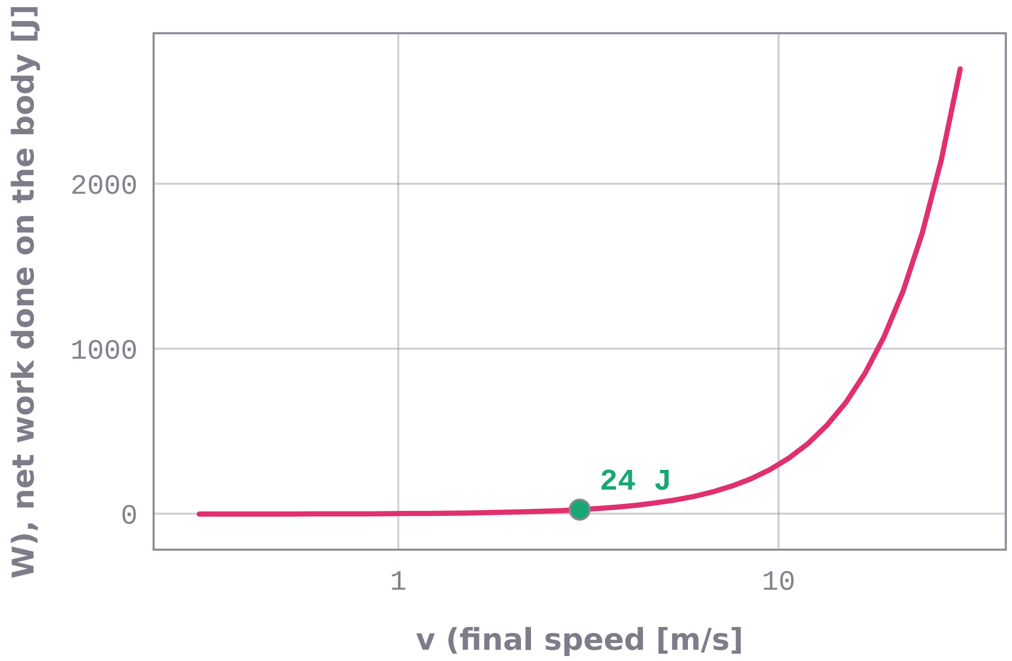
invalid call	outcome	message
without `k`	NEEDS_INPUT	spring_launch_speed needs k (spring constant, N/m). Example: solve('spring_launch_speed', k='... N/m', x='... m', m='... kg').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Work-energy theorem, `net_work_kinetic_energy`

$$W = \frac{m\left(v^2 - v_0^2\right)}{2}$$

input	meaning	kind / role	unit	default
`W`	net work done on the body, equal to its change in kinetic energy	energy / net_work	`J`	
`m`	mass	mass / plain	`kg`	
`v`	final speed	speed / final	`m/s`	
`v0`	initial speed	speed / initial	`m/s`	

- **Applies when:** the work stated is the net (total) work on the body
- **Fails when:** only the work of one of several forces is known
- **Assumes:** W is the work of ALL forces on the body; constant mass; speeds far below the speed of light
- **Evidence:** hand-checked pack



W (net work done on the body, equal to its change in kinetic energy) against v (final speed) from 0.3 m/s to 30 m/s on a logarithmic axis. Other inputs as in the worked example: m = 6 kg, v0 = 1 m/s. 41 points computed by classical-physics-ai999x; marked: the worked example, v = 3 m/s, W = 24 J.

```
classical_physics_ai999x.solve('net_work_kinetic_energy', m='6 kg', v0='1 m/s', v='3 m/s',
target='W')
# -> 24 J   (SOLVED)
```

$$W = \frac{m \left(v^{2} - v_{0}^{2}\right)}{2}$$
$$W = \frac{m \left(v^{2} - v_{0}^{2}\right)}{2} = 24\ \mathrm{J}$$

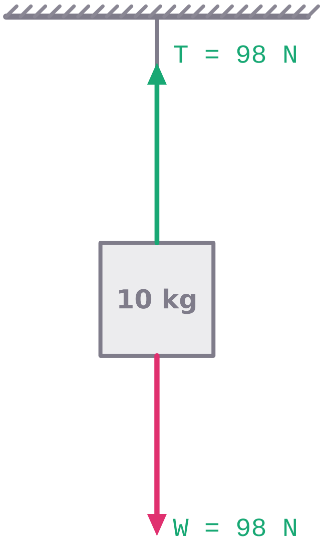
invalid call	outcome	message
without `m`	NEEDS_INPUT	net_work_kinetic_energy needs m (mass, kg). Example: solve('net_work_kinetic_energy', m='... kg', v='... m/s', v0='... m/s').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Tension in a rope holding a hanging body, `tension_hanging_at_rest`

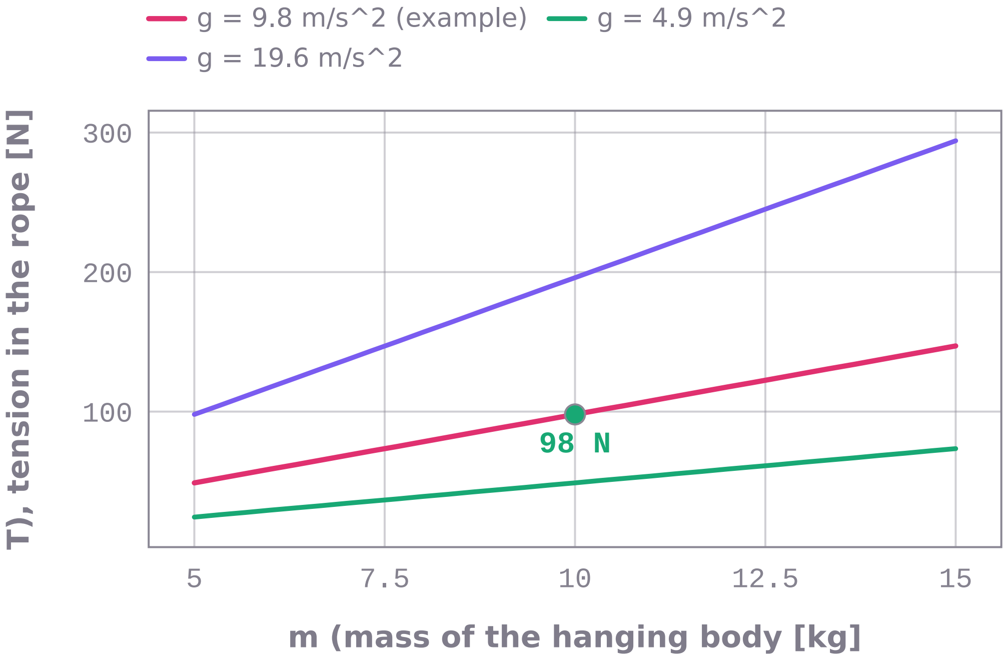
$$T = m g$$

input	meaning	kind / role	unit	default
`T`	tension in the rope	force / tension	`N`	
`m`	mass of the hanging body	mass / plain	`kg`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Conditions:** hanging
- **Not when:** accelerated, inclined
- **Applies when:** the body hangs at rest or moves at constant velocity from a single vertical rope
- **Fails when:** the body accelerates; several ropes or ropes at an angle share the load
- **Assumes:** one vertical rope carries the whole weight; the body is not accelerating
- **Evidence:** hand-checked pack



A 10 kg mass hanging at rest from a rope (g = 9.8 m/s²): the rope tension the package returns balances the weight (also from the package). Arrows to scale. Marked: T = 98 N, W = 98 N.



T (tension in the rope) against m (mass of the hanging body) from 5 kg to 15 kg. One curve for each value of g (gravitational acceleration): 9.8 m/s^2, 4.9 m/s^2, 19.6 m/s^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 10 kg, T = 98 N.

```
classical_physics_ai999x.solve('tension_hanging_at_rest', m='10 kg', g='9.8 m/s^2',
target='T')
# -> 98 N   (SOLVED)
```

$$T = mg$$
$$T = gm = 98\,\mathrm{N}$$

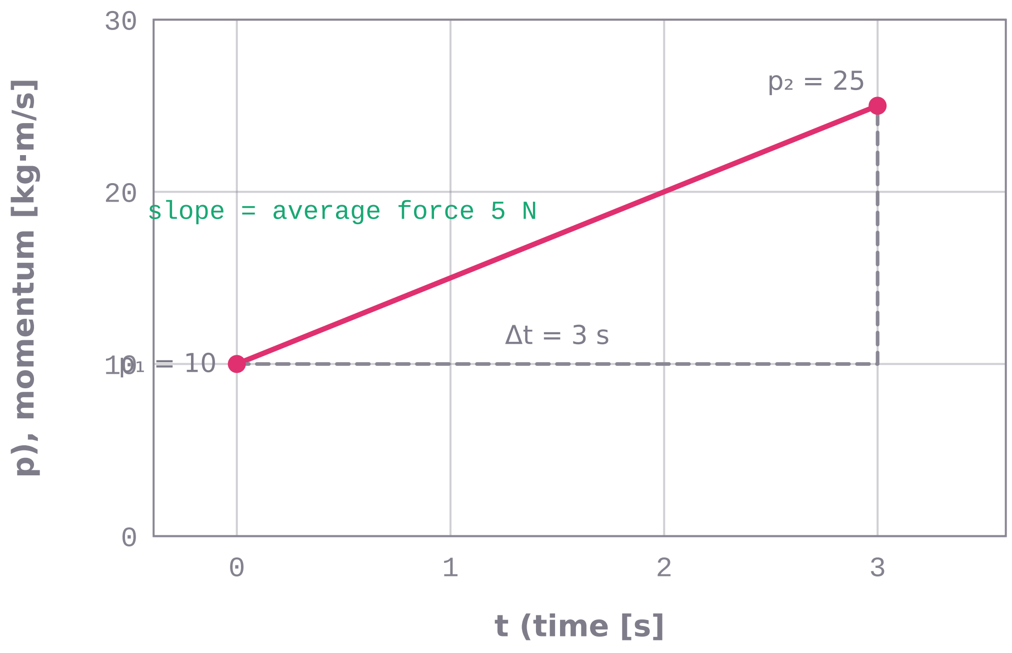
invalid call	outcome	message
without `m`	NEEDS_INPUT	tension_hanging_at_rest needs m (mass of the hanging body, kg). Example: solve('tension_hanging_at_rest', m='... kg', g='... m/s^2').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Average net force from a change of momentum, `average_force_from_momentum_change`

$$F = \frac{p_2 - p_1}{t}$$

input	meaning	kind / role	unit	default
`F`	average net force	force / net	`N`	
`p2`	final momentum	momentum / final	`kg*m/s`	
`p1`	initial momentum	momentum / initial	`kg*m/s`	
`t`	time over which the momentum changes	time / plain	`s`	

- **Applies when:** the momentum before and after is known
- **Fails when:** the force asked for is one of several forces
- **Assumes:** F is the net force averaged over the interval; motion along a line
- **Evidence:** hand-checked pack



Momentum rising from $p_1 = 10$ to $p_2 = 25$ kg·m/s in Δt = 3 s (inputs): the average net force is the slope of momentum against time. Marked: the average force the package returns, 5 N.

```
classical_physics_ai999x.solve('average_force_from_momentum_change', p2='25 kg*m/s', p1='10
kg*m/s', t='3 s', target='F')
# -> 5 N   (SOLVED)
```

$$F = \frac{p_2 - p_1}{t}$$

$$F = \frac{-p_1 + p_2}{t} = 5\ \mathrm{N}$$

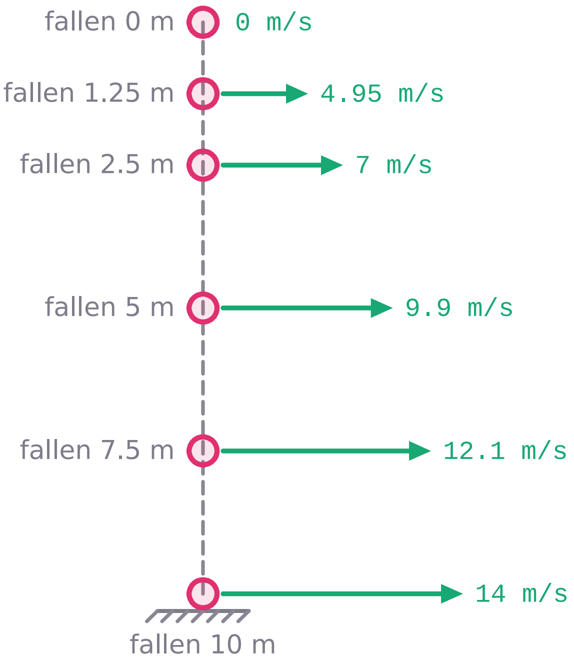
invalid call	outcome	message
without `p2`	NEEDS_INPUT	average_force_from_momentum_change needs p2 (final momentum, kg*m/s). Example: solve('average_force_from_momentum_change', p2='... kg*m/s',
`p2` in a wrong unit	INVALID_INPUT	p2 is a momentum: its unit must be like kg*m/s, not 's'.

Speed after falling from rest, `speed_after_fall_from_rest`

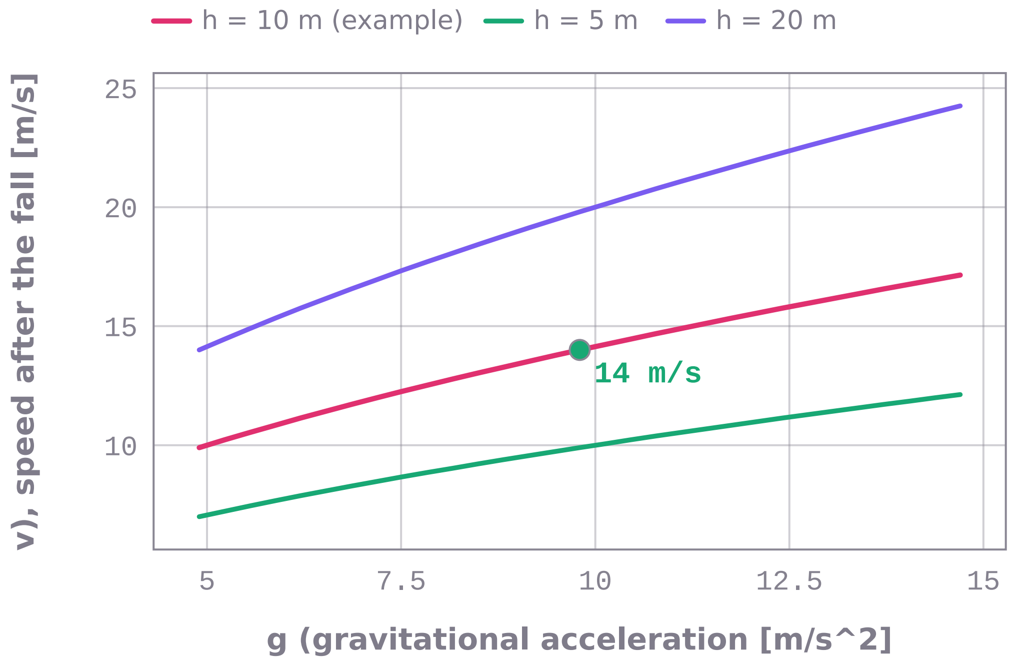
$$v = \sqrt{2gh}$$

input	meaning	kind / role	unit	default
`v`	speed after the fall	speed / final	`m/s`	
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	height fallen	length / height	`m`	

- **Conditions:** free_fall, from_rest
- **Applies when:** the body starts from rest and falls freely
- **Fails when:** it starts with a speed; air resistance matters
- **Assumes:** released from rest; no air resistance
- **Evidence:** hand-checked pack



A body released from rest 10 m above the ground (g = 9.8 m/s², no air resistance), drawn after falling 0, 1.25, 2.5, 5, 7.5 and 10 m; each arrow is to scale with its speed v = √(2 g h) from the package. Marked: 14 m/s on reaching the ground.



v (speed after the fall) against g (gravitational acceleration) from 4.9 m/s^2 to 14.7 m/s^2. One curve for each value of h (height fallen): 10 m, 5 m, 20 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, g = 9.8 m/s^2, v = 14 m/s.

```
classical_physics_ai999x.solve('speed_after_fall_from_rest', g='9.8 m/s^2', h='10 m',
target='v')
# -> 14 m/s   (SOLVED)
```

$$v = \sqrt{2gh}$$
$$v = \sqrt{2}\sqrt{g}\sqrt{h} = 14\,\mathrm{m/s}$$

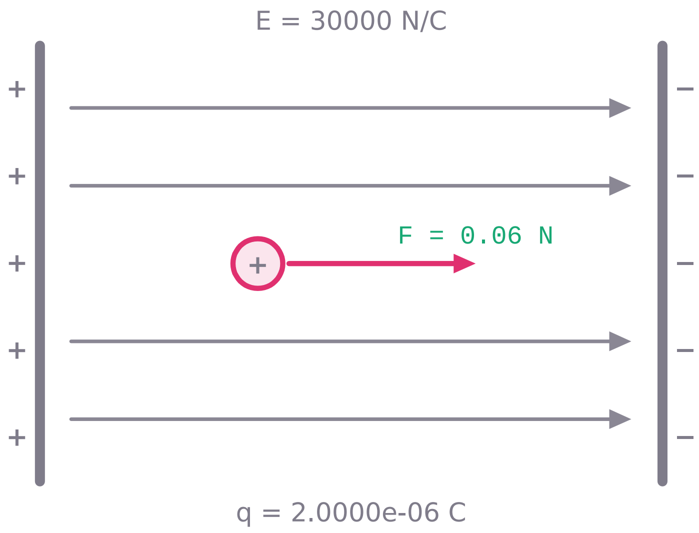
invalid call	outcome	message
without `g`	SOLVED	
`g` in a wrong unit	INVALID_INPUT	g is a acceleration: its unit must be like m/s^2, not 'kg'.

Force on a charge in an electric field, `electric_force_in_field`

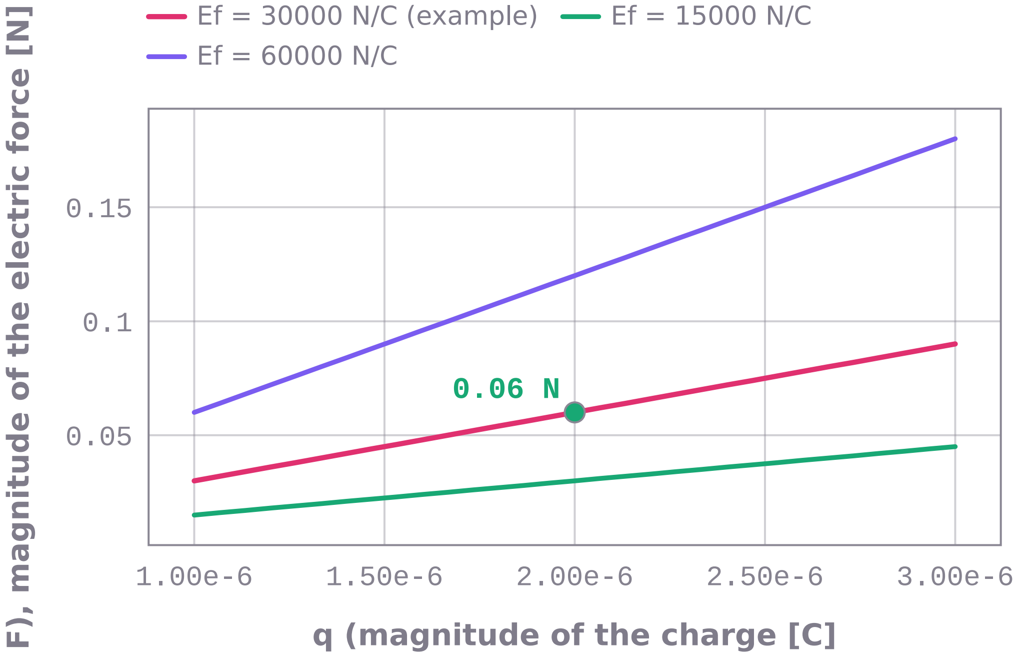
$$F = qE_f$$

input	meaning	kind / role	unit	default
`F`	magnitude of the electric force	force / electric	`N`	
`q`	magnitude of the charge	charge / plain	`C`	
`Ef`	electric field strength at the charge	electric_field / plain	`N/C`	

- **Conditions:** electric
- **Applies when:** a charge sits in a known electric field
- **Fails when:** the charge's own field is asked for
- **Assumes:** magnitudes only
- **Evidence:** definition



A charge q = 2.0000e-06 C in a uniform field E = 30000 N/C between two plates: it is pushed along the field lines (a negative charge would be pushed the other way). Marked: the force the package returns, F = qE = 0.06 N.



F (magnitude of the electric force) against q (magnitude of the charge) from 1.0000e-06 C to 3.0000e-06 C. One curve for each value of Ef (electric field strength at the charge): 30000 N/C, 15000 N/C, 60000 N/C. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, q = 2.0000e-06 C, F = 0.06 N.

```
classical_physics_ai999x.solve('electric_force_in_field', q='2e-06 C', Ef='30000 N/C',
target='F')
# -> 0.06 N   (SOLVED)
```

$$F = qE_f$$
$$F = E_f q = 0.06\ \mathrm{N}$$

invalid call	outcome	message
without `q`	NEEDS_INPUT	electric_force_in_field needs q (magnitude of the charge, C). Example: solve('electric_force_in_field', q='... C', Ef='... N/C').

invalid call	outcome	message
`q` in a wrong unit	INVALID_INPUT	q is a charge: its unit must be like C, not 'kg'.

Questions in words that work:

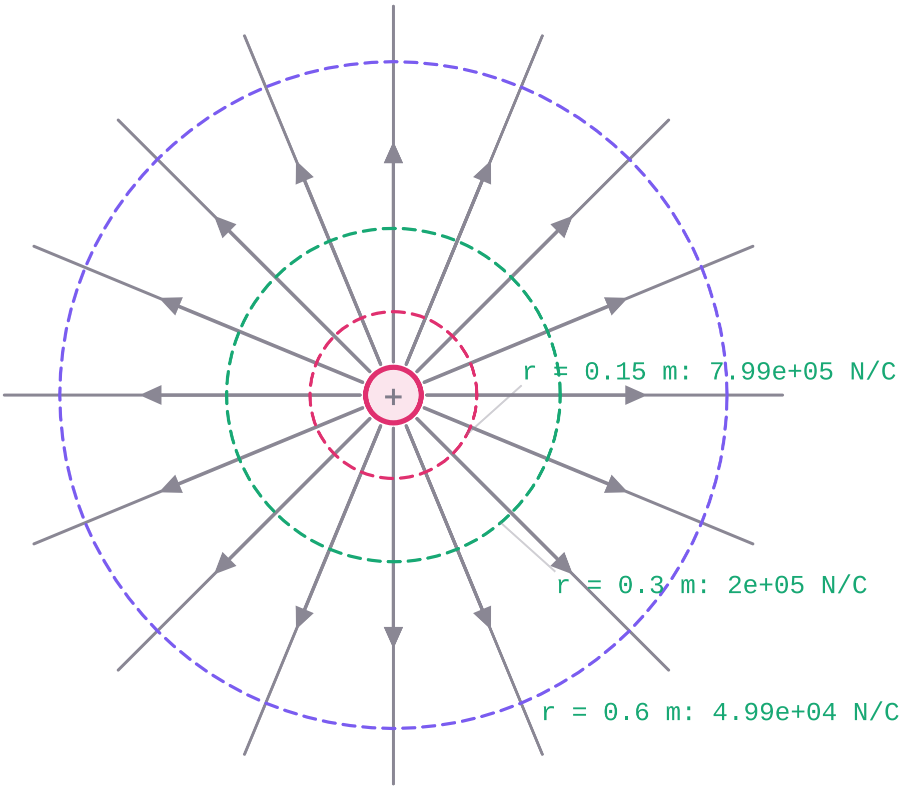
- "A charge of 2 uC is placed in an electric field of 30000 N/C. What is the electric force on it?" → 0.06 N
- "A 5 nC charged particle sits in an electric field of 4000 N/C. What electric force acts on it?" → 2e-05 N

Electric field of a point charge, `point_charge_field`

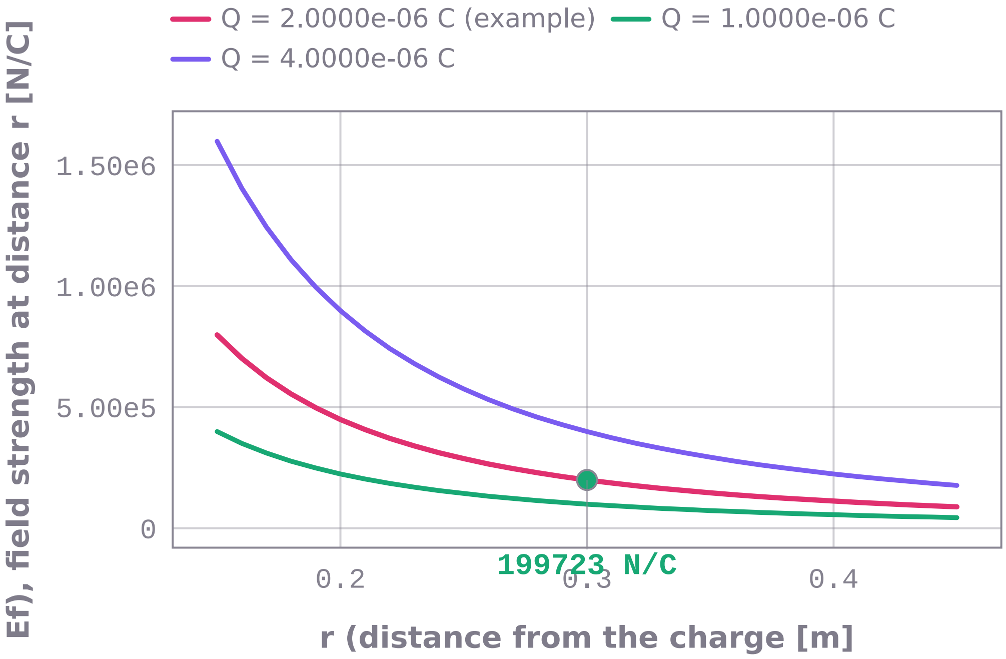
$$E_f = \frac{k_e Q}{r^2}$$

input	meaning	kind / role	unit	default
`Ef`	field strength at distance r	electric_field / plain	`N/C`	
`Q`	magnitude of the source charge	charge / plain	`C`	
`r`	distance from the charge	length / distance	`m`	
`ke`		coulomb_constant / constant	`N*m^2/C^2`	8987551792.3 N*m^2/C^2

- **Conditions:** electric
- **Applies when:** outside a point or spherical charge
- **Fails when:** inside a conductor or a dielectric
- **Assumes:** a point (or spherically symmetric) charge; vacuum or air
- **Evidence:** fundamental_law



Electric field lines of a point charge Q = 2.0000e-06 C (pointing away from a positive charge), with the field strength the package returns at three distances, doubling the distance quarters the field (inverse square). Marked: 798893 N/C at 0.15 m; 199723 N/C at 0.3 m; 49930.8 N/C at 0.6 m.



Ef (field strength at distance r) against r (distance from the charge) from 0.15 m to 0.45 m. One curve for each value of Q (magnitude of the source charge): 2.0000e-06 C, 1.0000e-06 C, 4.0000e-06 C. Other inputs as in the worked example: ke = 8987551792.3 N*m^2/C^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.3 m, Ef = 199723 N/C.

```
classical_physics_ai999x.solve('point_charge_field', Q='2e-06 C', r='0.3 m', target='Ef')
# -> 199723 N/C   (SOLVED)
```

$$E_f = \frac{k_e Q}{r^2}$$

$$E_f = \frac{Q k_e}{r^2} = 199723\ \mathrm{N/C}$$

invalid call	outcome	message
without `Q`	NEEDS_INPUT	point_charge_field needs Q (magnitude of the source charge, C). Example: solve('point_charge_field', Q='... C', r='... m', ke='... N*m^2/C^2
`Q` in a wrong unit	INVALID_INPUT	Q is a charge: its unit must be like C, not 'kg'.

Questions in words that work:

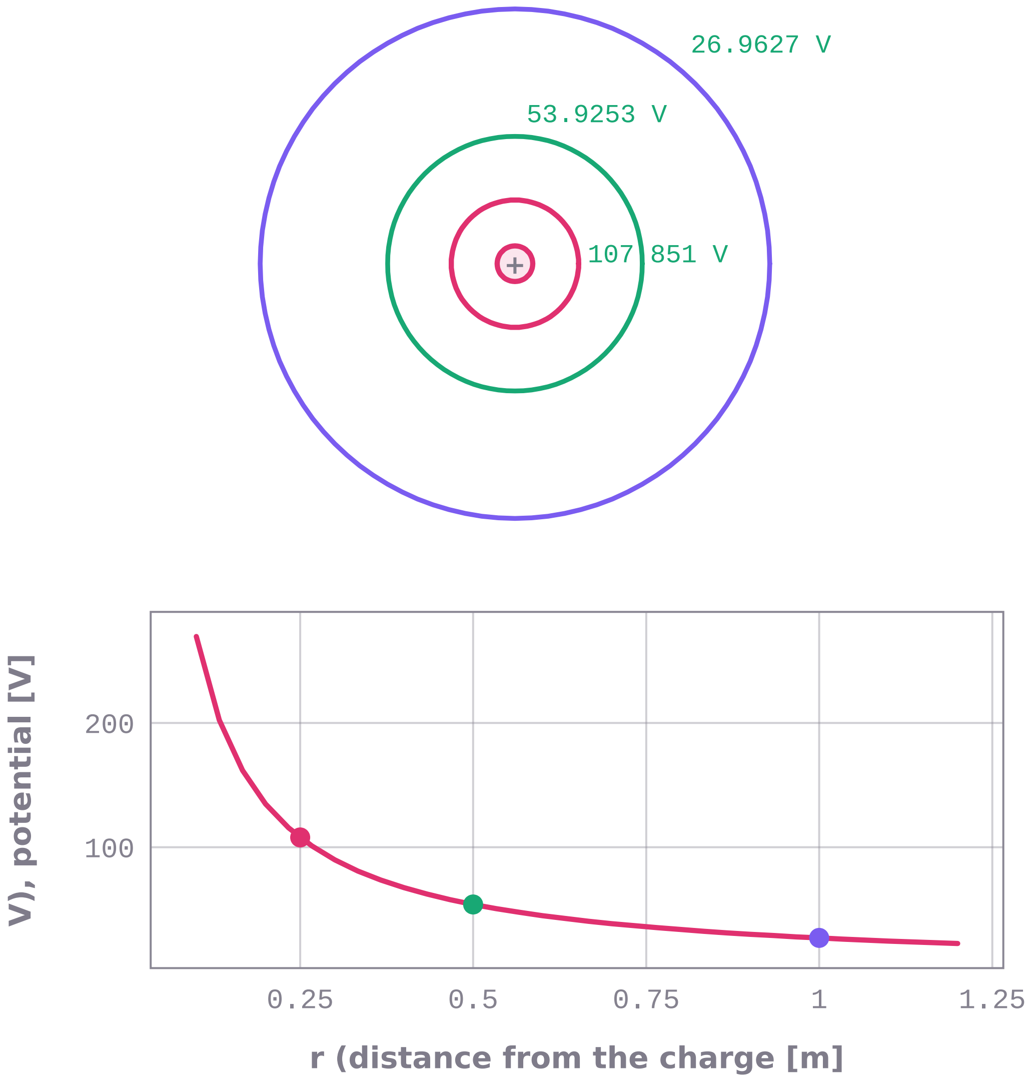
- "What is the electric field 0.3 m from a point charge of 2 uC?" → 199723 N/C
- "Find the electric field strength at a distance of 10 cm from a charged sphere carrying 1 nC." → 898.755 N/C

Electric potential of a point charge, `point_charge_potential`

$$V = \frac{k_e Q}{r}$$

input	meaning	kind / role	unit	default
`V`	electric potential relative to infinity	voltage / potential	`V`	
`Q`	source charge	charge / plain	`C`	
`r`	distance from the charge	length / distance	`m`	
`ke`		coulomb_constant / constant	`N*m^2/C^2`	8987551792.3 N*m^2/C^2

- **Conditions:** electric
- **Applies when:** outside a point or spherical charge
- **Fails when:** the reference point is not at infinity
- **Assumes:** a point charge; potential zero at infinity; vacuum or air
- **Evidence:** fundamental_law



Equipotential circles around a point charge Q = 3.0000e-09 C at r = 0.25, 0.5 and 1 m, labelled with the potential the package returns, and the potential against distance (34 distances): it falls as 1/r. Marked: 107.851 V at 0.25 m, 53.9253 V at 0.5 m, 26.9627 V at 1 m.

```
classical_physics_ai999x.solve('point_charge_potential', Q='3e-09 C', r='0.5 m',
target='V')
# -> 53.9253 V   (SOLVED)
```

$$V = \frac{k_e Q}{r}$$

$$V = \frac{Q k_e}{r} = 53.9253\ \mathrm{V}$$

invalid call	outcome	message
without `Q`	NEEDS_INPUT	point_charge_potential needs Q (source charge, C). Example: solve('point_charge_potential', Q='... C', r='... m', ke='... N*m^2/C^2').
`Q` in a wrong unit	INVALID_INPUT	Q is a charge: its unit must be like C, not 'kg'.

Questions in words that work:

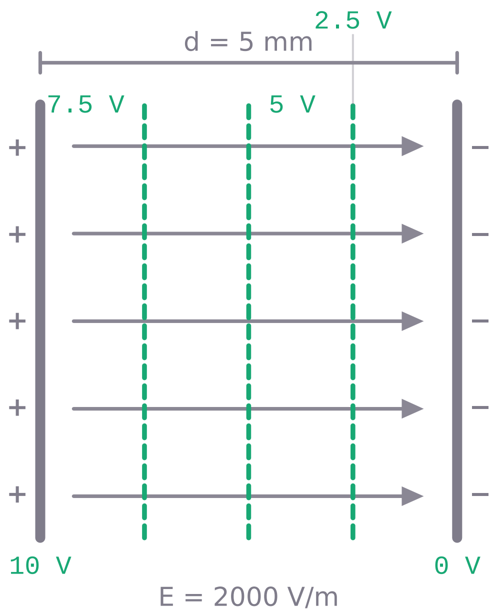
- "What is the electric potential 0.5 m from a point charge of 3 nC?" → 53.9253 V
- "Find the electric potential at 2 m from a point charge of 1 uC." → 4493.78 V

Potential difference in a uniform field, `uniform_field_potential`

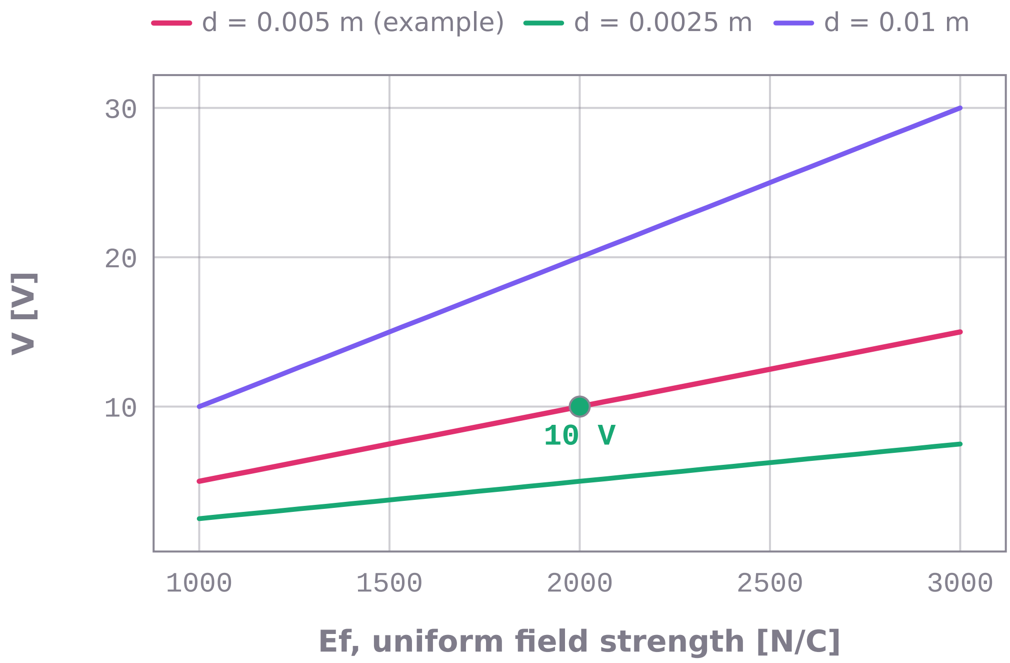
$$V = E_f d$$

input	meaning	kind / role	unit	default
`V`	potential difference between the plates	voltage / plain	`V`	
`Ef`	uniform field strength	electric_field / plain	`N/C`	
`d`	separation along the field	length / separation	`m`	

- **Conditions:** uniform_field
- **Applies when:** between parallel plates far from their edges
- **Fails when:** the field is not uniform
- **Assumes:** a uniform electric field
- **Evidence:** fundamental_law



A uniform field E = 2000 V/m between two plates d = 5 mm apart (field lines grey, from + to −), with equipotential planes (dashed) a quarter, half and three quarters of the way across. The potential difference the package returns over the whole gap is 10 V; the potential at each plane is that minus the drop over the distance from the + plate (drops returned: 2.5 V after 1.25 mm, 5 V after 2.5 mm, 7.5 V after 3.75 mm).



V (potential difference between the plates) against Ef (uniform field strength) from 1000 N/C to 3000 N/C. One curve for each value of d (separation along the field): 0.005 m, 0.0025 m, 0.01 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Ef = 2000 N/C, V = 10 V.

```
classical_physics_ai999x.solve('uniform_field_potential', Ef='2000 V/m', d='0.005 m',
target='V')
# -> 10 V   (SOLVED)
```

$$V = E_f d$$
$$V = E_f d = 10\ \mathrm{V}$$

invalid call	outcome	message
without `Ef`	NEEDS_INPUT	uniform_field_potential needs Ef (uniform field strength, N/C). Example: solve('uniform_field_potential', Ef='... N/C', d='... m').
`Ef` in a wrong unit	INVALID_INPUT	Ef is a electric field: its unit must be like N/C, not 'kg'.

Questions in words that work:

- "Parallel plates are 5 mm apart with a uniform electric field of 2000 V/m between the plates. What is the potential difference?" → 10 V
- "A 12 V battery is connected across parallel plates with a plate separation of 2 mm. What is the uniform electric field between the plates?" → 6000 N/C

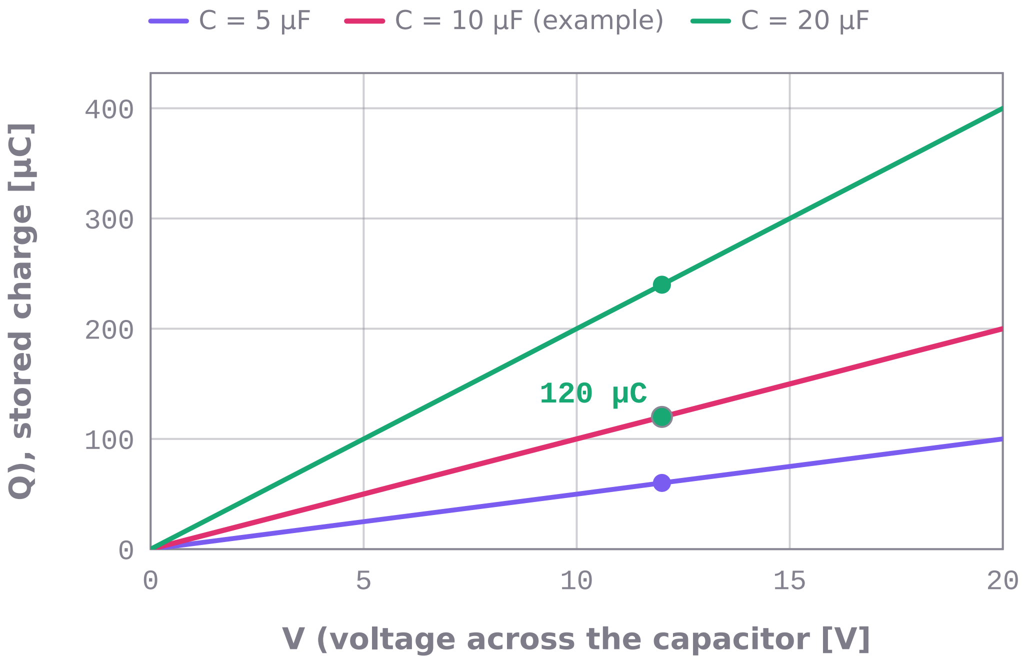
Charge on a capacitor, `capacitance_definition`

$$Q = CV$$

input	meaning	kind / role	unit	default
`Q`	charge on either plate	charge / plain	`C`	
`C`	capacitance	capacitance / plain	`F`	

input	meaning	kind / role	unit	default
`V`	voltage across the capacitor	voltage / plain	`V`	

- **Conditions:** capacitor
- **Applies when:** a charged capacitor
- **Fails when:** the dielectric breaks down
- **Assumes:** an ideal capacitor
- **Evidence:** definition



Charge stored on capacitors of 5, 10 and 20 μF against the voltage across them (10 voltages each from the package, charge shown in μC): Q = C V, so the capacitance is the slope. Marked at V = 12 V: C = 5.0000e-06 F: Q = 6.0000e-05 C; C = 1.0000e-05 F: Q = 0.00012 C; C = 2.0000e-05 F: Q = 0.00024 C.

```
classical_physics_ai999x.solve('capacitance_definition', C='1e-05 F', V='12 V', target='Q')
# -> 0.00012 C   (SOLVED)
```

$$Q = CV$$
$$Q = CV = 0.00012\ \mathrm{C}$$

invalid call	outcome	message
without `C`	NEEDS_INPUT	capacitance_definition needs C (capacitance, F). Example: solve('capacitance_definition', C='... F', V='... V').
`C` in a wrong unit	INVALID_INPUT	C is a capacitance: its unit must be like F, not 'kg'.

Questions in words that work:

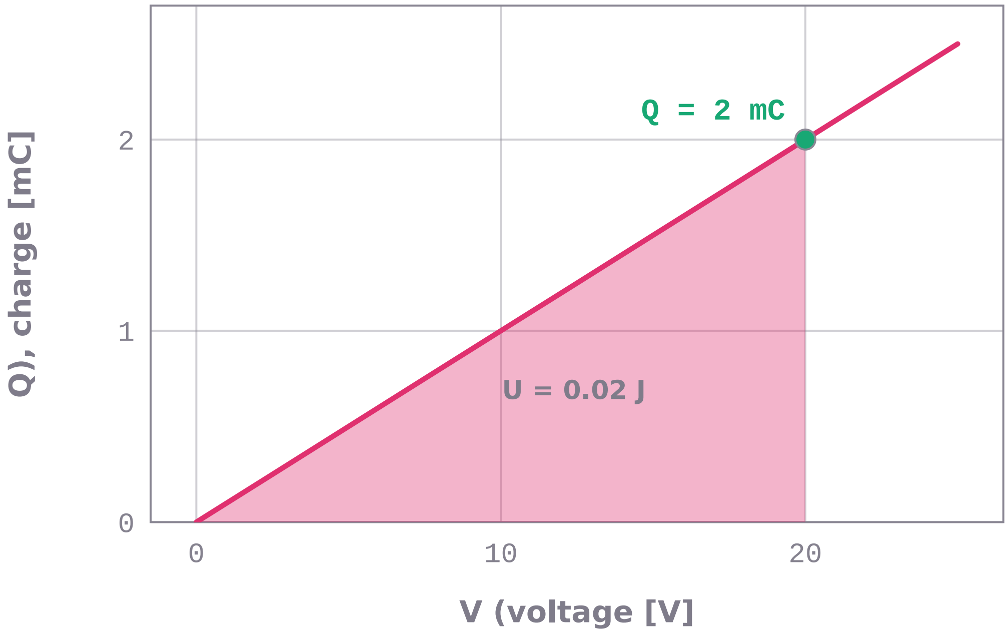
- "A 10 uF capacitor is charged to 12 V. How much charge does it hold?" → 0.00012 C
- "A capacitor holds a charge of 5 uC when connected to 100 V. What is its capacitance?" → 5e-08 F

Energy stored in a capacitor, `capacitor_energy`

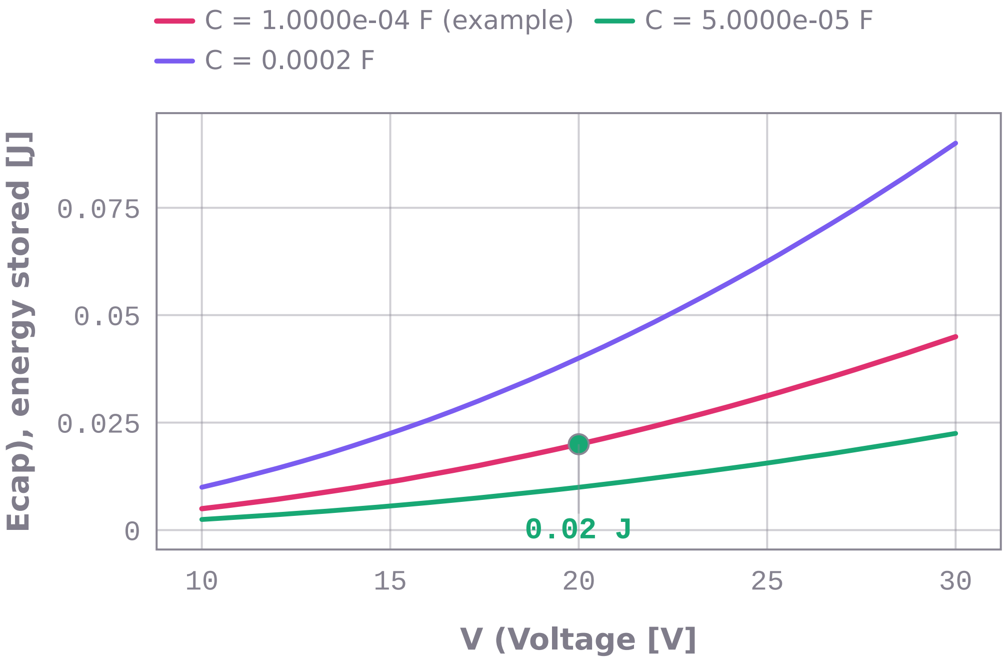
$$\text{Ecap} = \frac{CV^2}{2}$$

input	meaning	kind / role	unit	default
`Ecap`	energy stored	energy / capacitor	`J`	
`C`		capacitance / plain	`F`	
`V`		voltage / plain	`V`	

- **Conditions:** capacitor
- **Applies when:** a charged capacitor
- **Fails when:** the capacitor leaks or breaks down
- **Assumes:** an ideal capacitor
- **Evidence:** fundamental_law



Charge on a C = 0.0001 F capacitor against its voltage (from the package's Q = C V, charge in mC): the energy stored when it is charged to 20 V is the triangle under the line, ½ Q V. Marked: the energy the package returns, U = 0.02 J, and Q = 0.002 C.



Ecap (energy stored) against V (Voltage) from 10 V to 30 V. One curve for each value of C (Capacitance): 1.0000e-04 F, 5.0000e-05 F, 0.0002 F. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, V = 20 V, Ecap = 0.02 J.

```
classical_physics_ai999x.solve('capacitor_energy', C='0.0001 F', V='20 V', target='Ecap')
# -> 0.02 J   (SOLVED)
```

$$\mathrm{Ecap} = \frac{CV^2}{2}$$

$$\mathrm{Ecap} = \frac{CV^2}{2} = 0.02\ \mathrm{J}$$

invalid call	outcome	message
without `C`	NEEDS_INPUT	capacitor_energy needs C (capacitance, F). Example: solve('capacitor_energy', C='... F', V='... V').
`C` in a wrong unit	INVALID_INPUT	C is a capacitance: its unit must be like F, not 'kg'.

Questions in words that work:

- "A 100 uF capacitor is charged to 20 V. What is the energy stored in the capacitor?" → 0.02 J
- "How much energy is stored in a capacitor of 2 uF charged to 300 V?" → 0.09 J

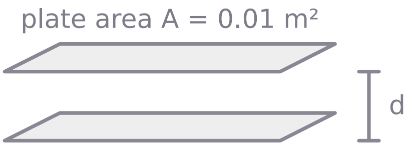
Capacitance of parallel plates, `parallel_plate_capacitor`

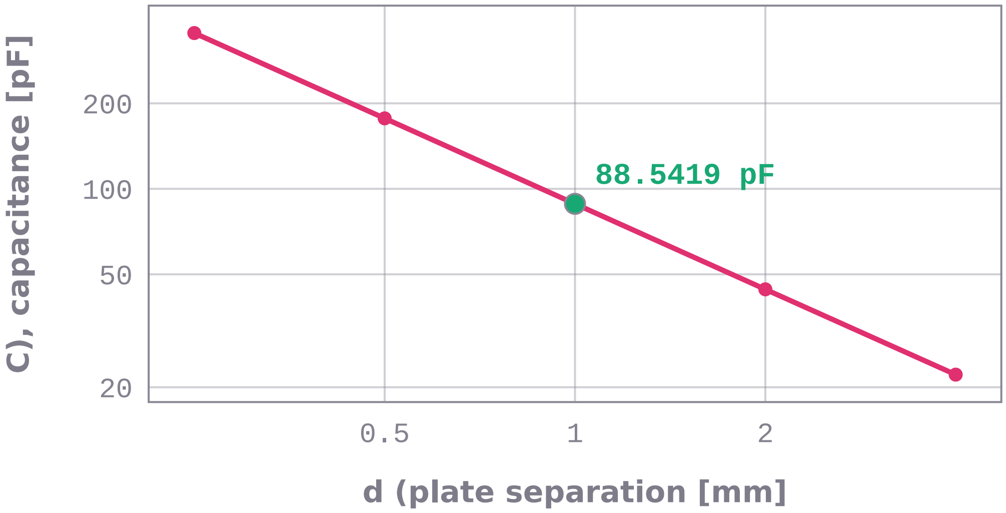
$$C = \frac{\varepsilon_0 A}{d}$$

input	meaning	kind / role	unit	default
`C`		capacitance / plain	`F`	
`A`	area of each plate	area / plain	`m^2`	

input	meaning	kind / role	unit	default
`d`	plate separation	length / separation	`m`	
`eps0`		permittivity / constant	`C^2/(N*m^2)`	8.8541878128e-12 C^2/(N*m^2)

- **Conditions:** capacitor
- **Applies when:** parallel plates with air or vacuum between them
- **Fails when:** a dielectric fills the gap; the plates are small compared with their separation
- **Assumes:** plates much wider than their separation; vacuum or air between the plates
- **Evidence:** fundamental_law





A parallel-plate capacitor of plate area A = 0.01 m² in vacuum with the gap d halved and doubled (0.25 to 4 mm): the capacitance the package returns is inversely proportional to the gap (a straight line of slope −1 on these logarithmic axes; values in pF). Marked: C = 8.8542e-11 F at d = 1 mm.

```
classical_physics_ai999x.solve('parallel_plate_capacitor', A='0.01 m^2', d='0.001 m',
target='C')
# -> 8.85419e-11 F   (SOLVED)
```

$$C = \frac{\varepsilon_0 A}{d}$$

$$C = \frac{A \varepsilon_0}{d} = 8.85419e - 11\ \mathrm{F}$$

invalid call	outcome	message
without `A`	NEEDS_INPUT	parallel_plate_capacitor needs A (area of each plate, m^2). Example: solve('parallel_plate_capacitor', A='... m^2', d='... m', eps0='... C^2
`A` in a wrong unit	INVALID_INPUT	A is a area: its unit must be like m^2, not 'kg'.

Questions in words that work:

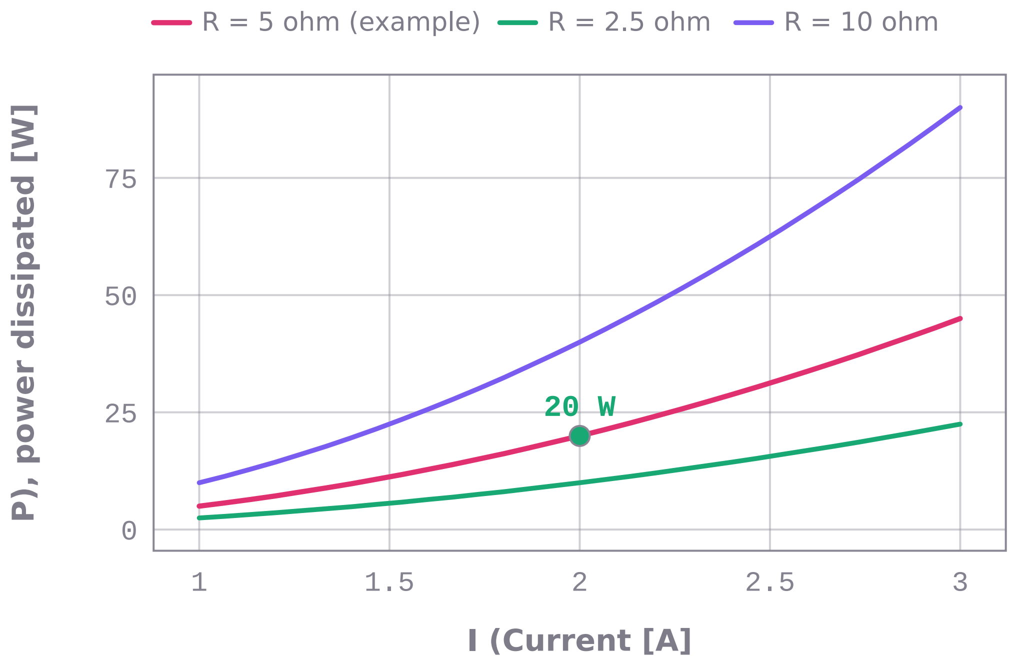
- "A capacitor has parallel plates of area 0.01 m^2 with a plate separation of 1 mm. What is its capacitance?" → 8.85419e-11 F
- "What is the capacitance of a capacitor with plates of area 2 m^2 and a separation of 5 mm?" → 3.54168e-09 F

Power from current and resistance, `ohm_power_current`

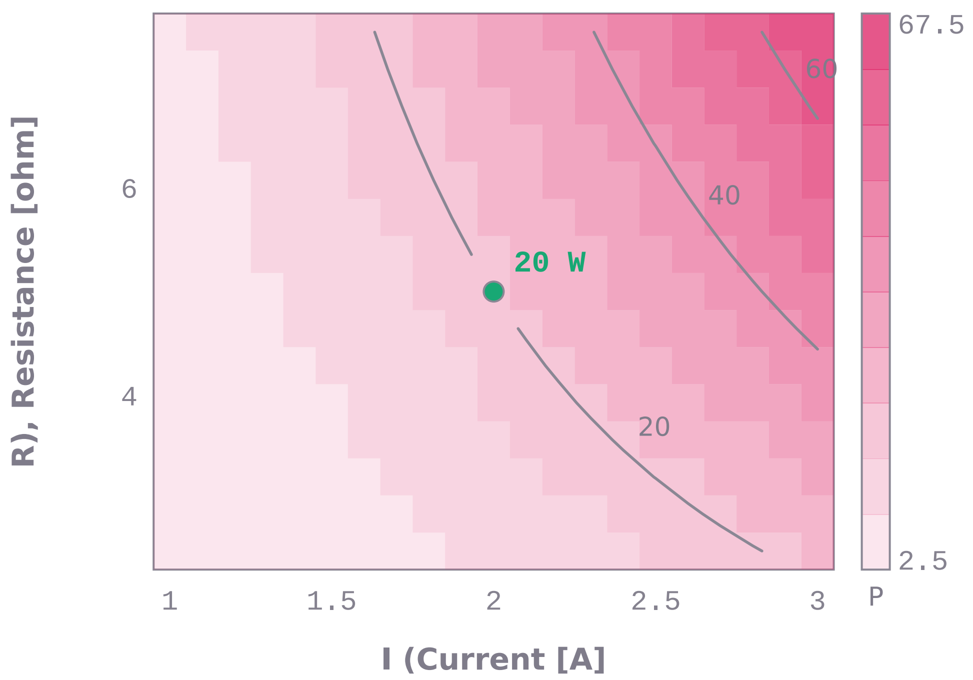
$$P = I^2 R$$

input	meaning	kind / role	unit	default
`P`	power dissipated	power / plain	`W`	
`I`		current / plain	`A`	
`R`		resistance / plain	`ohm`	

- **Conditions:** resistive
- **Not when:** alternating_current
- **Applies when:** a resistor carries a steady current
- **Fails when:** alternating current; a non-ohmic device
- **Assumes:** an ohmic resistor; steady direct current
- **Evidence:** derived



P (power dissipated) against I (Current) from 1 A to 3 A. One curve for each value of R (Resistance): 5 ohm, 2.5 ohm, 10 ohm. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I = 2 A, P = 20 W.



P (power dissipated) over I (Current) from 1 A to 3 A and R (Resistance) from 2.5 ohm to 7.5 ohm: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 2.5 W to dark 67.5 W); iso-lines at P = 20, 40, 60 W. Marked: the worked example, I = 2 A, R = 5 ohm, P = 20 W.

```
classical_physics_ai999x.solve('ohm_power_current', I='2 A', R='5 ohm', target='P')
# -> 20 W   (SOLVED)
```

$$P = I^2 R$$
$$P = I^2 R = 20 \text{ W}$$

invalid call	outcome	message
without `I`	NEEDS_INPUT	ohm_power_current needs I (current, A). Example: solve('ohm_power_current', I='... A', R='... ohm').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Questions in words that work:

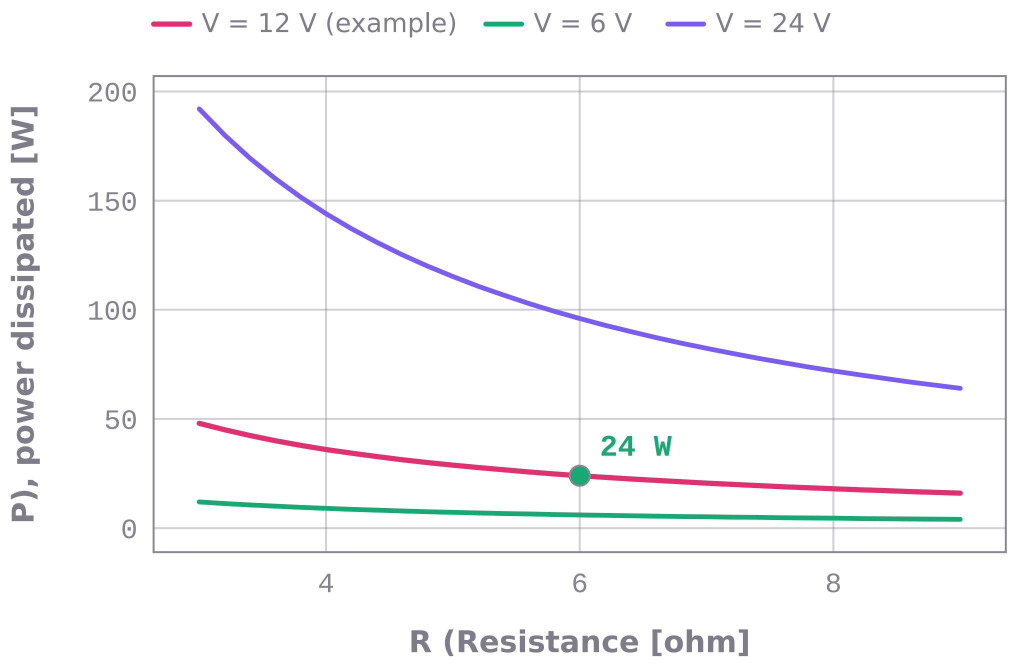
- "A current of 2 A flows through a 5 ohm resistor. What power does it dissipate?" → 20 W
- "A heater with a resistance of 4 ohm dissipates 100 W. What current flows through it?" → 5 A

Power from voltage and resistance, `ohm_power_voltage`

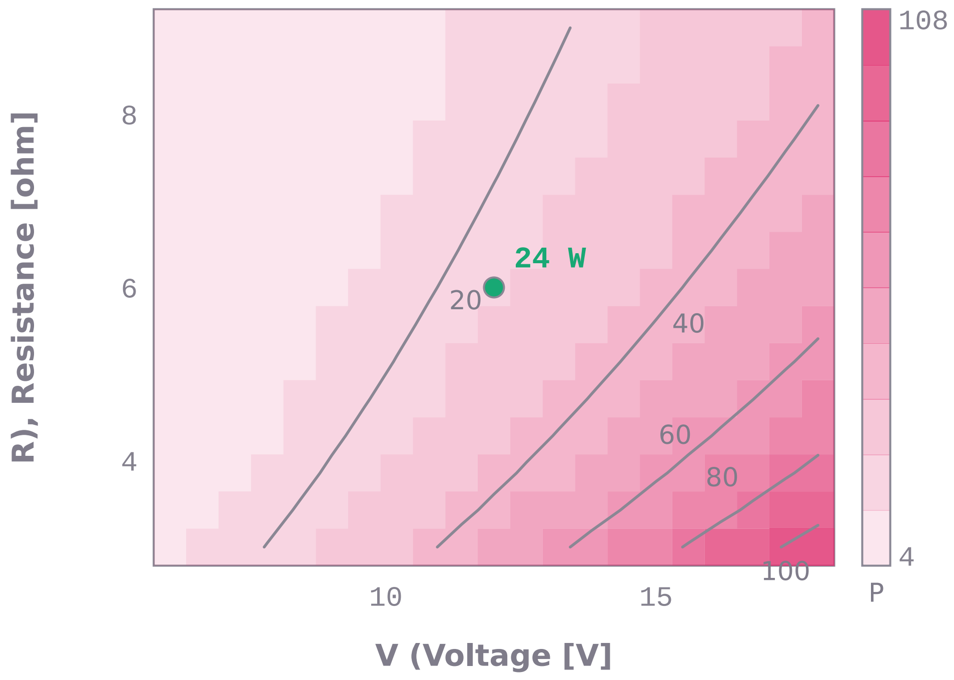
$$P = \frac{V^2}{R}$$

input	meaning	kind / role	unit	default
`P`	power dissipated	power / plain	`W`	
`V`		voltage / plain	`V`	
`R`		resistance / plain	`ohm`	

- **Conditions:** resistive
- **Not when:** alternating_current
- **Applies when:** a resistor has a steady voltage across it
- **Fails when:** alternating current; a non-ohmic device
- **Assumes:** an ohmic resistor; steady direct current
- **Evidence:** derived



P (power dissipated) against R (Resistance) from 3 ohm to 9 ohm. One curve for each value of V (Voltage): 12 V, 6 V, 24 V. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 6 ohm, P = 24 W.



P (power dissipated) over V (Voltage) from 6 V to 18 V and R (Resistance) from 3 ohm to 9 ohm: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 4 W to dark 108 W); iso-lines at P = 20, 40, 60, 80, 100 W. Marked: the worked example, V = 12 V, R = 6 ohm, P = 24 W.

```
classical_physics_ai999x.solve('ohm_power_voltage', V='12 V', R='6 ohm', target='P')
# -> 24 W   (SOLVED)
```

$$P = \frac{V^2}{R}$$

$$P = \frac{V^2}{R} = 24\ \mathrm{W}$$

invalid call	outcome	message
without `V`	NEEDS_INPUT	ohm_power_voltage needs V (voltage, V). Example: solve('ohm_power_voltage', V='... V', R='... ohm').
`V` in a wrong unit	INVALID_INPUT	V is a voltage: its unit must be like V, not 'kg'.

Questions in words that work:

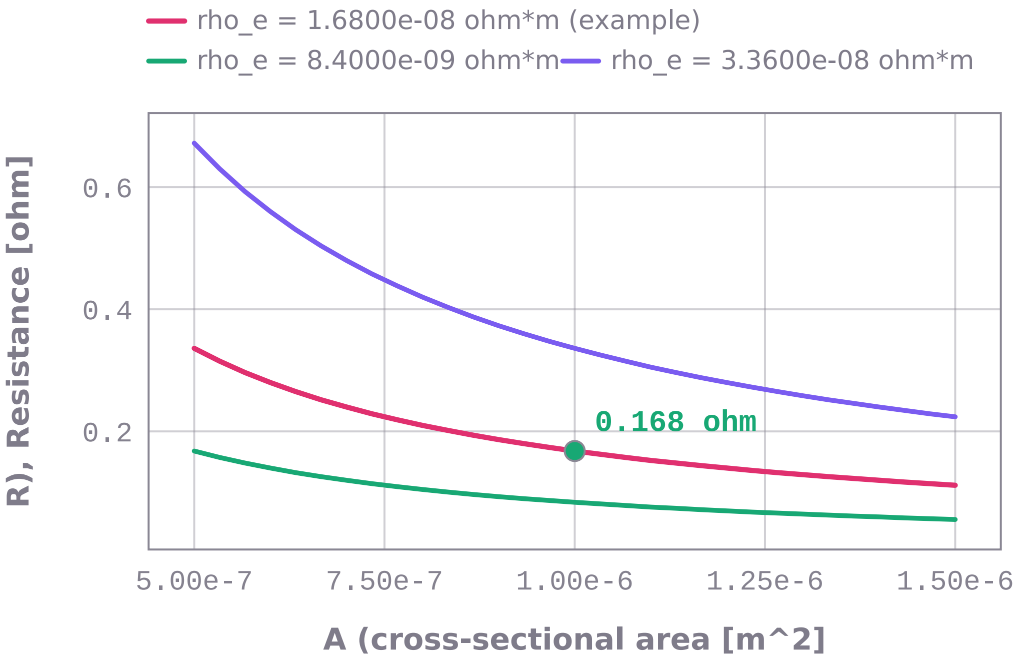
- "A 6 ohm resistor is connected across 12 V. What power does it dissipate?" → 24 W
- "A lamp dissipates 60 W when connected to 120 V. What is its resistance?" → 240 ohm

Resistance of a uniform wire, `resistivity`

$$R = \frac{\rho_e L}{A}$$

input	meaning	kind / role	unit	default
`R`		resistance / plain	`ohm`	
`rho_e`	resistivity of the material	resistivity / plain	`ohm*m`	
`L`	length of the wire	length / wire_length	`m`	
`A`	cross-sectional area	area / plain	`m^2`	

- **Conditions:** wire
- **Applies when:** an ohmic conductor of uniform cross-section
- **Fails when:** the temperature changes; non-uniform cross-section
- **Assumes:** a uniform wire; constant temperature
- **Evidence:** empirical



R (Resistance) against A (cross-sectional area) from 5.0000e-07 m^2 to 1.5000e-06 m^2. One curve for each value of rho_e (resistivity of the material): 1.6800e-08 ohm*m, 8.4000e-09 ohm*m, 3.3600e-08 ohm*m. Other inputs as in the worked example: L = 10.0 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 1.0000e-06 m^2, R = 0.168 ohm.

```
classical_physics_ai999x.solve('resistivity', rho_e='1.68e-08 ohm*m', L='10 m', A='1e-06
m^2', target='R')
# -> 0.168 ohm   (SOLVED)
```

$$R = \frac{\rho_e L}{A}$$

$$R = \frac{L \rho_e}{A} = 0.168 \text{ ohm}$$

invalid call	outcome	message
without `rho_e`	NEEDS_INPUT	resistivity needs rho_e (resistivity of the material, ohm*m). Example: solve('resistivity', rho_e='... ohm*m', L='... m', A='... m^2').
`rho_e` in a wrong unit	INVALID_INPUT	rho_e is a resistivity: its unit must be like ohm*m, not 'kg'.

Questions in words that work:

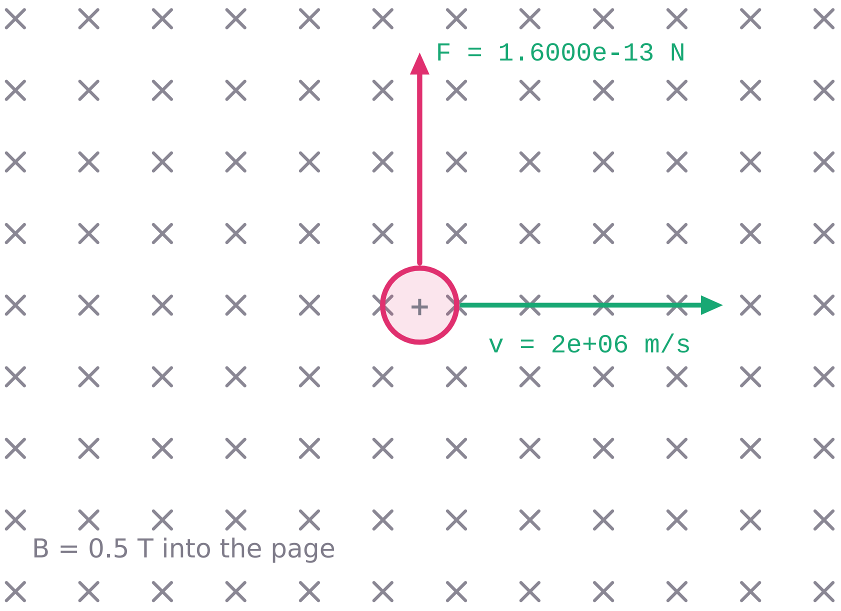
- "A copper wire of resistivity 1.68e-8 ohm*m has a wire length of 10 m and a cross-sectional area of 1e-6 m^2. What is its resistance?" → 0.168 ohm
- "What is the resistance of a wire with resistivity 5e-7 ohm*m, wire length 0.8 m and area 2e-7 m^2?" → 2 ohm

Magnetic force on a moving charge, `magnetic_force_charge`

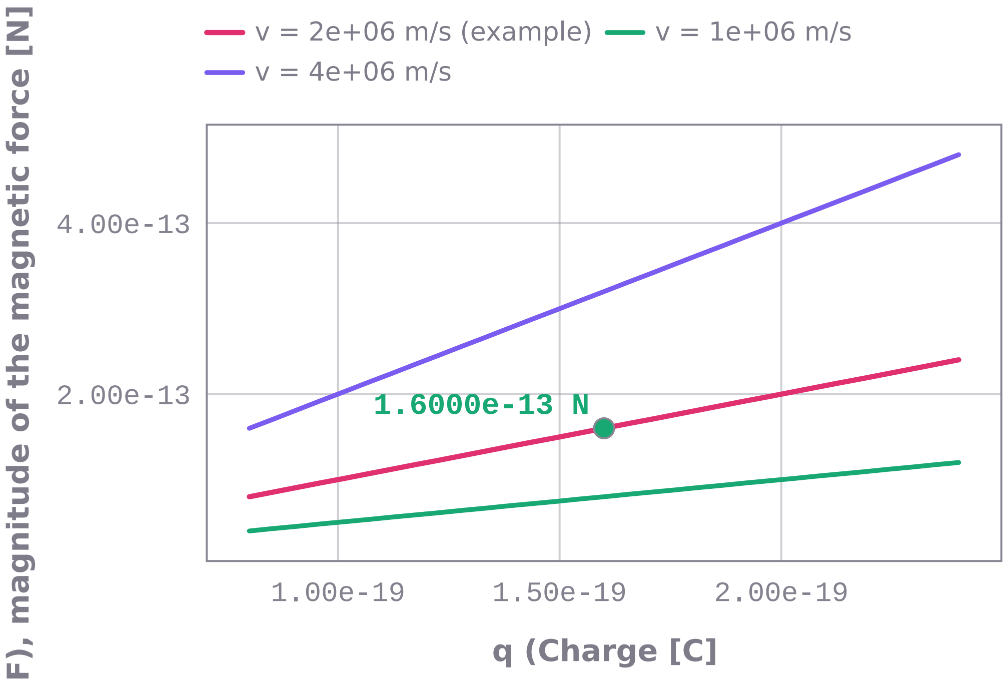
$$F = qvB \sin(\theta)$$

input	meaning	kind / role	unit	default
`F`	magnitude of the magnetic force	force / magnetic	`N`	
`q`		charge / plain	`C`	
`v`	speed of the charge	speed / plain	`m/s`	
`B`		magnetic_field / plain	`T`	
`theta`	angle between velocity and field	angle / plain	`rad`	

- **Conditions:** magnetic
- **Applies when:** a charge moving in a magnetic field
- **Fails when:** relativistic speeds; an electric field also acts
- **Assumes:** a uniform magnetic field; no electric field
- **Evidence:** fundamental_law



A charge q = 1.6000e-19 C moving at v = 2e+06 m/s through a uniform magnetic field B = 0.5 T pointing into the page (×), at 90° to the velocity: the force is perpendicular to both (F = q v × B). Marked: the force the package returns, F = q v B sin θ = 1.6000e-13 N.



F (magnitude of the magnetic force) against q (Charge) from 8.0000e-20 C to 2.4000e-19 C. One curve for each value of v (speed of the charge): 2e+06 m/s, 1e+06 m/s, 4e+06 m/s. Other inputs as in the worked example: B = 0.5 T, theta = 1.5707963267948966 rad. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, q = 1.6000e-19 C, F = 1.6000e-13 N.

```
classical_physics_ai999x.solve('magnetic_force_charge', q='1.6e-19 C', v='2e+06 m/s',
B='0.5 T', theta='90 deg', target='F')
# -> 1.6e-13 N   (SOLVED)
```

$$F = qvB\sin(\theta)$$
$$F = Bqv\sin(\theta) = 1.6e - 13\ \mathrm{N}$$

invalid call	outcome	message
without `q`	NEEDS_INPUT	magnetic_force_charge needs q (charge, C). Example: solve('magnetic_force_charge', q='... C', v='... m/s', B='... T', theta='... rad').
`q` in a wrong unit	INVALID_INPUT	q is a charge: its unit must be like C, not 'kg'.

Questions in words that work:

- "A proton with charge 1.6e-19 C moves at 2e6 m/s perpendicular to the field of a 0.5 T magnetic field. What is the magnetic force on it?" → 1.6e-13 N
- "A 3 uC charge moves at 400 m/s at an angle of 30 degrees to a 0.2 T magnetic field. What is the magnetic force?" → 0.00012 N

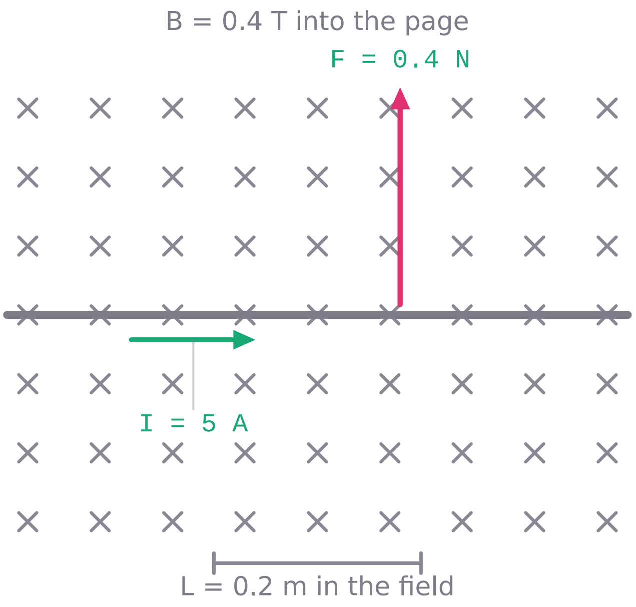
Magnetic force on a current-carrying wire, `magnetic_force_wire`

$$F = ILB\sin(\theta)$$

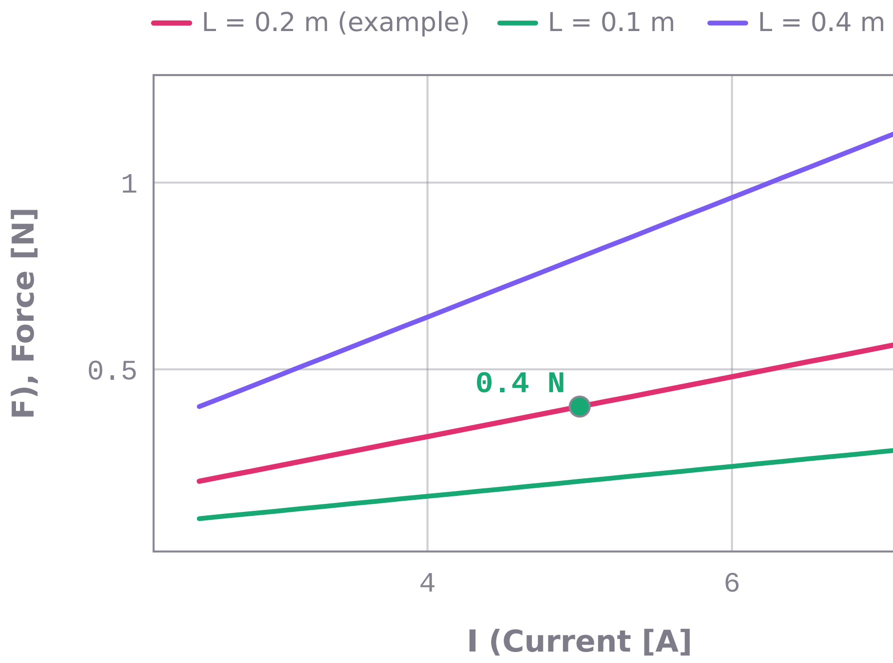
input	meaning	kind / role	unit	default
`F`		force / magnetic	`N`	

input	meaning	kind / role	unit	default
`I`		current / plain	`A`	
`L`	length of wire in the field	length / wire_length	`m`	
`B`		magnetic_field / plain	`T`	
`theta`	angle between the wire and the field	angle / plain	`rad`	

- **Conditions:** magnetic, wire
- **Applies when:** a straight wire in a uniform magnetic field
- **Fails when:** the field varies along the wire
- **Assumes:** a straight wire; a uniform field
- **Evidence:** fundamental_law



A straight wire carrying I = 5 A to the right, with L = 0.2 m of it in a uniform field B = 0.4 T into the page (×), at 90°: the force is perpendicular to wire and field (F = I L × B). Marked: the force the package returns, F = 0.4 N.



F (Force) against I (Current) from 2.5 A to 7.5 A. One curve for each value of L (length of wire in the field): 0.2 m, 0.1 m, 0.4 m. Other inputs as in the worked example: B = 0.4 T, theta = 1.5707963267948966 rad. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I = 5 A, F = 0.4 N.

```
classical_physics_ai999x.solve('magnetic_force_wire', I='5 A', L='0.2 m', B='0.4 T',
theta='90 deg', target='F')
# -> 0.4 N   (SOLVED)
```

$$F = ILB\sin\left(\theta\right)$$
$$F = BIL\sin\left(\theta\right) = 0.4\ \mathrm{N}$$

invalid call	outcome	message
without `I`	NEEDS_INPUT	magnetic_force_wire needs I (current, A). Example: solve('magnetic_force_wire', I='... A', L='... m', B='... T', theta='... rad').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Questions in words that work:

- "A wire carrying 5 A has a wire length of 0.2 m perpendicular to the field of a 0.4 T magnetic field. What is the magnetic force on the wire?" → 0.4 N
- "A wire length of 0.5 m carries 2 A at an angle of 30 degrees to a 0.1 T magnetic field. What is the magnetic force?" → 0.05 N

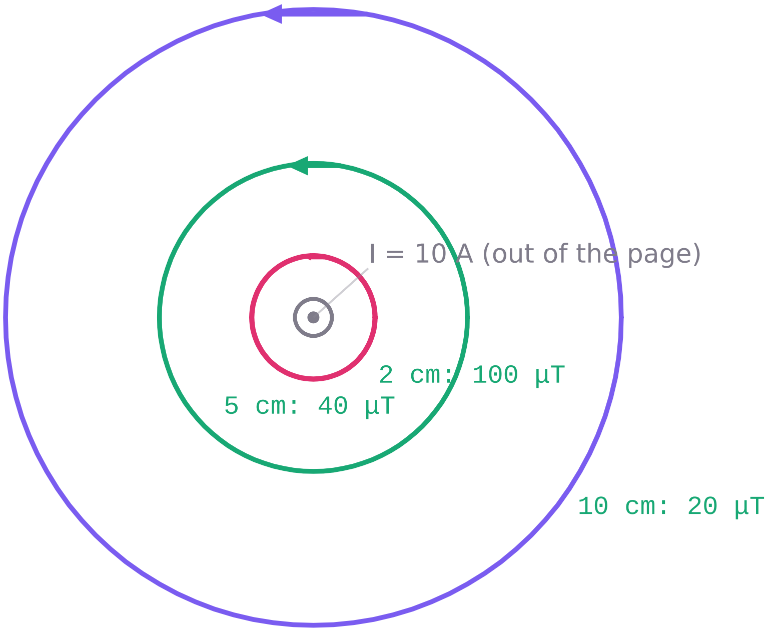
Magnetic field of a long straight wire, `long_wire_field`

$$B = \frac{\mu_0 I}{2\pi r}$$

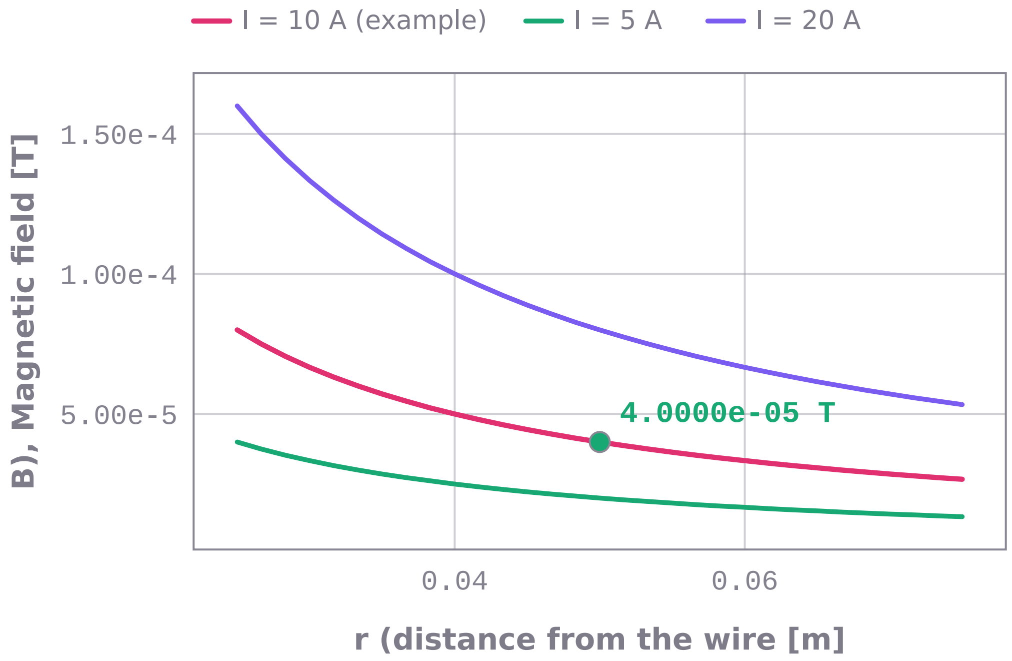
input	meaning	kind / role	unit	default
`B`		magnetic_field / plain	`T`	

input	meaning	kind / role	unit	default
`I`		current / plain	`A`	
`r`	distance from the wire	length / distance	`m`	
`mu0`		permeability / constant	`T*m/A`	1.25663706212e-06 T*m/A

- **Conditions:** wire
- **Not when:** loop, solenoid
- **Applies when:** near a long straight wire, far from its ends
- **Fails when:** near the ends of a short wire
- **Assumes:** a long straight wire; vacuum or air
- **Evidence:** fundamental_law



Magnetic field lines round a long straight wire carrying I = 10 A out of the page: circles, anticlockwise (right-hand rule). Marked: the field strength the package returns at 2, 5 and 10 cm (0.0001 T, 4.0000e-05 T, 2.0000e-05 T), falling as 1/r.



B (Magnetic field) against r (distance from the wire) from 0.025 m to 0.075 m. One curve for each value of I (Current): 10 A, 5 A, 20 A. Other inputs as in the worked example: mu0 = 1.25663706212e-06 T*m/A. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.05 m, B = 4.0000e-05 T.

```
classical_physics_ai999x.solve('long_wire_field', I='10 A', r='0.05 m', target='B')
# -> 4e-05 T   (SOLVED)
```

$$B = \frac{\mu_0 I}{2\pi r}$$

$$B = \frac{I\mu_0}{2\pi r} = 4e - 05 \text{ T}$$

invalid call	outcome	message
without `I`	NEEDS_INPUT	long_wire_field needs I (current, A). Example: solve('long_wire_field', I='... A', r='... m', mu0='... T*m/A').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Questions in words that work:

- "What is the magnetic field 5 cm from a long straight wire carrying 10 A?" → 4e-05 T
- "A long wire carries 2 A. What is the magnetic field at a distance of 1 cm?" → 4e-05 T

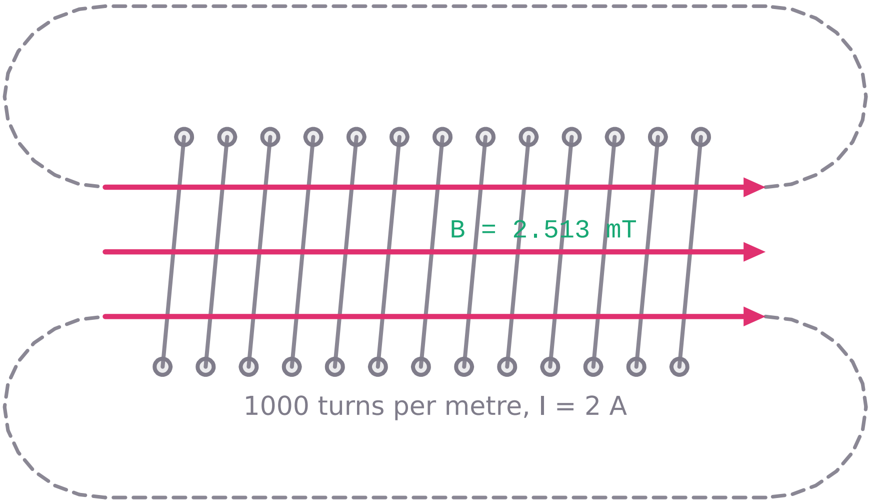
Magnetic field inside a long solenoid, `solenoid_field`

$$B = \mu_0 n_t I$$

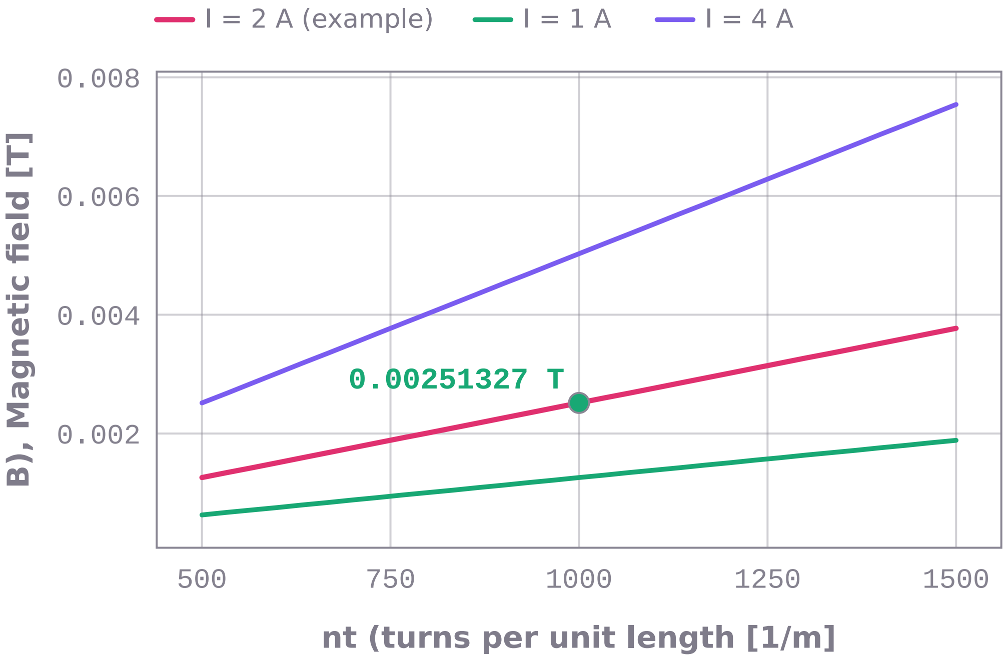
input	meaning	kind / role	unit	default
`B`		magnetic_field / plain	`T`	
`nt`	turns per unit length	turn_density / plain	`1/m`	
`I`		current / plain	`A`	

input	meaning	kind / role	unit	default
`mu0`		permeability / constant	`T*m/A`	1.25663706212e-06 T*m/A

- **Conditions:** solenoid
- **Applies when:** inside a long solenoid
- **Fails when:** near the ends; an iron core
- **Assumes:** a long solenoid; air core
- **Evidence:** fundamental_law



A long solenoid with n = 1000 turns per metre carrying I = 2 A: inside, the field is uniform and along the axis (solid arrows); outside it is weak and returns round the ends (dashed, schematic). Marked: the field inside the package returns, B = μ₀ n I = 0.00251327 T.



B (Magnetic field) against nt (turns per unit length) from 500 1/m to 1500 1/m. One curve for each value of I (Current): 2 A, 1 A, 4 A. Other inputs as in the worked example: mu0 = 1.25663706212e-06 T*m/A. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, nt = 1000 1/m, B = 0.00251327 T.

```
classical_physics_ai999x.solve('solenoid_field', nt='1000 1/m', I='2 A', target='B')
# -> 0.00251327 T   (SOLVED)
```

$$B = \mu_0 n_t I$$
$$B = I \mu_0 n_t = 0.00251327 \text{ T}$$

invalid call	outcome	message
without `nt`	NEEDS_INPUT	solenoid_field needs nt (turns per unit length, 1/m). Example: solve('solenoid_field', nt='... 1/m', I='... A', mu0='... T*m/A').
`nt` in a wrong unit	INVALID_INPUT	nt is a turn density: its unit must be like 1/m, not 'kg'.

Questions in words that work:

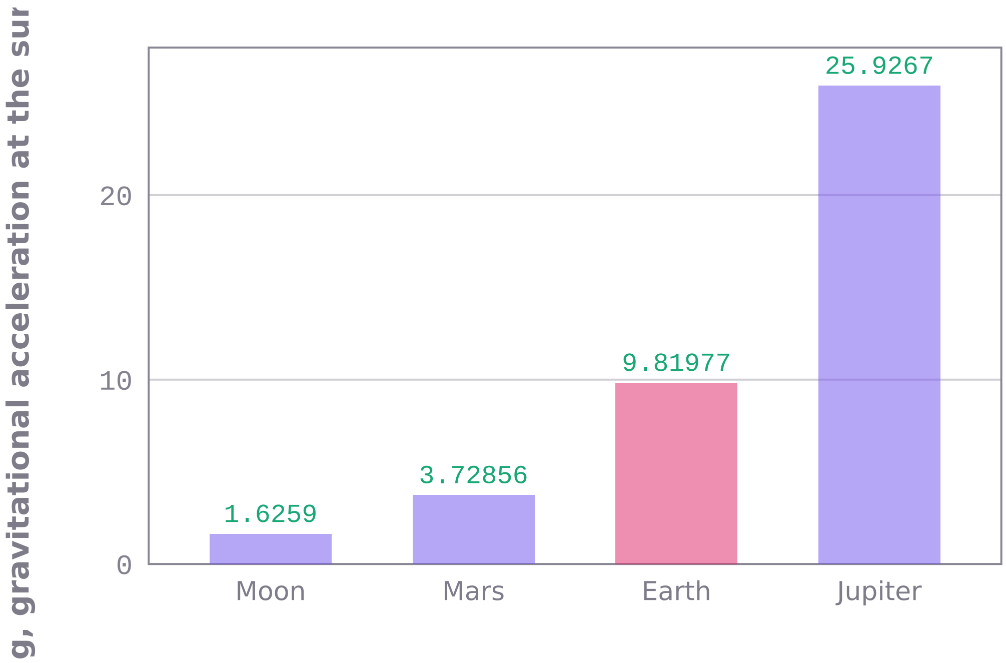
- "A solenoid has 1000 turns per meter and carries 2 A. What is the magnetic field inside it?" → 0.00251327 T
- "What current gives a magnetic field of 0.01 T inside a solenoid with 2000 turns per meter?" → 3.97887 A

Gravitational acceleration at a distance from a mass, `surface_gravity`

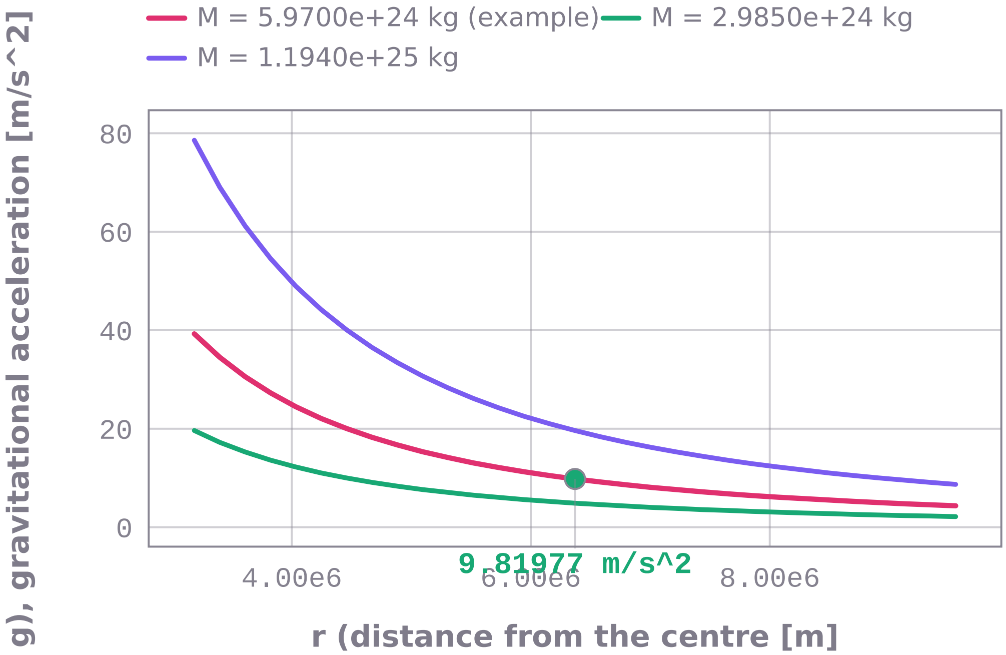
$$g = \frac{GM}{r^2}$$

input	meaning	kind / role	unit	default
`g`	gravitational acceleration	acceleration / gravity	`m/s^2`	
`M`	mass of the planet or star	mass / plain	`kg`	
`r`	distance from the centre	length / radius	`m`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2

- **Conditions:** celestial
- **Applies when:** at or above the surface of a planet or star
- **Fails when:** inside the body
- **Assumes:** a spherically symmetric body; outside the body
- **Evidence:** fundamental_law



Surface gravity of 4 bodies from their mass and radius (stated: Moon 7.3500e+22 kg, 1.737e+06 m; Mars 6.4200e+23 kg, 3.39e+06 m; Earth 5.9700e+24 kg, 6.37e+06 m; Jupiter 1.8980e+27 kg, 6.9900e+07 m; Earth is the worked example), each from the package. Marked: Moon 1.6259 m/s², Mars 3.72856 m/s², Earth 9.81977 m/s², Jupiter 25.9267 m/s².



g (gravitational acceleration) against r (distance from the centre) from 3.185e+06 m to 9.555e+06 m. One curve for each value of M (mass of the planet or star): 5.9700e+24 kg, 2.9850e+24 kg, 1.1940e+25 kg. Other inputs as in the worked example: G = 6.6743e-11 N*m^2/kg^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 6.37e+06 m, g = 9.81977 m/s^2.

```
classical_physics_ai999x.solve('surface_gravity', M='5.97e+24 kg', r='6.37e+06 m',
target='g')
# -> 9.81977 m/s^2   (SOLVED)
```

$$g = \frac{GM}{r^2}$$

$$g = \frac{GM}{r^2} = 9.81977\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	surface_gravity needs M (mass of the planet or star, kg). Example: solve('surface_gravity', M='... kg', r='... m', G='... N*m^2/kg^2').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

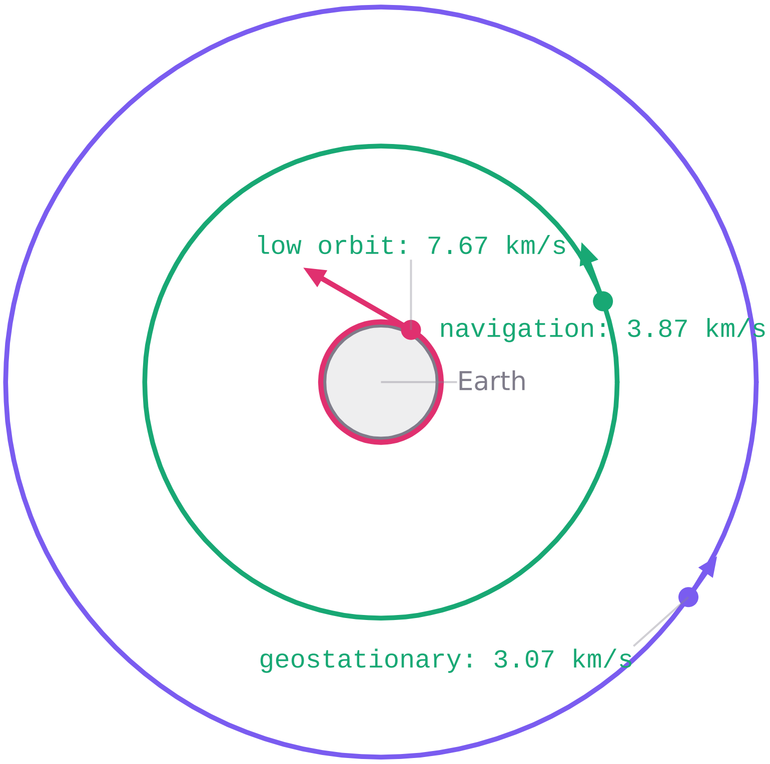
- "A planet has a mass of 5.97e24 kg and a radius of 6.37e6 m. What is the acceleration due to gravity at its surface?" → 9.81977 m/s^2
- "The moon has a mass of 7.35e22 kg and a radius of 1.74e6 m. What is its surface gravity?" → 1.6203 m/s^2

Speed in a circular orbit, `orbital_speed`

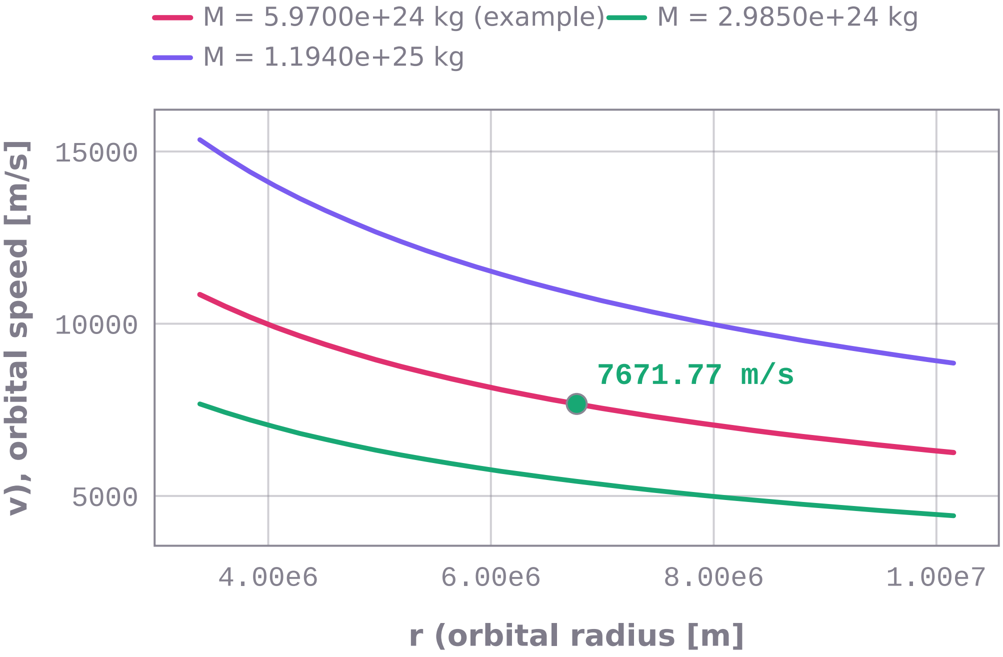
$$v = \sqrt{\frac{GM}{r}}$$

input	meaning	kind / role	unit	default
`v`	orbital speed	speed / orbital	`m/s`	
`M`	mass of the central body	mass / plain	`kg`	
`r`	orbital radius	length / radius	`m`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2

- **Conditions:** orbit
- **Applies when:** a satellite in a circular orbit
- **Fails when:** elliptical orbits
- **Assumes:** a circular orbit; the orbiting body much lighter than the central one
- **Evidence:** fundamental_law



Circular orbits round a body of mass M = 5.9700e+24 kg (the Earth, radius 6.371e+06 m) drawn to scale at orbital radii 6770 km, 26560 km, 42200 km (the first is the worked example), with velocity arrows to scale: the farther out, the slower. Marked: the orbital speeds the package returns, 7671.77 m/s, 3873.25 m/s, 3072.8 m/s.



v (orbital speed) against r (orbital radius) from 3.385e+06 m to 1.0155e+07 m. One curve for each value of M (mass of the central body): 5.9700e+24 kg, 2.9850e+24 kg, 1.1940e+25 kg. Other inputs as in the worked example: G = 6.6743e-11 N*m^2/kg^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 6.77e+06 m, v = 7671.77 m/s.

```
classical_physics_ai999x.solve('orbital_speed', M='5.97e+24 kg', r='6.77e+06 m',
target='v')
# -> 7671.77 m/s   (SOLVED)
```

$$v = \sqrt{\frac{GM}{r}}$$

$$v = \frac{\sqrt{G}\sqrt{M}}{\sqrt{r}} = 7671.77\ \mathrm{m/s}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	orbital_speed needs M (mass of the central body, kg). Example: solve('orbital_speed', M='... kg', r='... m', G='... N*m^2/kg^2').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

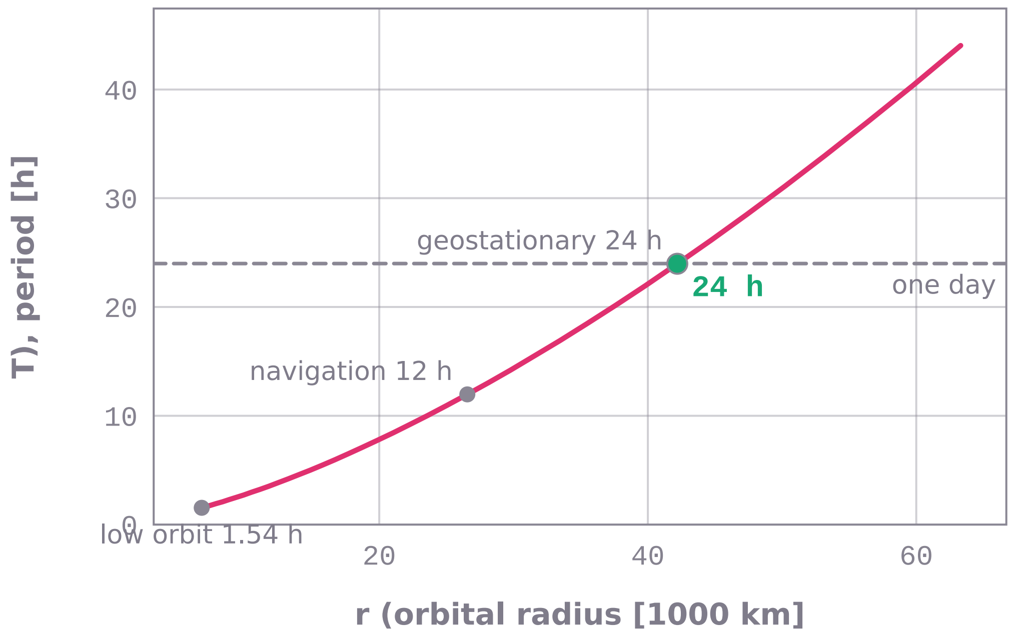
- "A satellite is in a circular orbit of radius 6.77e6 m around a planet of mass 5.97e24 kg. What is its orbital speed?" → 7671.77 m/s
- "A planet orbits a star of mass 1.99e30 kg at an orbital radius of 1.5e11 m. Find its orbital speed." → 29756.6 m/s

Period of a circular orbit, `orbital_period`

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

input	meaning	kind / role	unit	default
`T`	orbital period	time / period	`s`	
`r`	orbital radius	length / radius	`m`	
`M`	mass of the central body	mass / plain	`kg`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2

- **Conditions:** orbit
- **Applies when:** a satellite in a circular orbit
- **Fails when:** elliptical orbits
- **Assumes:** a circular orbit; the orbiting body much lighter than the central one
- **Evidence:** fundamental_law



Period of a circular orbit round a body of mass M = 5.9700e+24 kg against its radius, 40 radii from 6600 to 63300 km from the package (T ∝ r^(3/2); hours = seconds / 3600). Marked: low orbit 5544.63 s; navigation 43085.6 s; geostationary 86289.6 s; the given orbit 86289.6 s at r = 4.2200e+07 m.

```
classical_physics_ai999x.solve('orbital_period', r='4.22e+07 m', M='5.97e+24 kg',
target='T')
# -> 86289.6 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

$$T = \frac{2\pi r^{\frac{3}{2}}}{\sqrt{G}\sqrt{M}} = 86289.6\ \mathrm{s}$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	orbital_period needs r (orbital radius, m). Example: solve('orbital_period', r='... m', M='... kg', G='... N*m^2/kg^2').
`r` in a wrong unit	INVALID_INPUT	r is a length: its unit must be like m, not 'kg'.

Questions in words that work:

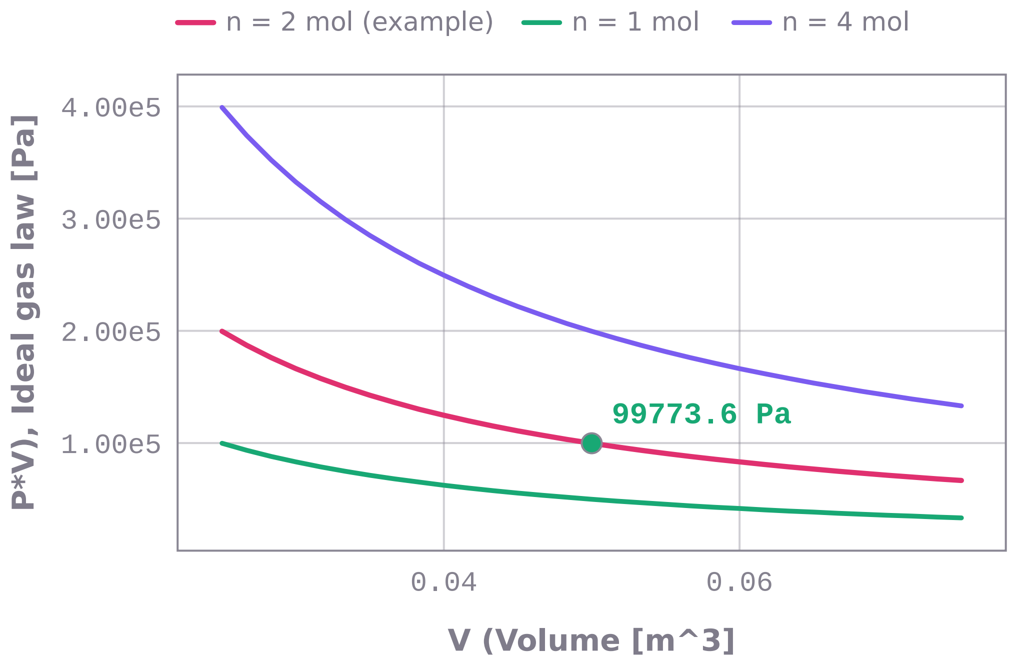
- "A satellite orbits a planet of mass 5.97e24 kg at an orbital radius of 4.22e7 m. What is its orbital period?" → 86289.6 s
- "What is the orbital period of a planet that orbits a star of mass 1.99e30 kg at a radius of 1.5e11 m?" → 3.16729e+07 s

Ideal gas law, `ideal_gas_moles`

$$PV = nRT$$

input	meaning	kind / role	unit	default
`P`	absolute pressure	pressure / plain	`Pa`	
`V`		volume / plain	`m^3`	
`n`	amount of gas	amount / plain	`mol`	
`T`	absolute temperature	temperature / plain	`K`	
`R`		gas_constant / constant	`J/(mol*K)`	8.314462618 J/(mol*K)

- **Conditions:** gas
- **Applies when:** a dilute gas far from condensation
- **Fails when:** high pressure or near condensation
- **Assumes:** an ideal gas; absolute pressure and temperature
- **Evidence:** fundamental_law



P*V (Ideal gas law) against V (Volume) from 0.025 m^3 to 0.075 m^3. One curve for each value of n (amount of gas): 2 mol, 1 mol, 4 mol. Other inputs as in the worked example: T = 300.0 K, R = 8.314462618 J/(mol*K). 31 points per curve computed by classical-physics-ai999x; marked: the worked example, V = 0.05 m^3, P*V = 99773.6 Pa.

```
classical_physics_ai999x.solve('ideal_gas_moles', n='1 mol', T='273.15 K', V='0.0224 m^3',
target='P')
# -> 101388 Pa   (SOLVED)
```

$$PV = nRT$$

$$P = \frac{RTn}{V} = 101388\ \mathrm{Pa}$$

invalid call	outcome	message
without `n`	NEEDS_INPUT	ideal_gas_moles needs n (amount of gas, mol). Example: solve('ideal_gas_moles', V='... m^3', n='... mol', T='... K', R='... J/(mol*K)').
`n` in a wrong unit	INVALID_INPUT	n is a amount: its unit must be like mol, not 'kg'.

Questions in words that work:

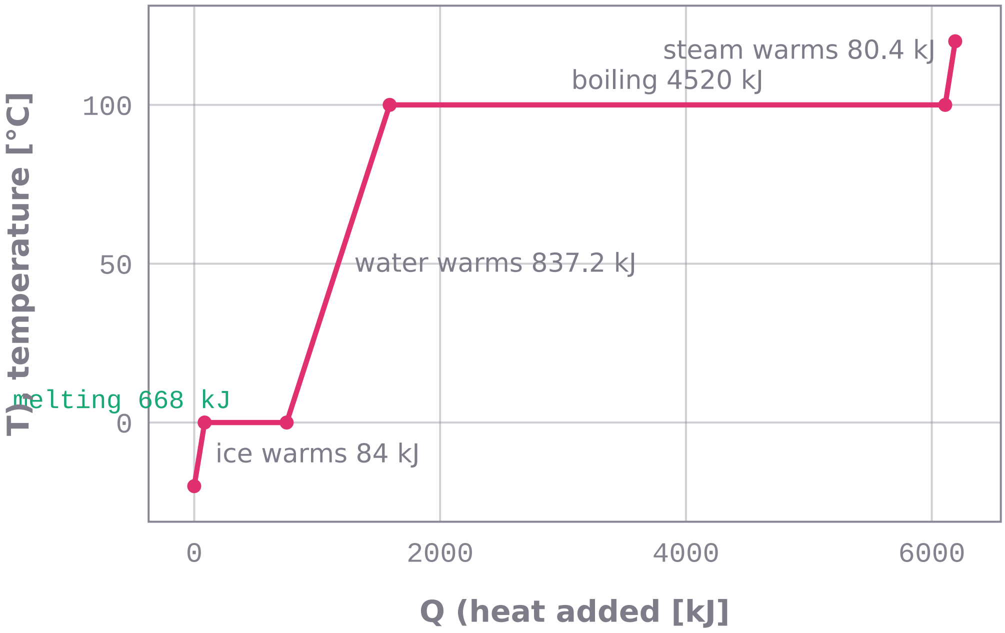
- "2 moles of an ideal gas are held at 300 K in a volume of 0.05 m^3. What is the pressure?" → 99773.6 Pa
- "A gas at a pressure of 200000 Pa occupies 0.01 m^3 at 300 K. How many moles of gas are there?" → 0.801816 mol

Heat to melt or freeze, `latent_heat_fusion`

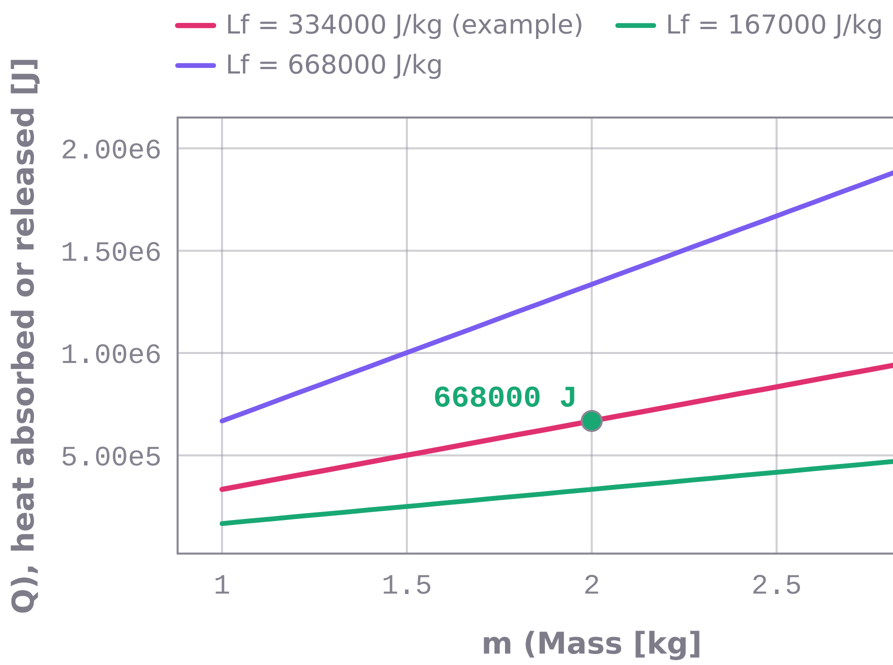
$$Q = mL_f$$

input	meaning	kind / role	unit	default
`Q`	heat absorbed or released	energy / heat	`J`	
`m`		mass / plain	`kg`	
`Lf`	latent heat of fusion	latent_heat / fusion	`J/kg`	

- **Conditions:** melting
- **Not when:** boiling
- **Applies when:** melting or freezing at the melting point
- **Fails when:** the temperature also changes
- **Assumes:** at the melting point; no temperature change during the phase change
- **Evidence:** fundamental_law



Heating curve of 2 kg of water from ice at −20 °C to steam at 120 °C at atmospheric pressure (stated: c_ice = 2100, c_water = 4186, c_steam = 2010 J/(kg·K), L_f = 334000 J/kg, L_v = 2.26e+06 J/kg): each stage's heat from the package (heat = m c ΔT; latent heats m L), accumulated; the temperature stays put while the phase changes. Marked: melting needs 668000 J, boiling 4.52e+06 J.



Q (heat absorbed or released) against m (Mass) from 1 kg to 3 kg. One curve for each value of Lf (latent heat of fusion): 334000 J/kg, 167000 J/kg, 668000 J/kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, Q = 668000 J.

```
classical_physics_ai999x.solve('latent_heat_fusion', m='2 kg', Lf='334000 J/kg',
target='Q')
# -> 668000 J   (SOLVED)
```

$$Q = mL_f$$
$$Q = L_f m = 668000\ \mathrm{J}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	latent_heat_fusion needs m (mass, kg). Example: solve('latent_heat_fusion', m='... kg', Lf='... J/kg').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "How much heat is needed to melt 2 kg of ice at 0 C? The latent heat of fusion is 334000 J/kg." → 668000 J
- "A block of lead melts completely when it absorbs 100000 J. The heat of fusion of lead is 25000 J/kg. What is its mass?" → 4 kg

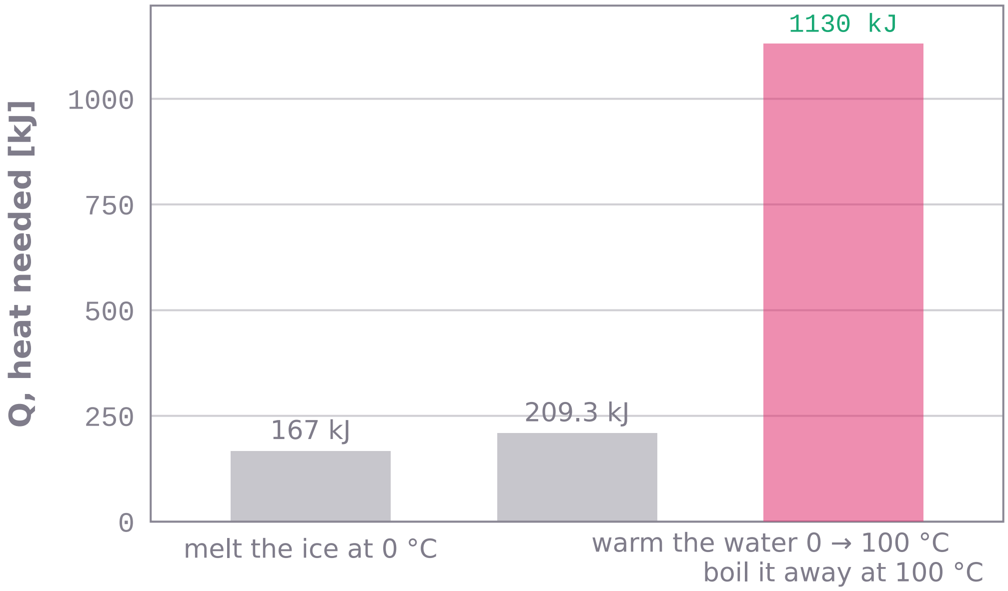
Heat to boil or condense, `latent_heat_vaporization`

$$Q = mL_v$$

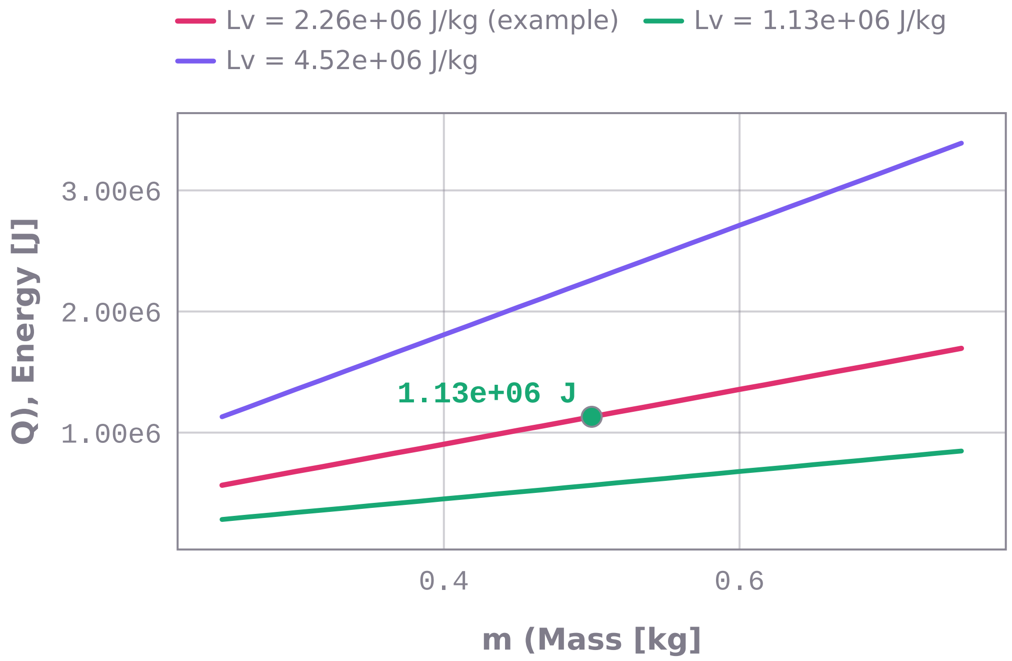
input	meaning	kind / role	unit	default
`Q`		energy / heat	`J`	
`m`		mass / plain	`kg`	

input	meaning	kind / role	unit	default
`Lv`	latent heat of vaporization	latent_heat / vaporization	`J/kg`	

- **Conditions:** boiling
- **Not when:** melting
- **Applies when:** boiling or condensing at the boiling point
- **Fails when:** the temperature also changes
- **Assumes:** at the boiling point; no temperature change during the phase change
- **Evidence:** fundamental_law



Heat needed by 0.5 kg of water (the worked example's mass) to melt (L_f = 334000 J/kg), to warm from 0 to 100 °C (c = 4186 J/(kg·K)) and to boil away (L_v = 2.26e+06 J/kg), each from the package: boiling takes more than five times the heat of warming it all the way. Marked: vaporisation 1.13e+06 J.



Q (Energy) against m (Mass) from 0.25 kg to 0.75 kg. One curve for each value of Lv (latent heat of vaporization): 2.26e+06 J/kg, 1.13e+06 J/kg, 4.52e+06 J/kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 0.5 kg, Q = 1.13e+06 J.

```
classical_physics_ai999x.solve('latent_heat_vaporization', m='0.5 kg', Lv='2.26e+06 J/kg',
target='Q')
# -> 1.13e+06 J   (SOLVED)
```

$$Q = mL_v$$
$$Q = L_v m = 1.13e + 06\text{ J}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	latent_heat_vaporization needs m (mass, kg). Example: solve('latent_heat_vaporization', m='... kg', Lv='... J/kg').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "How much heat is needed to boil 0.5 kg of water at 100 C if the latent heat of vaporization is 2.26e6 J/kg?" → 1.13e+06 J
- "Steam condenses and releases 4.52e6 J. The heat of vaporization is 2.26e6 J/kg. What mass of steam condenses?" → 2 kg

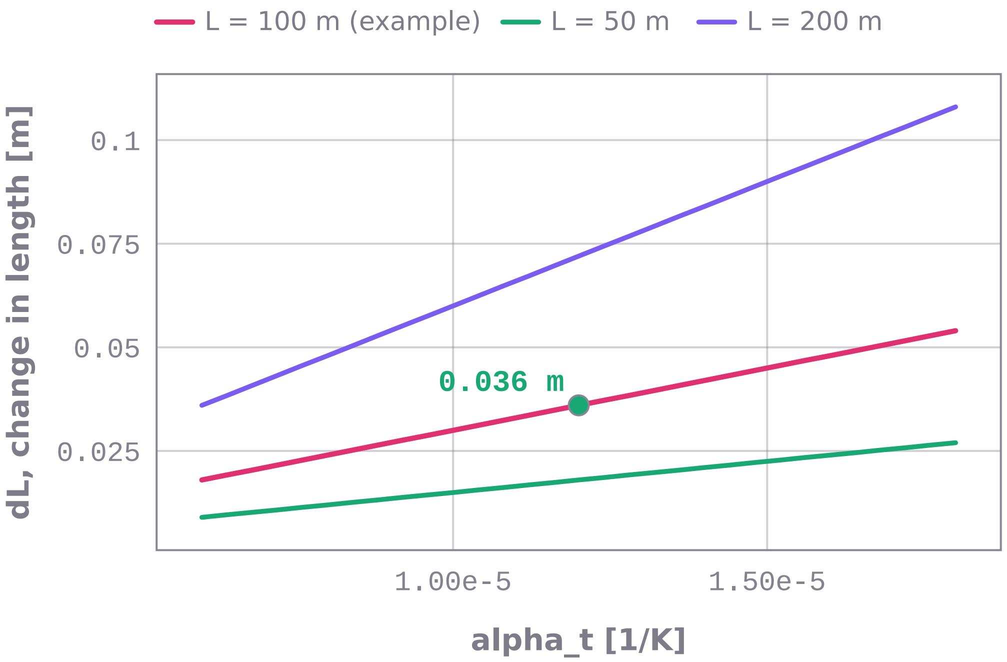
Linear thermal expansion, `linear_expansion`

$$d_L = \alpha_t L d_T$$

input	meaning	kind / role	unit	default
`dL`	change in length	length / expansion	`m`	
`alpha_t`	coefficient of linear expansion	expansion_coefficient / plain	`1/K`	

input	meaning	kind / role	unit	default
`L`	original length	length / original	`m`	
`dT`	change in temperature	temperature_difference / plain	`K`	

- **Conditions:** thermal_expansion
- **Applies when:** solids over moderate temperature ranges
- **Fails when:** phase changes; very large temperature ranges
- **Assumes:** a small temperature change; the coefficient constant over the range
- **Evidence:** empirical



dL (change in length) against alpha_t (coefficient of linear expansion) from 6.0000e-06 1/K to 1.8000e-05 1/K. One curve for each value of L (original length): 100 m, 50 m, 200 m. Other inputs as in the worked example: dT = 30.0 K. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, alpha_t = 1.2000e-05 1/K, dL = 0.036 m.

```
classical_physics_ai999x.solve('linear_expansion', alpha_t='1.2e-05 1/K', L='100 m', dT='30
K', target='dL')
# -> 0.036 m   (SOLVED)
```

$$d_L = \alpha_t L d_T$$
$$d_L = L\alpha_t d_T = 0.036\ \mathrm{m}$$

invalid call	outcome	message
without `alpha_t`	NEEDS_INPUT	linear_expansion needs alpha_t (coefficient of linear expansion, 1/K). Example: solve('linear_expansion', alpha_t='... 1/K', L='... m', dT='
`alpha_t` in a wrong unit	INVALID_INPUT	alpha_t is a expansion coefficient: its unit must be like 1/K, not 'kg'.

Questions in words that work:

- "A steel rail with an original length of 100 m expands as it is heated by 30 K. The coefficient of linear expansion is 12e-6 1/K. What is the change in length?" → 0.036 m

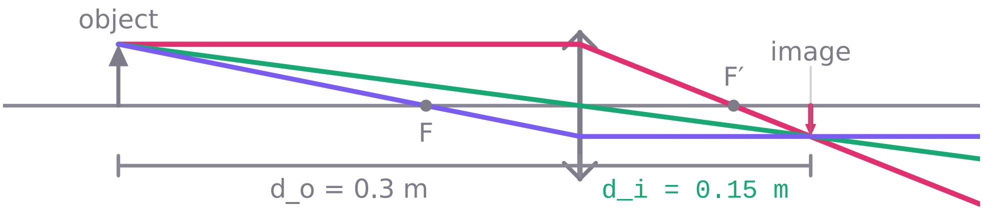
- "An aluminium rod with an initial length of 2 m expands by 0.002 m. Its coefficient of linear expansion is 2.4e-5 1/K. What temperature change caused the expansion?" → 41.6667 K

Thin lens equation, `thin_lens`

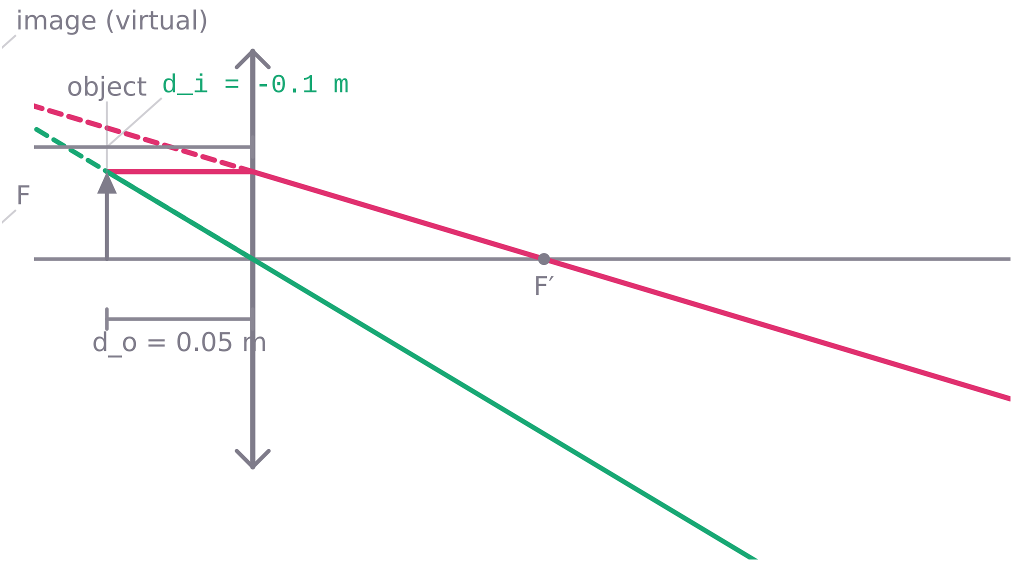
$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

input	meaning	kind / role	unit	default
`d_o`	object distance	length / object_distance	`m`	
`d_i`	image distance	length / image_distance	`m`	
`f`	focal length	length / focal	`m`	

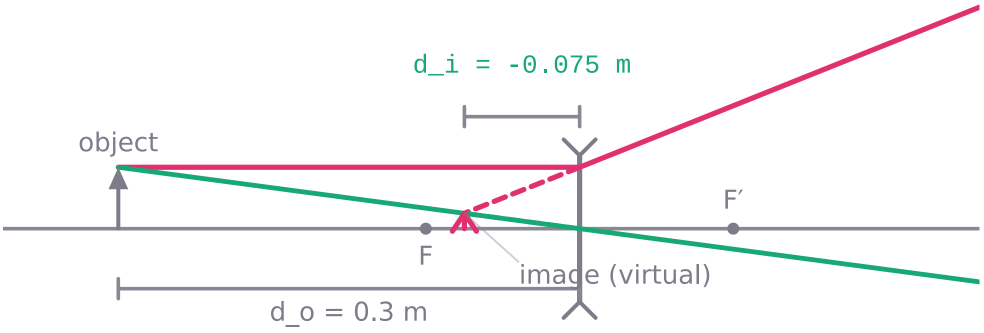
- **Conditions:** lens
- **Applies when:** a thin lens
- **Fails when:** thick lenses or wide-angle rays
- **Assumes:** a thin lens; paraxial rays
- **Evidence:** fundamental_law



Ray diagram of a thin converging lens of focal length f = 0.1 m with an object 0.3 m in front of it (drawn 4 cm tall): the ray parallel to the axis is bent through the focal point F′, the ray through the centre goes straight, the ray through F leaves parallel. Beyond the focal point the rays meet behind the lens. Marked: the image distance the package returns, d_i = 0.15 m (real, behind the lens); magnification −d_i/d_o = -0.5 and image height -2 cm (derived).



Ray diagram of a thin converging lens of focal length f = 0.1 m with an object 0.05 m in front of it (drawn 3 cm tall): the ray parallel to the axis is bent through the focal point F′, the ray through the centre goes straight. Inside the focal length the rays diverge; they appear to come from an enlarged upright image. Marked: the image distance the package returns, d_i = -0.1 m (negative: virtual, on the object side); magnification −d_i/d_o = 2 and image height 6 cm (derived).



Ray diagram of a thin diverging lens of focal length f = -0.1 m with an object 0.3 m in front of it (drawn 4 cm tall): the ray parallel to the axis is bent away from the focal point F′, the ray through the centre goes straight. A diverging lens always gives a smaller upright virtual image. Marked: the image distance the package returns, d_i = -0.075 m (negative: virtual, on the object side); magnification −d_i/d_o = 0.25 and image height 1 cm (derived).

```
classical_physics_ai999x.solve('thin_lens', d_o='0.3 m', f='0.1 m', target='d_i')
# -> 0.15 m   (SOLVED)
```

$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

$$d_i = \frac{d_o f}{d_o - f} = 0.15\ \mathrm{m}$$

invalid call	outcome	message
without `d_o`	NEEDS_INPUT	thin_lens needs d_o (object distance, m). Example: solve('thin_lens', d_o='... m', f='... m').
`d_o` in a wrong unit	INVALID_INPUT	d_o is a length: its unit must be like m, not 'kg'.

Questions in words that work:

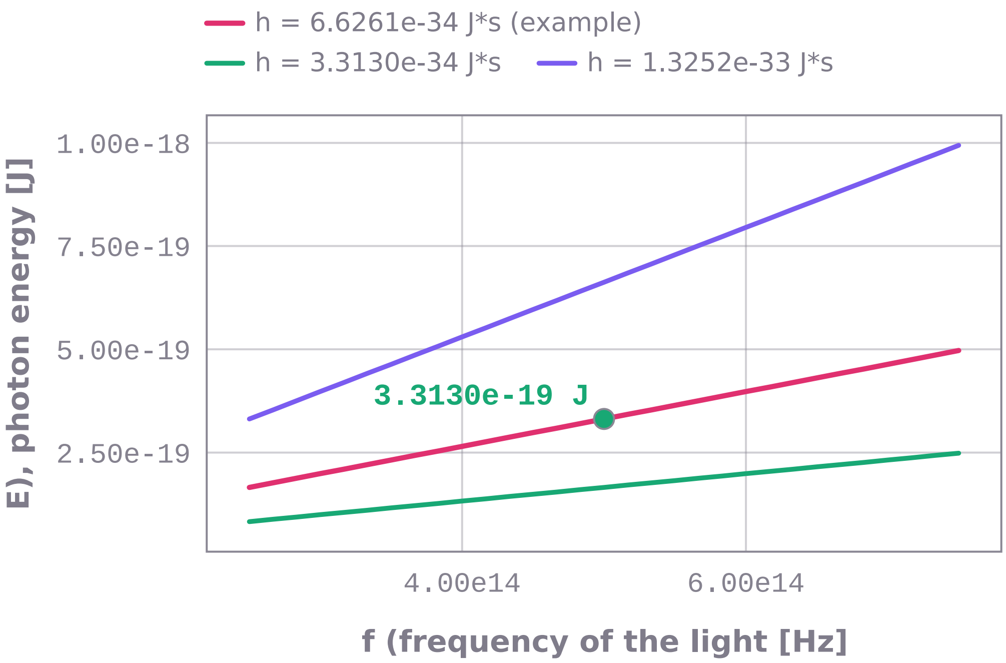
- "An object is placed 30 cm from a converging lens with a focal length of 10 cm. What is the image distance?" → 0.15 m
- "A lens forms an image at an image distance of 50 cm for an object distance of 50 cm. What is the focal length?" → 0.25 m

Energy of a photon, `photon_energy`

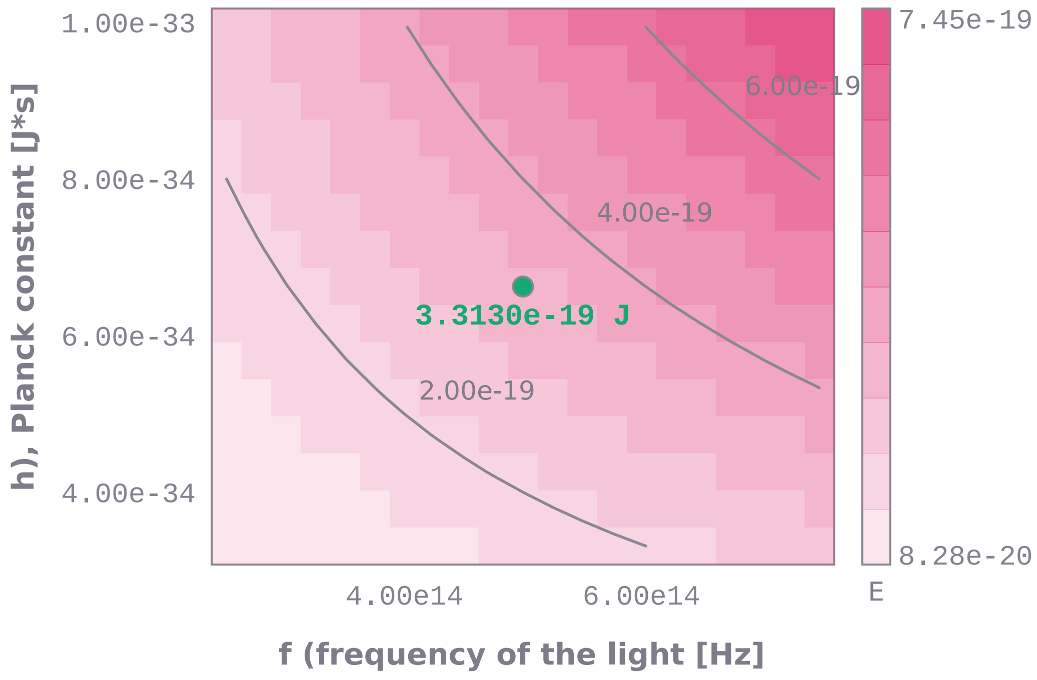
$$E = hf$$

input	meaning	kind / role	unit	default
`E`	photon energy	energy / photon	`J`	
`f`	frequency of the light	frequency / plain	`Hz`	
`h`		planck_constant / constant	`J*s`	6.62607015e-34 J*s

- **Conditions:** photon
- **Applies when:** light or other electromagnetic radiation
- **Fails when:** a classical wave energy is asked for
- **Assumes:** a single photon
- **Evidence:** fundamental_law



E (photon energy) against f (frequency of the light) from 2.5000e+14 Hz to 7.5000e+14 Hz. One curve for each value of h (Planck constant): 6.6261e-34 J*s, 3.3130e-34 J*s, 1.3252e-33 J*s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, f = 5.0000e+14 Hz, E = 3.3130e-19 J.



E (photon energy) over f (frequency of the light) from 2.5000e+14 Hz to 7.5000e+14 Hz and h (Planck constant) from 3.3130e-34 J*s to 9.9391e-34 J*s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 8.2826e-20 J to dark 7.4543e-19 J); iso-lines at E = 2.00e-19, 4.00e-19, 6.00e-19 J. Marked: the worked example, f = 5.0000e+14 Hz, h = 6.6261e-34 J*s, E = 3.3130e-19 J.

```
classical_physics_ai999x.solve('photon_energy', f='5e+14 Hz', target='E')
# -> 3.31304e-19 J   (SOLVED)
```

$$E = hf$$
$$E = fh = 3.31304e - 19 \text{ J}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	photon_energy needs f (frequency of the light, Hz). Example: solve('photon_energy', f='... Hz', h='... J*s').
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Questions in words that work:

- "What is the energy of a photon with a frequency of 5e14 Hz?" → 3.31304e-19 J
- "A photon has an energy of 3.3e-19 J. What is its frequency?" → 4.98033e+14 Hz

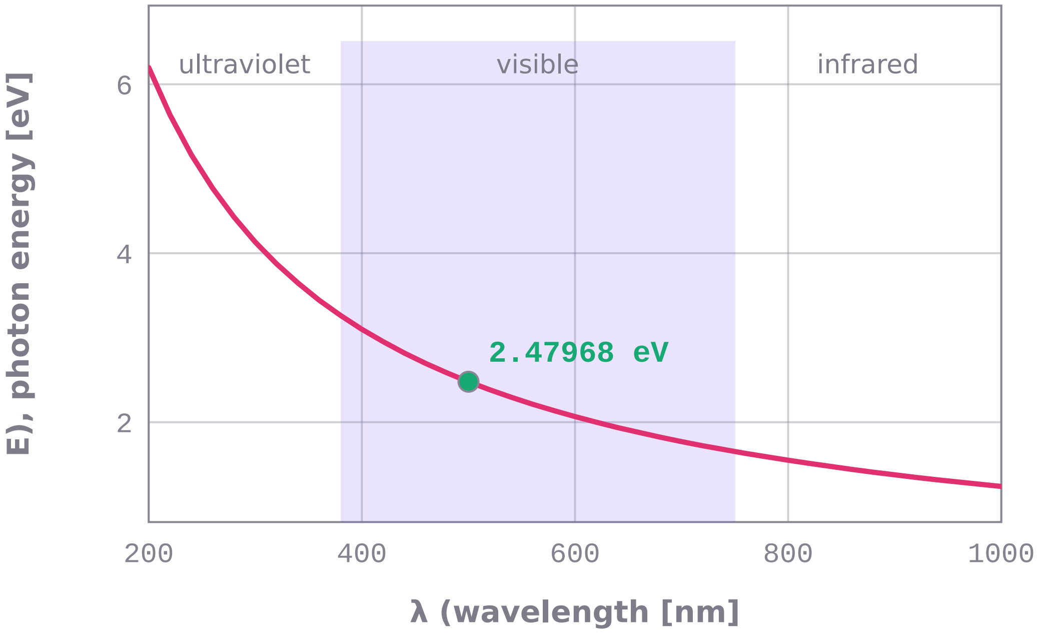
Photon energy from wavelength, `photon_energy_wavelength`

$$E = \frac{hc}{\lambda}$$

input	meaning	kind / role	unit	default
`E`		energy / photon	`J`	
`lam`	wavelength in vacuum	length / wavelength	`m`	

input	meaning	kind / role	unit	default
`h`		planck_constant / constant	`J*s`	6.62607015e-34 J*s
`c`		speed / light	`m/s`	299792458.0 m/s

- **Conditions:** photon
- **Applies when:** light in vacuum or air
- **Fails when:** the wavelength is measured in a medium
- **Assumes:** a single photon; wavelength in vacuum
- **Evidence:** fundamental_law



Energy of one photon against its wavelength from 200 to 1000 nm, 41 wavelengths from the package (E = h c / λ, converted to electronvolts with 1 eV = 1.6022e-19 J): shorter wavelength, more energy per photon. The visible band 380–750 nm is shaded. Marked: E = 3.9729e-19 J = 2.47968 eV at 500 nm.

```
classical_physics_ai999x.solve('photon_energy_wavelength', lam='5e-07 m', target='E')
# -> 3.97289e-19 J   (SOLVED)
```

$$E = \frac{hc}{\lambda}$$

$$E = \frac{ch}{\lambda} = 3.97289e - 19 \text{ J}$$

invalid call	outcome	message
without `lam`	NEEDS_INPUT	photon_energy_wavelength needs lam (wavelength in vacuum, m). Example: solve('photon_energy_wavelength', lam='... m', h='... J*s', c='... m/
`lam` in a wrong unit	INVALID_INPUT	lam is a length: its unit must be like m, not 'kg'.

Questions in words that work:

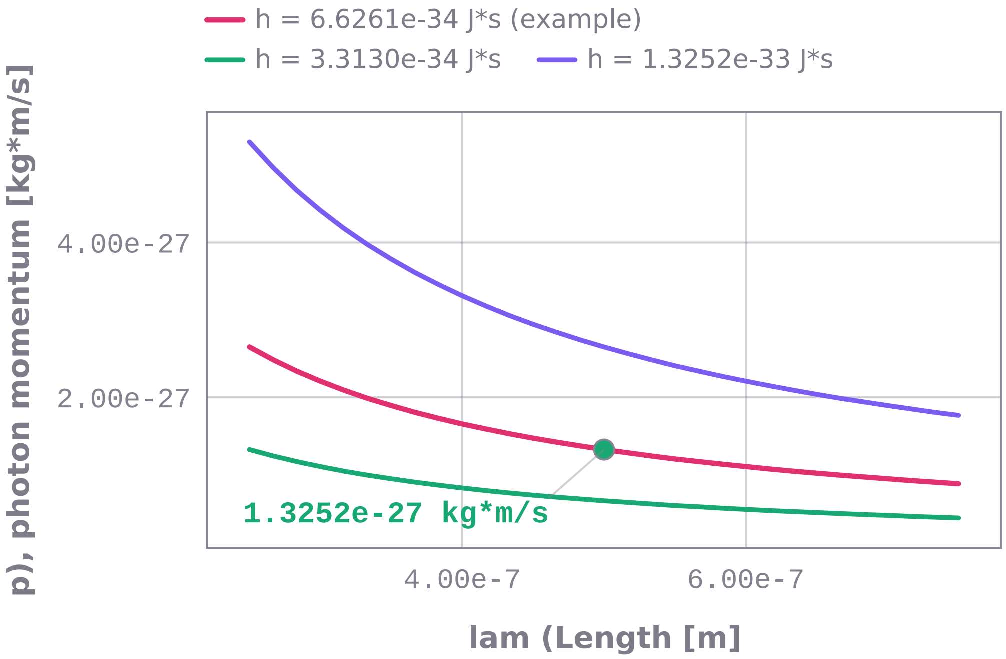
- "What is the energy of a photon of wavelength 500 nm?" → 3.97289e-19 J
- "A photon has an energy of 2e-19 J. What is its wavelength?" → 9.93223e-07 m

Momentum of a photon, `photon_momentum`

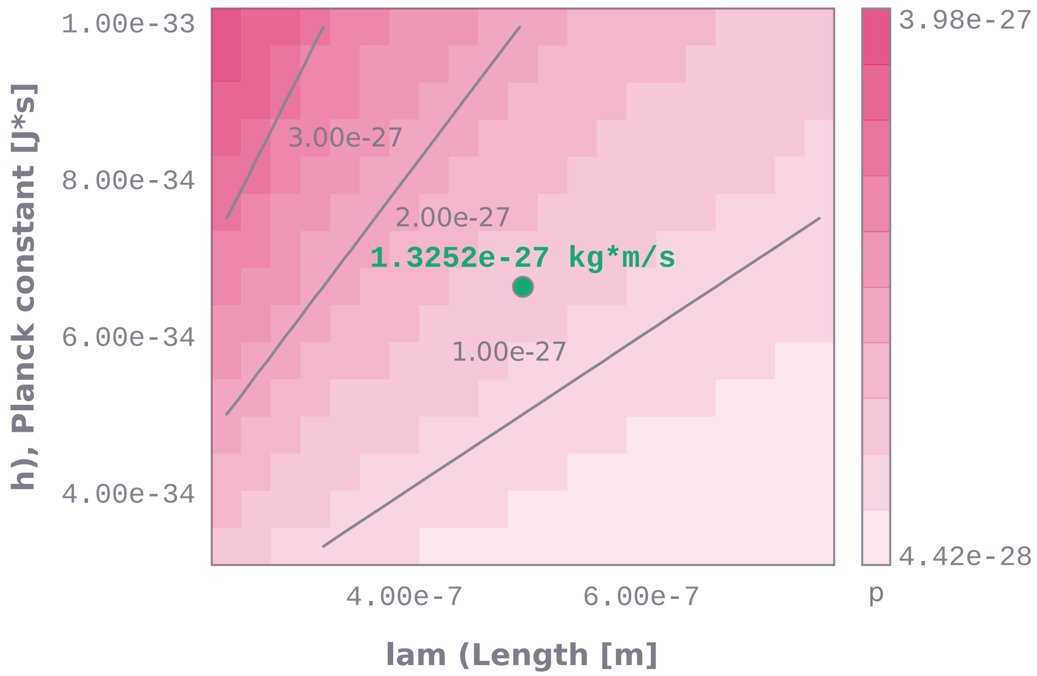
$$p = \frac{h}{\lambda}$$

input	meaning	kind / role	unit	default
`p`	photon momentum	momentum / plain	`kg*m/s`	
`lam`		length / wavelength	`m`	
`h`		planck_constant / constant	`J*s`	6.62607015e-34 J*s

- **Conditions:** photon
- **Applies when:** light
- **Fails when:** massive particles (use the de Broglie relation)
- **Assumes:** a single photon
- **Evidence:** fundamental_law



p (photon momentum) against lam (Length) from 2.5000e-07 m to 7.5000e-07 m. One curve for each value of h (Planck constant): 6.6261e-34 J*s, 3.3130e-34 J*s, 1.3252e-33 J*s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, lam = 5.0000e-07 m, p = 1.3252e-27 kg*m/s.



p (photon momentum) over lam (Length) from 2.5000e-07 m to 7.5000e-07 m and h (Planck constant) from 3.3130e-34 J*s to 9.9391e-34 J*s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 4.4174e-28 kg*m/s to dark 3.9756e-27 kg*m/s); iso-lines at p = 1.00e-27, 2.00e-27, 3.00e-27 kg*m/s. Marked: the worked example, lam = 5.0000e-07 m, h = 6.6261e-34 J*s, p = 1.3252e-27 kg*m/s.

```
classical_physics_ai999x.solve('photon_momentum', lam='5e-07 m', target='p')
# -> 1.32521e-27 kg*m/s   (SOLVED)
```

$$p = \frac{h}{\lambda}$$

$$p = \frac{h}{\lambda} = 1.32521e - 27\,\mathrm{kg\,m/s}$$

invalid call	outcome	message
without `lam`	NEEDS_INPUT	photon_momentum needs lam (length, m). Example: solve('photon_momentum', lam='... m', h='... J*s').
`lam` in a wrong unit	INVALID_INPUT	lam is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "What is the momentum of a photon with a wavelength of 500 nm?" → 1.32521e-27 kg*m/s
- "A photon has a momentum of 1e-27 kg*m/s. What is its wavelength?" → 6.62607e-07 m

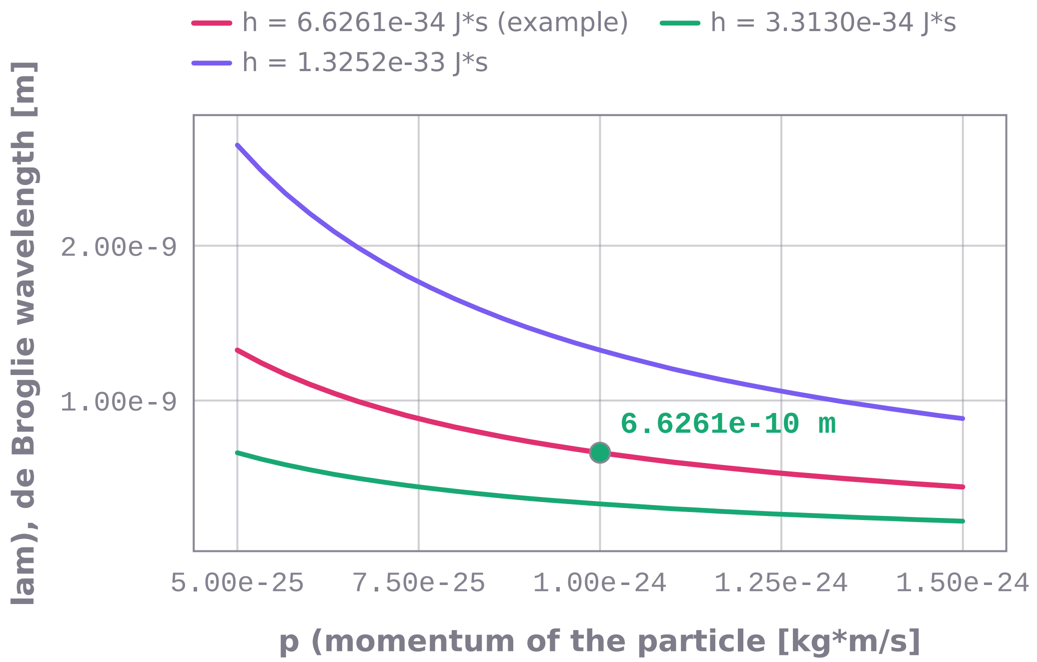
De Broglie wavelength, `de_broglie`

$$\lambda = \frac{h}{p}$$

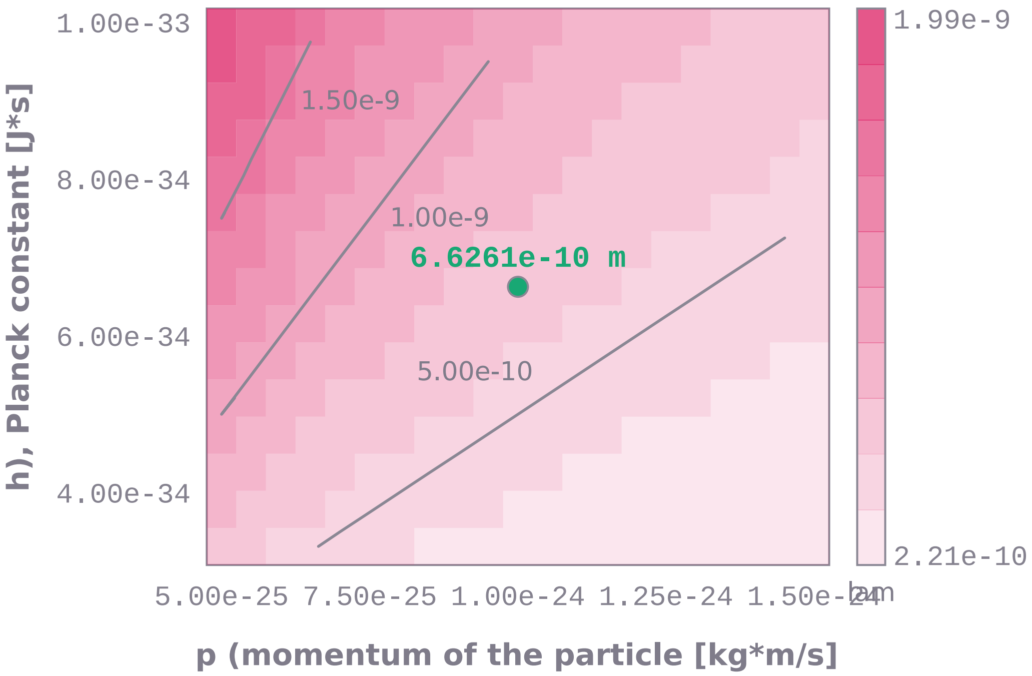
input	meaning	kind / role	unit	default
`lam`	de Broglie wavelength	length / wavelength	`m`	

input	meaning	kind / role	unit	default
p	momentum of the particle	momentum / plain	`kg*m/s`	
h		planck_constant / constant	`J*s`	6.62607015e-34 J*s

- **Conditions:** matter_wave
- **Not when:** photon
- **Applies when:** electrons, protons, neutrons and other particles
- **Fails when:** photons in a medium
- **Assumes:** a free particle
- **Evidence:** fundamental_law



lam (de Broglie wavelength) against p (momentum of the particle) from 5.0000e-25 kg*m/s to 1.5000e-24 kg*m/s. One curve for each value of h (Planck constant): 6.6261e-34 J*s, 3.3130e-34 J*s, 1.3252e-33 J*s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, p = 1.0000e-24 kg*m/s, lam = 6.6261e-10 m.



lam (de Broglie wavelength) over p (momentum of the particle) from 5.0000e-25 kg*m/s to 1.5000e-24 kg*m/s and h (Planck constant) from 3.3130e-34 J*s to 9.9391e-34 J*s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 2.2087e-10 m to dark 1.9878e-09 m); iso-lines at lam = 5.00e-10, 1.00e-9, 1.50e-9 m. Marked: the worked example, p = 1.0000e-24 kg*m/s, h = 6.6261e-34 J*s, lam = 6.6261e-10 m.

```
classical_physics_ai999x.solve('de_broglie', p='1e-24 kg*m/s', target='lam')
# -> 6.62607e-10 m   (SOLVED)
```

$$\lambda = \frac{h}{p}$$

$$\lambda = \frac{h}{p} = 6.62607e - 10\ \mathrm{m}$$

invalid call	outcome	message
without `p`	NEEDS_INPUT	de_broglie needs p (momentum of the particle, kg*m/s). Example: solve('de_broglie', p='... kg*m/s', h='... J*s').
`p` in a wrong unit	INVALID_INPUT	p is a momentum: its unit must be like kg*m/s, not 's'.

Questions in words that work:

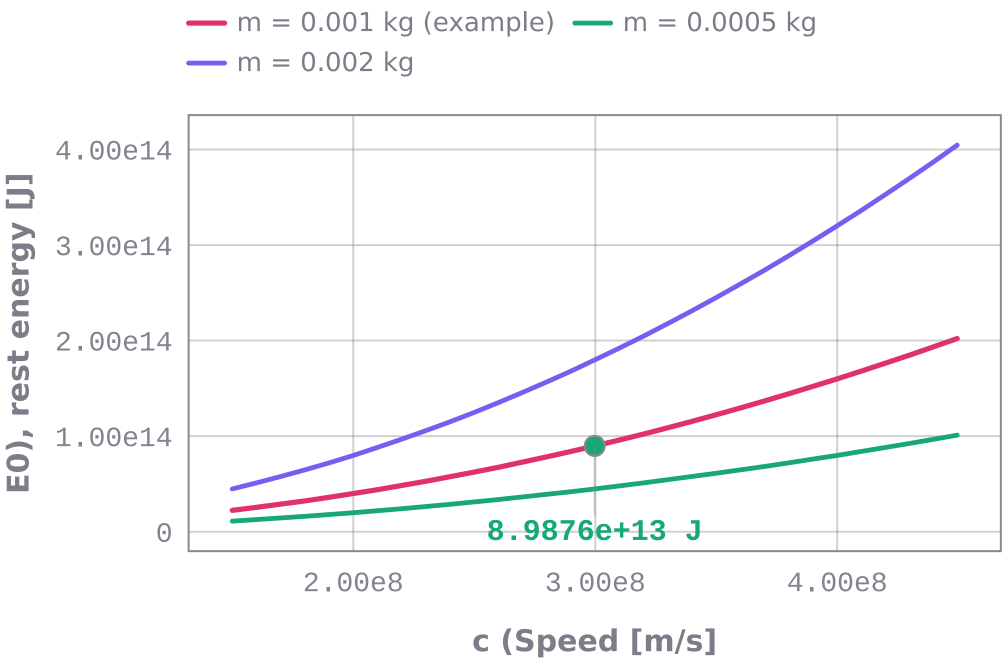
- "An electron has a momentum of 1e-24 kg*m/s. What is its de Broglie wavelength?" → 6.62607e-10 m
- "An electron with mass 9.11e-31 kg moves at 2e6 m/s. What is its de Broglie wavelength?" → 3.6367e-10 m

Rest energy, `rest_energy`

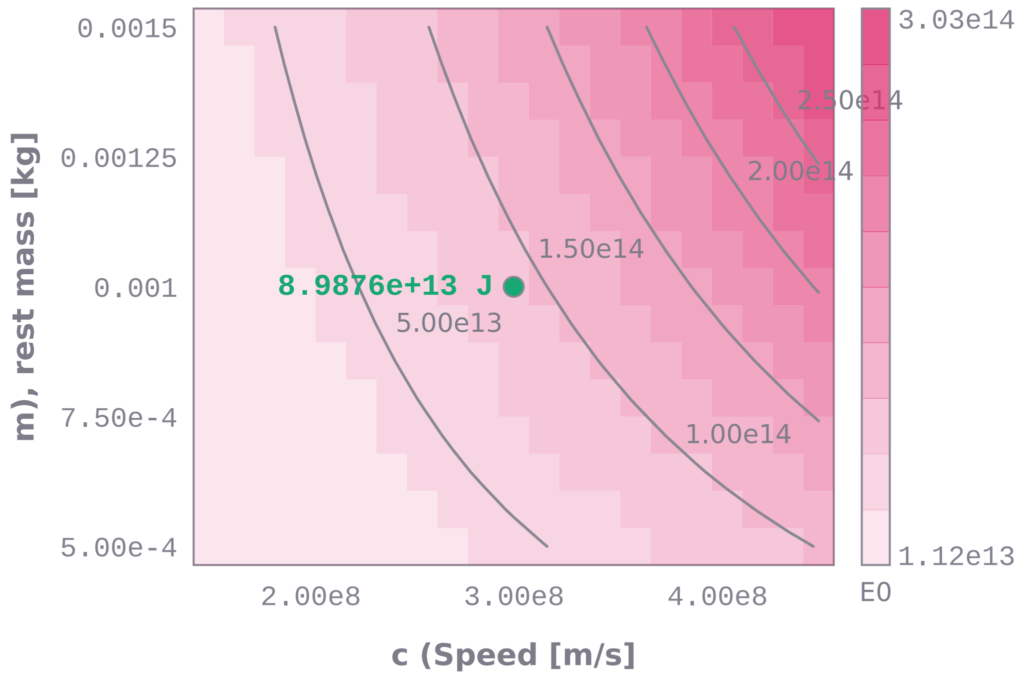
$$E_0 = mc^2$$

input	meaning	kind / role	unit	default
`E0`	rest energy	energy / rest	`J`	
`m`	rest mass	mass / plain	`kg`	
`c`		speed / light	`m/s`	299792458.0 m/s

- **Conditions:** mass_energy
- **Applies when:** mass converted entirely into energy, or the rest energy of a body
- **Fails when:** the kinetic energy is also wanted
- **Assumes:** energy of the mass at rest
- **Evidence:** fundamental_law



E0 (rest energy) against c (Speed) from 1.4990e+08 m/s to 4.4969e+08 m/s. One curve for each value of m (rest mass): 0.001 kg, 0.0005 kg, 0.002 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, c = 2.9979e+08 m/s, E0 = 8.9876e+13 J.



E0 (rest energy) over c (Speed) from 1.4990e+08 m/s to 4.4969e+08 m/s and m (rest mass) from 0.0005 kg to 0.0015 kg: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1.1234e+13 J to dark 3.0333e+14 J); iso-lines at E0 = 5.00e13, 1.00e14, 1.50e14, 2.00e14, 2.50e14 J. Marked: the worked example, c = 2.9979e+08 m/s, m = 0.001 kg, E0 = 8.9876e+13 J.

```
classical_physics_ai999x.solve('rest_energy', m='0.001 kg', target='E0')
# -> 8.98755e+13 J   (SOLVED)
```

$$E_0 = mc^2$$
$$E_0 = c^2 m = 8.98755e + 13 \text{ J}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	rest_energy needs m (rest mass, kg). Example: solve('rest_energy', m='... kg', c='... m/s').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

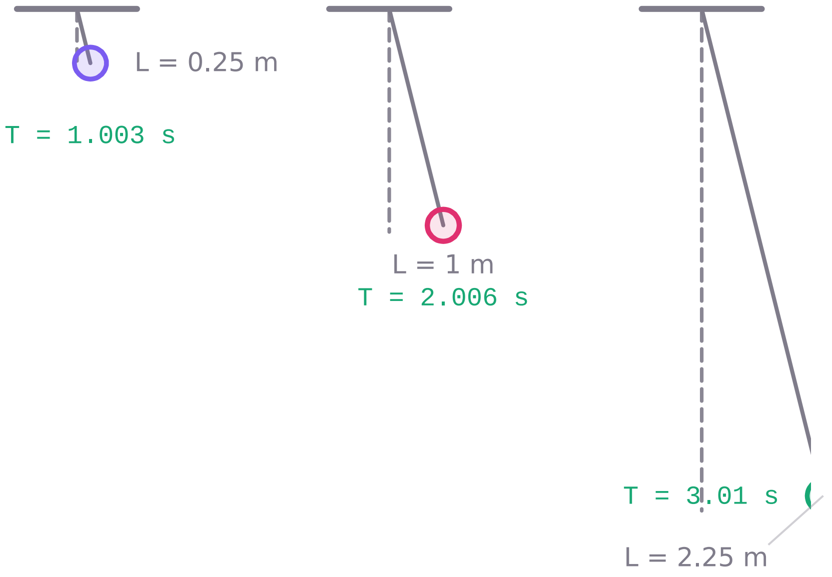
- "What is the rest energy of a 1 g mass?" → 8.98755e+13 J
- "A reaction releases 9e13 J when mass is converted entirely into energy. How much mass was converted?" → 0.00100139 kg

Period of a simple pendulum, `pendulum_period`

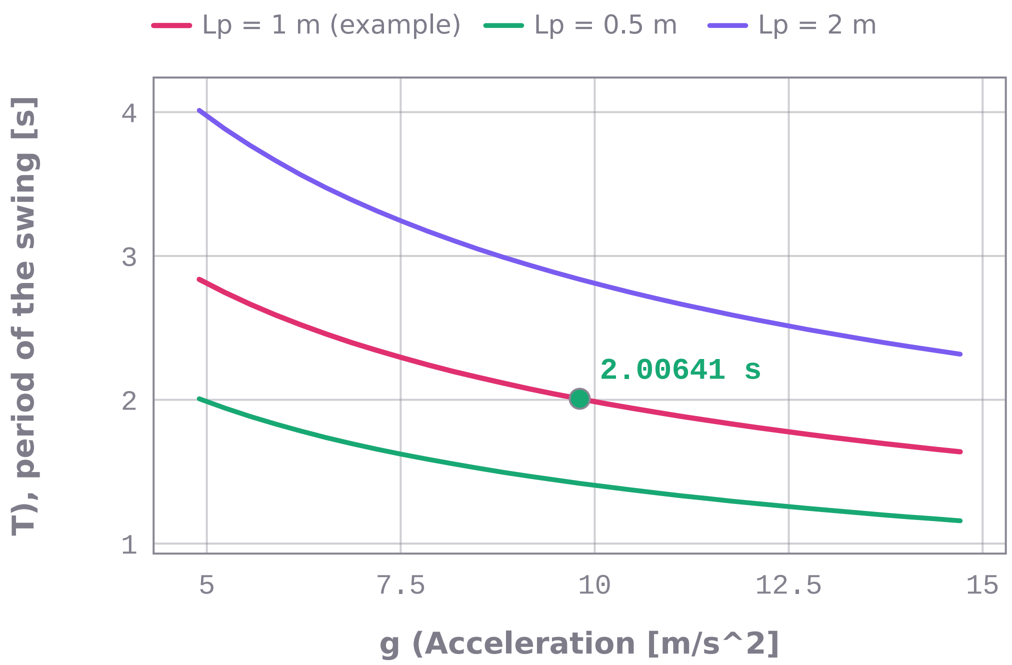
$$T = 2\pi\sqrt{\frac{L_p}{g}}$$

input	meaning	kind / role	unit	default
`T`	period of the swing	time / period	`s`	
`Lp`	length of the pendulum	length / pendulum	`m`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Conditions:** pendulum
- **Applies when:** small-angle swings
- **Fails when:** large amplitudes; a heavy rod
- **Assumes:** small swings; a point mass on a light string
- **Evidence:** fundamental_law



Simple pendulums of length 0.25, 1 and 2.25 m (g = 9.80665 m/s², small swings; the 1 m pendulum is the worked example), drawn to scale: the period grows with the square root of the length, so 9 times longer swings 3 times slower. Marked: the periods the package returns, 1.0032 s, 2.00641 s, 3.00961 s.



T (period of the swing) against g (Acceleration) from 4.90332 m/s^2 to 14.71 m/s^2. One curve for each value of Lp (length of the pendulum): 1 m, 0.5 m, 2 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, g = 9.80665 m/s^2, T = 2.00641 s.

```
classical_physics_ai999x.solve('pendulum_period', Lp='1 m', g='9.80665 m/s^2', target='T')
# -> 2.00641 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{L_p}{g}}$$

$$T = \frac{2\pi\sqrt{L_p}}{\sqrt{g}} = 2.00641\ \mathrm{s}$$

invalid call	outcome	message
without `Lp`	NEEDS_INPUT	pendulum_period needs Lp (length of the pendulum, m). Example: solve('pendulum_period', Lp='... m', g='... m/s^2').
`Lp` in a wrong unit	INVALID_INPUT	Lp is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "What is the period of a pendulum with a pendulum length of 1 m?" → 2.00641 s
- "A pendulum has a period of 2 s. What is the length of the pendulum?" → 0.993621 m

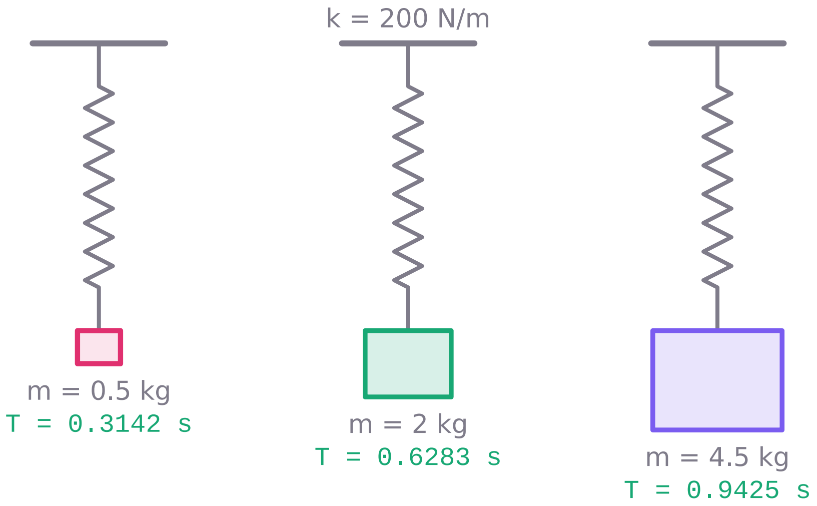
Period of a mass on a spring, `spring_oscillation_period`

$$T = 2\pi\sqrt{\frac{m}{k}}$$

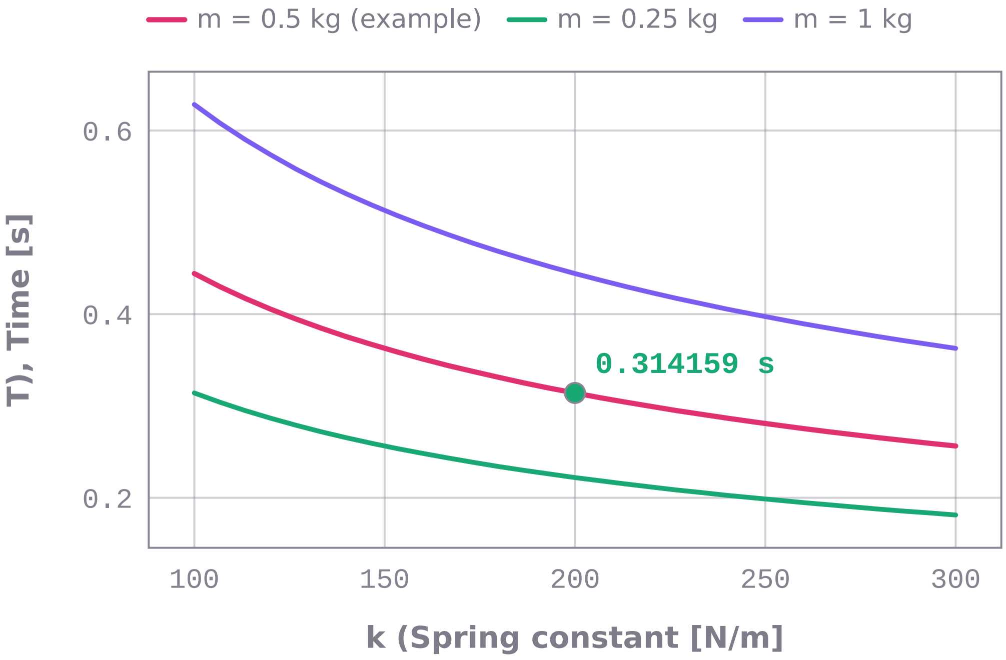
input	meaning	kind / role	unit	default
`T`		time / period	`s`	

input	meaning	kind / role	unit	default
`m`		mass / plain	`kg`	
`k`		spring_constant / plain	`N/m`	

- **Conditions:** spring_oscillation
- **Applies when:** a mass oscillating on an ideal spring
- **Fails when:** damping or a heavy spring
- **Assumes:** an ideal spring; small oscillations; no damping
- **Evidence:** fundamental_law



Masses of 0.5 kg (the worked example), 2 kg and 4.5 kg on identical springs of stiffness k = 200 N/m: the period of oscillation grows with the square root of the mass, independent of the amplitude. Marked: the periods the package returns, 0.314159 s, 0.628319 s, 0.942478 s.



T (Time) against k (Spring constant) from 100 N/m to 300 N/m. One curve for each value of m (Mass): 0.5 kg, 0.25 kg, 1 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, k = 200 N/m, T = 0.314159 s.

```
classical_physics_ai999x.solve('spring_oscillation_period', m='0.5 kg', k='200 N/m',
target='T')
# -> 0.314159 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{m}{k}}$$
$$T = \frac{2\pi\sqrt{m}}{\sqrt{k}} = 0.314159\ \mathrm{s}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	spring_oscillation_period needs m (mass, kg). Example: solve('spring_oscillation_period', m='... kg', k='... N/m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

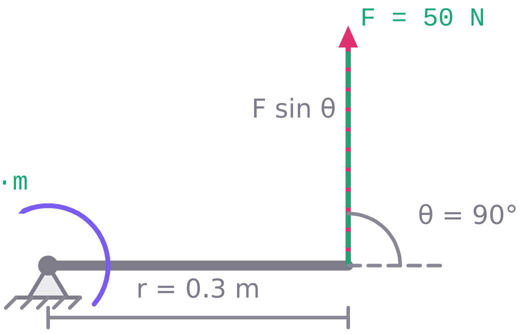
- "A 0.5 kg mass oscillates on a spring with a spring constant of 200 N/m. What is the period?" → 0.314159 s
- "A mass oscillates on a spring of stiffness 40 N/m with a period of 1 s. What is the mass?" → 1.01321 kg

Torque of a force, `torque_lever`

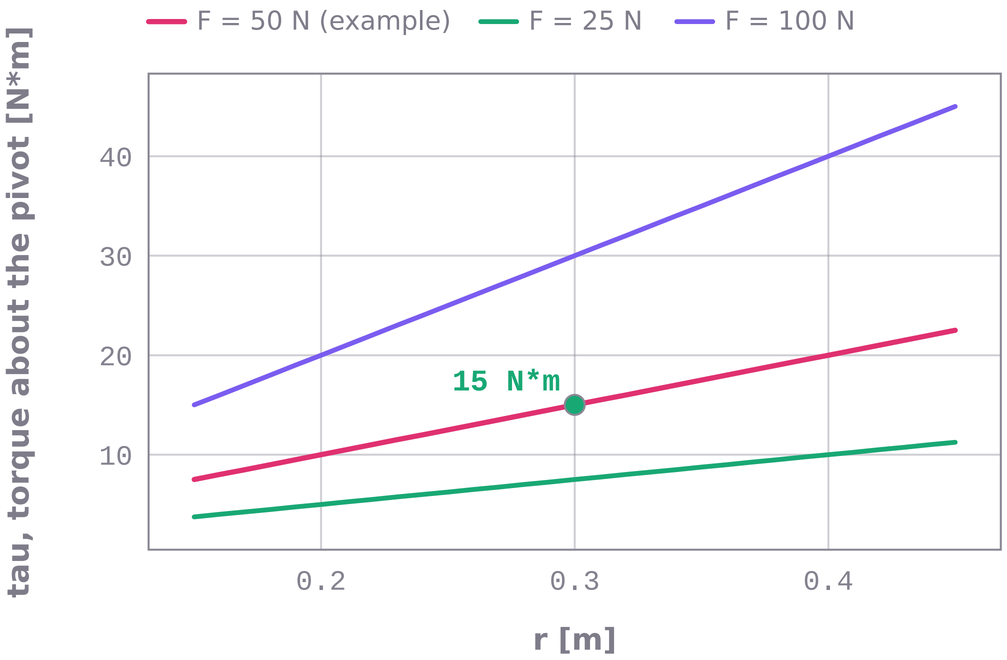
$$\tau = rF\sin\left(\theta\right)$$

input	meaning	kind / role	unit	default
`tau`	torque about the pivot	torque / plain	`N*m`	
`r`	distance from the pivot to the point of application	length / lever_arm	`m`	
`F`	applied force	force / applied	`N`	
`theta`	angle between r and F	angle / plain	`rad`	

- **Conditions:** pivot
- **Applies when:** a force applied at a distance from a pivot
- **Fails when:** the pivot moves
- **Assumes:** a rigid body; a fixed pivot
- **Evidence:** definition



A force F = 50 N applied at r = 0.3 m from a pivot at θ = 90° to the lever: only its component perpendicular to the lever, F sin θ, turns it. Marked: the torque the package returns, τ = 15 N·m (at 90° it would be 15 N·m, also returned).



tau (torque about the pivot) against r (distance from the pivot to the point of application) from 0.15 m to 0.45 m. One curve for each value of F (applied force): 50 N, 25 N, 100 N. Other inputs as in the worked example: theta = 1.5707963267948966 rad. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.3 m, tau = 15 N*m.

```
classical_physics_ai999x.solve('torque_lever', r='0.3 m', F='50 N', theta='90 deg',
target='tau')
# -> 15 N*m   (SOLVED)
```

$$\tau = rF\sin\left(\theta\right)$$
$$\tau = Fr\sin\left(\theta\right) = 15\,\mathrm{N\,m}$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	torque_lever needs r (distance from the pivot to the point of application, m). Example: solve('torque_lever', r='... m', F='... N', theta='.

invalid call	outcome	message
`r` in a wrong unit	INVALID_INPUT	r is a length: its unit must be like m, not 'kg'.

Questions in words that work:

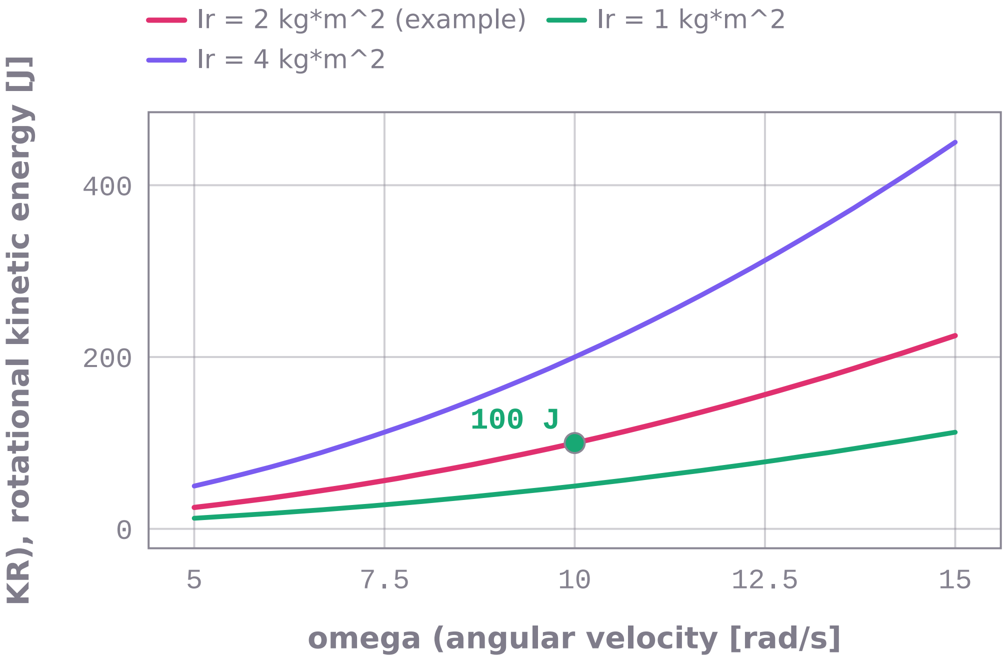
- "A wrench applies an applied force of 50 N at a lever arm of 0.3 m, at an angle of 90 degrees. What is the torque?" → 15 N*m
- "A force of 40 N pushes a door at a distance from the hinge of 0.2 m at an angle of 30 degrees. What is the torque?" → 4 N*m

Rotational kinetic energy, `rotational_kinetic_energy`

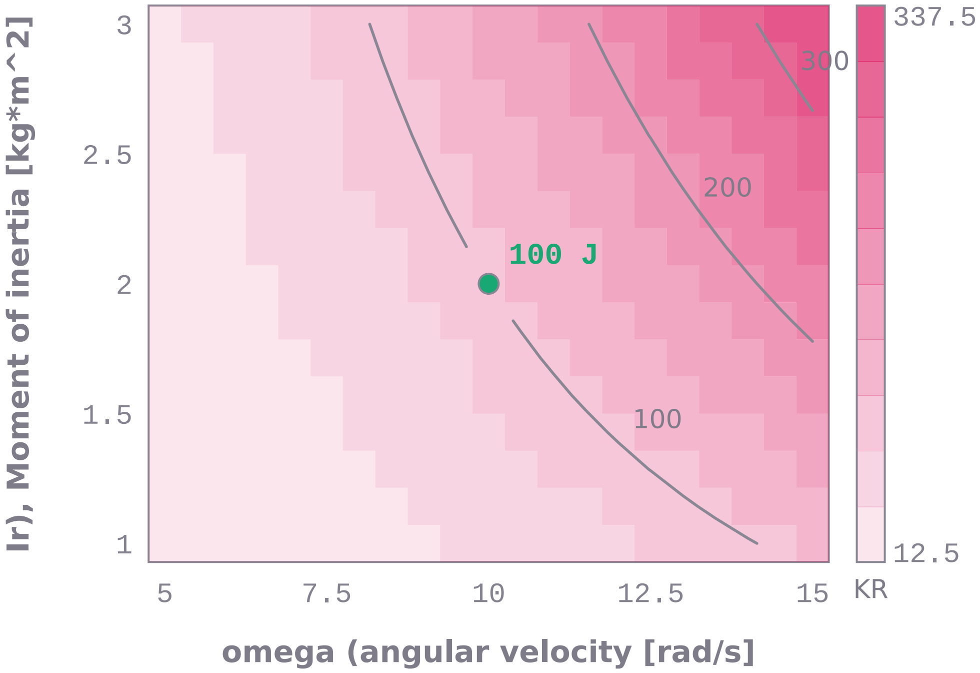
$$K_R = \frac{I_r \omega^2}{2}$$

input	meaning	kind / role	unit	default
`KR`	rotational kinetic energy	energy / rotational	`J`	
`Ir`		moment_of_inertia / plain	`kg*m^2`	
`omega`	angular velocity	angular_frequency / plain	`rad/s`	

- **Conditions:** rotating
- **Applies when:** a rotating rigid body
- **Fails when:** the axis moves
- **Assumes:** a rigid body rotating about a fixed axis
- **Evidence:** definition



KR (rotational kinetic energy) against omega (angular velocity) from 5 rad/s to 15 rad/s. One curve for each value of Ir (Moment of inertia): 2 kg*m^2, 1 kg*m^2, 4 kg*m^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, omega = 10 rad/s, KR = 100 J.



KR (rotational kinetic energy) over omega (angular velocity) from 5 rad/s to 15 rad/s and Ir (Moment of inertia) from 1 kg*m^2 to 3 kg*m^2: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 12.5 J to dark 337.5 J); iso-lines at KR = 100, 200, 300 J. Marked: the worked example, omega = 10 rad/s, Ir = 2 kg*m^2, KR = 100 J.

```
classical_physics_ai999x.solve('rotational_kinetic_energy', Ir='2 kg*m^2', omega='10
rad/s', target='KR')
# -> 100 J   (SOLVED)
```

$$K_R = \frac{I_r \omega^2}{2}$$

$$K_R = \frac{I_r \omega^2}{2} = 100\ \mathrm{J}$$

invalid call	outcome	message
without `Ir`	NEEDS_INPUT	rotational_kinetic_energy needs Ir (moment_of_inertia, kg*m^2). Example: solve('rotational_kinetic_energy', Ir='... kg*m^2', omega='... rad/
`Ir` in a wrong unit	INVALID_INPUT	Ir is a moment of inertia: its unit must be like kg*m^2, not 's'.

Questions in words that work:

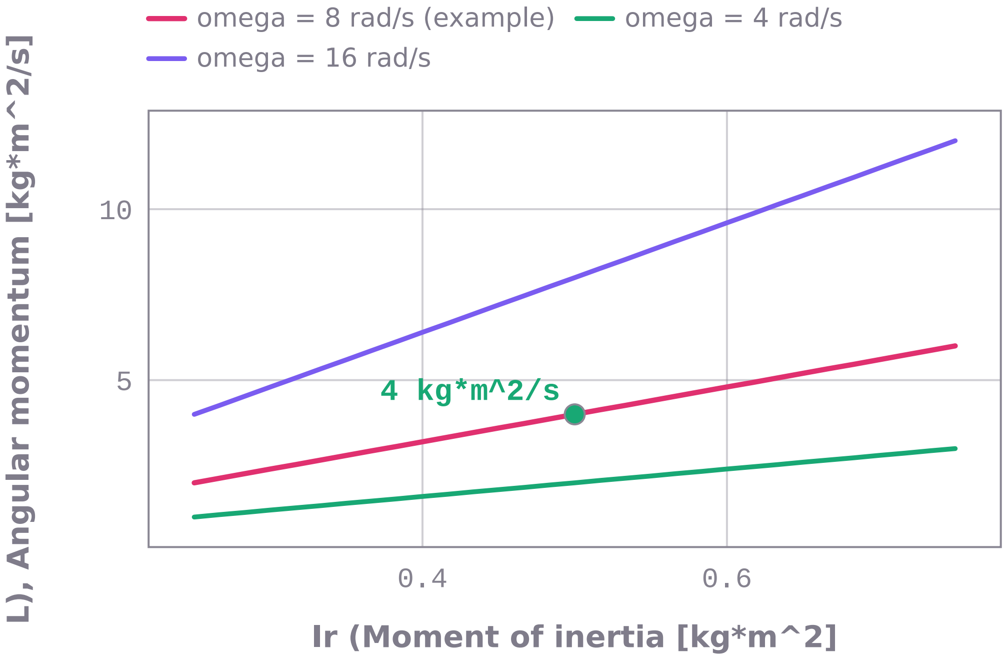
- "A flywheel with a moment of inertia of 2 kg*m^2 rotates at 10 rad/s. What is its rotational kinetic energy?" → 100 J
- "A spinning wheel rotating at 5 rad/s has a rotational kinetic energy of 50 J. What is its moment of inertia?" → 4 kg*m^2

Angular momentum of a rotating body, `angular_momentum`

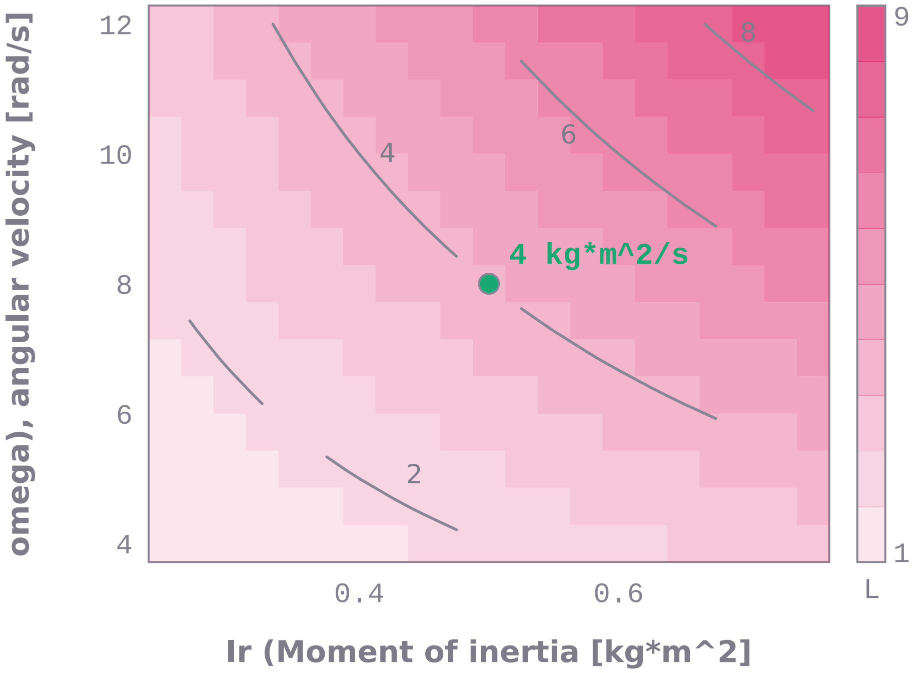
$$L = I_r \omega$$

input	meaning	kind / role	unit	default
`L`		angular_momentum / plain	`kg*m^2/s`	
`Ir`		moment_of_inertia / plain	`kg*m^2`	
`omega`	angular velocity	angular_frequency / plain	`rad/s`	

- **Conditions:** rotating
- **Applies when:** a rotating rigid body
- **Fails when:** the axis moves
- **Assumes:** a rigid body rotating about a fixed axis
- **Evidence:** definition



L (Angular momentum) against Ir (Moment of inertia) from 0.25 kg*m^2 to 0.75 kg*m^2. One curve for each value of omega (angular velocity): 8 rad/s, 4 rad/s, 16 rad/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Ir = 0.5 kg*m^2, L = 4 kg*m^2/s.



L (Angular momentum) over Ir (Moment of inertia) from 0.25 kg*m^2 to 0.75 kg*m^2 and omega (angular velocity) from 4 rad/s to 12 rad/s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1 kg*m^2/s to dark 9 kg*m^2/s); iso-lines at L = 2, 4, 6, 8 kg*m^2/s. Marked: the worked example, Ir = 0.5 kg*m^2, omega = 8 rad/s, L = 4 kg*m^2/s.

```
classical_physics_ai999x.solve('angular_momentum', Ir='0.5 kg*m^2', omega='8 rad/s',
target='L')
# -> 4 kg*m^2/s    (SOLVED)
```

$$L = I_r \omega$$
$$L = I_r \omega = 4\,\mathrm{kg\,m^2/s}$$

invalid call	outcome	message
without `Ir`	NEEDS_INPUT	angular_momentum needs Ir (moment_of_inertia, kg*m^2). Example: solve('angular_momentum', Ir='... kg*m^2', omega='... rad/s').
`Ir` in a wrong unit	INVALID_INPUT	Ir is a moment of inertia: its unit must be like kg*m^2, not 's'.

Questions in words that work:

- "A turntable with a moment of inertia of 0.5 kg*m^2 rotates at 8 rad/s. What is its angular momentum?" → 4 kg*m^2/s
- "A rotating wheel has an angular momentum of 12 kg*m^2/s and a moment of inertia of 3 kg*m^2. What is its angular velocity?" → 4 rad/s

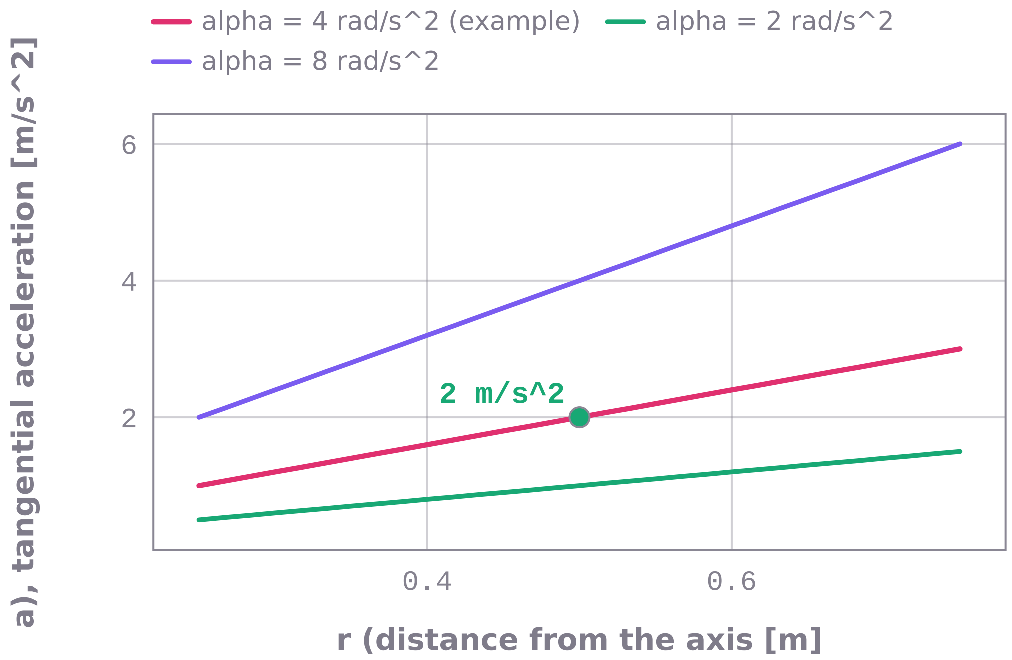
Tangential acceleration of a point on a rotating body, `tangential_acceleration`

$$a = r\alpha$$

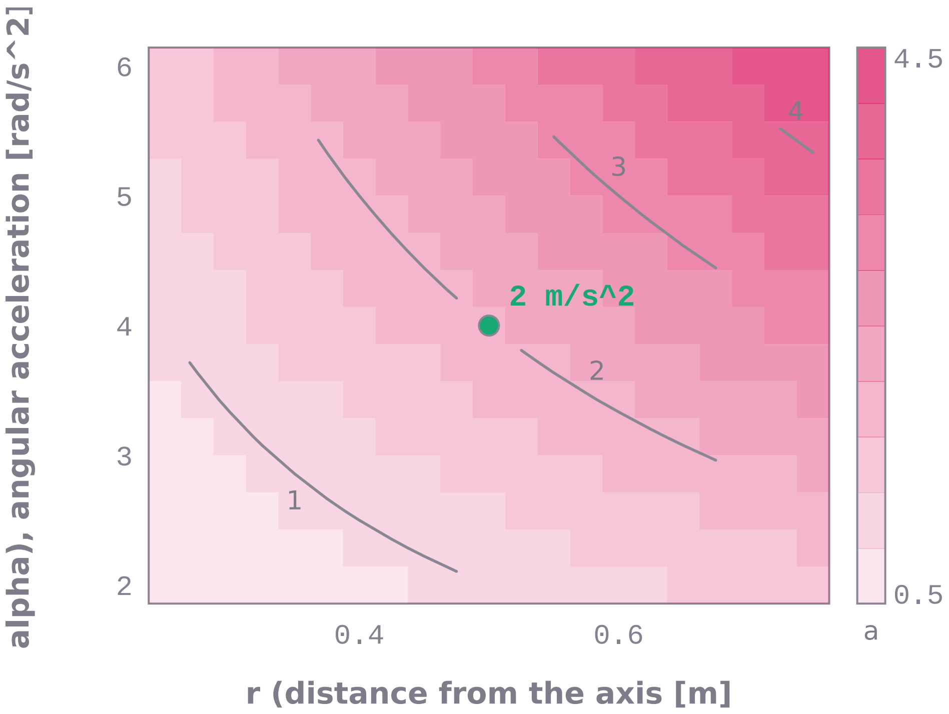
input	meaning	kind / role	unit	default
`a`	tangential acceleration	acceleration / tangential	`m/s^2`	

input	meaning	kind / role	unit	default
`r`	distance from the axis	length / radius	`m`	
`alpha`	angular acceleration	angular_acceleration / plain	`rad/s^2`	

- **Conditions:** rotating
- **Applies when:** a point on a rotating rigid body
- **Fails when:** the radius changes
- **Assumes:** a rigid body about a fixed axis
- **Evidence:** definition



a (tangential acceleration) against r (distance from the axis) from 0.25 m to 0.75 m. One curve for each value of alpha (angular acceleration): 4 rad/s^2, 2 rad/s^2, 8 rad/s^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.5 m, a = 2 m/s^2.



a (tangential acceleration) over r (distance from the axis) from 0.25 m to 0.75 m and alpha (angular acceleration) from 2 rad/s^2 to 6 rad/s^2: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0.5 m/s^2 to dark 4.5 m/s^2); iso-lines at a = 1, 2, 3, 4 m/s^2. Marked: the worked example, r = 0.5 m, alpha = 4 rad/s^2, a = 2 m/s^2.

```
classical_physics_ai999x.solve('tangential_acceleration', r='0.5 m', alpha='4 rad/s^2',
target='a')
# -> 2 m/s^2   (SOLVED)
```

$$a = r\alpha$$
$$a = \alpha r = 2\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	tangential_acceleration needs r (distance from the axis, m). Example: solve('tangential_acceleration', r='... m', alpha='... rad/s^2').
`r` in a wrong unit	INVALID_INPUT	r is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "A point on a rotating wheel is at a radius of 0.5 m. The wheel has an angular acceleration of 4 rad/s^2. What is the tangential acceleration of the point?" → 2 m/s^2
- "A flywheel of radius 0.3 m spins up with an angular acceleration of 10 rad/s^2. Find the tangential acceleration at its rim." → 3 m/s^2

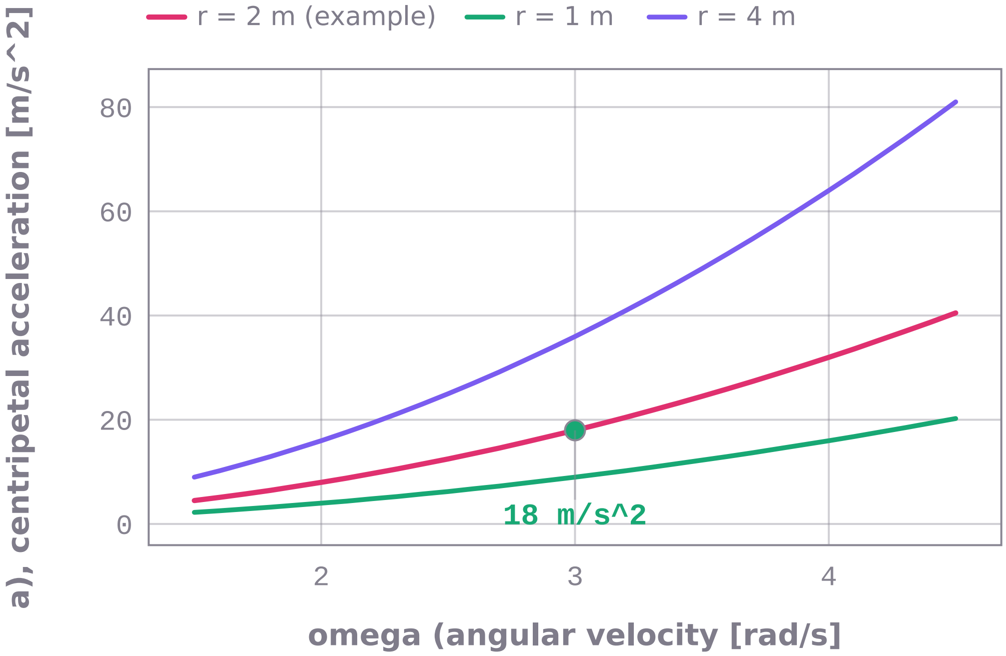
Centripetal acceleration from angular velocity, `centripetal_acceleration_omega`

$$a = r\omega^2$$

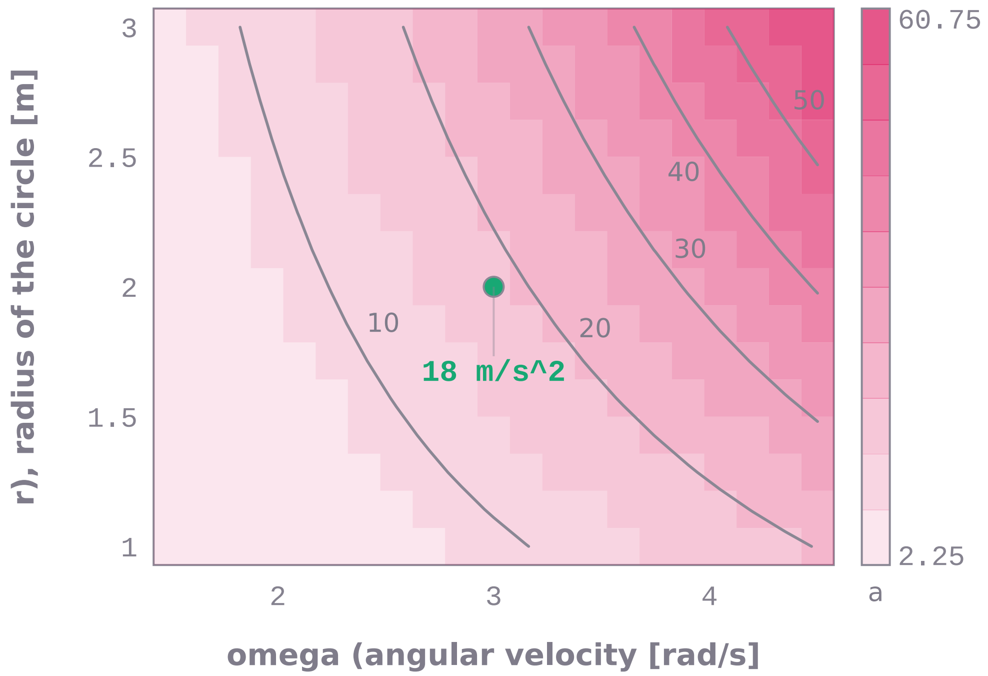
input	meaning	kind / role	unit	default
`a`	centripetal acceleration	acceleration / centripetal	`m/s^2`	

input	meaning	kind / role	unit	default
`r`	radius of the circle	length / radius	`m`	
`omega`	angular velocity	angular_frequency / plain	`rad/s`	

- **Conditions:** rotating
- **Applies when:** motion on a circle at constant angular velocity
- **Fails when:** the radius changes
- **Assumes:** uniform circular motion
- **Evidence:** derived



a (centripetal acceleration) against omega (angular velocity) from 1.5 rad/s to 4.5 rad/s. One curve for each value of r (radius of the circle): 2 m, 1 m, 4 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, omega = 3 rad/s, a = 18 m/s^2.



a (centripetal acceleration) over omega (angular velocity) from 1.5 rad/s to 4.5 rad/s and r (radius of the circle) from 1 m to 3 m: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 2.25 m/s^2 to dark 60.75 m/s^2); iso-lines at a = 10, 20, 30, 40, 50 m/s^2. Marked: the worked example, omega = 3 rad/s, r = 2 m, a = 18 m/s^2.

```
classical_physics_ai999x.solve('centripetal_acceleration_omega', r='2 m', omega='3 rad/s',
target='a')
# -> 18 m/s^2   (SOLVED)
```

$$a = r\omega^2$$
$$a = \omega^2 r = 18\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	centripetal_acceleration_omega needs r (radius of the circle, m). Example: solve('centripetal_acceleration_omega', r='... m', omega='... rad
`r` in a wrong unit	INVALID_INPUT	r is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "A centrifuge rotates at an angular velocity of 3 rad/s. What is the centripetal acceleration at a radius of 2 m?" → 18 m/s^2
- "A sample spins at an angular velocity of 100 rad/s at a radius of 0.1 m. What is its centripetal acceleration?" → 1000 m/s^2

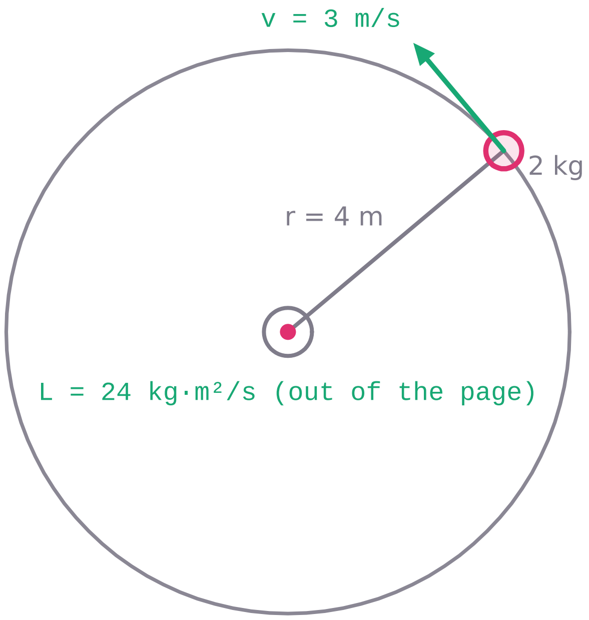
Angular momentum of a point mass on a circle, `angular_momentum_point_mass`

$$L = mvr$$

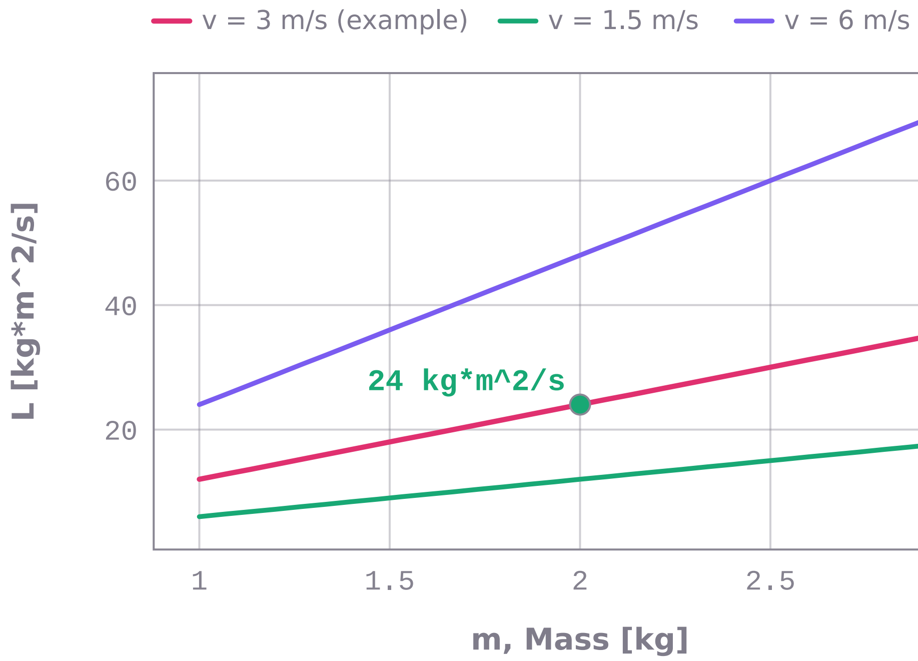
input	meaning	kind / role	unit	default
`L`	angular momentum about the centre	angular_momentum / plain	`kg*m^2/s`	

input	meaning	kind / role	unit	default
`m`		mass / plain	`kg`	
`v`	speed on the circle	speed / plain	`m/s`	
`r`	radius of the circle	length / radius	`m`	

- **Conditions:** orbit
- **Applies when:** a small body moving on a circle
- **Fails when:** an extended body
- **Assumes:** a point mass moving perpendicular to the radius
- **Evidence:** definition



A 2 kg body moving at v = 3 m/s perpendicular to its radius r = 4 m about a centre: its angular momentum points out of the page (right-hand rule, ⊙ at the centre). Marked: the value the package returns, L = m v r = 24 kg·m²/s.



L (angular momentum about the centre) against m (Mass) from 1 kg to 3 kg. One curve for each value of v (speed on the circle): 3 m/s, 1.5 m/s, 6 m/s. Other inputs as in the worked example: r = 4.0 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, L = 24 kg*m^2/s.

```
classical_physics_ai999x.solve('angular_momentum_point_mass', m='2 kg', v='3 m/s', r='4 m',
target='L')
# -> 24 kg*m^2/s   (SOLVED)
```

$$L = mvr$$
$$L = mrv = 24\,\mathrm{kg\,m^2/s}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	angular_momentum_point_mass needs m (mass, kg). Example: solve('angular_momentum_point_mass', m='... kg', v='... m/s', r='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A 2 kg satellite moves at 3 m/s in a circular orbit of radius 4 m. What is its angular momentum?" → 24 kg*m^2/s
- "A planet of mass 5.97e24 kg orbits at 2.98e4 m/s at an orbital radius of 1.496e11 m. What is its angular momentum?" → 2.66147e+40 kg*m^2/s

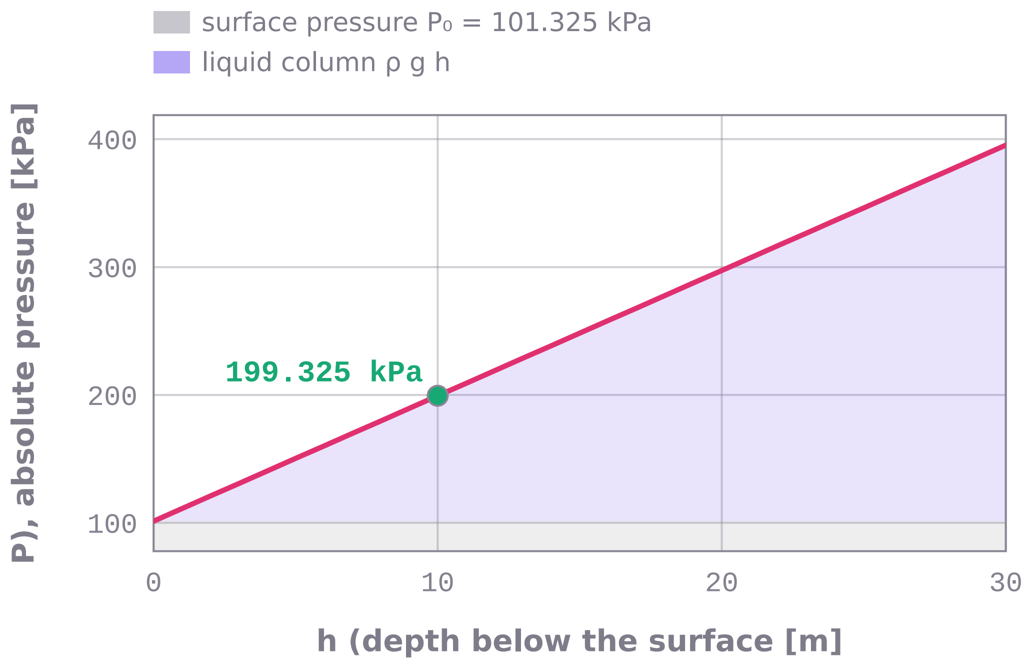
Absolute pressure at a depth in a fluid, `absolute_pressure_depth`

$$P = P_0 + \rho g h$$

input	meaning	kind / role	unit	default
`P`	absolute pressure at the depth	pressure / absolute	`Pa`	
`P0`	pressure at the surface	pressure / atmospheric	`Pa`	

input	meaning	kind / role	unit	default
`rho`	density of the fluid	density / plain	`kg/m^3`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	depth below the surface	length / depth	`m`	

- **Conditions:** fluid_depth
- **Applies when:** a liquid at rest
- **Fails when:** the fluid is accelerating or compressible over the depth
- **Assumes:** an incompressible fluid at rest; uniform gravity
- **Evidence:** fundamental_law



Absolute pressure in a liquid (ρ = 1000 kg/m³, g = 9.8 m/s²) with P₀ = 101325 Pa at the surface, 31 depths from the package (shown in kPa): the surface pressure's share is constant, the liquid column adds ρ g h. Marked: 199325 Pa at 10 m.

```
classical_physics_ai999x.solve('absolute_pressure_depth', P0='101325 Pa', rho='1000
kg/m^3', h='10 m', g='9.8 m/s^2', target='P')
# -> 199325 Pa   (SOLVED)
```

$$P = P_0 + \rho g h$$
$$P = P_0 + g h \rho = 199325\ \mathrm{Pa}$$

invalid call	outcome	message
without `P0`	NEEDS_INPUT	absolute_pressure_depth needs P0 (pressure at the surface, Pa). Example: solve('absolute_pressure_depth', P0='... Pa', rho='... kg/m^3', g='
`P0` in a wrong unit	INVALID_INPUT	P0 is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

- "The atmospheric pressure is 101325 Pa. What is the absolute pressure 10 m below the surface of water of density 1000 kg/m^3?" → 199392 Pa

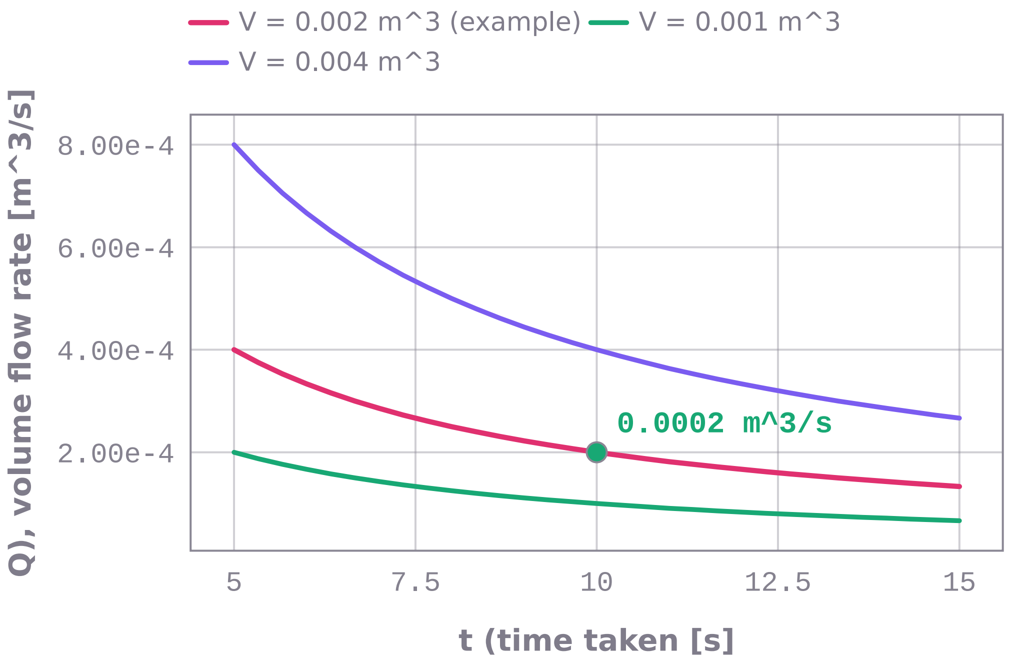
- "A diver is at a depth of 20 m in sea water of density 1025 kg/m^3 where the atmospheric pressure is 1e5 Pa. What is the absolute pressure on the diver?" → 301036 Pa

Flow rate as volume per time, `flow_rate_volume_time`

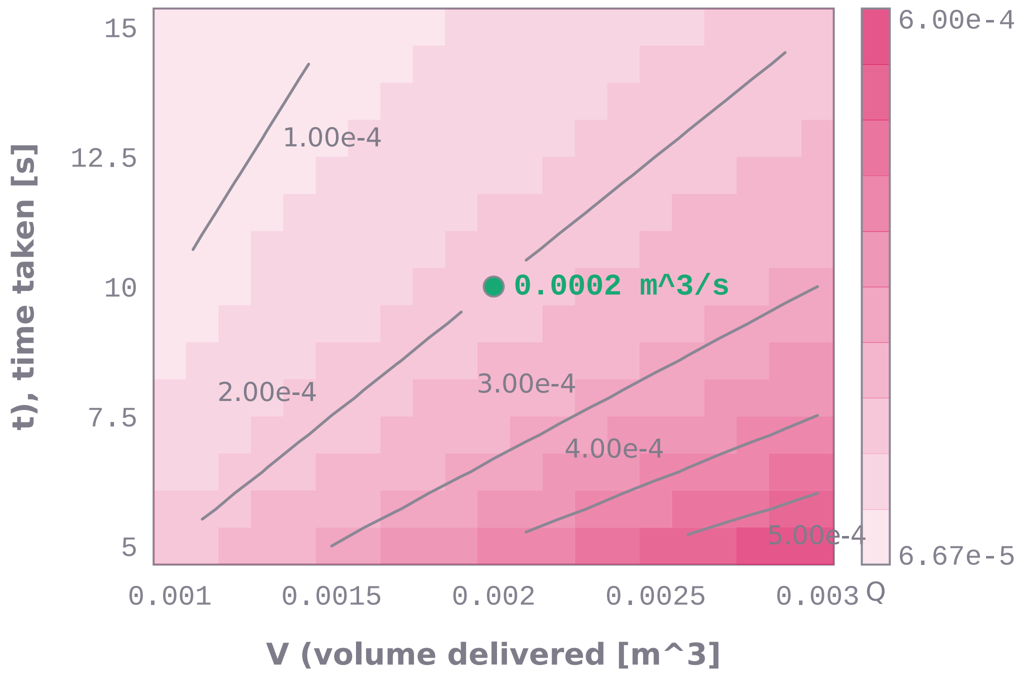
$$Q = \frac{V}{t}$$

input	meaning	kind / role	unit	default
`Q`	volume flow rate	flow_rate / plain	`m^3/s`	
`V`	volume delivered	volume / plain	`m^3`	
`t`	time taken	time / plain	`s`	

- **Conditions:** flow
- **Applies when:** a fluid flows through a pipe or opening
- **Fails when:** the flow is unsteady and an instantaneous rate is asked
- **Assumes:** a steady flow
- **Evidence:** definition



Q (volume flow rate) against t (time taken) from 5 s to 15 s. One curve for each value of V (volume delivered): 0.002 m^3, 0.001 m^3, 0.004 m^3. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, t = 10 s, Q = 0.0002 m^3/s.



Q (volume flow rate) over V (volume delivered) from 0.001 m^3 to 0.003 m^3 and t (time taken) from 5 s to 15 s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 6.6667e-05 m^3/s to dark 0.0006 m^3/s); iso-lines at Q = 1.00e-4, 2.00e-4, 3.00e-4, 4.00e-4, 5.00e-4 m^3/s. Marked: the worked example, V = 0.002 m^3, t = 10 s, Q = 0.0002 m^3/s.

```
classical_physics_ai999x.solve('flow_rate_volume_time', V='0.002 m^3', t='10 s',
target='Q')
# -> 0.0002 m^3/s   (SOLVED)
```

$$Q = \frac{V}{t}$$

$$Q = \frac{V}{t} = 0.0002\,\mathrm{m}^3/\mathrm{s}$$

invalid call	outcome	message
without `V`	NEEDS_INPUT	flow_rate_volume_time needs V (volume delivered, m^3). Example: solve('flow_rate_volume_time', V='... m^3', t='... s').
`V` in a wrong unit	INVALID_INPUT	V is a volume: its unit must be like m^3, not 'kg'.

Questions in words that work:

- "A hose fills a 2 L bucket in 10 s with water. What is the flow rate?" → 0.0002 m^3/s
- "Water flows through a pipe and 300 L flows out in 60 s. What is the volume flow rate?" → 0.005 m^3/s

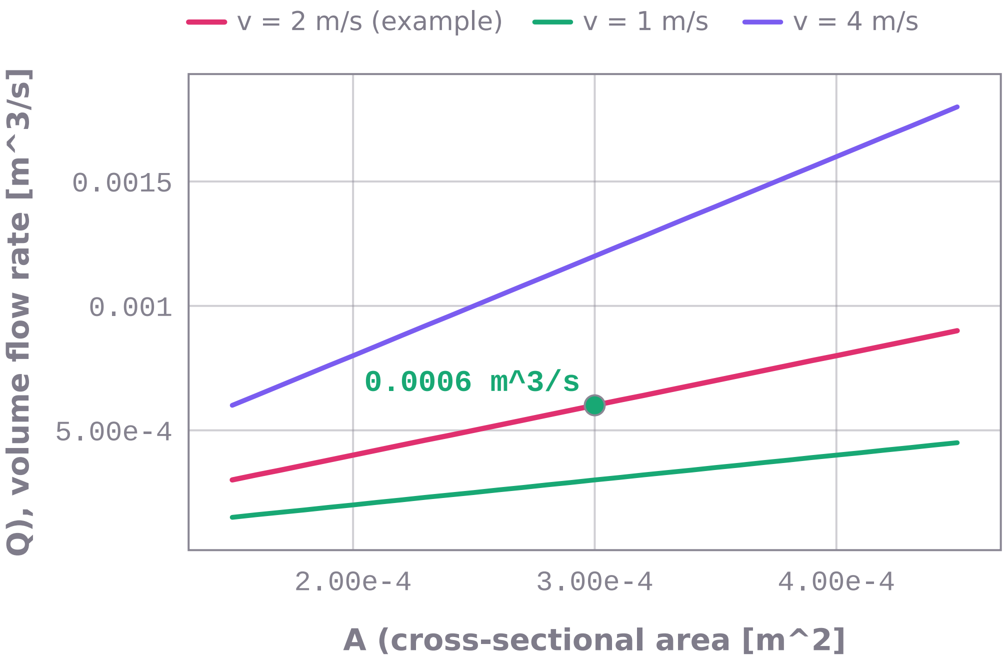
Flow rate through a cross section, `flow_rate_area_speed`

$$Q = Av$$

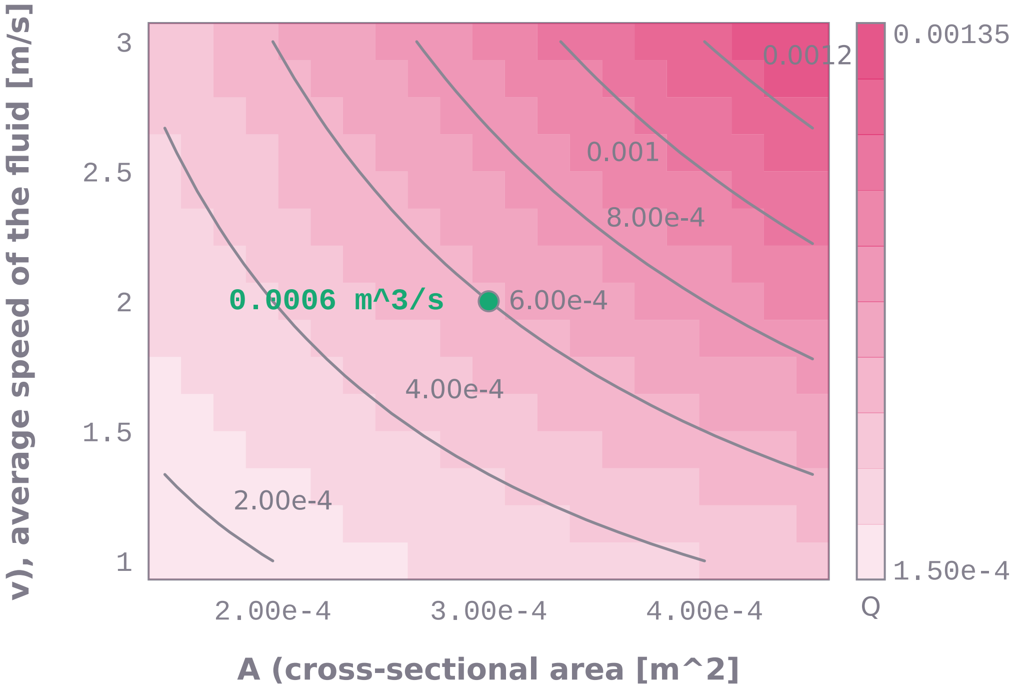
input	meaning	kind / role	unit	default
`Q`	volume flow rate	flow_rate / plain	`m^3/s`	

input	meaning	kind / role	unit	default
`A`	cross-sectional area	area / plain	`m^2`	
`v`	average speed of the fluid	speed / plain	`m/s`	

- **Conditions:** flow
- **Applies when:** a fluid flows through a pipe
- **Fails when:** the flow is not along the pipe
- **Assumes:** a steady flow; the average speed over the cross section
- **Evidence:** definition



Q (volume flow rate) against A (cross-sectional area) from 0.00015 m^2 to 0.00045 m^2. One curve for each value of v (average speed of the fluid): 2 m/s, 1 m/s, 4 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 0.0003 m^2, Q = 0.0006 m^3/s.



Q (volume flow rate) over A (cross-sectional area) from 0.00015 m^2 to 0.00045 m^2 and v (average speed of the fluid) from 1 m/s to 3 m/s: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0.00015 m^3/s to dark 0.00135 m^3/s); iso-lines at Q = 2.00e-4, 4.00e-4, 6.00e-4, 8.00e-4, 0.001, 0.0012 m^3/s. Marked: the worked example, A = 0.0003 m^2, v = 2 m/s, Q = 0.0006 m^3/s.

```
classical_physics_ai999x.solve('flow_rate_area_speed', A='0.0003 m^2', v='2 m/s',
target='Q')
# -> 0.0006 m^3/s   (SOLVED)
```

$$Q = Av$$
$$Q = Av = 0.0006\ \mathrm{m}^3/\mathrm{s}$$

invalid call	outcome	message
without `A`	NEEDS_INPUT	flow_rate_area_speed needs A (cross-sectional area, m^2). Example: solve('flow_rate_area_speed', A='... m^2', v='... m/s').
`A` in a wrong unit	INVALID_INPUT	A is a area: its unit must be like m^2, not 'kg'.

Questions in words that work:

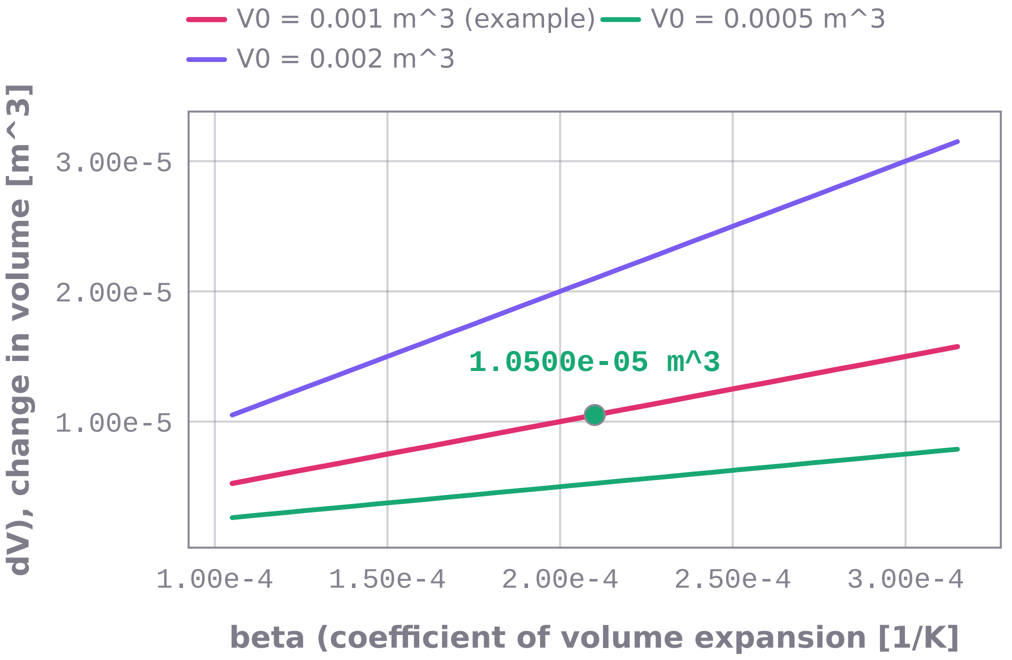
- "Water flows through a pipe with a cross-sectional area of 3e-4 m^2 at a speed of 2 m/s. What is the flow rate?" → 0.0006 m^3/s
- "Blood is pumped at a flow rate of 8.3e-5 m^3/s into the aorta of cross-sectional area 3.14e-4 m^2. What is the speed of the blood?" → 0.264331 m/s

Thermal expansion of volume, `volume_expansion`

$$d_V = \beta V_0 d_T$$

input	meaning	kind / role	unit	default
`dV`	change in volume	volume / expansion	`m^3`	
`beta`	coefficient of volume expansion	expansion_coefficient / volume	`1/K`	
`V0`	original volume	volume / original	`m^3`	
`dT`	temperature change	temperature_difference / plain	`K`	

- **Conditions:** thermal_expansion
- **Applies when:** solids and liquids over moderate temperature changes
- **Fails when:** phase changes or large temperature ranges
- **Assumes:** a small temperature change; a constant coefficient
- **Evidence:** empirical



dV (change in volume) against beta (coefficient of volume expansion) from 0.000105 1/K to 0.000315 1/K. One curve for each value of V0 (original volume): 0.001 m^3, 0.0005 m^3, 0.002 m^3. Other inputs as in the worked example: dT = 50.0 K. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, beta = 0.00021 1/K, dV = 1.0500e-05 m^3.

```
classical_physics_ai999x.solve('volume_expansion', beta='0.00021 1/K', V0='0.001 m^3',
dT='50 K', target='dV')
# -> 1.05e-05 m^3   (SOLVED)
```

$$d_V = \beta V_0 d_T$$
$$d_V = V_0 \beta d_T = 1.05e - 05\ \mathrm{m}^3$$

invalid call	outcome	message
without `beta`	NEEDS_INPUT	volume_expansion needs beta (coefficient of volume expansion, 1/K). Example: solve('volume_expansion', beta='... 1/K', V0='... m^3', dT='...
`beta` in a wrong unit	INVALID_INPUT	beta is a expansion coefficient: its unit must be like 1/K, not 'kg'.

Questions in words that work:

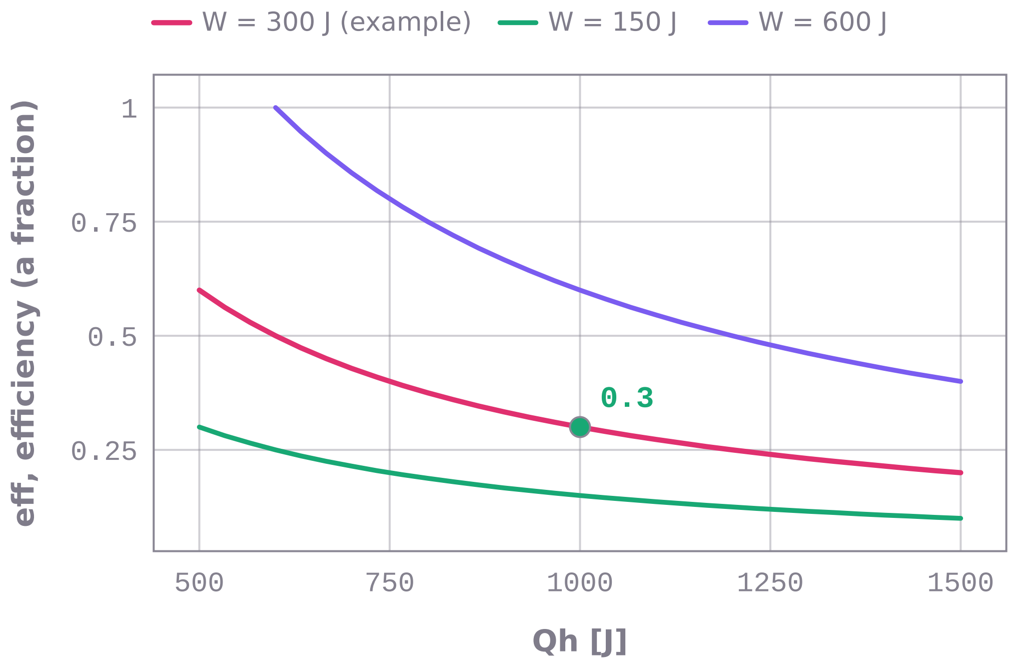
- "Water with an original volume of 1 L and a coefficient of volume expansion of 2.1e-4 1/K is heated by 50 K. What is the change in volume as it expands?" → 1.05e-05 m^3
- "Gasoline has a coefficient of volume expansion of 9.5e-4 1/K. A tank holds an initial volume of 60 L and is warmed by 20 K so it expands. What is the increase in volume?" → 0.00114 m^3

Efficiency of a heat engine, `engine_efficiency`

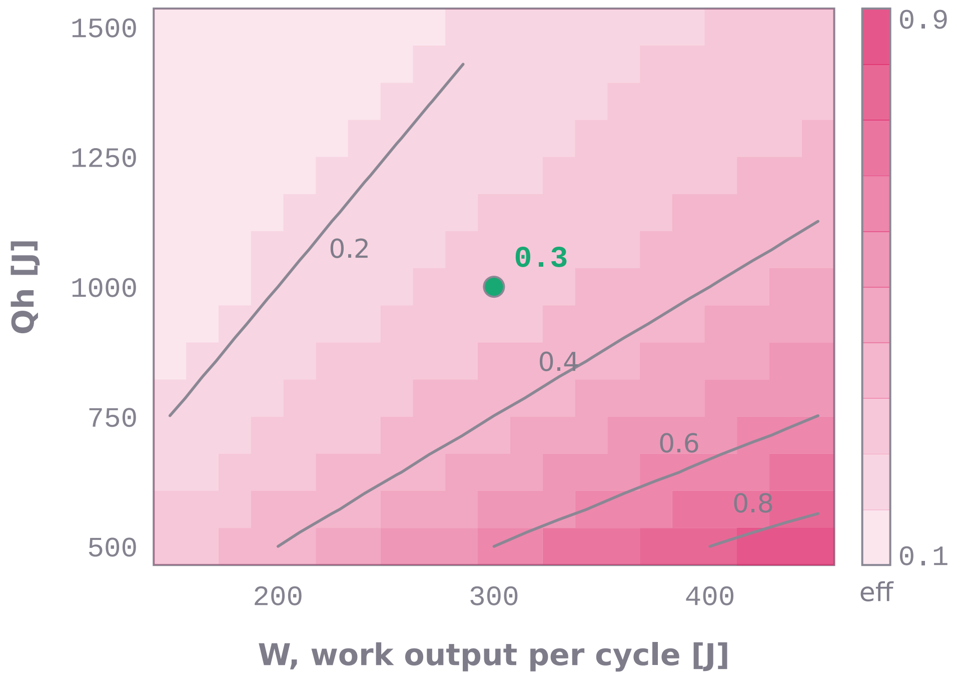
$$\mathrm{eff} = \frac{W}{Q_h}$$

input	meaning	kind / role	unit	default
`eff`	efficiency (a fraction)	efficiency / plain	`1`	
`W`	work output per cycle	energy / work	`J`	
`Qh`	heat taken in from the hot reservoir	energy / heat_hot	`J`	

- **Conditions:** heat_engine
- **Applies when:** a heat engine over whole cycles
- **Fails when:** a single non-cyclic process
- **Assumes:** a cyclic engine; efficiency as a fraction, not a percentage
- **Evidence:** definition



eff (efficiency (a fraction)) against Qh (heat taken in from the hot reservoir) from 500 J to 1500 J. One curve for each value of W (work output per cycle): 300 J, 150 J, 600 J. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Qh = 1000 J, eff = 0.3.



eff (efficiency (a fraction)) over W (work output per cycle) from 150 J to 450 J and Qh (heat taken in from the hot reservoir) from 500 J to 1500 J: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0.1 to dark 0.9); iso-lines at eff = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, W = 300 J, Qh = 1000 J, eff = 0.3.

```
classical_physics_ai999x.solve('engine_efficiency', W='300 J', Qh='1000 J', target='eff')
# -> 0.3 1   (SOLVED)
```

$$\mathrm{eff} = \frac{W}{Q_h}$$

$$\mathrm{eff} = \frac{W}{Q_h} = 0.3\ 1$$

invalid call	outcome	message
without `W`	NEEDS_INPUT	engine_efficiency needs W (work output per cycle, J). Example: solve('engine_efficiency', W='... J', Qh='... J').
`W` in a wrong unit	INVALID_INPUT	W is a energy: its unit must be like J, not 'kg'.

Questions in words that work:

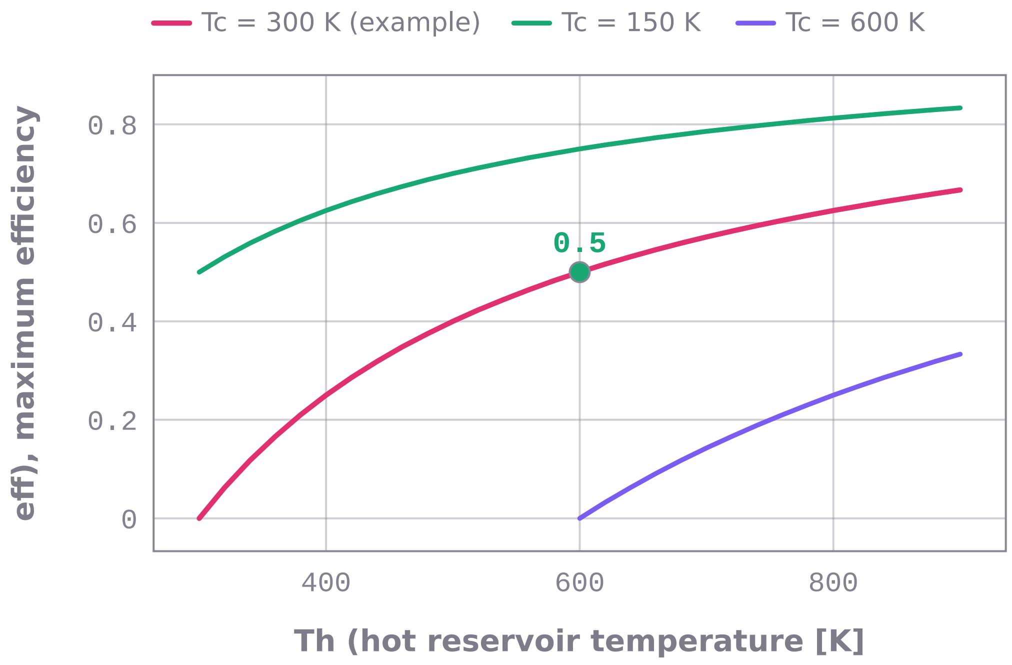
- "A heat engine takes in a heat input of 1000 J and produces a work output of 300 J. What is its efficiency?" → 0.3 1
- "An engine with an efficiency of 0.25 has a heat input of 4000 J. What is its work output?" → 1000 J

Carnot (maximum) efficiency, `carnot_efficiency`

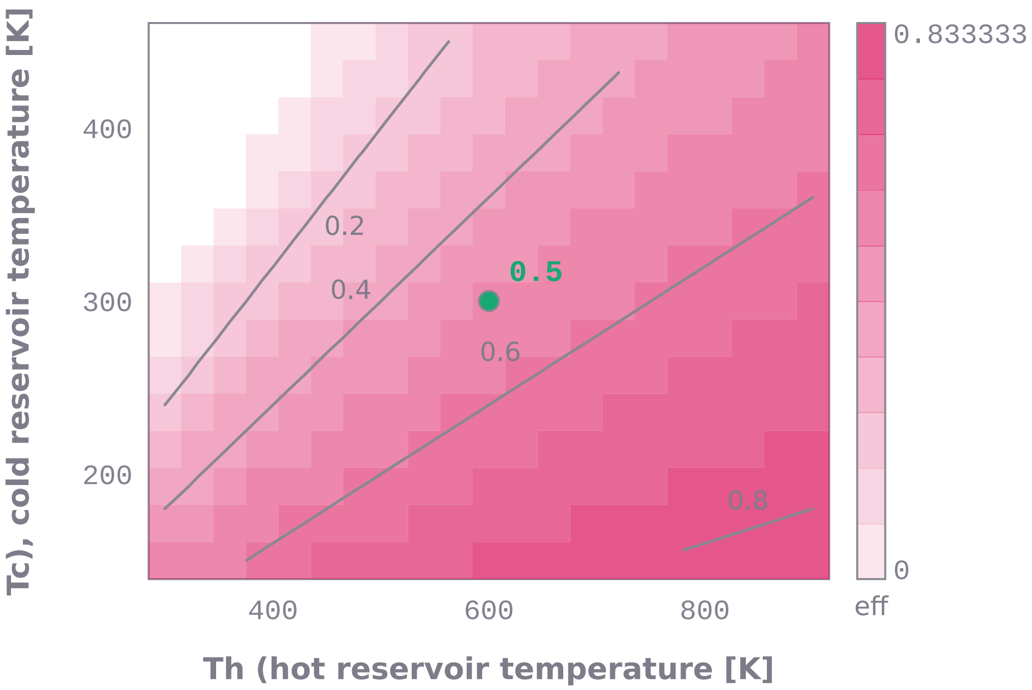
$$\mathrm{eff} = 1 - \frac{T_c}{T_h}$$

input	meaning	kind / role	unit	default
`eff`	maximum efficiency	efficiency / carnot	`1`	
`Tc`	cold reservoir temperature (absolute)	temperature / cold	`K`	
`Th`	hot reservoir temperature (absolute)	temperature / hot	`K`	

- **Conditions:** heat_engine
- **Applies when:** an ideal engine between two reservoirs
- **Fails when:** temperatures in degrees Celsius used as absolute
- **Assumes:** a reversible engine between two reservoirs; absolute temperatures
- **Evidence:** fundamental_law



eff (maximum efficiency) against Th (hot reservoir temperature (absolute)) from 300 K to 900 K. One curve for each value of Tc (cold reservoir temperature (absolute)): 300 K, 150 K, 600 K. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Th = 600 K, eff = 0.5.



eff (maximum efficiency) over Th (hot reservoir temperature (absolute)) from 300 K to 900 K and Tc (cold reservoir temperature (absolute)) from 150 K to 450 K: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0 to dark 0.833333); iso-lines at eff = 0.2, 0.4, 0.6, 0.8; blank cells: inputs the capability refuses. Marked: the worked example, Th = 600 K, Tc = 300 K, eff = 0.5.

```
classical_physics_ai999x.solve('carnot_efficiency', Tc='300 K', Th='600 K', target='eff')
# -> 0.5 1   (SOLVED)
```

$$\mathrm{eff} = 1 - \frac{T_c}{T_h}$$

$$\mathrm{eff} = \frac{-T_c + T_h}{T_h} = 0.5\ 1$$

invalid call	outcome	message
without `Tc`	NEEDS_INPUT	carnot_efficiency needs Tc (cold reservoir temperature (absolute), K). Example: solve('carnot_efficiency', Tc='... K', Th='... K').
`Tc` in a wrong unit	INVALID_INPUT	Tc is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

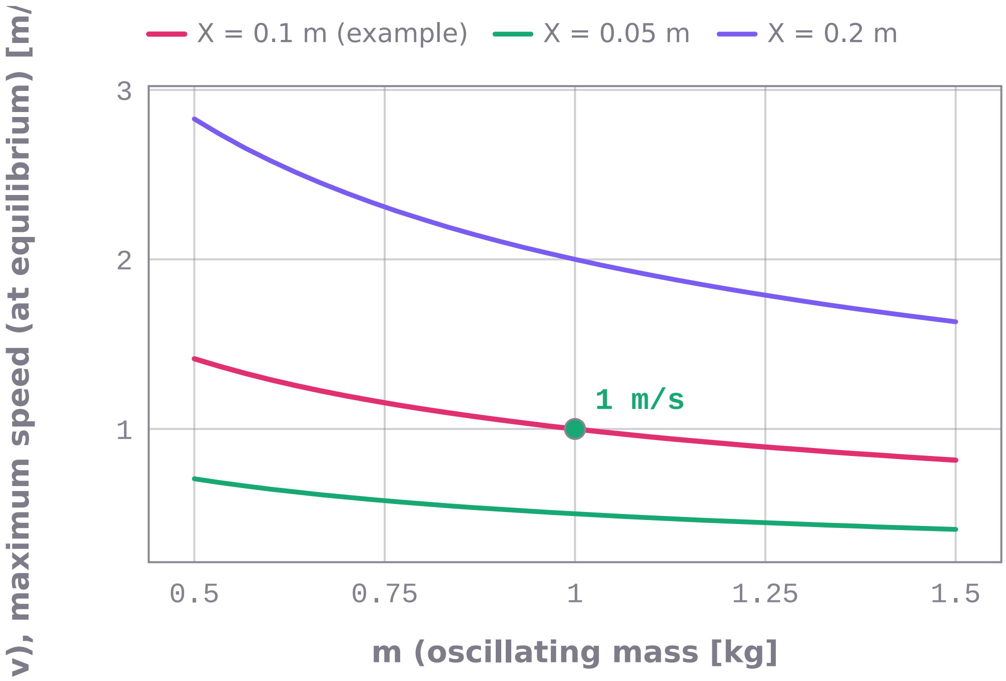
- "A heat engine works between a hot reservoir temperature of 600 K and a cold reservoir temperature of 300 K. What is its maximum efficiency?" → 0.5 1
- "What is the carnot efficiency of an engine with a hot reservoir temperature of 873 K and a cold reservoir temperature of 293 K?" → 0.664376 1

Maximum speed of a mass on a spring, `shm_max_speed`

$$v = X\sqrt{\frac{k}{m}}$$

input	meaning	kind / role	unit	default
`v`	maximum speed (at equilibrium)	speed / max	`m/s`	
`X`	amplitude	length / amplitude	`m`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	

- **Conditions:** shm
- **Applies when:** a mass oscillating on a spring
- **Fails when:** damped or driven motion
- **Assumes:** undamped simple harmonic motion; an ideal massless spring
- **Evidence:** fundamental_law



v (maximum speed (at equilibrium)) against m (oscillating mass) from 0.5 kg to 1.5 kg. One curve for each value of X (amplitude): 0.1 m, 0.05 m, 0.2 m. Other inputs as in the worked example: k = 100.0 N/m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 1 kg, v = 1 m/s.

```
classical_physics_ai999x.solve('shm_max_speed', X='0.1 m', k='100 N/m', m='1 kg',
target='v')
# -> 1 m/s   (SOLVED)
```

$$v = X\sqrt{\frac{k}{m}}$$

$$v = \frac{X\sqrt{k}}{\sqrt{m}} = 1\ \mathrm{m/s}$$

invalid call	outcome	message
without `X`	NEEDS_INPUT	shm_max_speed needs X (amplitude, m). Example: solve('shm_max_speed', X='... m', k='... N/m', m='... kg').
`X` in a wrong unit	INVALID_INPUT	X is a length: its unit must be like m, not 'kg'.

Questions in words that work:

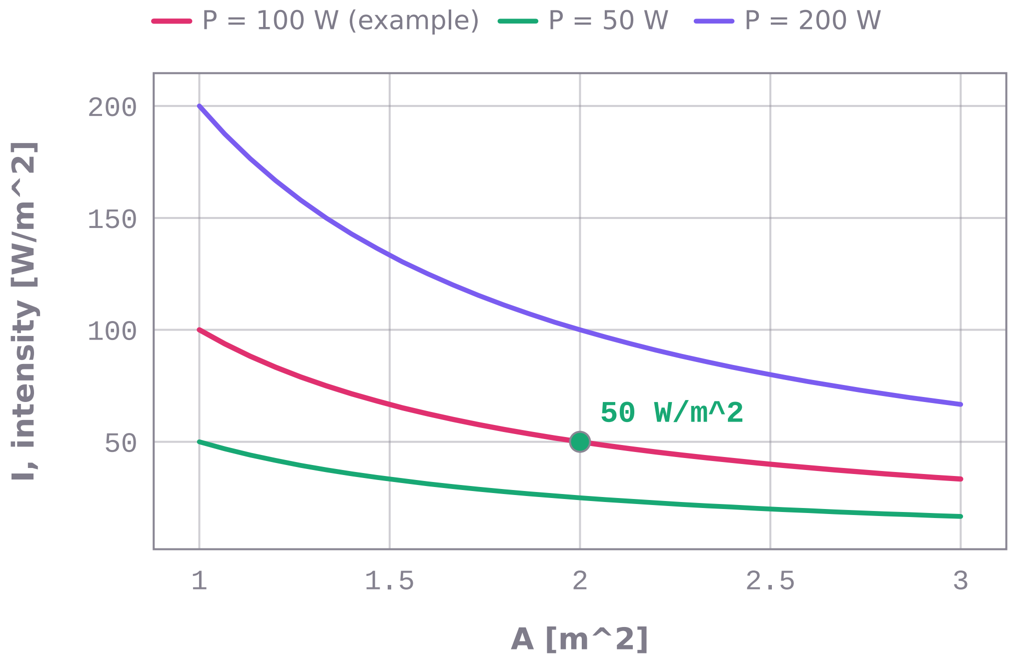
- "A 1 kg mass oscillates in simple harmonic motion on a spring with a spring constant of 100 N/m and an amplitude of 0.1 m. What is its maximum speed?" → 1 m/s
- "A 0.5 kg block on a spring of spring constant 200 N/m oscillates with an amplitude of 5 cm. What is its maximum velocity?" → 1 m/s

Intensity as power per area, `intensity_power_area`

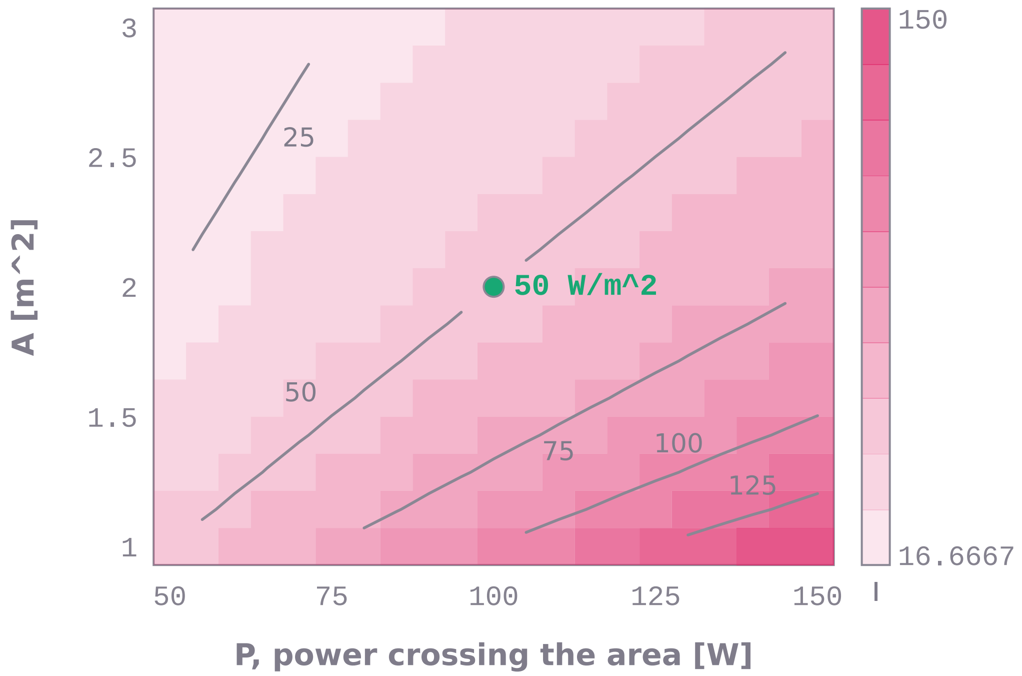
$$I = \frac{P}{A}$$

input	meaning	kind / role	unit	default
`I`	intensity	intensity / plain	`W/m^2`	
`P`	power crossing the area	power / plain	`W`	
`A`	area perpendicular to the flow of energy	area / plain	`m^2`	

- **Conditions:** radiation
- **Applies when:** waves or radiation falling on a surface
- **Fails when:** non-uniform beams
- **Assumes:** power spread uniformly over the area
- **Evidence:** definition



I (intensity) against A (area perpendicular to the flow of energy) from 1 m^2 to 3 m^2. One curve for each value of P (power crossing the area): 100 W, 50 W, 200 W. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 2 m^2, I = 50 W/m^2.



I (intensity) over P (power crossing the area) from 50 W to 150 W and A (area perpendicular to the flow of energy) from 1 m^2 to 3 m^2: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 16.6667 W/m^2 to dark 150 W/m^2); iso-lines at I = 25, 50, 75, 100, 125 W/m^2. Marked: the worked example, P = 100 W, A = 2 m^2, I = 50 W/m^2.

```
classical_physics_ai999x.solve('intensity_power_area', P='100 W', A='2 m^2', target='I')
# -> 50 W/m^2   (SOLVED)
```

$$I = \frac{P}{A}$$

$$I = \frac{P}{A} = 50\ \mathrm{W/m^2}$$

invalid call	outcome	message
without `P`	NEEDS_INPUT	intensity_power_area needs P (power crossing the area, W). Example: solve('intensity_power_area', P='... W', A='... m^2').
`P` in a wrong unit	INVALID_INPUT	P is a power: its unit must be like W, not 'kg'.

Questions in words that work:

- "A power of 100 W falls uniformly on an area of 2 m^2. What is the intensity?" → 50 W/m^2
- "Sunlight with an intensity of 1000 W/m^2 falls on a panel with an area of 0.5 m^2. What power does it receive?" → 500 W

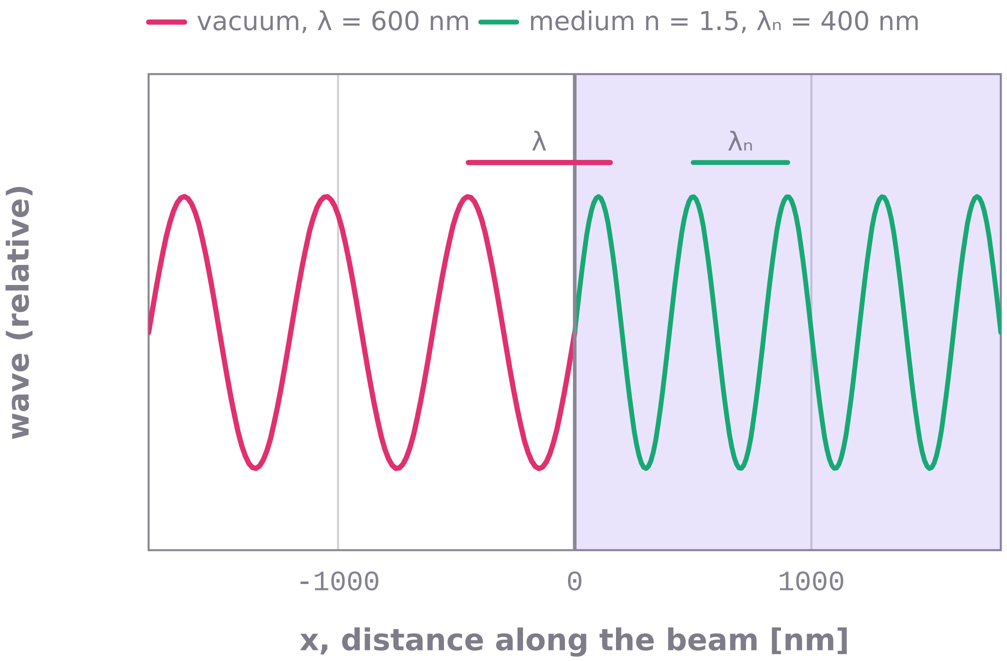
Wavelength of light in a medium, `wavelength_in_medium`

$$\lambda_n = \frac{\lambda}{n}$$

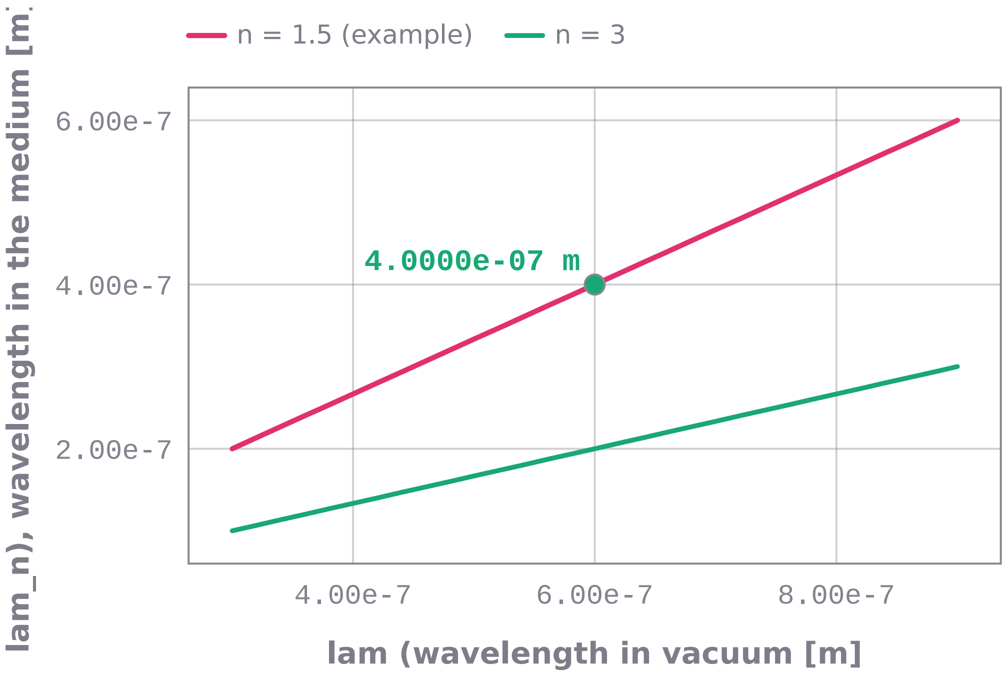
input	meaning	kind / role	unit	default
`lam_n`	wavelength in the medium	length / wavelength_medium	`m`	

input	meaning	kind / role	unit	default
`lam`	wavelength in vacuum	length / wavelength	`m`	
`n`	index of refraction	refractive_index / plain	`1`	

- **Conditions:** medium
- **Applies when:** light entering a transparent medium
- **Fails when:** absorbing media
- **Assumes:** the frequency is unchanged across the boundary
- **Evidence:** definition



Light of vacuum wavelength λ = 6.0000e-07 m entering a medium of refractive index n = 1.5 (shaded) at x = 0, drawn as a sine wave of each wavelength: the frequency is unchanged, so the waves are compressed. Marked: the wavelength in the medium the package returns, $\lambda_n = \lambda/n$ = 4.0000e-07 m.



lam_n (wavelength in the medium) against lam (wavelength in vacuum) from 3.0000e-07 m to 9.0000e-07 m. One curve for each value of n (index of refraction): 1.5, 3. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, lam = 6.0000e-07 m, lam_n = 4.0000e-07 m.

```
classical_physics_ai999x.solve('wavelength_in_medium', lam='6e-07 m', n='1.5 1',
target='lam_n')
# -> 4e-07 m   (SOLVED)
```

$$\lambda_n = \frac{\lambda}{n}$$
$$\lambda_n = \frac{\lambda}{n} = 4e - 07\,\mathrm{m}$$

invalid call	outcome	message
without `lam`	NEEDS_INPUT	wavelength_in_medium needs lam (wavelength in vacuum, m). Example: solve('wavelength_in_medium', lam='... m', n='... 1').
`lam` in a wrong unit	INVALID_INPUT	lam is a length: its unit must be like m, not 'kg'.

Questions in words that work:

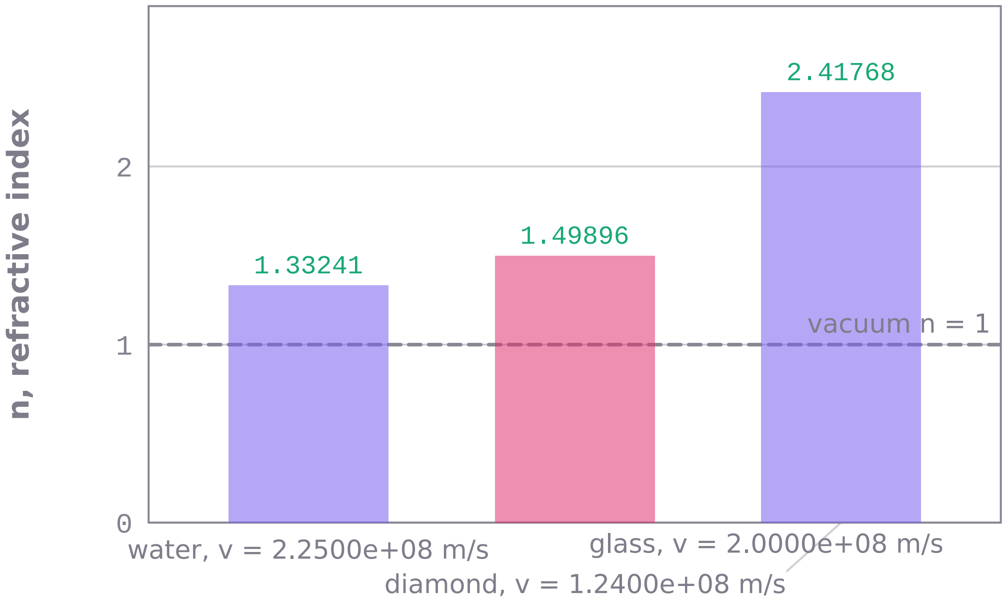
- "Light with a wavelength in vacuum of 600 nm enters glass with an index of refraction of 1.5. What is the wavelength in the medium?" → 4e-07 m
- "Yellow light of wavelength in air 589 nm enters water, refractive index 1.333. What is the wavelength in the medium?" → 4.4186e-07 m

Index of refraction, `index_of_refraction`

$$n = \frac{c}{v}$$

input	meaning	kind / role	unit	default
`n`	index of refraction	refractive_index / plain	`1`	
`c`		speed / light	`m/s`	299792458.0 m/s
`v`	speed of light in the medium	speed / medium	`m/s`	

- **Conditions:** medium
- **Applies when:** transparent materials
- **Fails when:** absorbing media
- **Assumes:** light in a transparent medium
- **Evidence:** definition



Refractive index n = c/v of transparent media from the speed of light in each (stated: water 2.2500e+08 m/s, glass 2.0000e+08 m/s, diamond 1.2400e+08 m/s; glass is the worked example), each from the package. Marked: water 1.33241, glass 1.49896, diamond 2.41768.

```
classical_physics_ai999x.solve('index_of_refraction', v='2e+08 m/s', target='n')
# -> 1.49896 1   (SOLVED)
```

$$n = \frac{c}{v}$$
$$n = \frac{c}{v} = 1.49896\ 1$$

invalid call	outcome	message
without `v`	NEEDS_INPUT	index_of_refraction needs v (speed of light in the medium, m/s). Example: solve('index_of_refraction', c='... m/s', v='... m/s').
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

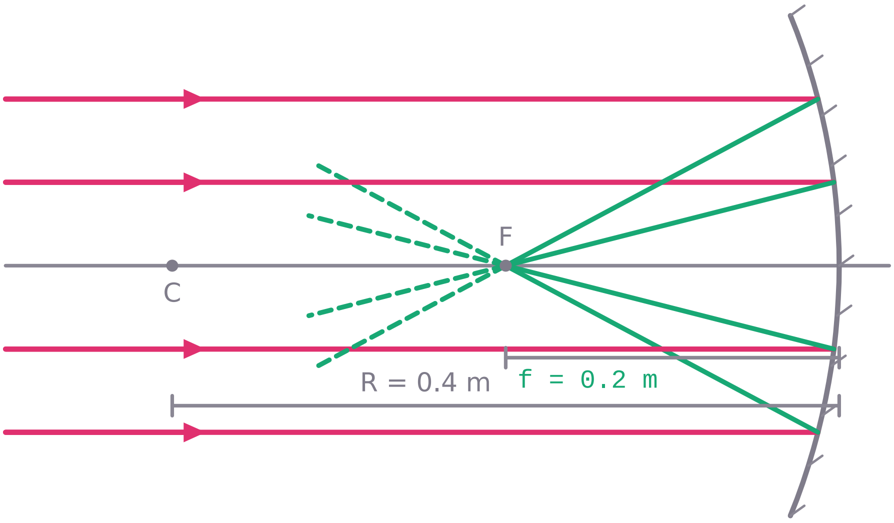
- "The speed of light in the medium is 2e8 m/s for a certain glass. What is the index of refraction of the glass?" → 1.49896 1
- "Diamond has an index of refraction of 2.42. What is the speed of light in the material?" → 1.23881e+08 m/s

Focal length of a spherical mirror, `mirror_focal_length`

$$f = \frac{R}{2}$$

input	meaning	kind / role	unit	default
`f`	focal length	length / focal	`m`	
`R`	radius of curvature	length / curvature	`m`	

- **Conditions:** mirror
- **Applies when:** spherical mirrors with small apertures
- **Fails when:** rays far from the axis
- **Assumes:** paraxial rays
- **Evidence:** fundamental_law



A concave mirror with radius of curvature R = 0.4 m (centre C): rays arriving parallel to the axis reflect through the focal point F, half-way to C (paraxial rays). Marked: the focal length the package returns, f = R/2 = 0.2 m.

```
classical_physics_ai999x.solve('mirror_focal_length', R='0.4 m', target='f')
# -> 0.2 m   (SOLVED)
```

$$f = \frac{R}{2}$$
$$f = \frac{R}{2} = 0.2\,\mathrm{m}$$

invalid call	outcome	message
without `R`	NEEDS_INPUT	mirror_focal_length needs R (radius of curvature, m). Example: solve('mirror_focal_length', R='... m').
`R` in a wrong unit	INVALID_INPUT	R is a length: its unit must be like m, not 'kg'.

Questions in words that work:

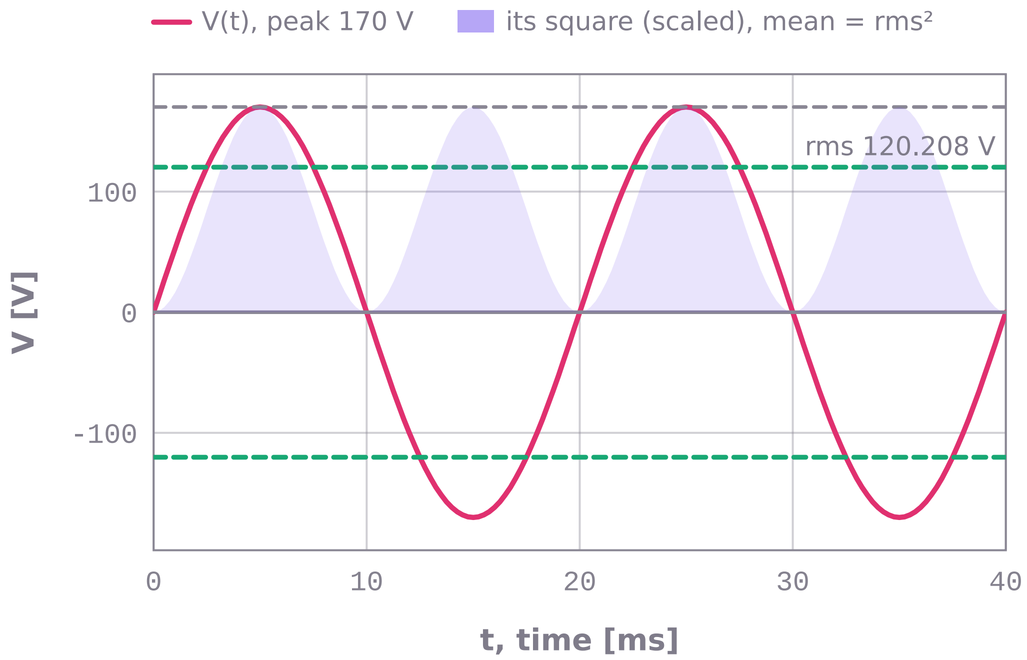
- "A concave mirror has a radius of curvature of 0.4 m. What is its focal length?" → 0.2 m
- "A mirror has a focal length of 15 cm. What is its radius of curvature?" → 0.3 m

Rms voltage of a sinusoidal supply, `rms_voltage`

$$V_{\mathrm{rms}} = \frac{V_0}{\sqrt{2}}$$

input	meaning	kind / role	unit	default
`Vrms`	rms voltage	voltage / rms	`V`	
`V0`	peak voltage	voltage / peak	`V`	

- **Conditions:** alternating_current
- **Applies when:** sinusoidal AC
- **Fails when:** square or other waveforms
- **Assumes:** a sinusoidal waveform
- **Evidence:** definition



An alternating voltage of peak 170 V (drawn at 50 Hz for two cycles, a stated frequency) with the root-mean-square value the package returns: the steady value that delivers the same average power. The shaded curve is the square of the wave divided by the peak, whose mean height is rms²/peak (derived). Marked: rms = 120.208 V = peak/√2.

```
classical_physics_ai999x.solve('rms_voltage', V0='170 V', target='Vrms')
# -> 120.208 V   (SOLVED)
```

$$V_{\mathrm{rms}} = \frac{V_0}{\sqrt{2}}$$

$$V_{\mathrm{rms}} = \frac{\sqrt{2}V_0}{2} = 120.208\ \mathrm{V}$$

invalid call	outcome	message
without `V0`	NEEDS_INPUT	rms_voltage needs V0 (peak voltage, V). Example: solve('rms_voltage', V0='... V').
`V0` in a wrong unit	INVALID_INPUT	V0 is a voltage: its unit must be like V, not 'kg'.

Questions in words that work:

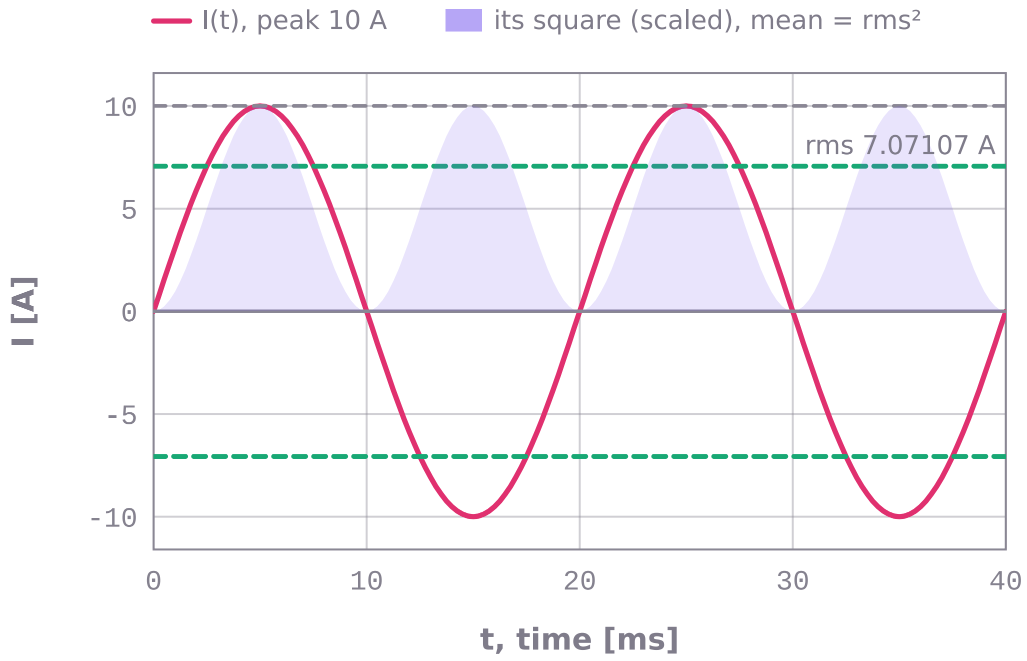
- "An AC supply has a peak voltage of 170 V. What is the rms voltage?" → 120.208 V
- "Mains AC has an rms voltage of 230 V. What is its peak voltage?" → 325.269 V

Rms current of a sinusoidal supply, `rms_current`

$$I_{\mathrm{rms}} = \frac{I_0}{\sqrt{2}}$$

input	meaning	kind / role	unit	default
`Irms`	rms current	current / rms	`A`	
`I0`	peak current	current / peak	`A`	

- **Conditions:** alternating_current
- **Applies when:** sinusoidal AC
- **Fails when:** square or other waveforms
- **Assumes:** a sinusoidal waveform
- **Evidence:** definition



An alternating current of peak 10 A (drawn at 50 Hz for two cycles, a stated frequency) with the root-mean-square value the package returns: the steady value that delivers the same average power. The shaded curve is the square of the wave divided by the peak, whose mean height is rms²/peak (derived). Marked: rms = 7.07107 A = peak/√2.

```
classical_physics_ai999x.solve('rms_current', I0='10 A', target='Irms')
# -> 7.07107 A   (SOLVED)
```

$$I_{\text{rms}} = \frac{I_0}{\sqrt{2}}$$

$$I_{\text{rms}} = \frac{\sqrt{2} I_0}{2} = 7.07107\ \text{A}$$

invalid call	outcome	message
without `I0`	NEEDS_INPUT	rms_current needs I0 (peak current, A). Example: solve('rms_current', I0='... A').
`I0` in a wrong unit	INVALID_INPUT	I0 is a current: its unit must be like A, not 'kg'.

Questions in words that work:

- "An AC circuit has a peak current of 10 A. What is the rms current?" → 7.07107 A
- "An AC appliance draws an rms current of 2 A. What is the peak current?" → 2.82843 A

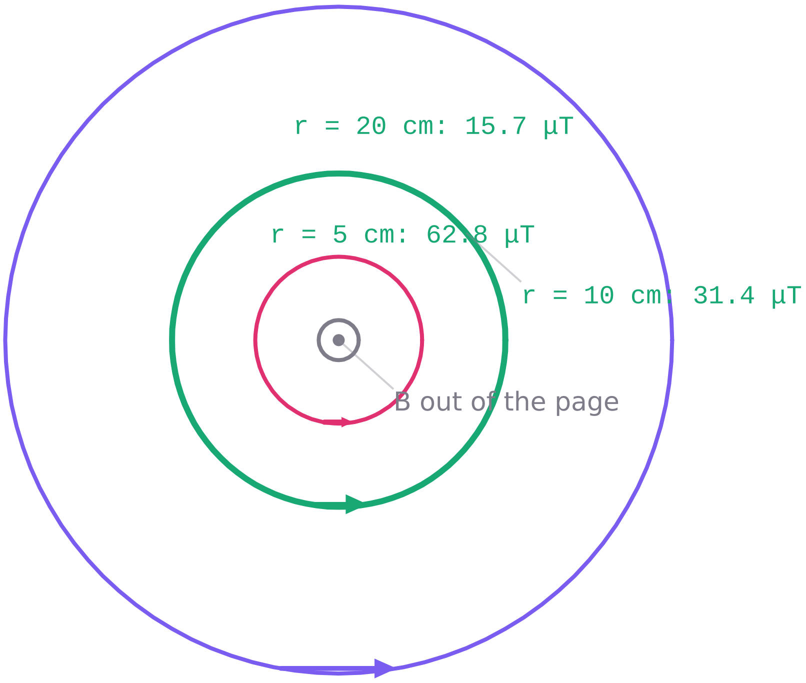
Magnetic field at the centre of a circular loop, `loop_center_field`

$$B = \frac{\mu_0 I}{2r}$$

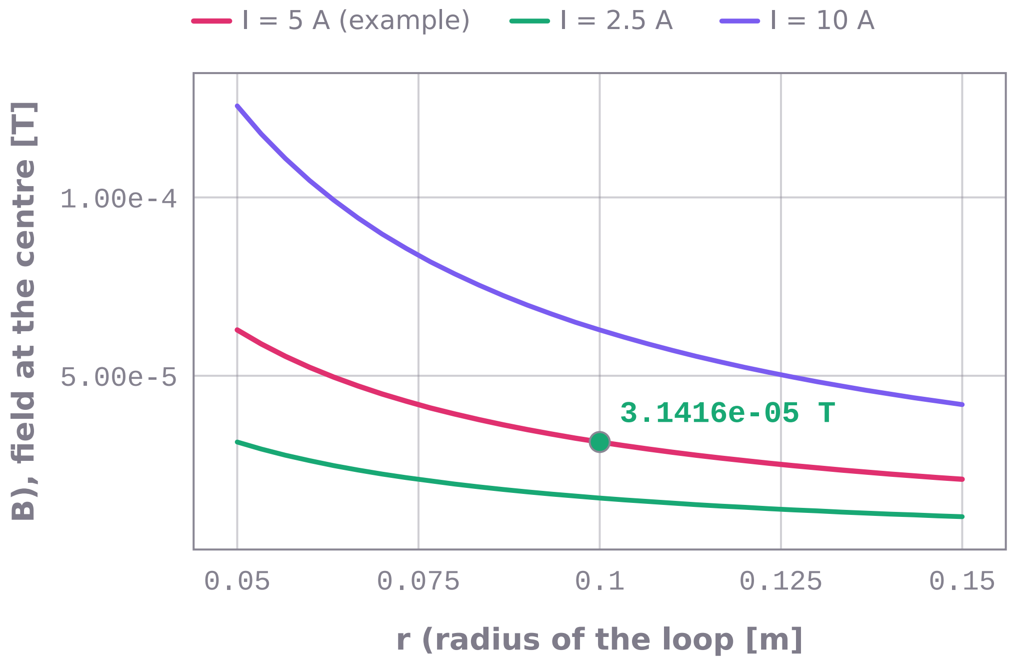
input	meaning	kind / role	unit	default
`B`	field at the centre	magnetic_field / plain	`T`	
`I`		current / plain	`A`	
`r`	radius of the loop	length / radius	`m`	

input	meaning	kind / role	unit	default
`mu0`		permeability / constant	`T*m/A`	1.25663706212e-06 T*m/A

- **Conditions:** loop
- **Applies when:** the centre of a circular current loop
- **Fails when:** points off the centre
- **Assumes:** a single circular loop; vacuum or air
- **Evidence:** fundamental_law



Circular loops of radius 5, 10 and 20 cm each carrying I = 5 A anticlockwise: the field at the centre points out of the page (⊙). Marked: the field strength at the centre the package returns, 6.2832e-05 T for r = 0.05 m, 3.1416e-05 T for r = 0.1 m, 1.5708e-05 T for r = 0.2 m, halving the radius doubles it.



B (field at the centre) against r (radius of the loop) from 0.05 m to 0.15 m. One curve for each value of I (Current): 5 A, 2.5 A, 10 A. Other inputs as in the worked example: mu0 = 1.25663706212e-06 T*m/A. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.1 m, B = 3.1416e-05 T.

```
classical_physics_ai999x.solve('loop_center_field', I='5 A', r='0.1 m', target='B')
# -> 3.14159e-05 T   (SOLVED)
```

$$B = \frac{\mu_0 I}{2r}$$
$$B = \frac{I \mu_0}{2r} = 3.14159e-05\ \mathrm{T}$$

invalid call	outcome	message
without `I`	NEEDS_INPUT	loop_center_field needs I (current, A). Example: solve('loop_center_field', I='... A', r='... m', mu0='... T*m/A').
`I` in a wrong unit	INVALID_INPUT	I is a current: its unit must be like A, not 'kg'.

Questions in words that work:

- "A circular loop of wire with a radius of 0.1 m carries a current of 5 A. What is the magnetic field at its centre?" → 3.14159e-05 T
- "A loop of radius 5 cm carries a current of 8 A. What magnetic field does it produce at the centre?" → 0.000100531 T

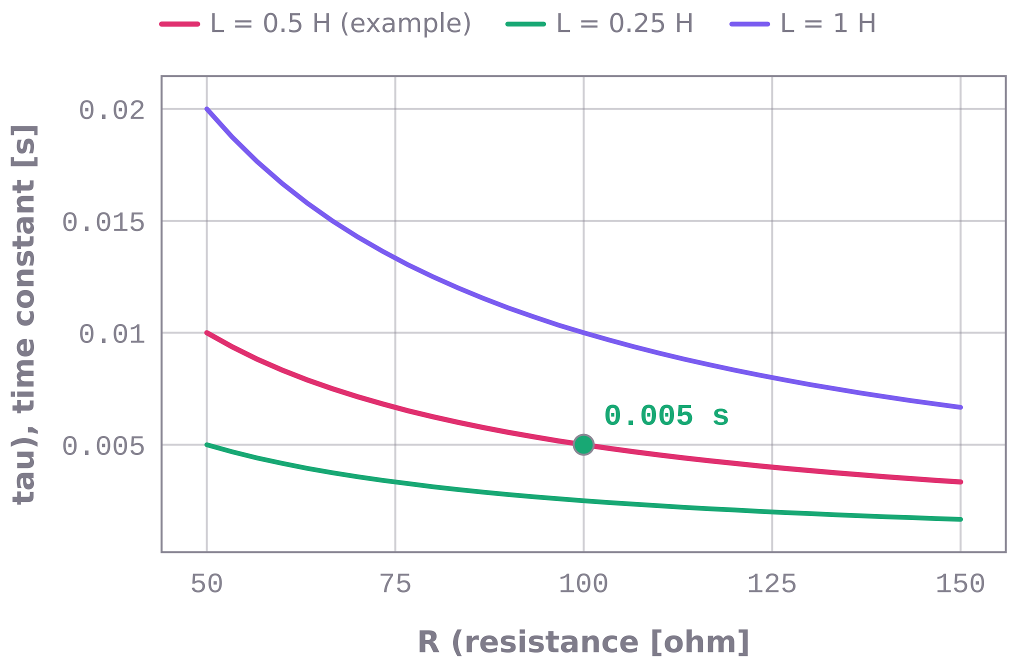
Time constant of an RL circuit, `rl_time_constant`

$$\tau = \frac{L}{R}$$

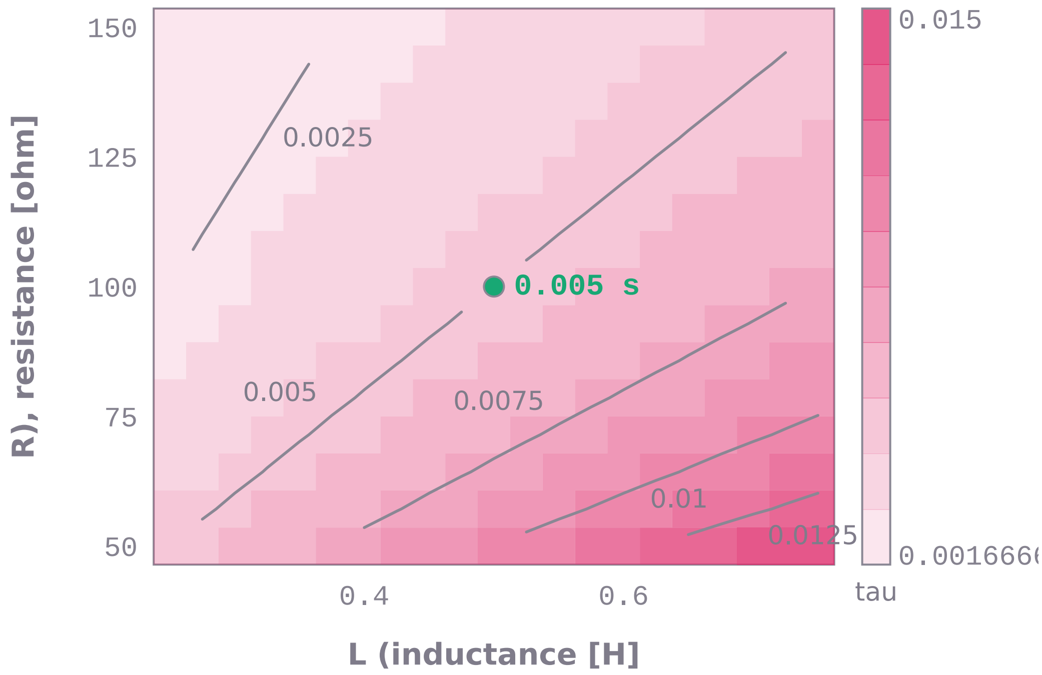
input	meaning	kind / role	unit	default
`tau`	time constant	time / time_constant	`s`	

input	meaning	kind / role	unit	default
`L`	inductance	inductance / plain	`H`	
`R`	resistance	resistance / plain	`ohm`	

- **Conditions:** inductor
- **Applies when:** an RL circuit switched on or off
- **Fails when:** other elements present
- **Assumes:** a single inductor and resistor
- **Evidence:** definition



tau (time constant) against R (resistance) from 50 ohm to 150 ohm. One curve for each value of L (inductance): 0.5 H, 0.25 H, 1 H. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 100 ohm, tau = 0.005 s.



tau (time constant) over L (inductance) from 0.25 H to 0.75 H and R (resistance) from 50 ohm to 150 ohm: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0.00166667 s to dark 0.015 s); iso-lines at tau = 0.0025, 0.005, 0.0075, 0.01, 0.0125 s. Marked: the worked example, L = 0.5 H, R = 100 ohm, tau = 0.005 s.

```
classical_physics_ai999x.solve('rl_time_constant', L='0.5 H', R='100 ohm', target='tau')
# -> 0.005 s   (SOLVED)
```

$$\tau = \frac{L}{R}$$

$$\tau = \frac{L}{R} = 0.005 \text{ s}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	rl_time_constant needs L (inductance, H). Example: solve('rl_time_constant', L='... H', R='... ohm').
`L` in a wrong unit	INVALID_INPUT	L is a inductance: its unit must be like H, not 'kg'.

Questions in words that work:

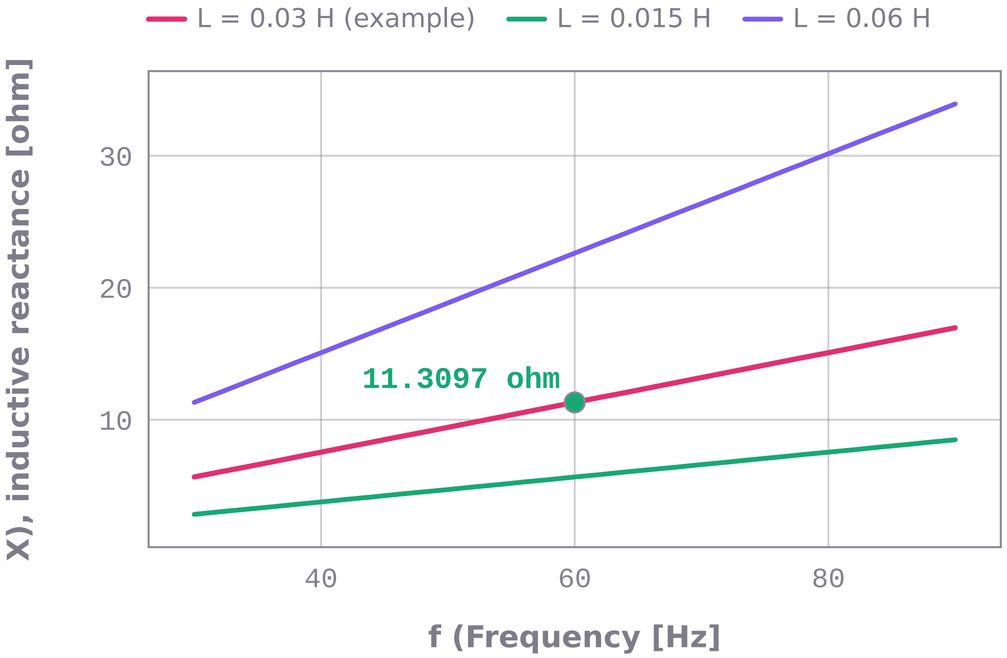
- "An inductor of inductance 0.5 H is connected in series with a 100 ohm resistor. What is the time constant?" → 0.005 s
- "You want a time constant of 1 s using an inductor and a 500 ohm resistor. What inductance is needed?" → 500 H

Reactance of an inductor, `inductive_reactance`

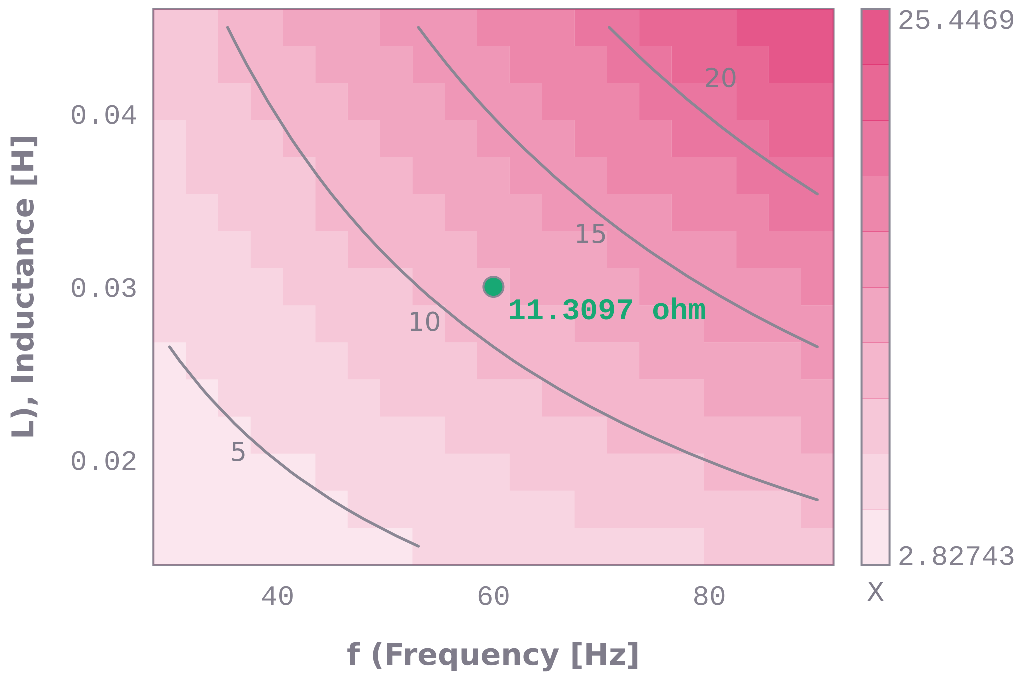
$$X = 2\pi f L$$

input	meaning	kind / role	unit	default
`X`	inductive reactance	resistance / reactance	`ohm`	
`f`		frequency / plain	`Hz`	
`L`		inductance / plain	`H`	

- **Conditions:** inductor
- **Not when:** capacitor
- **Applies when:** an inductor in an AC circuit
- **Fails when:** DC
- **Assumes:** sinusoidal AC
- **Evidence:** definition



X (inductive reactance) against f (Frequency) from 30 Hz to 90 Hz. One curve for each value of L (Inductance): 0.03 H, 0.015 H, 0.06 H. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, f = 60 Hz, X = 11.3097 ohm.



X (inductive reactance) over f (Frequency) from 30 Hz to 90 Hz and L (Inductance) from 0.015 H to 0.045 H: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 2.82743 ohm to dark 25.4469 ohm); iso-lines at X = 5, 10, 15, 20 ohm. Marked: the worked example, f = 60 Hz, L = 0.03 H, X = 11.3097 ohm.

```
classical_physics_ai999x.solve('inductive_reactance', f='60 Hz', L='0.03 H', target='X')
# -> 11.3097 ohm    (SOLVED)
```

$$X = 2\pi f L$$
$$X = 2\pi L f = 11.3097\ \mathrm{ohm}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	inductive_reactance needs f (frequency, Hz). Example: solve('inductive_reactance', f='... Hz', L='... H').
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Questions in words that work:

- "An inductor of inductance 0.03 H is in an AC circuit at a frequency of 60 Hz. What is its reactance?" → 11.3097 ohm
- "At what frequency will an inductor of inductance 30 mH have a reactance of 100 ohm?" → 530.516 Hz

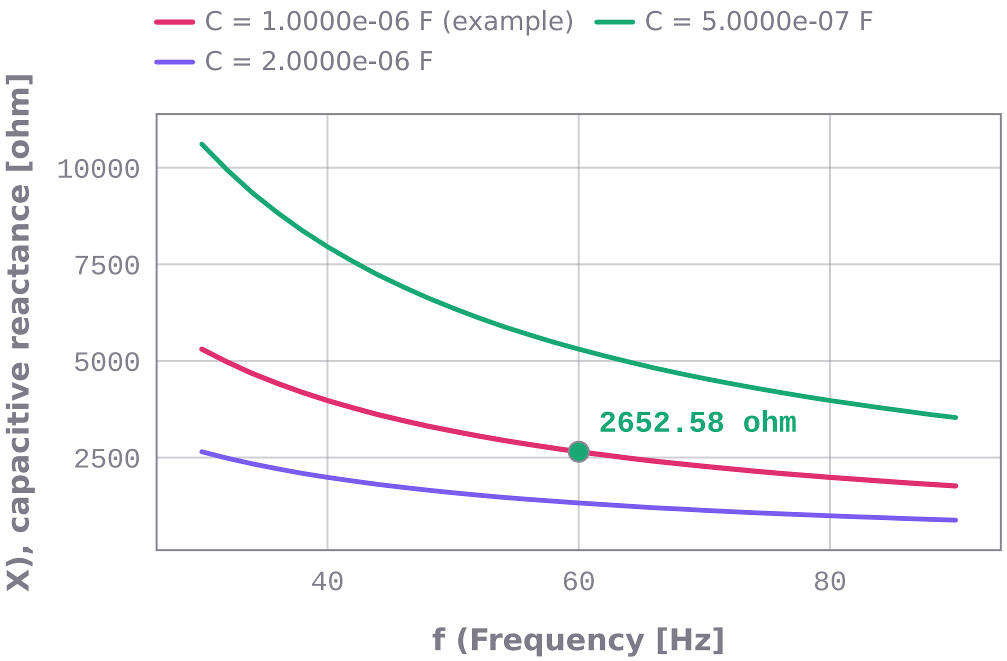
Reactance of a capacitor, `capacitive_reactance`

$$X = \frac{1}{2\pi f C}$$

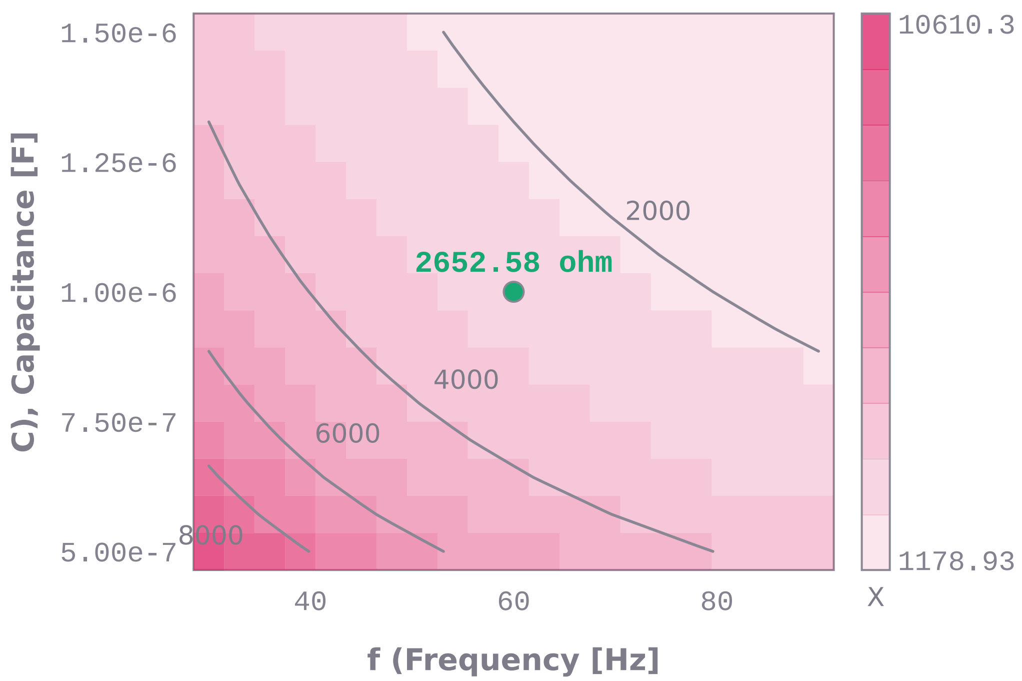
input	meaning	kind / role	unit	default
`X`	capacitive reactance	resistance / reactance	`ohm`	
`f`		frequency / plain	`Hz`	

input	meaning	kind / role	unit	default
`C`		capacitance / plain	`F`	

- **Conditions:** capacitor
- **Not when:** inductor
- **Applies when:** a capacitor in an AC circuit
- **Fails when:** DC
- **Assumes:** sinusoidal AC
- **Evidence:** definition



X (capacitive reactance) against f (Frequency) from 30 Hz to 90 Hz. One curve for each value of C (Capacitance): 1.0000e-06 F, 5.0000e-07 F, 2.0000e-06 F. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, f = 60 Hz, X = 2652.58 ohm.



X (capacitive reactance) over f (Frequency) from 30 Hz to 90 Hz and C (Capacitance) from 5.0000e-07 F to 1.5000e-06 F: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1178.93 ohm to dark 10610.3 ohm); iso-lines at X = 2000, 4000, 6000, 8000 ohm. Marked: the worked example, f = 60 Hz, C = 1.0000e-06 F, X = 2652.58 ohm.

```
classical_physics_ai999x.solve('capacitive_reactance', f='60 Hz', C='1e-06 F', target='X')
# -> 2652.58 ohm   (SOLVED)
```

$$X = \frac{1}{2\pi f C}$$

$$X = \frac{1}{2\pi C f} = 2652.58\ \mathrm{ohm}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	capacitive_reactance needs f (frequency, Hz). Example: solve('capacitive_reactance', f='... Hz', C='... F').
`f` in a wrong unit	INVALID_INPUT	f is a frequency: its unit must be like Hz, not 'kg'.

Questions in words that work:

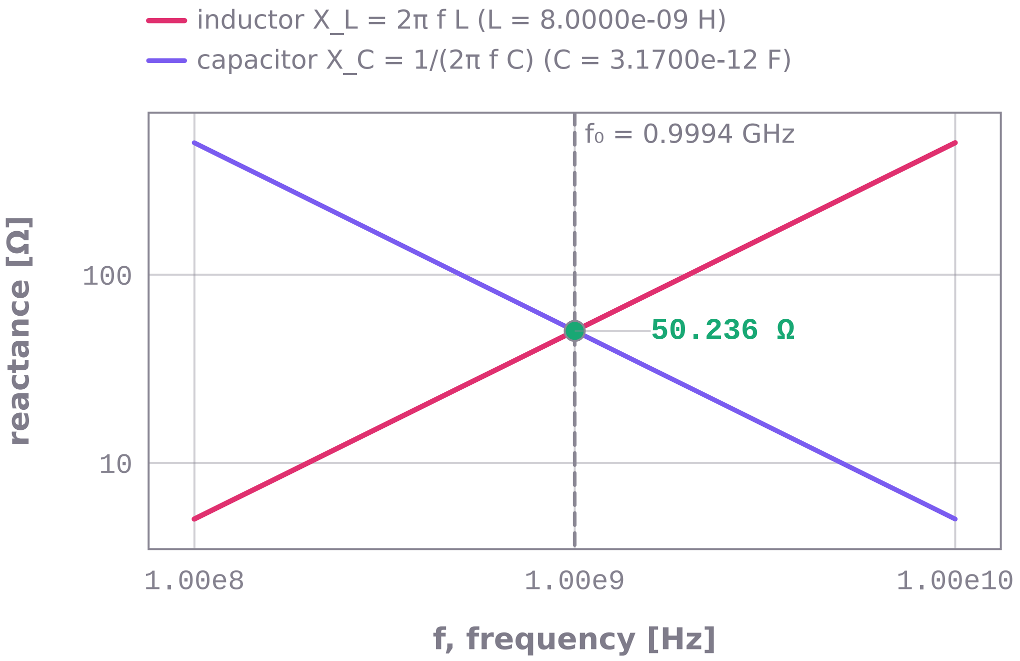
- "A capacitor with a capacitance of 1 uF is in an AC circuit at a frequency of 60 Hz. What is its reactance?" → 2652.58 ohm
- "What capacitance should a capacitor have to give a reactance of 2e6 ohm at 60 Hz?" → 1.32629e-09 F

Resonant frequency of an LC circuit, `lc_resonance`

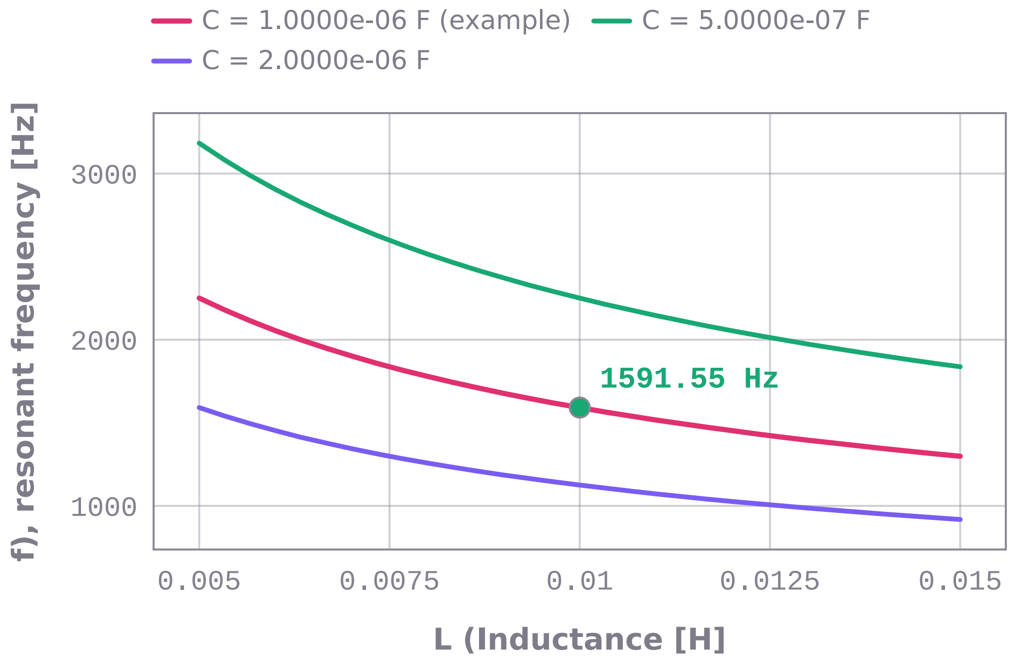
$$f = \frac{1}{2\pi\sqrt{LC}}$$

input	meaning	kind / role	unit	default
`f`	resonant frequency	frequency / resonant	`Hz`	
`L`		inductance / plain	`H`	
`C`		capacitance / plain	`F`	

- **Conditions:** lc_circuit
- **Applies when:** LC and series RLC circuits
- **Fails when:** strong damping
- **Assumes:** ideal components; negligible resistance for the frequency
- **Evidence:** fundamental_law



Reactance of an inductor L = 8.0000e-09 H and a capacitor C = 3.1700e-12 F against frequency over two decades (21 frequencies each from the package's reactance formulas): they are equal where the LC circuit resonates. Marked: the resonant frequency the package returns, f_0 = 9.9941e+08 Hz, where both reactances are 50.236 Ω.



f (resonant frequency) against L (Inductance) from 0.005 H to 0.015 H. One curve for each value of C (Capacitance): 1.0000e-06 F, 5.0000e-07 F, 2.0000e-06 F. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, L = 0.01 H, f = 1591.55 Hz.

```
classical_physics_ai999x.solve('lc_resonance', L='8e-09 H', C='3.17e-12 F', target='f')
# -> 9.99414e+08 Hz   (SOLVED)
```

$$f = \frac{1}{2\pi\sqrt{LC}}$$
$$f = \frac{1}{2\pi\sqrt{C}\sqrt{L}} = 9.99414e+08\ \mathrm{Hz}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	lc_resonance needs L (inductance, H). Example: solve('lc_resonance', L='... H', C='... F').
`L` in a wrong unit	INVALID_INPUT	L is a inductance: its unit must be like H, not 'kg'.

Questions in words that work:

- "A resonant circuit has an inductance of 0.01 H and a capacitance of 1 uF. What is the resonant frequency?" → 1591.55 Hz
- "What capacitance gives a resonant frequency of 1 GHz with an inductance of 8 nH in an LC circuit?" → 3.16629e-12 F

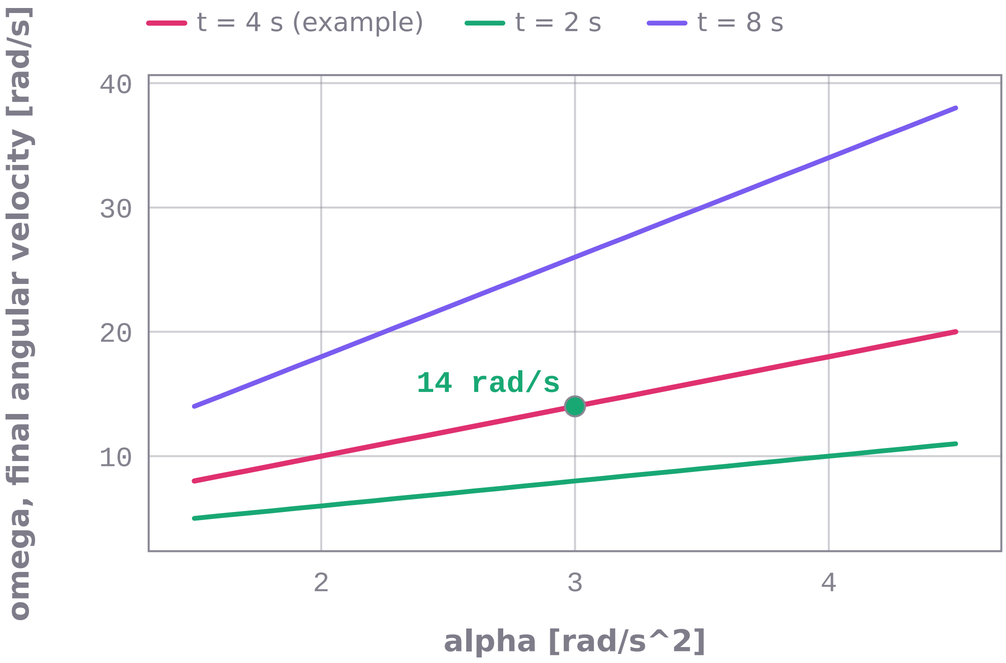
Angular velocity under constant angular acceleration, `angular_velocity_time`

$$\omega = \omega_0 + \alpha t$$

input	meaning	kind / role	unit	default
`omega`	final angular velocity	angular_frequency / final	`rad/s`	

input	meaning	kind / role	unit	default
`omega0`	initial angular velocity	angular_frequency / initial	`rad/s`	
`alpha`	constant angular acceleration	angular_acceleration / plain	`rad/s^2`	
`t`	elapsed time	time / plain	`s`	

- **Conditions:** rotating
- **Applies when:** a body spins up or down about a fixed axis
- **Fails when:** the angular acceleration varies
- **Assumes:** constant angular acceleration; a fixed axis
- **Evidence:** definition



omega (final angular velocity) against alpha (constant angular acceleration) from 1.5 rad/s^2 to 4.5 rad/s^2. One curve for each value of t (elapsed time): 4 s, 2 s, 8 s. Other inputs as in the worked example: omega0 = 2.0 rad/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, alpha = 3 rad/s^2, omega = 14 rad/s.

```
classical_physics_ai999x.solve('angular_velocity_time', omega0='2 rad/s', alpha='3
rad/s^2', t='4 s', target='omega')
# -> 14 rad/s   (SOLVED)
```

$$\omega = \omega_0 + \alpha t$$
$$\omega = \alpha t + \omega_0 = 14\,\mathrm{rad/s}$$

invalid call	outcome	message
without `omega0`	NEEDS_INPUT	angular_velocity_time needs omega0 (initial angular velocity, rad/s). Example: solve('angular_velocity_time', omega0='... rad/s', alpha='...
`omega0` in a wrong unit	INVALID_INPUT	omega0 is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

- "A flywheel rotating at an initial angular velocity of 2 rad/s has an angular acceleration of 3 rad/s^2 for 4 s. What is its final angular velocity?" → 14 rad/s

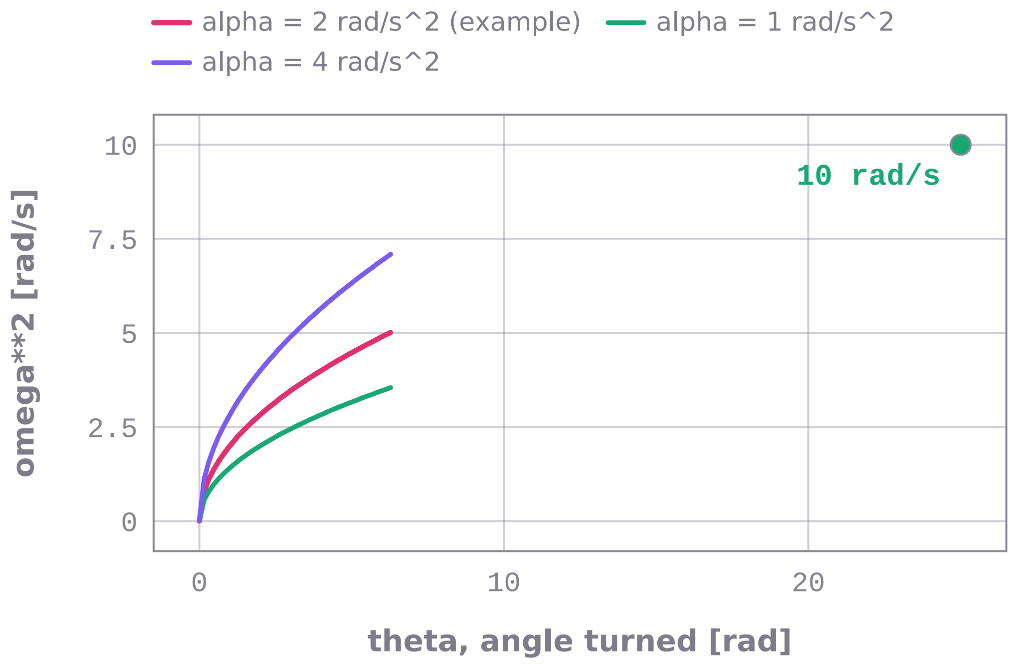
- "A turntable starts from rest and spins up with an angular acceleration of 0.5 rad/s^2. What is its final angular velocity after 6 s?" → 3 rad/s

Angular velocity and angle turned under constant angular acceleration, `angular_velocity_angle`

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$

input	meaning	kind / role	unit	default
`omega`	final angular speed	angular_frequency / final	`rad/s`	
`omega0`	initial angular speed	angular_frequency / initial	`rad/s`	
`alpha`	constant angular acceleration	angular_acceleration / plain	`rad/s^2`	
`theta`	angle turned	angle / angular_displacement	`rad`	

- **Conditions:** rotating
- **Applies when:** a body spins up or down about a fixed axis
- **Fails when:** the angular acceleration varies
- **Assumes:** constant angular acceleration; a fixed axis
- **Evidence:** fundamental_law



omega**2 (Angular velocity and angle turned under constant angular acceleration) against theta (angle turned) from 0 rad to 6.28319 rad. One curve for each value of alpha (constant angular acceleration): 2 rad/s^2, 1 rad/s^2, 4 rad/s^2. Other inputs as in the worked example: omega0 = 0.0 rad/s. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 25 rad, omega**2 = 10 rad/s.

```
classical_physics_ai999x.solve('angular_velocity_angle', omega0='0 rad/s', alpha='2
rad/s^2', theta='25 rad', target='omega')
# -> 10 rad/s   (SOLVED)
```

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$
$$\omega = -\sqrt{2\alpha\theta + \omega_0^2} = 10\ \mathrm{rad/s}$$

invalid call	outcome	message
without `omega0`	NEEDS_INPUT	angular_velocity_angle needs omega0 (initial angular speed, rad/s). Example: solve('angular_velocity_angle', omega0='... rad/s', alpha='...
`omega0` in a wrong unit	INVALID_INPUT	omega0 is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

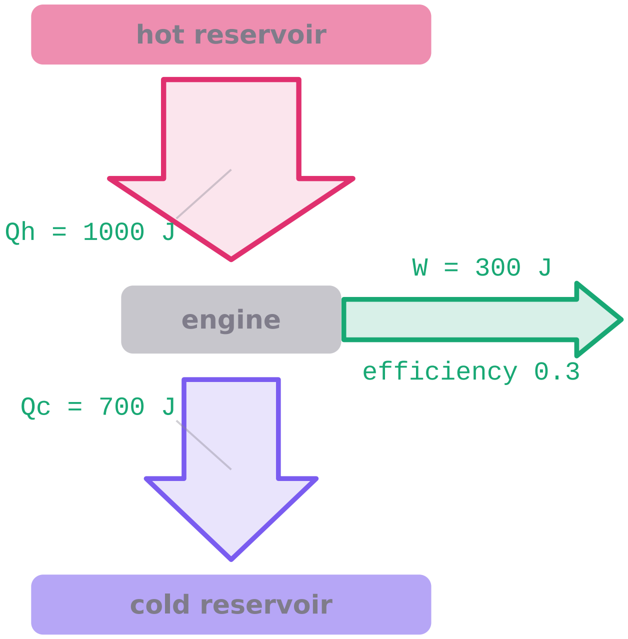
- "A flywheel starts from rest and spins up with an angular acceleration of 2 rad/s^2 through an angular displacement of 25 rad. What is its final angular velocity?" → 10 rad/s
- "A wheel rotating with an initial angular velocity of 4 rad/s has an angular acceleration of 1.5 rad/s^2 over an angular displacement of 10 rad. What is its final angular velocity?" → 6.78233 rad/s

Energy balance of a cyclic heat engine, `engine_energy_balance`

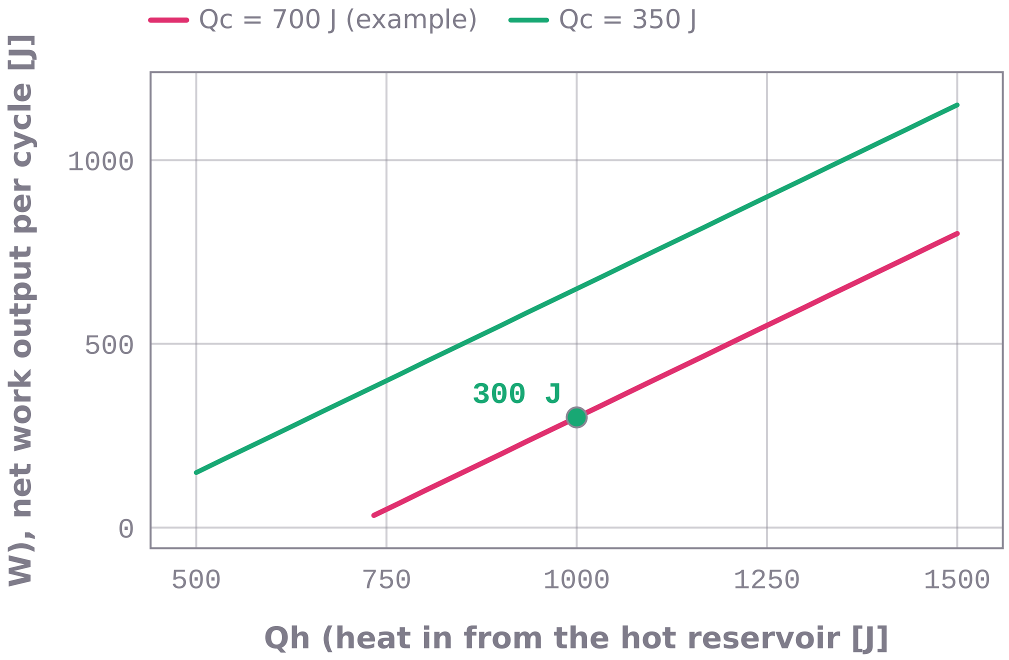
$$W = Q_h - Q_c$$

input	meaning	kind / role	unit	default
`W`	net work output per cycle	energy / work	`J`	
`Qh`	heat in from the hot reservoir	energy / heat_hot	`J`	
`Qc`	heat out to the cold reservoir	energy / heat_cold	`J`	

- **Conditions:** heat_engine
- **Applies when:** a heat engine over whole cycles
- **Fails when:** a single non-cyclic process
- **Assumes:** a cyclic engine (no net change in internal energy)
- **Evidence:** fundamental_law


hot reservoir
Qh = 1000 J
W = 300 J
engine
efficiency 0.3
Qc = 700 J
cold reservoir

Energy flow through a heat engine that takes Qh = 1000 J from a hot reservoir and rejects Qc = 700 J to a cold one; arrow widths to scale. Marked: the work the package returns, W = Qh − Qc = 300 J, and the efficiency W/Qh = 0.3 from its efficiency formula.



W (net work output per cycle) against Qh (heat in from the hot reservoir) from 500 J to 1500 J. One curve for each value of Qc (heat out to the cold reservoir): 700 J, 350 J. 24 points per curve computed by classical-physics-ai999x; marked: the worked example, Qh = 1000 J, W = 300 J.

```
classical_physics_ai999x.solve('engine_energy_balance', Qh='1000 J', Qc='700 J',
target='W')
# -> 300 J   (SOLVED)
```

$$W = Q_h - Q_c$$
$$W = -Q_c + Q_h = 300\ \mathrm{J}$$

invalid call	outcome	message
without `Qh`	NEEDS_INPUT	engine_energy_balance needs Qh (heat in from the hot reservoir, J). Example: solve('engine_energy_balance', Qh='... J', Qc='... J').
`Qh` in a wrong unit	INVALID_INPUT	Qh is a energy: its unit must be like J, not 'kg'.

Questions in words that work:

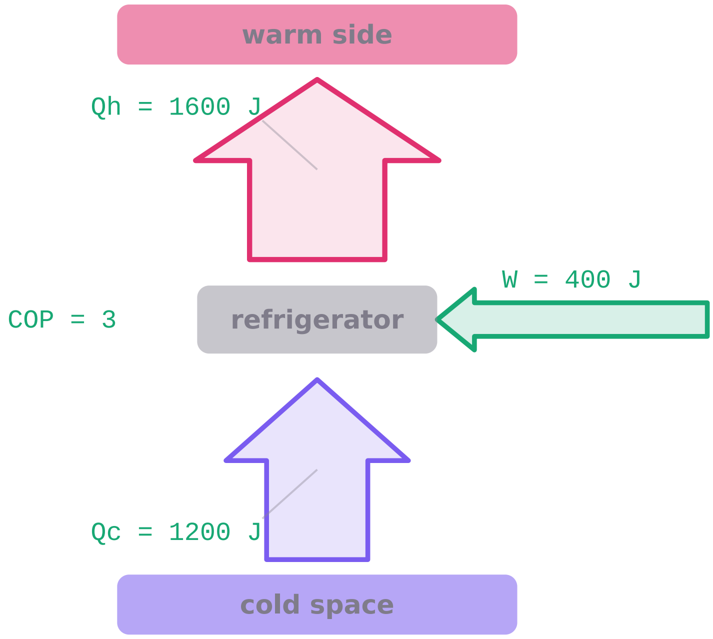
- "A heat engine takes in a heat input of 1000 J and has 700 J of heat rejected per cycle. What is its work output?" → 300 J
- "A power plant engine has a heat input of 6e5 J and a work output of 2.5e5 J. How much heat is rejected?" → 350000 J

Coefficient of performance of a refrigerator, `cop_refrigerator`

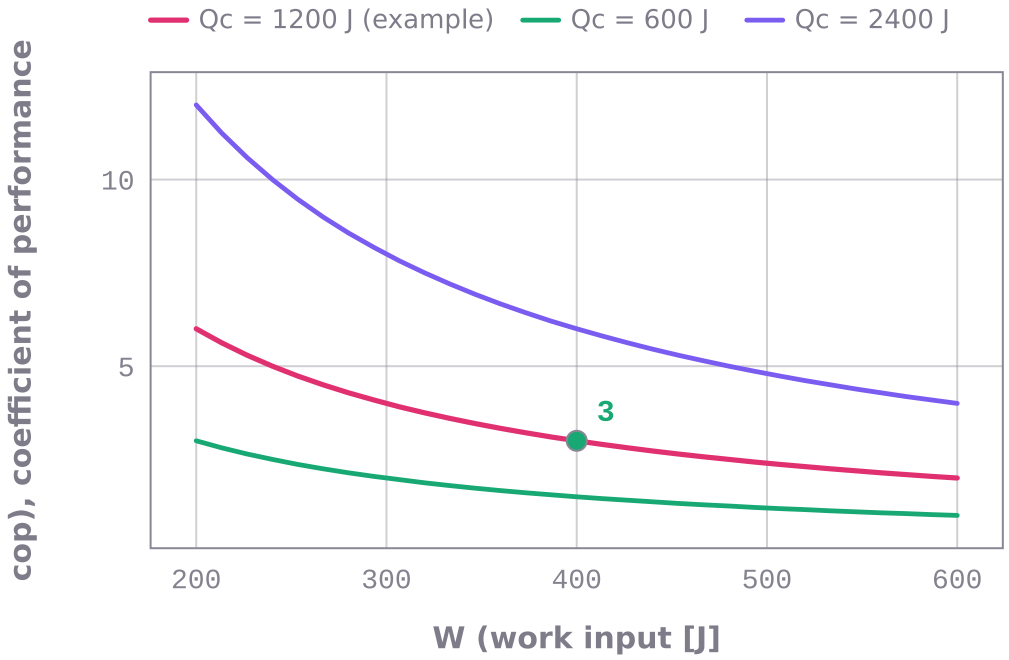
$$\mathrm{cop} = \frac{Q_c}{W}$$

input	meaning	kind / role	unit	default
`cop`	coefficient of performance	efficiency / cop	`1`	
`Qc`	heat removed from the cold space	energy / heat_cold	`J`	
`W`	work input	energy / work	`J`	

- **Conditions:** refrigerator
- **Not when:** heat_pump
- **Applies when:** refrigerators and air conditioners
- **Fails when:** a heat pump (heat delivered is the benefit)
- **Assumes:** a cyclic refrigerator
- **Evidence:** definition



A refrigerator moving Qc = 1200 J out of the cold space with W = 400 J of work; it rejects Qh = Qc + W = 1600 J (derived) to the room. Marked: the coefficient of performance the package returns, COP = Qc/W = 3. Arrow widths to scale.



cop (coefficient of performance) against W (work input) from 200 J to 600 J. One curve for each value of Qc (heat removed from the cold space): 1200 J, 600 J, 2400 J. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, W = 400 J, cop = 3.

```
classical_physics_ai999x.solve('cop_refrigerator', Qc='1200 J', W='400 J', target='cop')
# -> 3 1   (SOLVED)
```

$$\mathrm{cop} = \frac{Q_c}{W}$$

$$\mathrm{cop} = \frac{Q_c}{W} = 3\,1$$

invalid call	outcome	message
without `Qc`	NEEDS_INPUT	cop_refrigerator needs Qc (heat removed from the cold space, J). Example: solve('cop_refrigerator', Qc='... J', W='... J').
`Qc` in a wrong unit	INVALID_INPUT	Qc is a energy: its unit must be like J, not 'kg'.

Questions in words that work:

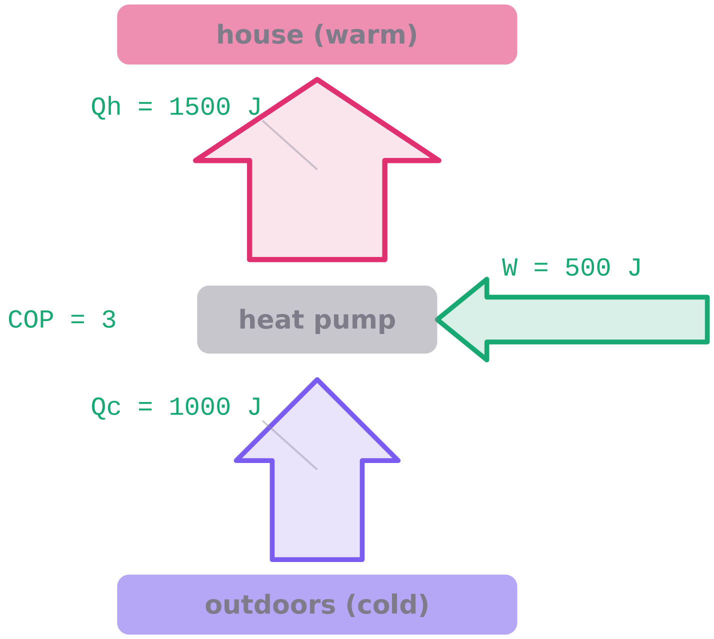
- "A refrigerator has 1200 J of heat removed using a work input of 400 J. What is its coefficient of performance?" → 3 1
- "A freezer with a coefficient of performance of 4 uses a work input of 250 J. How much heat is removed?" → 1000 J

Coefficient of performance of a heat pump, `cop_heat_pump`

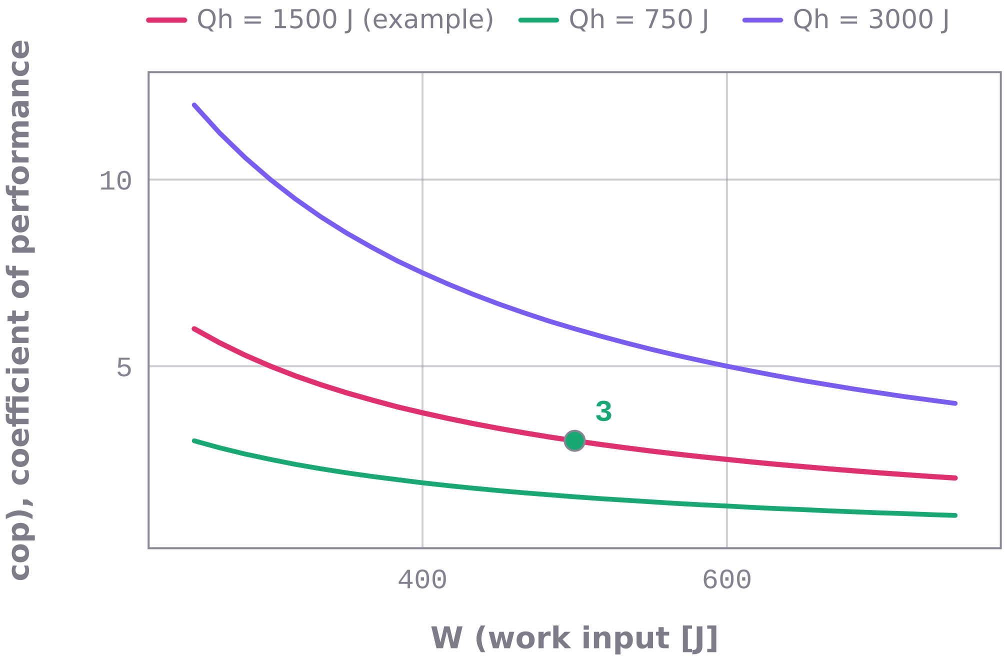
$$\mathrm{cop} = \frac{Q_h}{W}$$

input	meaning	kind / role	unit	default
`cop`	coefficient of performance	efficiency / cop	`1`	
`Qh`	heat delivered to the warm space	energy / heat_hot	`J`	
`W`	work input	energy / work	`J`	

- **Conditions:** heat_pump
- **Not when:** refrigerator
- **Applies when:** heat pumps heating a building
- **Fails when:** a refrigerator
- **Assumes:** a cyclic heat pump
- **Evidence:** definition


house (warm)
Qh = 1500 J
W = 500 J
COP = 3
heat pump
Qc = 1000 J
outdoors (cold)

A heat pump delivering Qh = 1500 J into a house with W = 500 J of work; it draws Qc = Qh − W = 1000 J (derived) from outdoors. Marked: the coefficient of performance the package returns, COP = Qh/W = 3. Arrow widths to scale.



cop (coefficient of performance) against W (work input) from 250 J to 750 J. One curve for each value of Qh (heat delivered to the warm space): 1500 J, 750 J, 3000 J. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, W = 500 J, cop = 3.

```
classical_physics_ai999x.solve('cop_heat_pump', Qh='1500 J', W='500 J', target='cop')
# -> 3 1   (SOLVED)
```

$$\mathrm{cop} = \frac{Q_h}{W}$$

$$\mathrm{cop} = \frac{Q_h}{W} = 3\,1$$

invalid call	outcome	message
without `Qh`	NEEDS_INPUT	cop_heat_pump needs Qh (heat delivered to the warm space, J). Example: solve('cop_heat_pump', Qh='... J', W='... J').
`Qh` in a wrong unit	INVALID_INPUT	Qh is a energy: its unit must be like J, not 'kg'.

Questions in words that work:

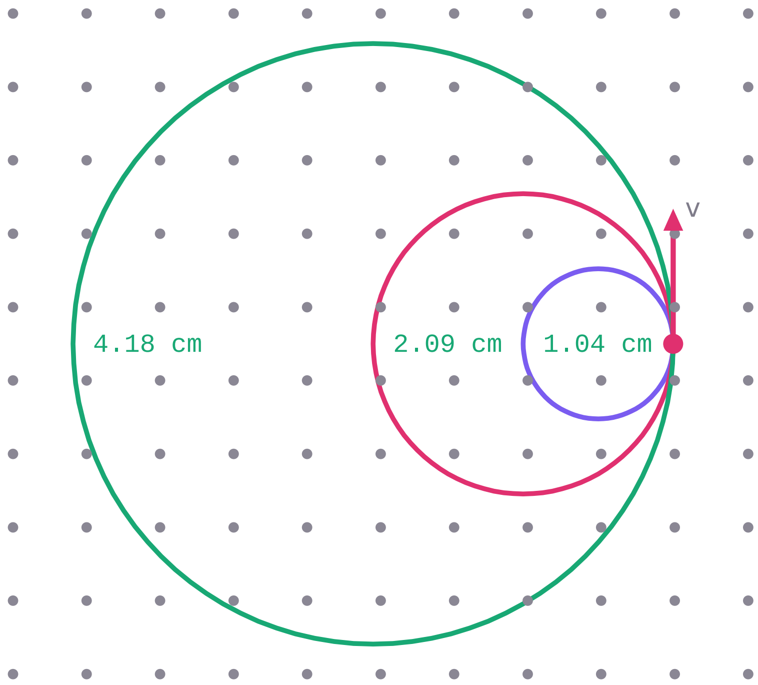
- "A heat pump delivers 1500 J of heat delivered for a work input of 500 J. What is its coefficient of performance?" → 3 1
- "A heat pump with a coefficient of performance of 3.5 uses a work input of 2000 J. How much heat is delivered?" → 7000 J

Radius of a charged particle's circle in a magnetic field, `cyclotron_radius`

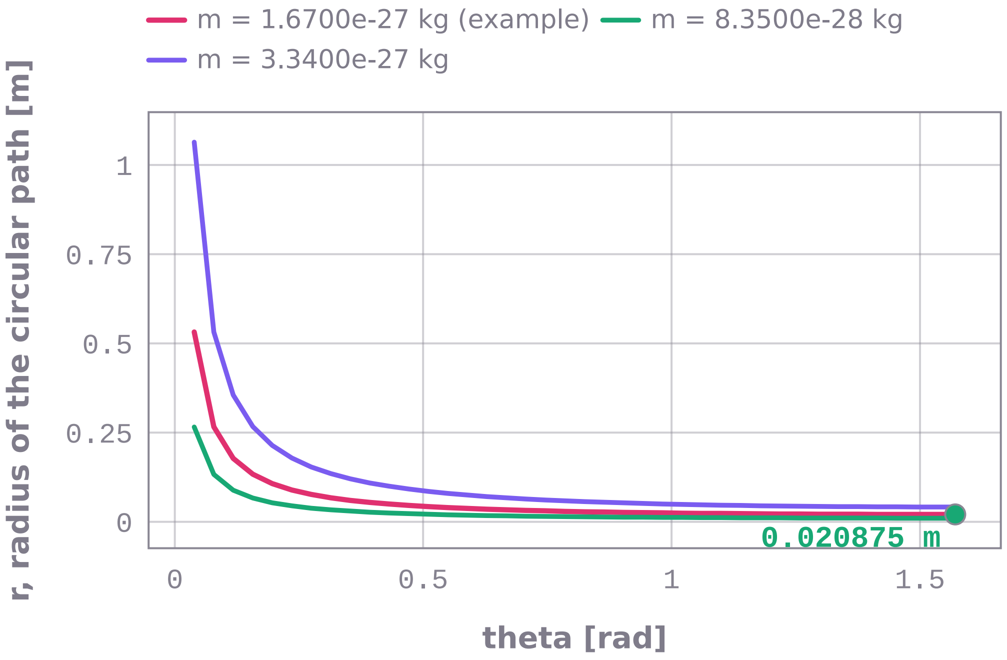
$$r = \frac{mv}{qB \sin\left(\theta\right)}$$

input	meaning	kind / role	unit	default
`r`	radius of the circular path	length / radius	`m`	
`m`	mass of the particle	mass / plain	`kg`	
`v`	speed perpendicular to the field	speed / plain	`m/s`	
`q`	magnitude of the charge	charge / plain	`C`	
`B`	field strength	magnetic_field / plain	`T`	
`theta`	angle between the velocity and the field	angle / plain	`rad`	

- **Conditions:** magnetic, charged_particle
- **Applies when:** a charged particle in a uniform magnetic field
- **Fails when:** relativistic speeds or a velocity component along the field
- **Assumes:** a uniform field; non-relativistic speed; the path is the circle traced by the perpendicular velocity component
- **Evidence:** derived



Circular paths of a charged particle (m = 1.6700e-27 kg, q = 1.6000e-19 C) launched upwards from the same point at 500000, 1e+06 (the worked example) and 2e+06 m/s across a uniform field B = 0.5 T out of the page (·), drawn to scale: the radius grows in proportion to the speed. Marked: the radii the package returns, 0.0104375 m, 0.020875 m, 0.04175 m.



r (radius of the circular path) against theta (angle between the velocity and the field) from 0 rad to 1.5708 rad. One curve for each value of m (mass of the particle): 1.6700e-27 kg, 8.3500e-28 kg, 3.3400e-27 kg. Other inputs as in the worked example: v = 1000000.0 m/s, q = 1.6e-19 C, B = 0.5 T. 40 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 1.5708 rad, r = 0.020875 m.

```
classical_physics_ai999x.solve('cyclotron_radius', m='1.67e-27 kg', v='1e+06 m/s', q='1.6e-19 C', B='0.5 T', theta='90 deg', target='r')
# -> 0.020875 m   (SOLVED)
```

$$r = \frac{mv}{qB\sin(\theta)}$$
$$r = \frac{mv}{Bq\sin(\theta)} = 0.020875\ \mathrm{m}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	cyclotron_radius needs m (mass of the particle, kg). Example: solve('cyclotron_radius', m='... kg', v='... m/s', q='... C', B='... T', theta
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

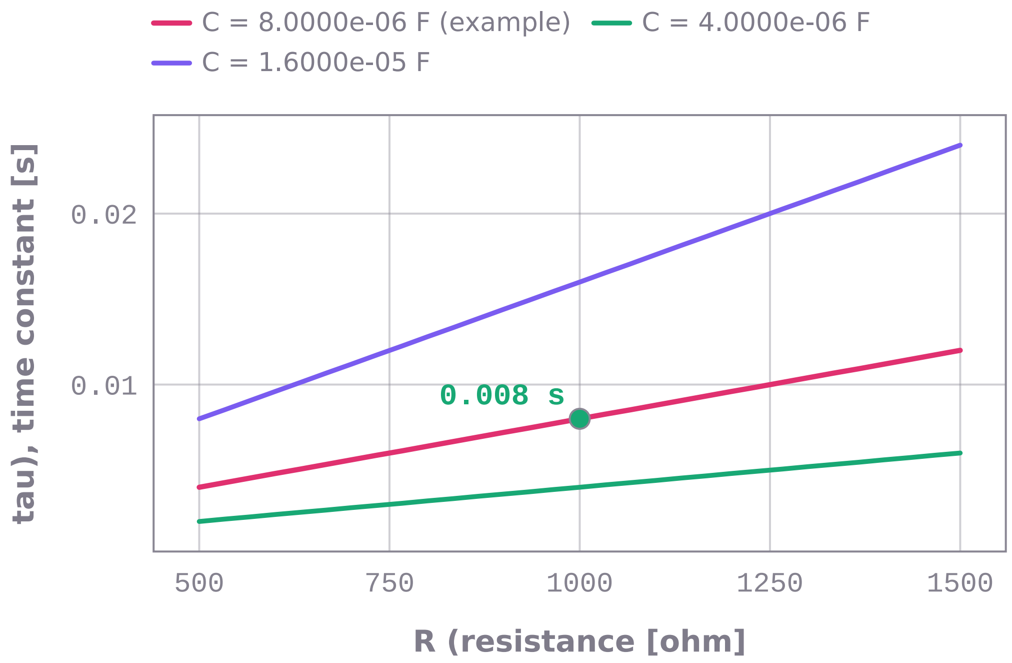
- "A proton of mass 1.67e-27 kg and charge 1.6e-19 C moves at 1e6 m/s perpendicular to a magnetic field of 0.5 T. What is the radius of the circular path?" → 0.020875 m
- "An electron of mass 9.11e-31 kg and charge 1.6e-19 C moves at 7.5e6 m/s perpendicular to a magnetic field of 1e-5 T. What is the radius of its path?" → 4.27031 m

Time constant of an RC circuit, `rc_time_constant`

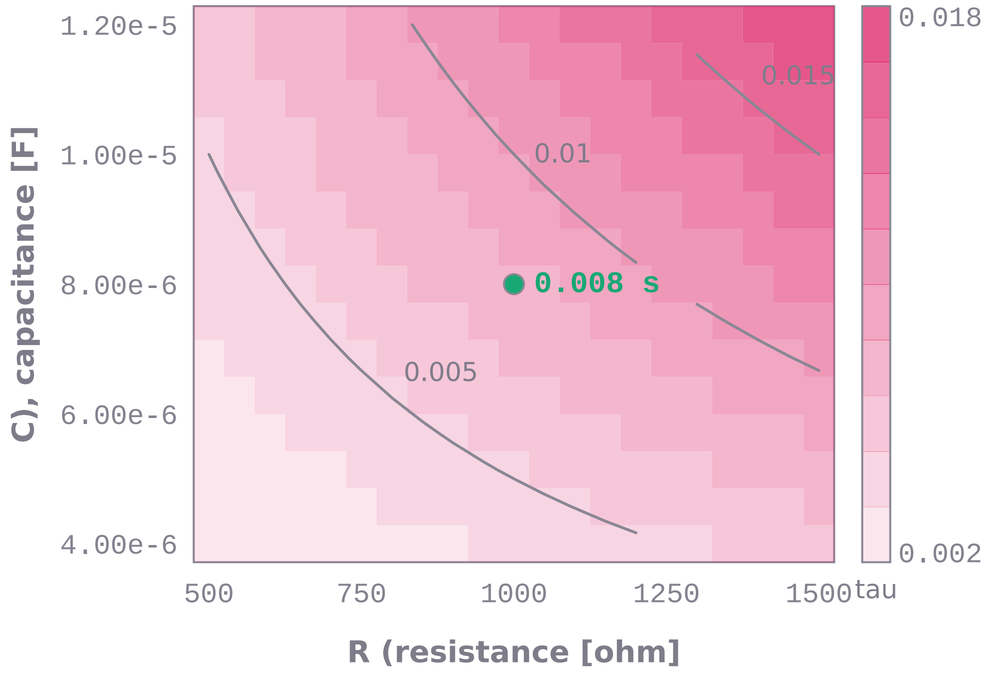
$$\tau = RC$$

input	meaning	kind / role	unit	default
`tau`	time constant	time / time_constant	`s`	
`R`	resistance	resistance / plain	`ohm`	
`C`	capacitance	capacitance / plain	`F`	

- **Conditions:** rc_circuit
- **Not when:** inductor
- **Applies when:** an RC circuit charging or discharging
- **Fails when:** other elements present
- **Assumes:** a single resistor and capacitor
- **Evidence:** definition



tau (time constant) against R (resistance) from 500 ohm to 1500 ohm. One curve for each value of C (capacitance): 8.0000e-06 F, 4.0000e-06 F, 1.6000e-05 F. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 1000 ohm, tau = 0.008 s.



tau (time constant) over R (resistance) from 500 ohm to 1500 ohm and C (capacitance) from 4.0000e-06 F to 1.2000e-05 F: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 0.002 s to dark 0.018 s); iso-lines at tau = 0.005, 0.01, 0.015 s. Marked: the worked example, R = 1000 ohm, C = 8.0000e-06 F, tau = 0.008 s.

```
classical_physics_ai999x.solve('rc_time_constant', R='1000 ohm', C='8e-06 F', target='tau')
# -> 0.008 s   (SOLVED)
```

$$\tau = RC$$
$$\tau = CR = 0.008\ \mathrm{s}$$

invalid call	outcome	message
without `R`	NEEDS_INPUT	rc_time_constant needs R (resistance, ohm). Example: solve('rc_time_constant', R='... ohm', C='... F').
`R` in a wrong unit	INVALID_INPUT	R is a resistance: its unit must be like ohm, not 'kg'.

Questions in words that work:

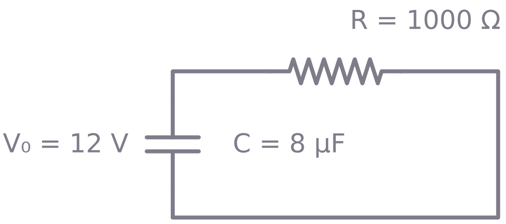
- "An RC circuit has a resistor of 1000 ohm and a capacitor of 8 uF. What is the time constant?" → 0.008 s
- "A capacitor of 4 uF discharges through a resistor in an RC circuit with a time constant of 2 ms. What is the resistance?" → 500 ohm

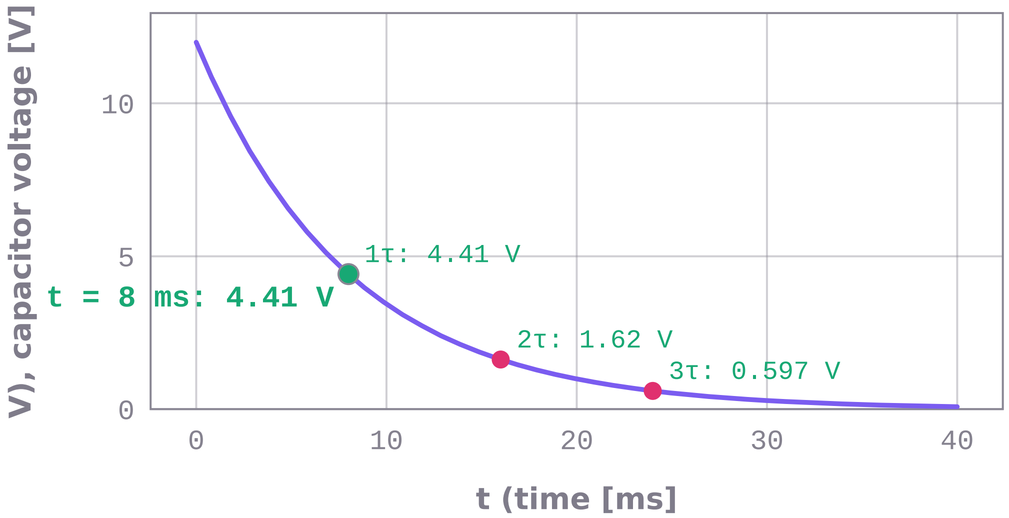
Voltage of a discharging capacitor, `rc_discharging`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	kind / role	unit	default
`V`	voltage across the capacitor at time t	voltage / capacitor	`V`	
`V0`	initial voltage across the capacitor	voltage / initial	`V`	
`t`	time since the discharge began	time / plain	`s`	
`R`	resistance	resistance / plain	`ohm`	
`C`	capacitance	capacitance / plain	`F`	

- **Conditions:** rc_circuit, discharging
- **Not when:** charging, inductor
- **Applies when:** a charged capacitor discharging through a resistor
- **Fails when:** a source keeps charging it
- **Assumes:** a single resistor and capacitor; the discharge starts at $t = 0$
- **Evidence:** fundamental_law





A capacitor C = 8.0000e-06 F charged to V_0 = 12 V discharging through R = 1000 Ω (sketch above), and its voltage against time, 40 times from the package (time shown in ms); time constant τ = RC = 0.008 s from its formula. Marked: 1τ: 4.41455 V, 2τ: 1.62402 V, 3τ: 0.597445 V, at t = 0.008 s: 4.41455 V, about 37 % is left after each τ.

```
classical_physics_ai999x.solve('rc_discharging', V0='12 V', t='0.008 s', R='1000 ohm',
C='8e-06 F', target='V')
# -> 4.41455 V   (SOLVED)
```

$$V = V_0 e^{\frac{-t}{RC}}$$

$$V = V_0 e^{-\frac{t}{CR}} = 4.41455\ \mathrm{V}$$

invalid call	outcome	message
without `V0`	NEEDS_INPUT	rc_discharging needs V0 (initial voltage across the capacitor, V). Example: solve('rc_discharging', V0='... V', t='... s', R='... ohm', C='.
`V0` in a wrong unit	INVALID_INPUT	V0 is a voltage: its unit must be like V, not 'kg'.

Questions in words that work:

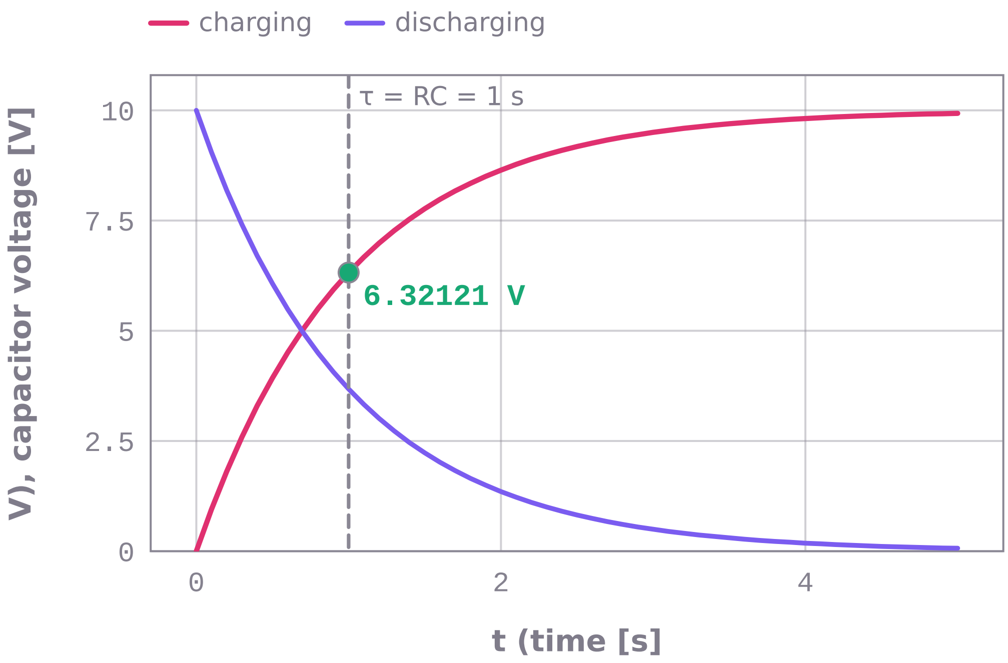
- "A capacitor of 8 uF with an initial voltage of 12 V discharges through a 1000 ohm resistor. What is the voltage across the capacitor after 8 ms?" → 4.41455 V
- "A 2 uF capacitor discharges through a 500 ohm resistor from an initial voltage of 10 V. What is the voltage across the capacitor after 2 ms?" → 1.35335 V

Voltage of a charging capacitor, `rc_charging`

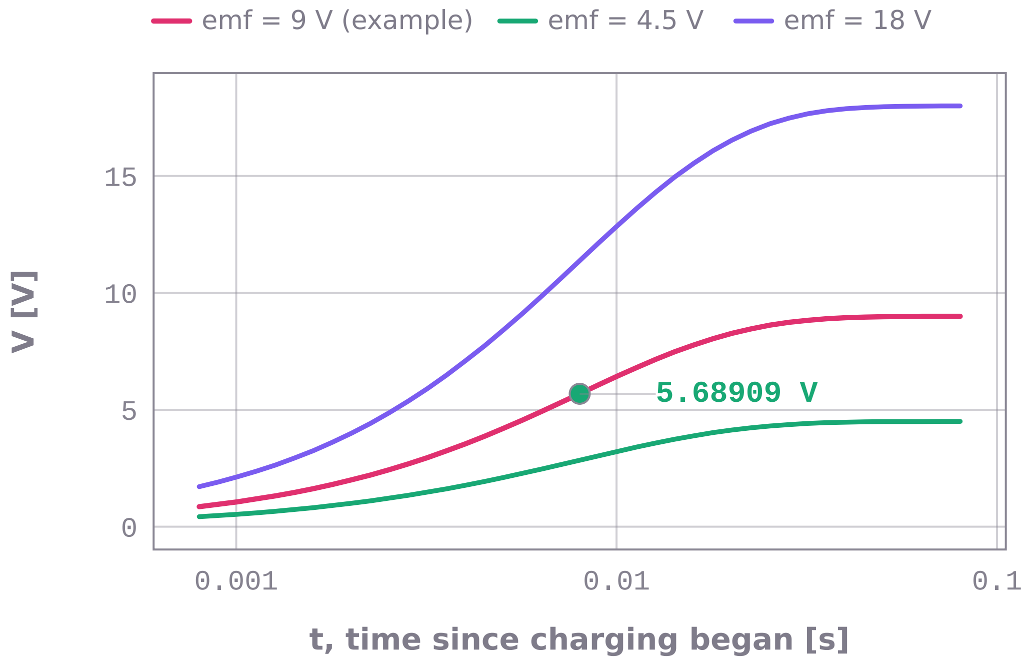
$$V = \mathrm{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	kind / role	unit	default
`V`	voltage across the capacitor at time t	voltage / capacitor	`V`	
`emf`	emf of the source	voltage / emf	`V`	
`t`	time since charging began	time / plain	`s`	
`R`	resistance	resistance / plain	`ohm`	
`C`	capacitance	capacitance / plain	`F`	

- **Conditions:** rc_circuit, charging
- **Not when:** discharging, inductor
- **Applies when:** an uncharged capacitor charged through a resistor by a constant emf
- **Fails when:** the capacitor starts charged
- **Assumes:** a single resistor and capacitor; the capacitor is uncharged at $t = 0$
- **Evidence:** fundamental_law



Capacitor voltage in an RC circuit (R = 1000 Ω, C = 0.001 F, source 10 V) charging from 0 and discharging from 10 V, 50 times each from the package; time constant τ = 1 s; marked: t = τ, charging voltage 6.32121 V (63.2121 % of the source).



V (voltage across the capacitor at time t) against t (time since charging began) from 0.0008 s to 0.08 s on a logarithmic axis. One curve for each value of emf (emf of the source): 9 V, 4.5 V, 18 V. Other inputs as in the worked example: R = 1000.0 ohm, C = 8e-06 F. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, t = 0.008 s, V = 5.68909 V.

```
classical_physics_ai999x.solve('rc_charging', emf='9 V', t='0.008 s', R='1000 ohm', C='8e-06 F', target='V')
# -> 5.68909 V   (SOLVED)
```

$$V = \mathrm{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

$$V = \mathrm{emf} - \mathrm{emf}e^{-\frac{t}{CR}} = 5.68909\ \mathrm{V}$$

invalid call	outcome	message
without `emf`	NEEDS_INPUT	rc_charging needs emf (emf of the source, V). Example: solve('rc_charging', emf='... V', t='... s', R='... ohm', C='... F').
`emf` in a wrong unit	INVALID_INPUT	emf is a voltage: its unit must be like V, not 'kg'.

Questions in words that work:

- "An uncharged 8 uF capacitor starts charging through a 1000 ohm resistor from a 9 V emf. What is the voltage across the capacitor after 8 ms?" → 5.68909 V
- "A 1 uF capacitor is charged through a 1500 ohm resistor by a 20 V emf. What is the voltage across the capacitor after 3 ms?" → 17.2933 V

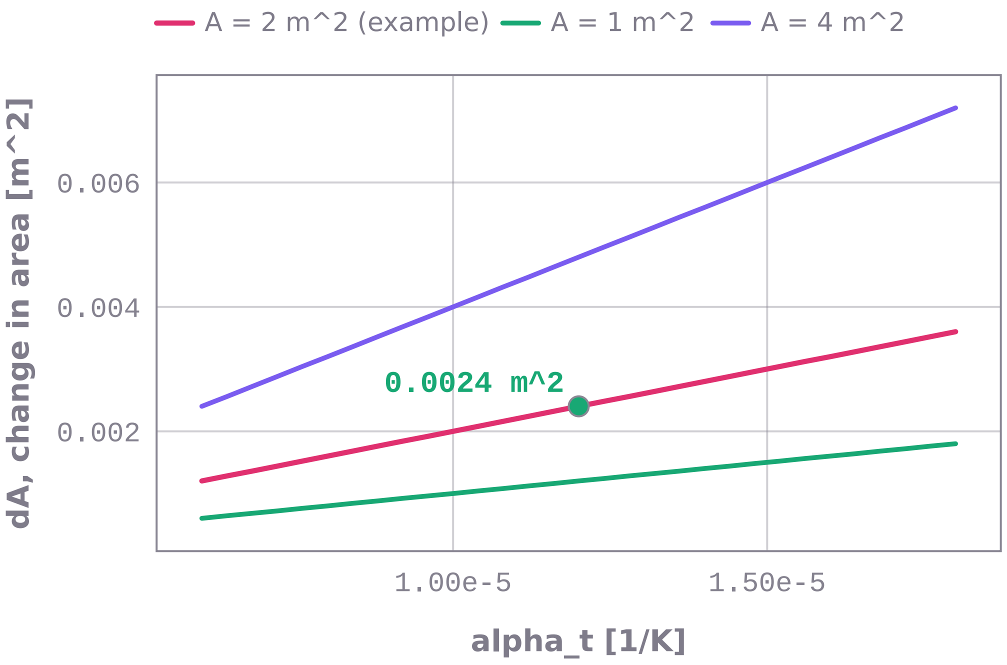
Thermal expansion of an area, `area_expansion`

$$d_A = 2\alpha_t A d_T$$

input	meaning	kind / role	unit	default
`dA`	change in area	area / expansion	`m^2`	

input	meaning	kind / role	unit	default
`alpha_t`	coefficient of linear expansion	expansion_coefficient / plain	`1/K`	
`A`	original area	area / original	`m^2`	
`dT`	change in temperature	temperature_difference / plain	`K`	

- **Conditions:** area_expansion
- **Applies when:** flat solids over moderate temperature ranges
- **Fails when:** phase changes; very large temperature ranges
- **Assumes:** a small temperature change; an isotropic material; the coefficient constant over the range
- **Evidence:** empirical



dA (change in area) against alpha_t (coefficient of linear expansion) from 6.0000e-06 1/K to 1.8000e-05 1/K. One curve for each value of A (original area): 2 m^2, 1 m^2, 4 m^2. Other inputs as in the worked example: dT = 50.0 K. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, alpha_t = 1.2000e-05 1/K, dA = 0.0024 m^2.

```
classical_physics_ai999x.solve('area_expansion', alpha_t='1.2e-05 1/K', A='2 m^2', dT='50
K', target='dA')
# -> 0.0024 m^2   (SOLVED)
```

$$d_A = 2\alpha_t A d_T$$
$$d_A = 2A\alpha_t d_T = 0.0024\ \mathrm{m}^2$$

invalid call	outcome	message
without `alpha_t`	NEEDS_INPUT	area_expansion needs alpha_t (coefficient of linear expansion, 1/K). Example: solve('area_expansion', alpha_t='... 1/K', A='... m^2', dT='..
`alpha_t` in a wrong unit	INVALID_INPUT	alpha_t is a expansion coefficient: its unit must be like 1/K, not 'kg'.

Questions in words that work:

- "A steel plate with an original area of 2.0 m^2 and a coefficient of linear expansion of 12e-6 1/K is heated by 50 K. What is the change in area?" → 0.0024 m^2

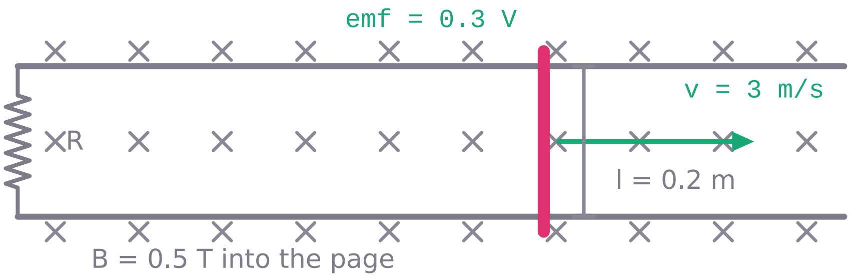
- "An aluminium sheet has an original area of 0.5 m^2 and a coefficient of linear expansion of 24e-6 1/K. By how much does its area increase when it warms by 100 K? What is the change in area?" → 0.0024 m^2

Emf of a rod moving through a magnetic field, `motional_emf`

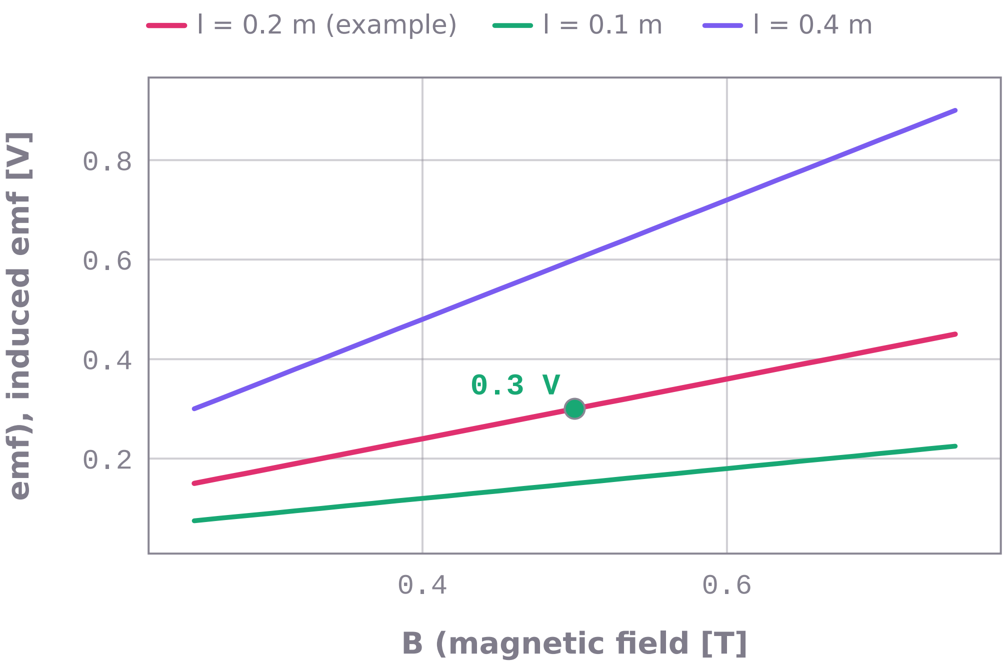
$$\text{emf} = Blv$$

input	meaning	kind / role	unit	default
`emf`	induced emf	voltage / emf	`V`	
`B`	magnetic field	magnetic_field / plain	`T`	
`l`	length of the rod	length / wire_length	`m`	
`v`	speed of the rod	speed / plain	`m/s`	

- **Conditions:** moving_rod
- **Applies when:** a straight rod moving through a uniform magnetic field
- **Fails when:** the motion is not perpendicular
- **Assumes:** B, the rod and its velocity mutually perpendicular; a uniform field
- **Evidence:** fundamental_law



A rod of length l = 0.2 m sliding at v = 3 m/s along two rails through a uniform field B = 0.5 T into the page; the rails are joined by a resistor, closing the circuit. Marked: the induced emf the package returns, ε = B l v = 0.3 V.



emf (induced emf) against B (magnetic field) from 0.25 T to 0.75 T. One curve for each value of l (length of the rod): 0.2 m, 0.1 m, 0.4 m. Other inputs as in the worked example: v = 3.0 m/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, B = 0.5 T, emf = 0.3 V.

```
classical_physics_ai999x.solve('motional_emf', B='0.5 T', l='0.2 m', v='3 m/s',
target='emf')
# -> 0.3 V   (SOLVED)
```

$$\mathrm{emf} = Blv$$
$$\mathrm{emf} = Blv = 0.3\ \mathrm{V}$$

invalid call	outcome	message
without `B`	NEEDS_INPUT	motional_emf needs B (magnetic field, T). Example: solve('motional_emf', B='... T', l='... m', v='... m/s').
`B` in a wrong unit	INVALID_INPUT	B is a magnetic field: its unit must be like T, not 'kg'.

Questions in words that work:

- "A rod of length 0.2 m slides along rails at 3 m/s through a magnetic field of 0.5 T. What is the motional emf?" → 0.3 V
- "A rod of length 0.5 m slides along rails through a magnetic field of 0.4 T and an emf of 1.2 V is induced. What is its speed?" → 6 m/s

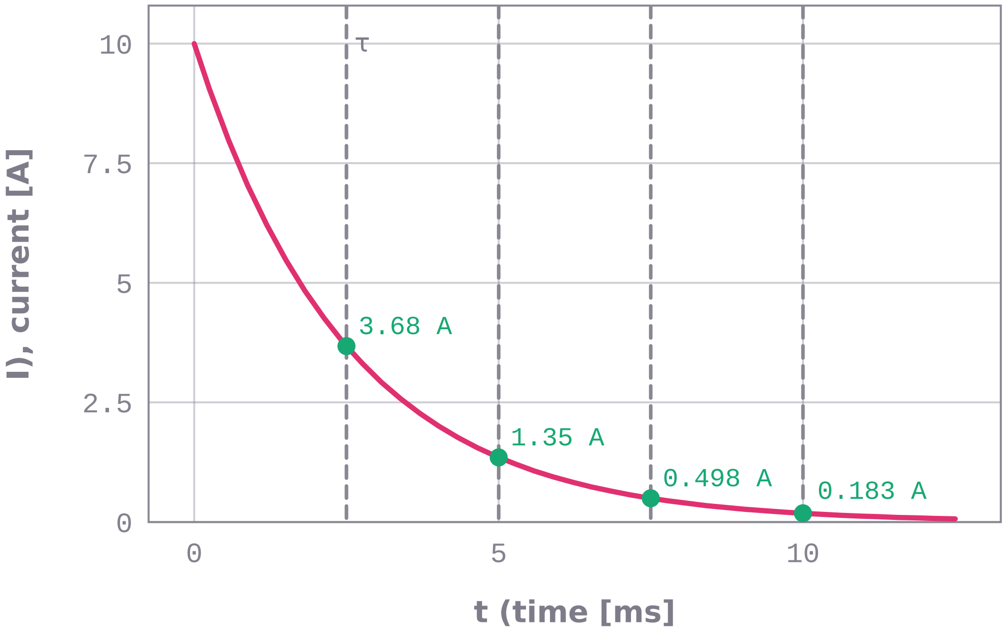
Current dying away in an RL circuit, `rl_current_decay`

$$I = I_0 e^{\frac{-t}{\tau}}$$

input	meaning	kind / role	unit	default
`I`	current at time t	current / plain	A	
`I0`	initial current	current / initial	A	

input	meaning	kind / role	unit	default
`t`	time since the circuit was switched off	time / plain	`s`	
`tau`	time constant	time / time_constant	`s`	

- **Conditions:** inductor, current_decay
- **Applies when:** an RL circuit after the source is switched off
- **Fails when:** the source is still connected
- **Assumes:** a single inductor and resistor; the source is removed at $t = 0$
- **Evidence:** fundamental_law



Current dying away in an inductive circuit from I_0 = 10 A with time constant τ = 0.0025 s, 40 times from the package (time in ms): each τ leaves e^{-1} ≈ 37 % of the current before it. Marked: 3.67879 A at 1τ, 1.35335 A at 2τ, 0.497871 A at 3τ, 0.183156 A at 4τ.

```
classical_physics_ai999x.solve('rl_current_decay', I0='10 A', t='0.0025 s', tau='0.0025 s',
target='I')
# -> 3.67879 A   (SOLVED)
```

$$I = I_0 e^{\frac{-t}{\tau}}$$

$$I = I_0 e^{-\frac{t}{\tau}} = 3.67879\ \mathrm{A}$$

invalid call	outcome	message
without `I0`	NEEDS_INPUT	rl_current_decay needs I0 (initial current, A). Example: solve('rl_current_decay', I0='... A', t='... s', tau='... s').
`I0` in a wrong unit	INVALID_INPUT	I0 is a current: its unit must be like A, not 'kg'.

Questions in words that work:

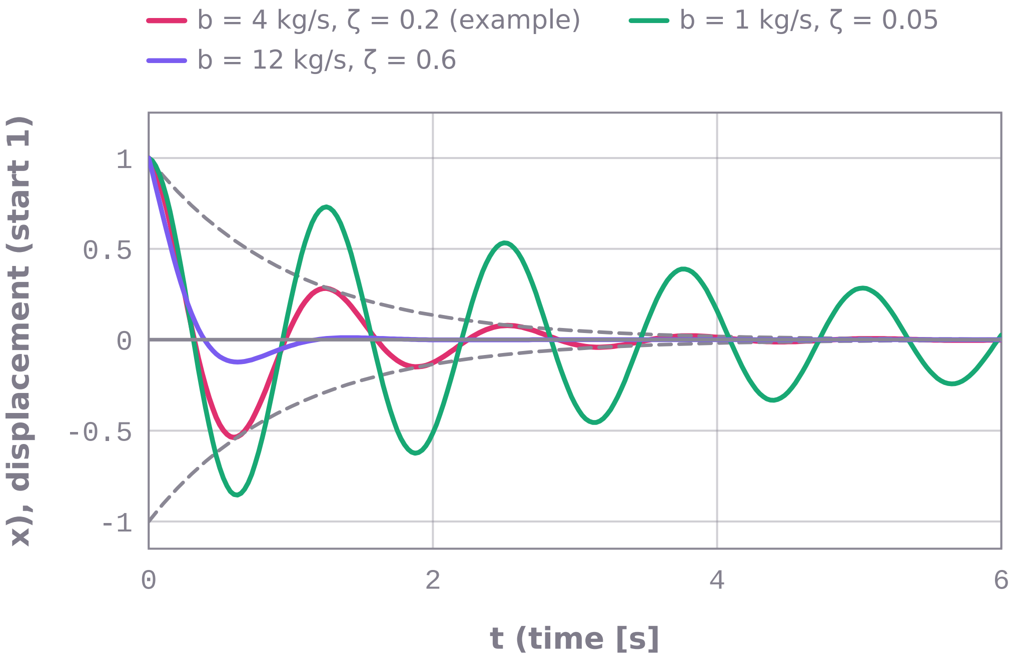
- "In an RL circuit with an inductor and a time constant of 2.5 ms, an initial current of 10 A is switched off. What is the current after 2.5 ms?" → 3.67879 A
- "An inductor carries an initial current of 4 A when the circuit is switched off; the time constant is 2 ms. What is the current after 4 ms?" → 0.541341 A

Angular frequency of a damped oscillator, `damped_angular_frequency`

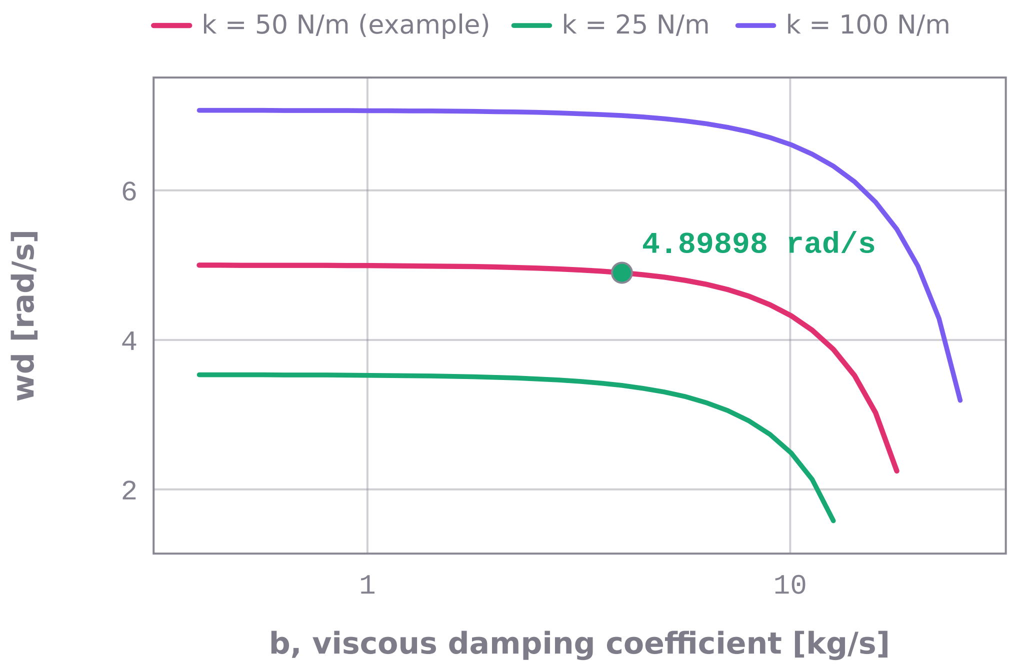
$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	kind / role	unit	default
`wd`	angular frequency of the damped oscillation	angular_frequency / damped	`rad/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** damped
- **Applies when:** a mass on a linear spring with viscous damping
- **Fails when:** critical or over-damping
- **Assumes:** linear viscous damping; underdamped (b^2 < 4 k m)
- **Evidence:** theorem



Free motion of a damped oscillator (k = 50 N/m, m = 2 kg) starting at x = 1 for three damping constants, drawn as e^(−b t / 2m) cos(ω_d t), with the damped angular frequency ω_d and damping ratio ζ returned by the package and the decay rate b/2m from the inputs; dashed: the decay envelope of the example). Marked: b = 4 kg/s: ω_d = 4.89898 rad/s, ζ = 0.2; b = 1 kg/s: ω_d = 4.99375 rad/s, ζ = 0.05; b = 12 kg/s: ω_d = 4 rad/s, ζ = 0.6.



wd (angular frequency of the damped oscillation) against b (viscous damping coefficient) from 0.4 kg/s to 40 kg/s on a logarithmic axis. One curve for each value of k (spring constant): 50 N/m, 25 N/m, 100 N/m. Other inputs as in the worked example: m = 2.0 kg. 34 points per curve computed by classical-physics-ai999x; marked: the worked example, b = 4 kg/s, wd = 4.89898 rad/s.

```
classical_physics_ai999x.solve('damped_angular_frequency', k='50 N/m', m='2 kg', b='4
kg/s', target='wd')
# -> 4.89898 rad/s   (SOLVED)
```

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

$$\omega_d = \frac{\sqrt{-b^2 + 4km}}{2m} = 4.89898\,\mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	damped_angular_frequency needs k (spring constant, N/m). Example: solve('damped_angular_frequency', k='... N/m', m='... kg', b='... kg/s').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

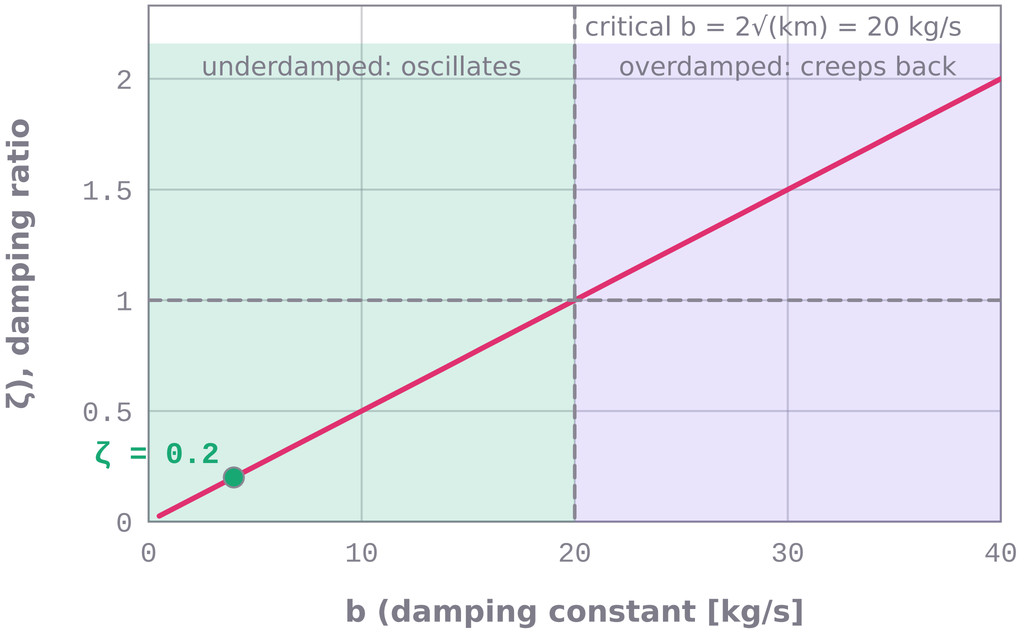
- "A 2 kg mass on a spring with a spring constant of 50 N/m has a damping coefficient of 4 kg/s. What is the damped angular frequency?" → 4.89898 rad/s
- "A damped oscillator has mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s. Find its damped angular frequency." → 3 rad/s

Damping ratio, `damping_ratio`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	kind / role	unit	default
`zeta`	damping ratio	damping_ratio / plain	`1`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	

- **Conditions:** damped
- **Applies when:** a damped mass-spring system
- **Fails when:** nonlinear damping
- **Assumes:** linear viscous damping
- **Evidence:** theorem



Damping ratio of an oscillator with k = 50 N/m and m = 2 kg against its damping constant, 40 values from the package: below ζ = 1 the motion oscillates, above it it does not; the critical damping constant 2√(k m) = 20 kg/s is derived from the inputs. Marked: ζ = 0.2 at b = 4 kg/s.

```
classical_physics_ai999x.solve('damping_ratio', b='4 kg/s', k='50 N/m', m='2 kg',
target='zeta')
# -> 0.2 1   (SOLVED)
```

$$\zeta = \frac{b}{2\sqrt{km}}$$

$$\zeta = \frac{b}{2\sqrt{k}\sqrt{m}} = 0.2\;1$$

invalid call	outcome	message
without `b`	NEEDS_INPUT	damping_ratio needs b (viscous damping coefficient, kg/s). Example: solve('damping_ratio', b='... kg/s', k='... N/m', m='... kg').
`b` in a wrong unit	INVALID_INPUT	b is a damping: its unit must be like kg/s, not 's'.

Questions in words that work:

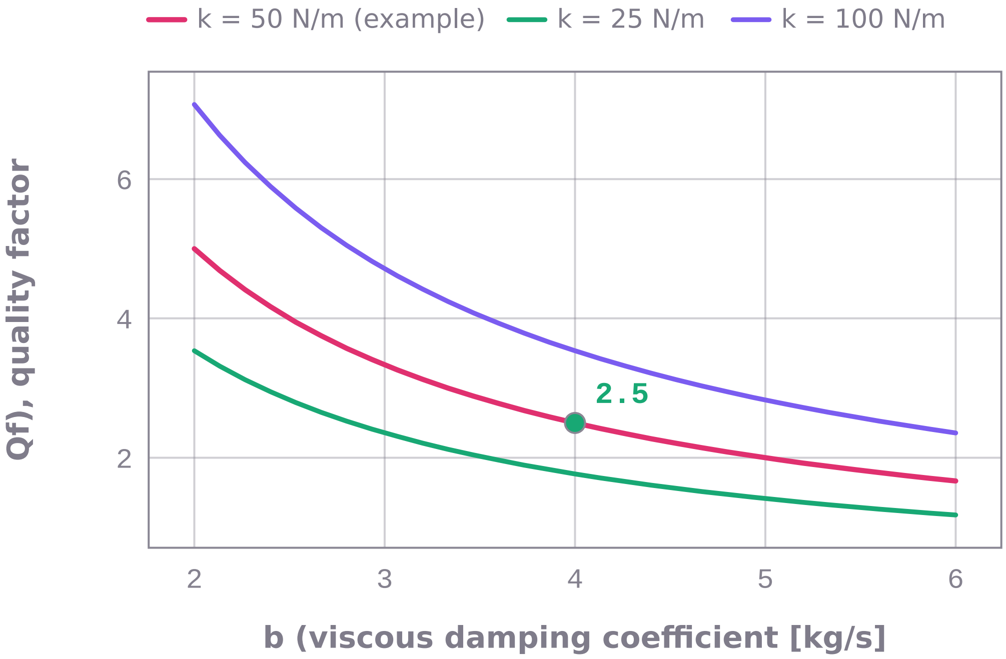
- "A damped oscillator has a 2 kg mass, a spring constant of 50 N/m and a damping coefficient of 4 kg/s. What is the damping ratio?" → 0.2 1
- "Find the damping ratio of a damped spring system with mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s." → 0.316228 1

Quality factor of a damped oscillator, `quality_factor`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	kind / role	unit	default
`Qf`	quality factor	quality_factor / plain	`1`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** damped
- **Applies when:** a damped mass-spring system
- **Fails when:** nonlinear damping
- **Assumes:** linear viscous damping
- **Evidence:** theorem



Qf (quality factor) against b (viscous damping coefficient) from 2 kg/s to 6 kg/s. One curve for each value of k (spring constant): 50 N/m, 25 N/m, 100 N/m. Other inputs as in the worked example: m = 2.0 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, b = 4 kg/s, Qf = 2.5.

```
classical_physics_ai999x.solve('quality_factor', k='50 N/m', m='2 kg', b='4 kg/s',
target='Qf')
# -> 2.5 1   (SOLVED)
```

$$Q_f = \frac{\sqrt{km}}{b}$$

$$Q_f = \frac{\sqrt{k}\sqrt{m}}{b} = 2.5\ 1$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	quality_factor needs k (spring constant, N/m). Example: solve('quality_factor', k='... N/m', m='... kg', b='... kg/s').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

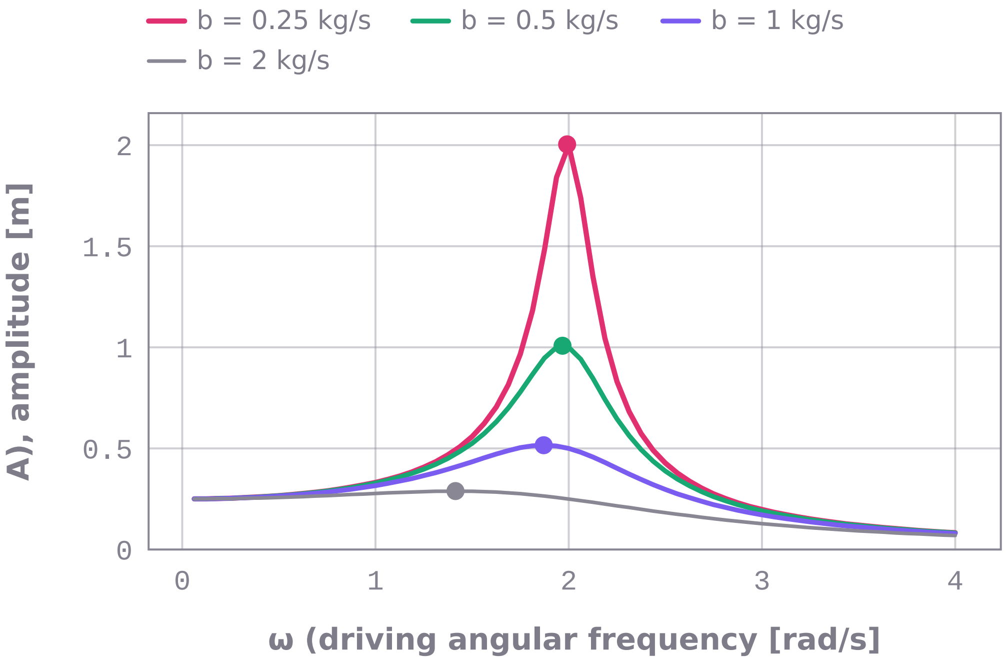
- "What is the quality factor of a damped oscillator with mass 2 kg, spring constant 50 N/m and damping coefficient 4 kg/s?" → 2.5 1
- "A damped oscillator has a 1 kg mass, a spring constant of 100 N/m and a damping coefficient of 0.5 kg/s. Find its quality factor." → 20 1

Steady-state amplitude of a driven damped oscillator, `driven_amplitude`

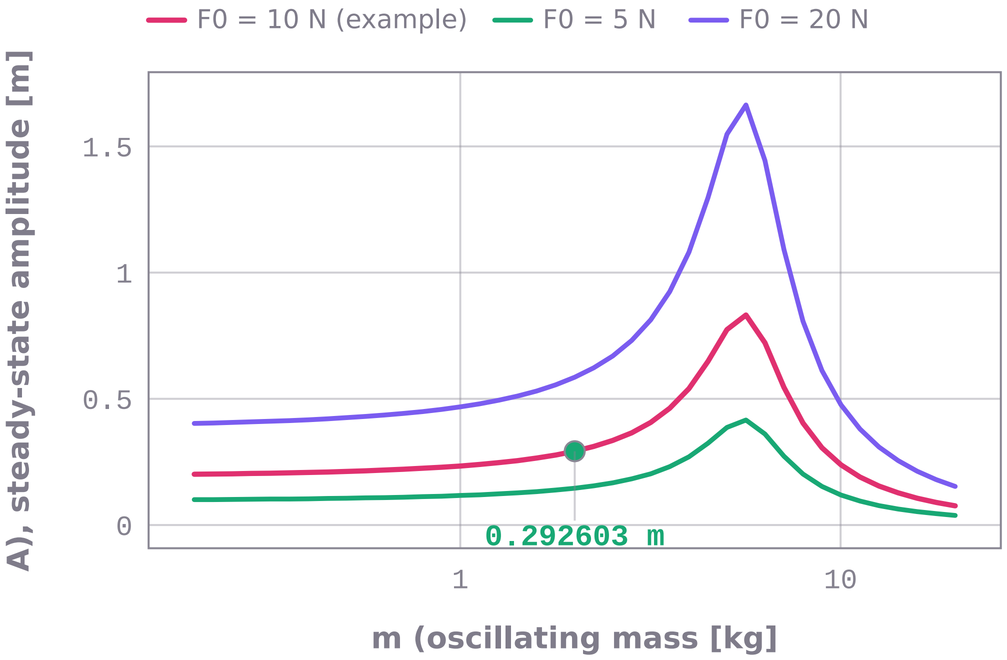
$$A = \frac{F_0}{\sqrt{\left(k - m\omega^2\right)^2 + \left(b\omega\right)^2}}$$

input	meaning	kind / role	unit	default
`A`	steady-state amplitude	length / amplitude	`m`	
`F0`	driving force amplitude	force / driving	`N`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	
`w`	driving angular frequency	angular_frequency / driving	`rad/s`	

- **Conditions:** driven
- **Applies when:** a damped mass-spring system driven by a sinusoidal force
- **Fails when:** before the transients decay
- **Assumes:** sinusoidal driving force; linear viscous damping; steady state (transients have died out)
- **Evidence:** theorem



Steady-state amplitude of a driven damped oscillator (F_0 = 1 N, k = 4 N/m, m = 1 kg) against the driving frequency for four damping constants, 64 frequencies each from the package; dots: the amplitude-resonance frequency it computes, b = 0.25: ω_r = 1.99217 rad/s, A = 2.00392 m; b = 0.5: ω_r = 1.9685 rad/s, A = 1.00791 m; b = 1: ω_r = 1.87083 rad/s, A = 0.516398 m; b = 2: ω_r = 1.41421 rad/s, A = 0.288675 m.



A (steady-state amplitude) against m (oscillating mass) from 0.2 kg to 20 kg on a logarithmic axis. One curve for each value of F0 (driving force amplitude): 10 N, 5 N, 20 N. Other inputs as in the worked example: k = 50.0 N/m, b = 4.0 kg/s, w = 3.0 rad/s. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, A = 0.292603 m.

```
classical_physics_ai999x.solve('driven_amplitude', F0='10 N', k='50 N/m', m='2 kg', b='4
kg/s', w='3 rad/s', target='A')
# -> 0.292603 m   (SOLVED)
```

$$A = \frac{F_0}{\sqrt{\left(k - m\omega^2\right)^2 + \left(b\omega\right)^2}}$$

$$A = \frac{F_0}{\sqrt{b^2\omega^2 + \left(k - m\omega^2\right)^2}} = 0.292603\ \mathrm{m}$$

invalid call	outcome	message
without `F0`	NEEDS_INPUT	driven_amplitude needs F0 (driving force amplitude, N). Example: solve('driven_amplitude', F0='... N', k='... N/m', m='... kg', b='... kg/s'
`F0` in a wrong unit	INVALID_INPUT	F0 is a force: its unit must be like N, not 'kg'.

Questions in words that work:

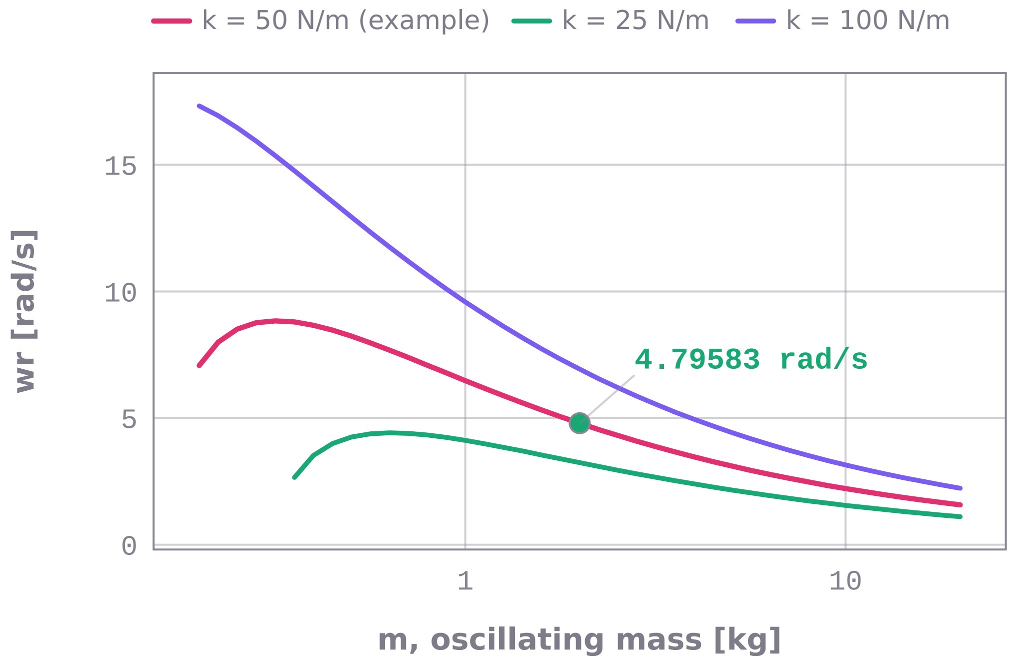
- "A 2 kg mass on a spring with spring constant 50 N/m and damping coefficient 4 kg/s is driven by a periodic force of amplitude 10 N at a driving frequency of 3 rad/s. What is the steady-state amplitude?" → 0.292603 m
- "A driven oscillator of mass 1 kg has spring constant 20 N/m and damping coefficient 1 kg/s. The driving force amplitude is 2 N and the driving frequency is 4 rad/s. Find the steady-state amplitude." → 0.353553 m

Amplitude-resonance frequency of a driven damped oscillator, `amplitude_resonance_frequency`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	kind / role	unit	default
`wr`	driving frequency of maximum amplitude	angular_frequency / resonance	`rad/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** driven
- **Applies when:** the driving frequency that maximises the steady-state amplitude
- **Fails when:** $b^2 \geq 2$ k m (no resonance peak)
- **Assumes:** sinusoidal driving; linear viscous damping; $b^2 < 2$ k m
- **Evidence:** theorem



wr (driving frequency of maximum amplitude) against m (oscillating mass) from 0.2 kg to 20 kg on a logarithmic axis. One curve for each value of k (spring constant): 50 N/m, 25 N/m, 100 N/m. Other inputs as in the worked example: b = 4.0 kg/s. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, wr = 4.79583 rad/s.

```
classical_physics_ai999x.solve('amplitude_resonance_frequency', k='50 N/m', m='2 kg', b='4
kg/s', target='wr')
# -> 4.79583 rad/s   (SOLVED)
```

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

$$\omega_r = \frac{\sqrt{-2b^2 + 4km}}{2m} = 4.79583\,\mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	amplitude_resonance_frequency needs k (spring constant, N/m). Example: solve('amplitude_resonance_frequency', k='... N/m', m='... kg', b='..
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

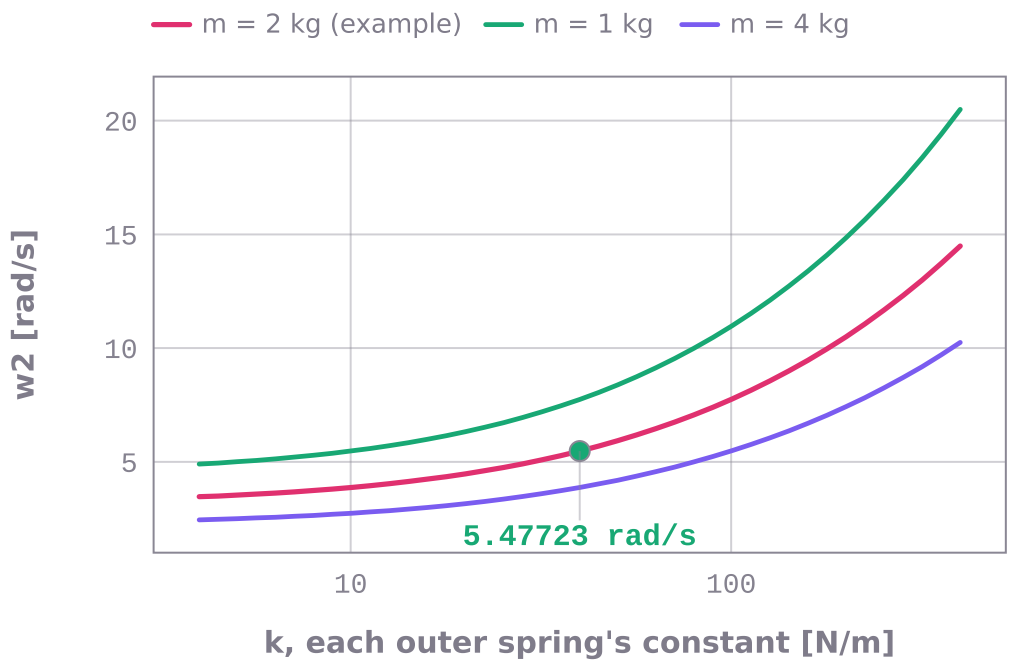
- "A driven oscillator has mass 2 kg, spring constant 50 N/m and damping coefficient 4 kg/s. What is its amplitude resonance frequency?" → 4.79583 rad/s
- "Find the amplitude resonance frequency of a driven damped oscillator with mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s." → 2.82843 rad/s

Higher normal-mode frequency of two coupled oscillators, `coupled_fast_mode`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	kind / role	unit	default
`w2`	higher normal-mode angular frequency	angular_frequency / mode_fast	`rad/s`	
`k`	each outer spring's constant	spring_constant / plain	`N/m`	
`kc`	coupling spring constant	spring_constant / coupling	`N/m`	
`m`	each mass	mass / plain	`kg`	

- **Conditions:** coupled
- **Applies when:** two equal masses joined by a coupling spring, each also tied to a wall
- **Fails when:** unequal masses or springs
- **Assumes:** two identical masses; identical outer springs; small oscillations, no damping
- **Evidence:** theorem



w2 (higher normal-mode angular frequency) against k (each outer spring's constant) from 4 N/m to 400 N/m on a logarithmic axis. One curve for each value of m (each mass): 2 kg, 1 kg, 4 kg. Other inputs as in the worked example: kc = 10.0 N/m. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, k = 40 N/m, w2 = 5.47723 rad/s.

```
classical_physics_ai999x.solve('coupled_fast_mode', k='40 N/m', kc='10 N/m', m='2 kg',
target='w2')
# -> 5.47723 rad/s   (SOLVED)
```

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

$$\omega_2 = \frac{\sqrt{k + 2k_c}}{\sqrt{m}} = 5.47723\ \mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	coupled_fast_mode needs k (each outer spring's constant, N/m). Example: solve('coupled_fast_mode', k='... N/m', kc='... N/m', m='... kg').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

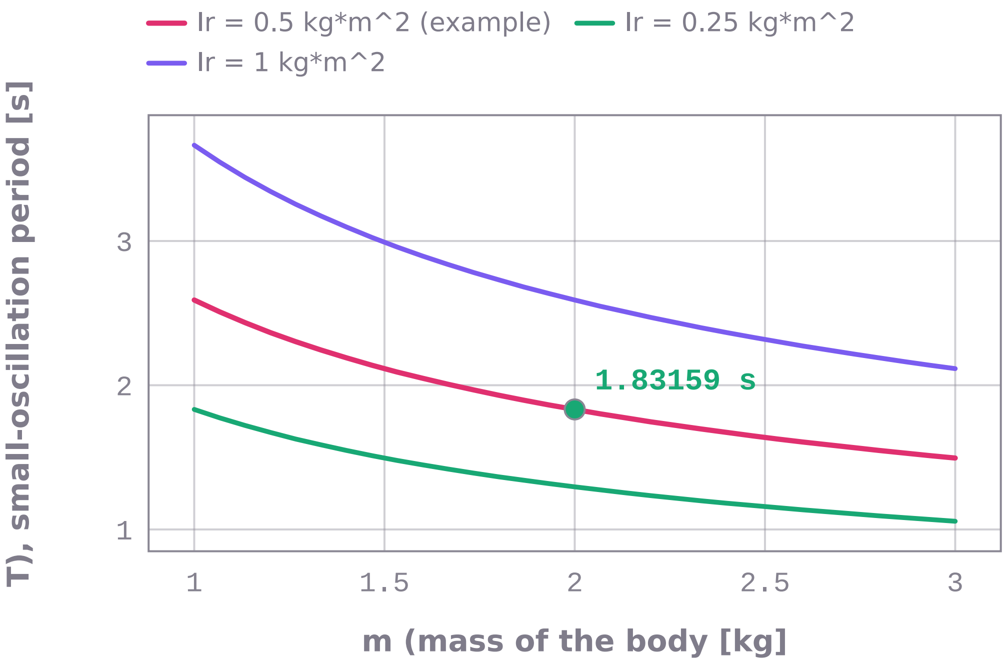
- "Two identical masses of 2 kg are each attached to a wall by a spring of spring constant 40 N/m and joined by a coupling spring of 10 N/m. What is the higher normal mode frequency?" → 5.47723 rad/s
- "Two coupled oscillators have masses of 1 kg each, outer spring constant 8 N/m and coupling spring constant 4 N/m. Find the higher normal-mode frequency." → 4 rad/s

Period of a physical pendulum, `physical_pendulum_period`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	kind / role	unit	default
`T`	small-oscillation period	time / period	`s`	
`Ir`	moment of inertia about the pivot	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the body	mass / plain	`kg`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`d`	distance from the pivot to the center of mass	length / pivot_cm	`m`	

- **Conditions:** physical_pendulum
- **Applies when:** a rigid body swinging about a fixed pivot
- **Fails when:** large amplitudes
- **Assumes:** small oscillations; no friction at the pivot
- **Evidence:** theorem



T (small-oscillation period) against m (mass of the body) from 1 kg to 3 kg. One curve for each value of Ir (moment of inertia about the pivot): 0.5 kg*m^2, 0.25 kg*m^2, 1 kg*m^2. Other inputs as in the worked example: g = 9.80665 m/s^2, d = 0.3 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m = 2 kg, T = 1.83159 s.

```
classical_physics_ai999x.solve('physical_pendulum_period', Ir='0.5 kg*m^2', m='2 kg',
d='0.3 m', target='T')
# -> 1.83159 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

$$T = \frac{2\pi\sqrt{I_r}}{\sqrt{d}\sqrt{g}\sqrt{m}} = 1.83159\ \mathrm{s}$$

invalid call	outcome	message
without `Ir`	NEEDS_INPUT	physical_pendulum_period needs Ir (moment of inertia about the pivot, kg*m^2). Example: solve('physical_pendulum_period', Ir='... kg*m^2', m
`Ir` in a wrong unit	INVALID_INPUT	Ir is a moment of inertia: its unit must be like kg*m^2, not 's'.

Questions in words that work:

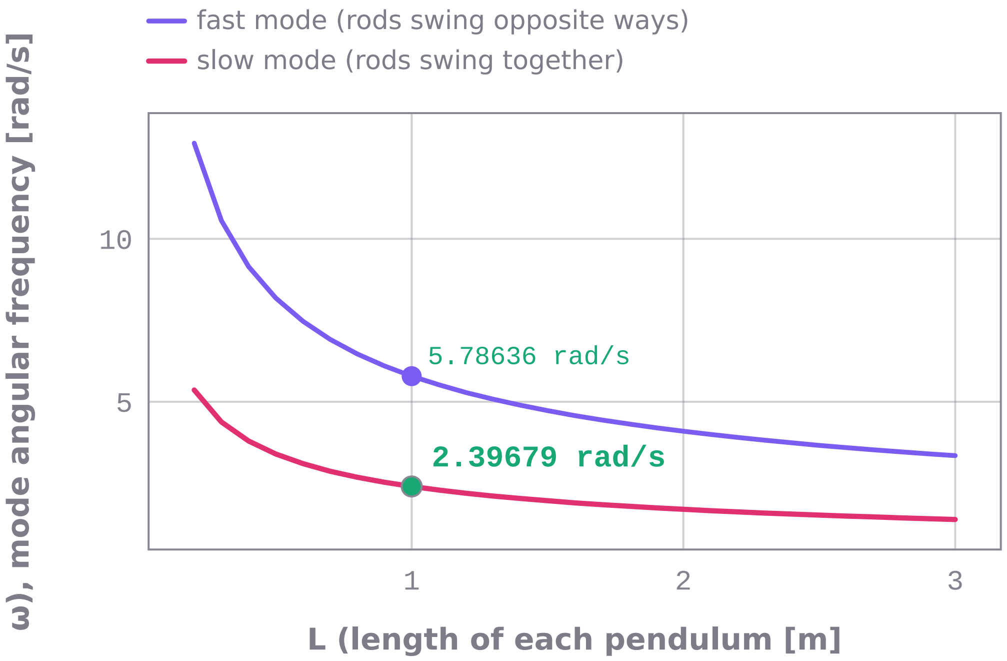
- "A physical pendulum of mass 2 kg has a moment of inertia of 0.5 kg*m^2 about its pivot, and the distance from the pivot to the center of mass is 0.3 m. What is its period?" → 1.83159 s
- "Find the period of a compound pendulum of mass 3 kg with moment of inertia 1.2 kg*m^2 and a distance from the pivot to the center of mass of 0.25 m." → 2.53793 s

Lower normal mode of a double pendulum, `double_pendulum_slow_mode`

$$\omega = \sqrt{\frac{\left(2 - \sqrt{2}\right)g}{L}}$$

input	meaning	kind / role	unit	default
`w`	lower normal-mode angular frequency	angular_frequency / mode_slow	`rad/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`L`	length of each arm	length / pendulum	`m`	

- **Conditions:** double_pendulum
- **Applies when:** a double pendulum with equal masses and arms
- **Fails when:** large amplitudes (chaotic motion)
- **Assumes:** equal masses and arm lengths; small oscillations
- **Evidence:** theorem



Small-oscillation normal-mode angular frequencies of a double pendulum of two equal masses on equal lengths L (standard gravity), 29 lengths from 0.2 to 3 m from the package for each mode. Marked at L = 1 m: slow mode 2.39679 rad/s, fast mode 5.78636 rad/s.

```
classical_physics_ai999x.solve('double_pendulum_slow_mode', L='1 m', target='w')
# -> 2.39679 rad/s   (SOLVED)
```

$$\omega = \sqrt{\frac{\left(2 - \sqrt{2}\right) g}{L}}$$

$$\omega = \frac{\sqrt{g}\sqrt{2 - \sqrt{2}}}{\sqrt{L}} = 2.39679\ \mathrm{rad/s}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	double_pendulum_slow_mode needs L (length of each arm, m). Example: solve('double_pendulum_slow_mode', g='... m/s^2', L='... m').
`L` in a wrong unit	INVALID_INPUT	L is a length: its unit must be like m, not 'kg'.

Questions in words that work:

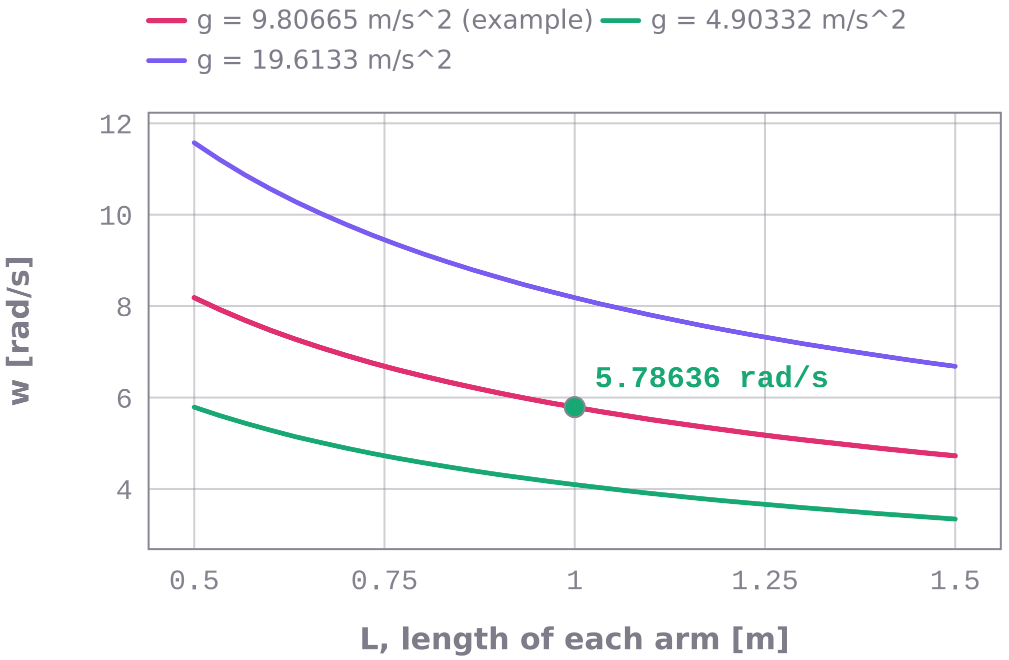
- "A double pendulum has equal masses and a length of each arm of 1 m. What is the lower normal mode frequency?" → 2.39679 rad/s
- "Find the lower normal-mode frequency of a double pendulum whose arm length is 0.5 m." → 3.38957 rad/s

Higher normal mode of a double pendulum, `double_pendulum_fast_mode`

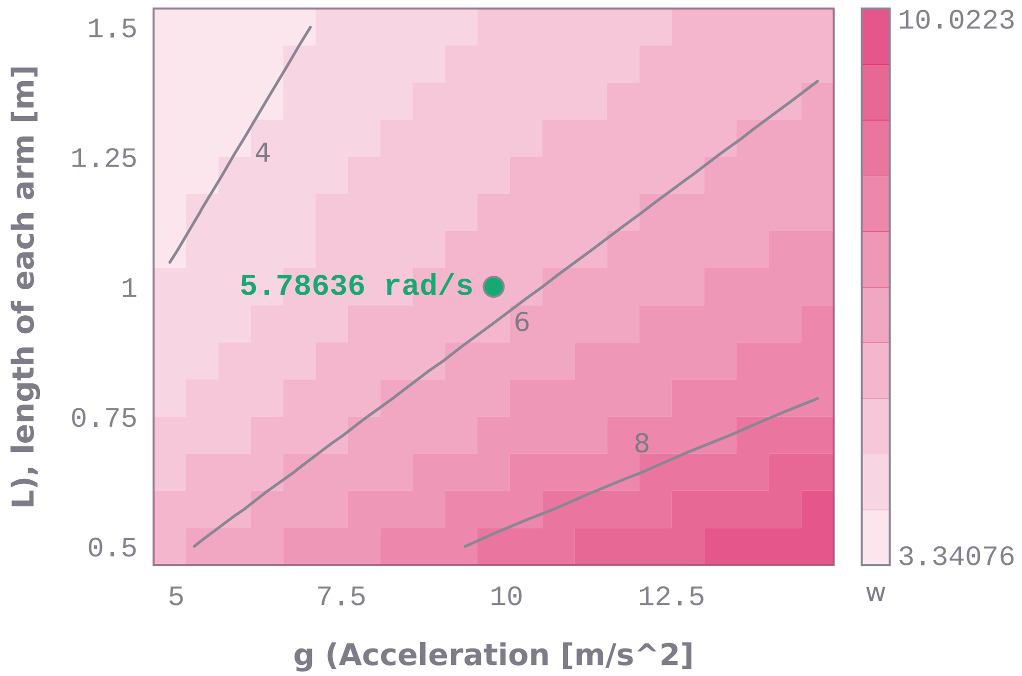
$$\omega = \sqrt{\frac{\left(2 + \sqrt{2}\right) g}{L}}$$

input	meaning	kind / role	unit	default
`w`	higher normal-mode angular frequency	angular_frequency / mode_fast	`rad/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`L`	length of each arm	length / pendulum	`m`	

- **Conditions:** double_pendulum
- **Applies when:** a double pendulum with equal masses and arms
- **Fails when:** large amplitudes (chaotic motion)
- **Assumes:** equal masses and arm lengths; small oscillations
- **Evidence:** theorem



w (higher normal-mode angular frequency) against L (length of each arm) from 0.5 m to 1.5 m. One curve for each value of g (Acceleration): 9.80665 m/s^2, 4.90332 m/s^2, 19.6133 m/s^2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, L = 1 m, w = 5.78636 rad/s.



w (higher normal-mode angular frequency) over g (Acceleration) from 4.90332 m/s^2 to 14.71 m/s^2 and L (length of each arm) from 0.5 m to 1.5 m: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 3.34076 rad/s to dark 10.0223 rad/s); iso-lines at w = 4, 6, 8 rad/s. Marked: the worked example, g = 9.80665 m/s^2, L = 1 m, w = 5.78636 rad/s.

```
classical_physics_ai999x.solve('double_pendulum_fast_mode', L='1 m', target='w')
# -> 5.78636 rad/s   (SOLVED)
```

$$\omega = \sqrt{\frac{\left(2 + \sqrt{2}\right) g}{L}}$$

$$\omega = \frac{\sqrt{g}\sqrt{\sqrt{2} + 2}}{\sqrt{L}} = 5.78636\ \mathrm{rad/s}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	double_pendulum_fast_mode needs L (length of each arm, m). Example: solve('double_pendulum_fast_mode', g='... m/s^2', L='... m').
`L` in a wrong unit	INVALID_INPUT	L is a length: its unit must be like m, not 'kg'.

Questions in words that work:

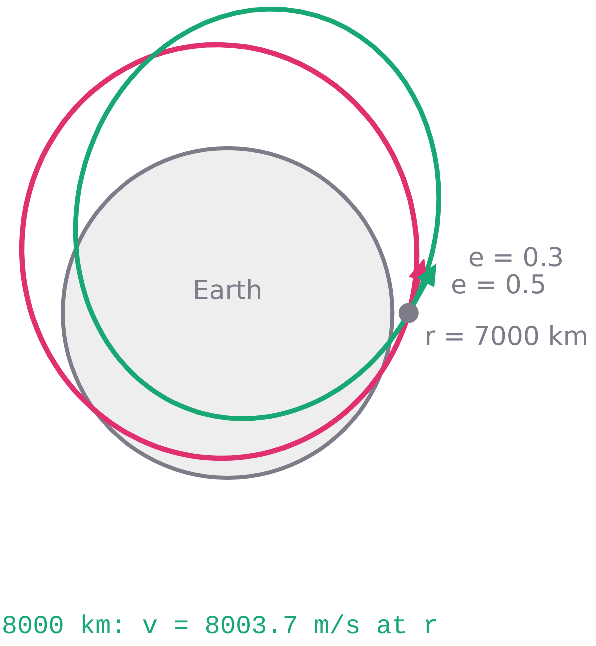
- "A double pendulum has equal masses and a length of each arm of 1 m. What is the higher normal mode frequency?" → 5.78636 rad/s
- "Find the higher normal-mode frequency of a double pendulum whose arm length is 0.5 m." → 8.18315 rad/s

Orbital speed on an elliptical orbit (vis-viva), `vis_viva`

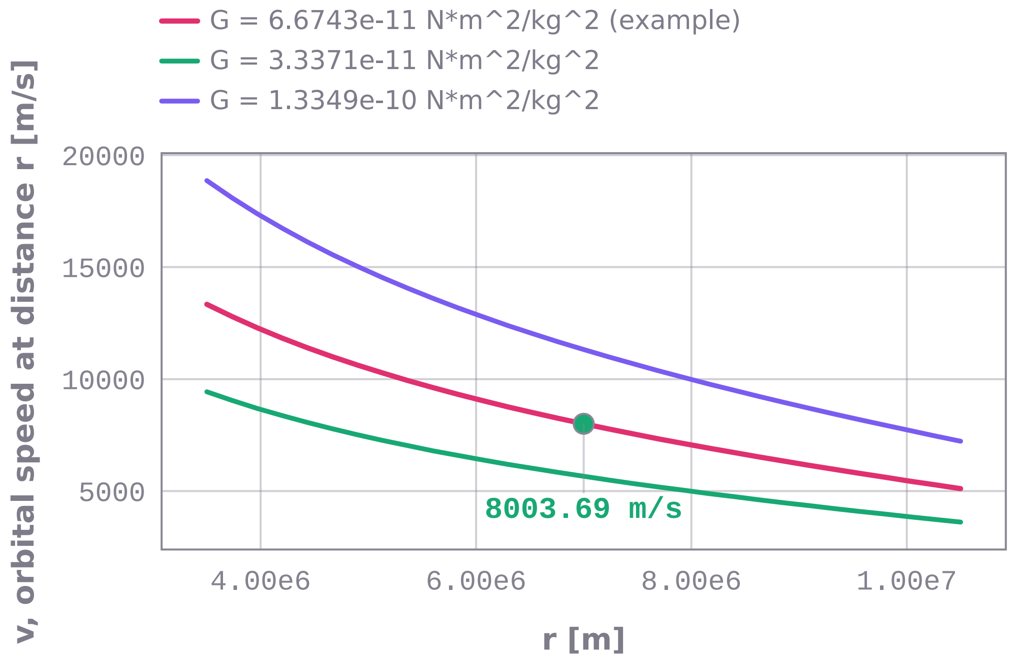
$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	kind / role	unit	default
`v`	orbital speed at distance r	speed / orbital	`m/s`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the central body	mass / plain	`kg`	
`r`	distance from the center of the central body	length / radius	`m`	
`a`	semi-major axis	length / semi_major	`m`	

- **Conditions:** orbit
- **Applies when:** a satellite or planet on an elliptical orbit
- **Fails when:** unbound orbits; comparable masses
- **Assumes:** two bodies, the orbiting one much lighter; a bound (elliptical) orbit
- **Evidence:** theorem



2 orbits round a body of mass M = 5.9720e+24 kg with the same semi-major axis a = 8e+06 m but eccentricities 0.3, 0.5, each passing through the point at r = 7e+06 m (the ellipses and the directions of motion there are drawn from that geometry). The vis-viva equation gives every one the same speed at that radius, whatever its shape. Marked: v = 8003.69 m/s (returned).



v (orbital speed at distance r) against r (distance from the center of the central body) from 3.5e+06 m to 1.0500e+07 m. One curve for each value of G (Gravitational constant): 6.6743e-11 N*m^2/kg^2, 3.3371e-11 N*m^2/kg^2, 1.3349e-10 N*m^2/kg^2. Other inputs as in the worked example: M = 5.972e+24 kg, a = 8000000.0 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 7e+06 m, v = 8003.69 m/s.

```
classical_physics_ai999x.solve('vis_viva', M='5.972e+24 kg', r='7e+06 m', a='8e+06 m',
target='v')
# -> 8003.69 m/s   (SOLVED)
```

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

$$v = \frac{\sqrt{G}\sqrt{M}\sqrt{2a - r}}{\sqrt{a}\sqrt{r}} = 8003.69\ \mathrm{m/s}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	vis_viva needs M (mass of the central body, kg). Example: solve('vis_viva', G='... N*m^2/kg^2', M='... kg', r='... m', a='... m').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

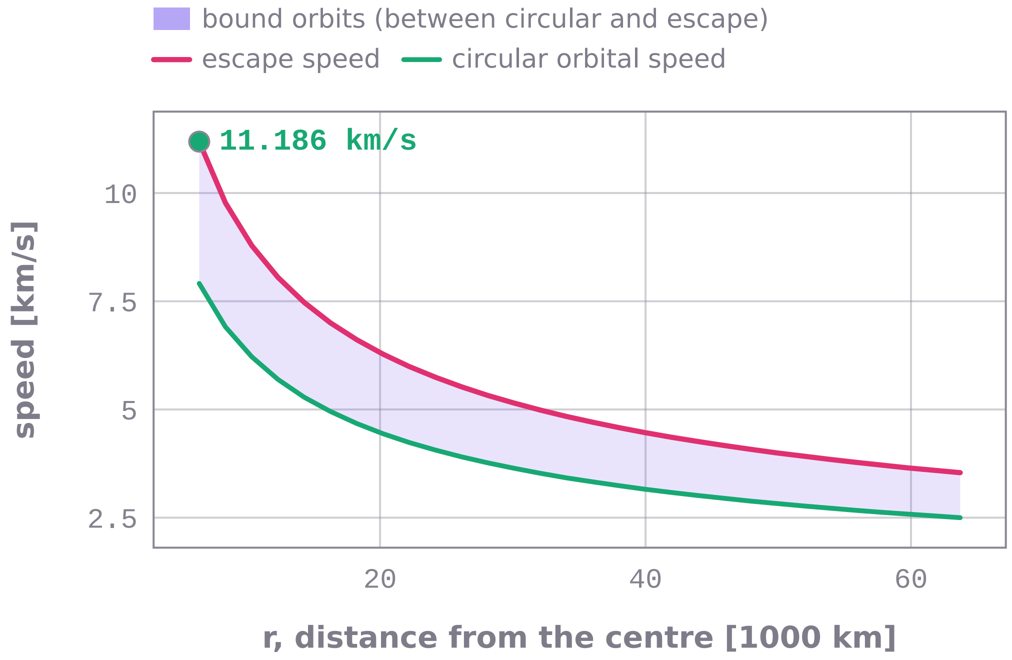
- "A satellite on an elliptical orbit around a planet of mass 5.972e24 kg has a semi-major axis of 8.0e6 m. What is its orbital speed at a distance from the center of 7.0e6 m?" → 8003.69 m/s
- "A comet orbits a star of mass 1.989e30 kg with a semi-major axis of 2.0e11 m. Find its orbital speed at a distance from the center of 1.5e11 m." → 33260.6 m/s

Escape speed, `escape_speed`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	kind / role	unit	default
`v`	escape speed	speed / escape	`m/s`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the body	mass / plain	`kg`	
`r`	distance from its center	length / radius	`m`	

- **Conditions:** escape
- **Applies when:** leaving a body's gravity from distance r
- **Fails when:** atmospheric drag; other bodies' gravity
- **Assumes:** no air resistance; a spherically symmetric body; starting from rest relative to it
- **Evidence:** theorem



Escape speed and circular orbital speed against distance from the centre of a body of mass M = 5.9720e+24 kg, from r = 6.371e+06 m to ten times that, 30 distances each from the package (km/s): the escape speed is always √2 times the circular speed. Marked: escape speed 11186 m/s at r = 6.371e+06 m.

```
classical_physics_ai999x.solve('escape_speed', M='5.972e+24 kg', r='6.371e+06 m',
target='v')
# -> 11186 m/s   (SOLVED)
```

$$v = \sqrt{\frac{2GM}{r}}$$

$$v = \frac{\sqrt{2}\sqrt{G}\sqrt{M}}{\sqrt{r}} = 11186\ \mathrm{m/s}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	escape_speed needs M (mass of the body, kg). Example: solve('escape_speed', G='... N*m^2/kg^2', M='... kg', r='... m').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

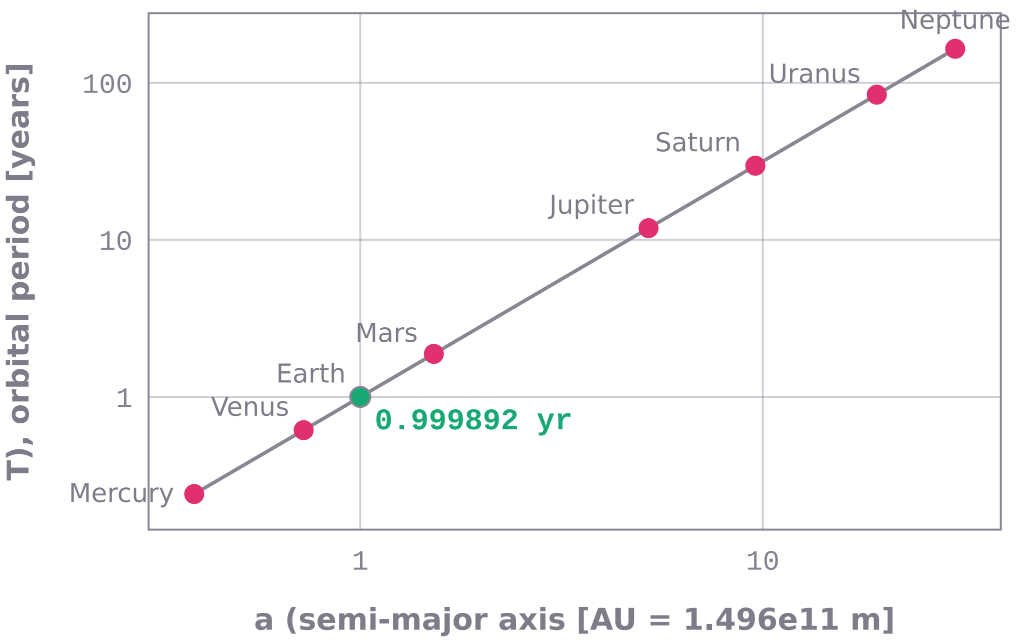
- "What is the escape speed from a planet of mass 5.972e24 kg and radius 6.371e6 m?" → 11186 m/s
- "Find the escape velocity at the surface of a moon with a mass of 7.342e22 kg and a radius of 1.737e6 m." → 2375.34 m/s

Orbital period from the semi-major axis (Kepler's third law), `kepler_period_semi_major`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	kind / role	unit	default
`T`	orbital period	time / period	`s`	
`a`	semi-major axis	length / semi_major	`m`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the central body	mass / plain	`kg`	

- **Conditions:** orbit
- **Applies when:** any bound elliptical orbit
- **Fails when:** comparable masses (use the total mass)
- **Assumes:** two bodies, the orbiting one much lighter
- **Evidence:** theorem



Kepler's third law for the eight planets' distances: orbital period against semi-major axis (stated, from Mercury 5.79e10 m to Neptune 4.495e12 m) round a central mass M = 1.9890e+30 kg, each period from the package (shown in years of 3.1558e+07 s, distances in AU): on logarithmic axes the points lie on one straight line of slope 3/2. Marked: Earth, T = 3.1554e+07 s = 0.999892 years.

```
classical_physics_ai999x.solve('kepler_period_semi_major', a='1.496e+11 m', M='1.989e+30
kg', target='T')
# -> 3.15542e+07 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{a^{3}}{GM}}$$

$$T = \frac{2\pi a^{\frac{3}{2}}}{\sqrt{G}\sqrt{M}} = 3.15542e + 07\ \mathrm{s}$$

invalid call	outcome	message
without `a`	NEEDS_INPUT	kepler_period_semi_major needs a (semi-major axis, m). Example: solve('kepler_period_semi_major', a='... m', G='... N*m^2/kg^2', M='... kg')
`a` in a wrong unit	INVALID_INPUT	a is a length: its unit must be like m, not 'kg'.

Questions in words that work:

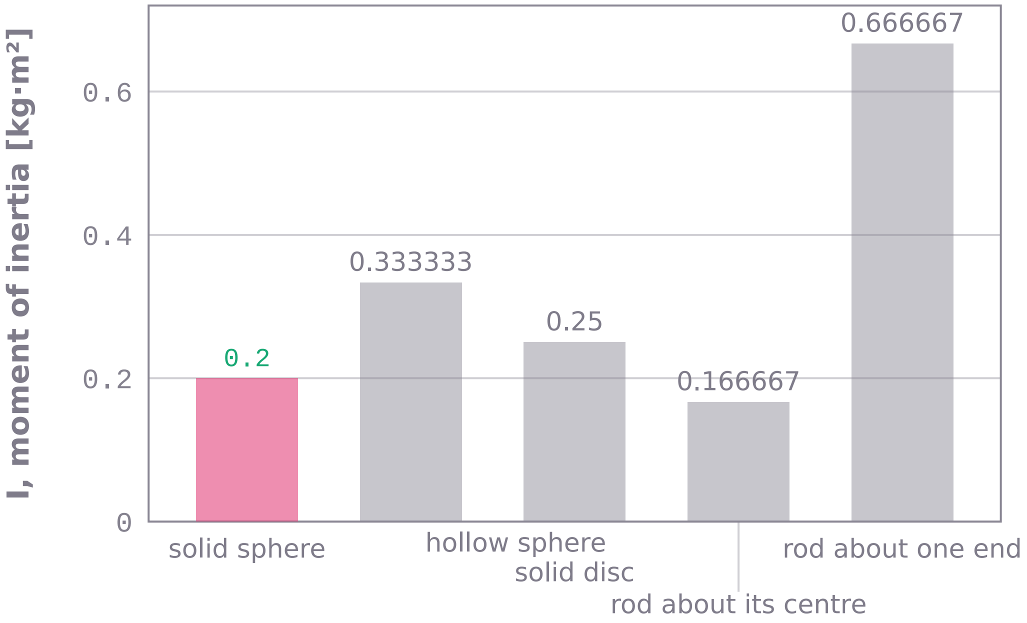
- "A planet orbits a star of mass 1.989e30 kg on an elliptical orbit with a semi-major axis of 1.496e11 m. What is its orbital period?" → 3.15542e+07 s
- "Find the orbital period of a satellite on an elliptical orbit around a planet of mass 5.972e24 kg with a semi-major axis of 4.2164e7 m." → 86164.8 s

Moment of inertia of a solid sphere, `inertia_solid_sphere`

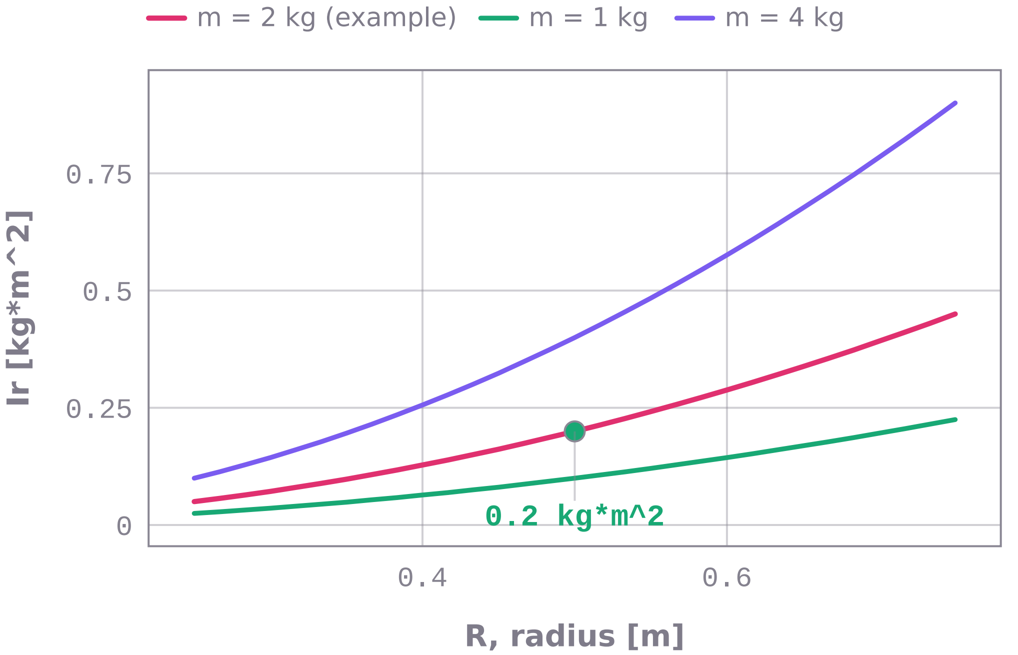
$$I_{r} = \frac{2mR^{2}}{5}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** solid_sphere
- **Applies when:** a uniform solid sphere
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem



Moments of inertia of bodies of the same mass m = 2 kg: spheres and a disc of radius R = 0.5 m, rods of length L = 1 m (= 2R) about their centre and about one end, each from the package; the highlighted bar is this formula's, the farther the mass from the axis, the larger I. Marked: I = 0.2 kg·m².



Ir (moment of inertia about the central axis) against R (radius) from 0.25 m to 0.75 m. One curve for each value of m (mass): 2 kg, 1 kg, 4 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 0.5 m, Ir = 0.2 kg*m^2.

```
classical_physics_ai999x.solve('inertia_solid_sphere', m='2 kg', R='0.5 m', target='Ir')
# -> 0.2 kg*m^2   (SOLVED)
```

$$I_r = \frac{2mR^2}{5}$$

$$I_r = \frac{2R^2m}{5} = 0.2\,\mathrm{kg}\,\mathrm{m}^2$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_solid_sphere needs m (mass, kg). Example: solve('inertia_solid_sphere', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

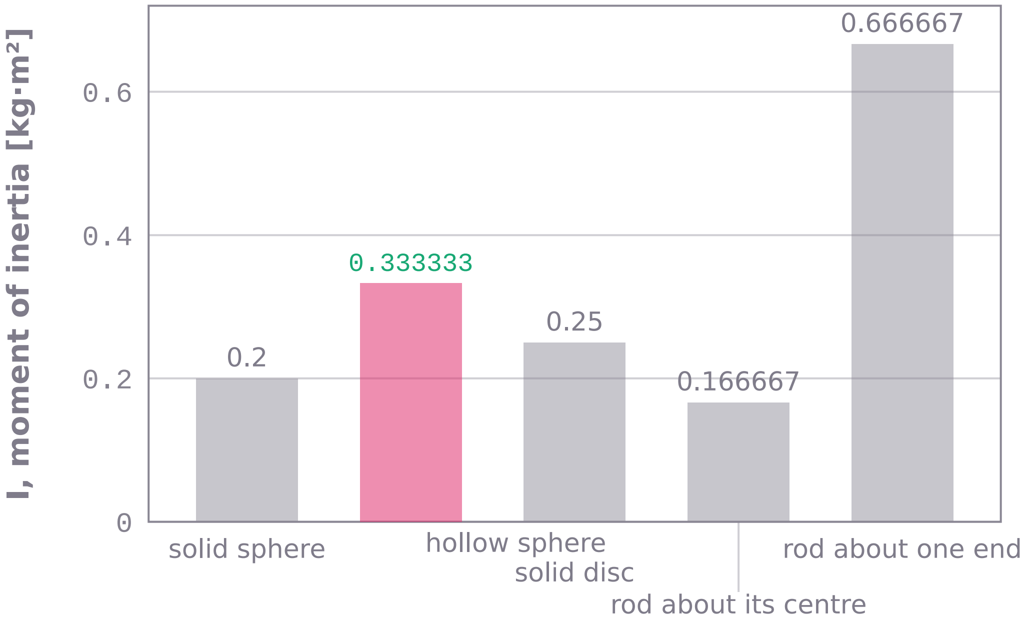
- "A uniform solid sphere has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.2 kg*m^2
- "Find the moment of inertia of a solid sphere of mass 5 kg and radius 0.2 m about its central axis." → 0.08 kg*m^2

Moment of inertia of a thin spherical shell, `inertia_hollow_sphere`

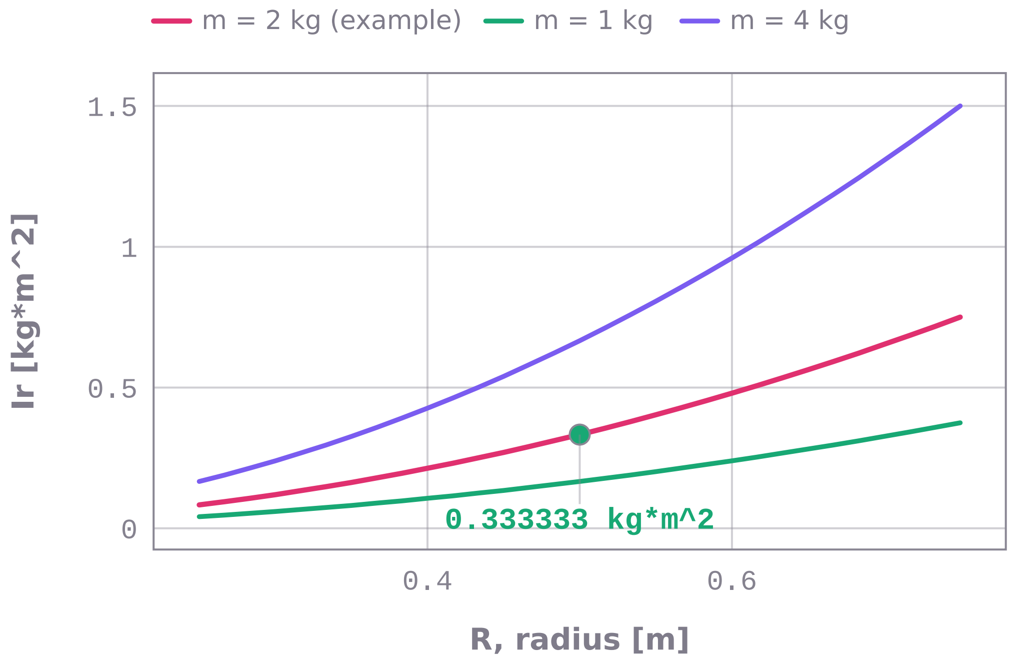
$$I_r = \frac{2mR^2}{3}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** hollow_sphere
- **Applies when:** a uniform hollow sphere
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem



Moments of inertia of bodies of the same mass m = 2 kg: spheres and a disc of radius R = 0.5 m, rods of length L = 1 m (= 2R) about their centre and about one end, each from the package; the highlighted bar is this formula's, the farther the mass from the axis, the larger I. Marked: I = 0.333333 kg·m².



Ir (moment of inertia about the central axis) against R (radius) from 0.25 m to 0.75 m. One curve for each value of m (mass): 2 kg, 1 kg, 4 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 0.5 m, Ir = 0.333333 kg*m^2.

```
classical_physics_ai999x.solve('inertia_hollow_sphere', m='2 kg', R='0.5 m', target='Ir')
# -> 0.333333 kg*m^2   (SOLVED)
```

$$I_r = \frac{2mR^2}{3}$$

$$I_r = \frac{2R^2m}{3} = 0.333333\,\mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_hollow_sphere needs m (mass, kg). Example: solve('inertia_hollow_sphere', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

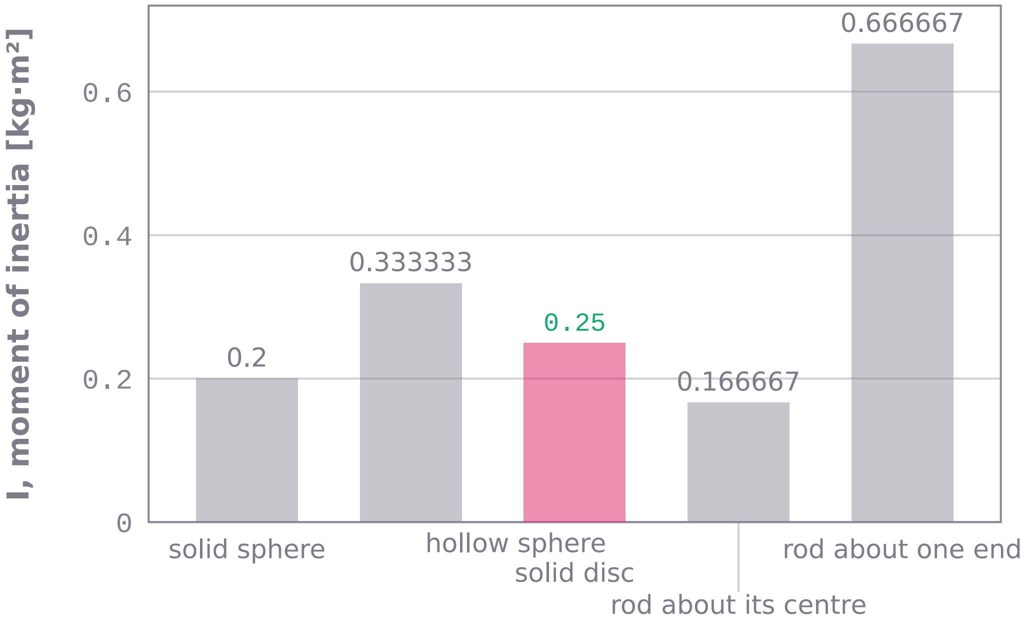
- "A uniform hollow sphere has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.333333 kg*m^2
- "Find the moment of inertia of a hollow sphere of mass 5 kg and radius 0.2 m about its central axis." → 0.133333 kg*m^2

Moment of inertia of a solid disk about its axis, `inertia_solid_disk`

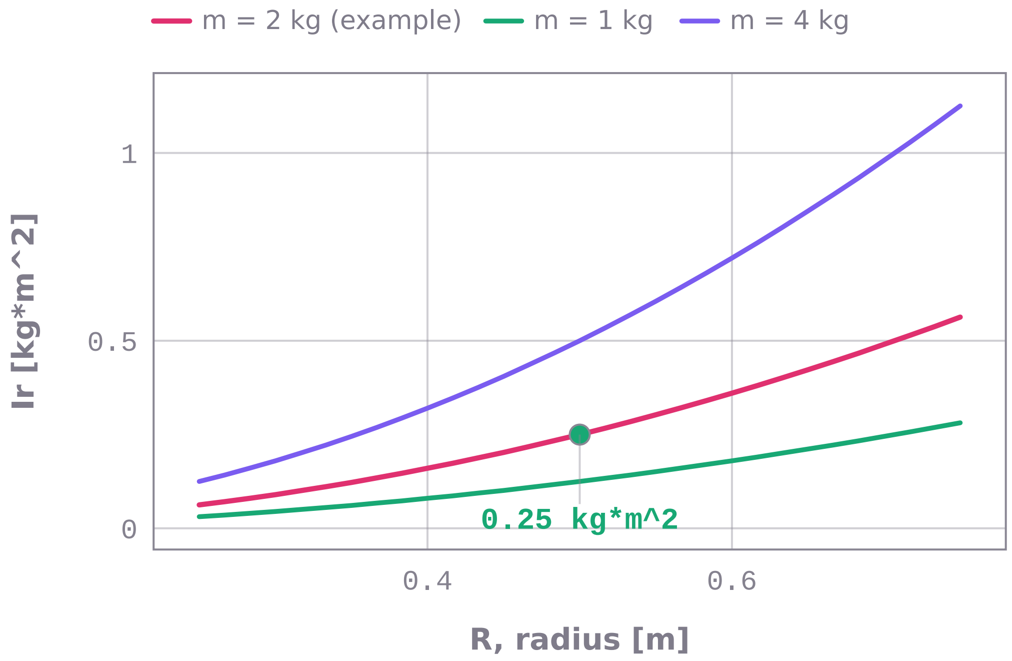
$$I_r = \frac{mR^2}{2}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** solid_disk
- **Applies when:** a uniform solid disk
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem



Moments of inertia of bodies of the same mass m = 2 kg: spheres and a disc of radius R = 0.5 m, rods of length L = 1 m (= 2R) about their centre and about one end, each from the package; the highlighted bar is this formula's, the farther the mass from the axis, the larger I. Marked: I = 0.25 kg·m².



Ir (moment of inertia about the central axis) against R (radius) from 0.25 m to 0.75 m. One curve for each value of m (mass): 2 kg, 1 kg, 4 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 0.5 m, Ir = 0.25 kg*m^2.

```
classical_physics_ai999x.solve('inertia_solid_disk', m='2 kg', R='0.5 m', target='Ir')
# -> 0.25 kg*m^2   (SOLVED)
```

$$I_r = \frac{mR^2}{2}$$
$$I_r = \frac{R^2 m}{2} = 0.25\ \mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_solid_disk needs m (mass, kg). Example: solve('inertia_solid_disk', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

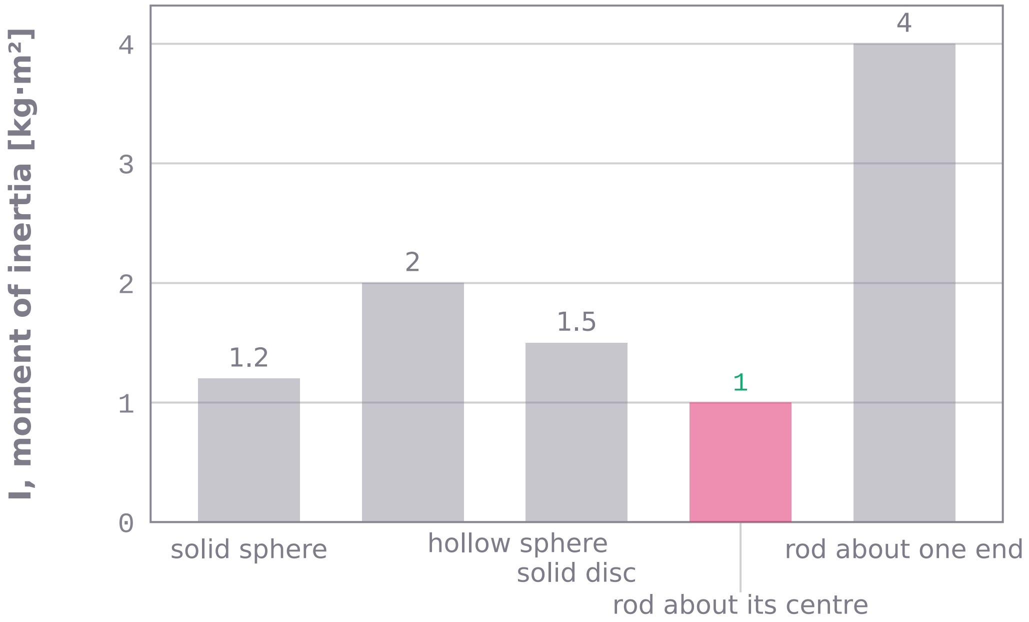
- "A uniform solid disk has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.25 kg*m^2
- "Find the moment of inertia of a solid disk of mass 5 kg and radius 0.2 m about its central axis." → 0.1 kg*m^2

Moment of inertia of a thin rod about its center, `inertia_rod_center`

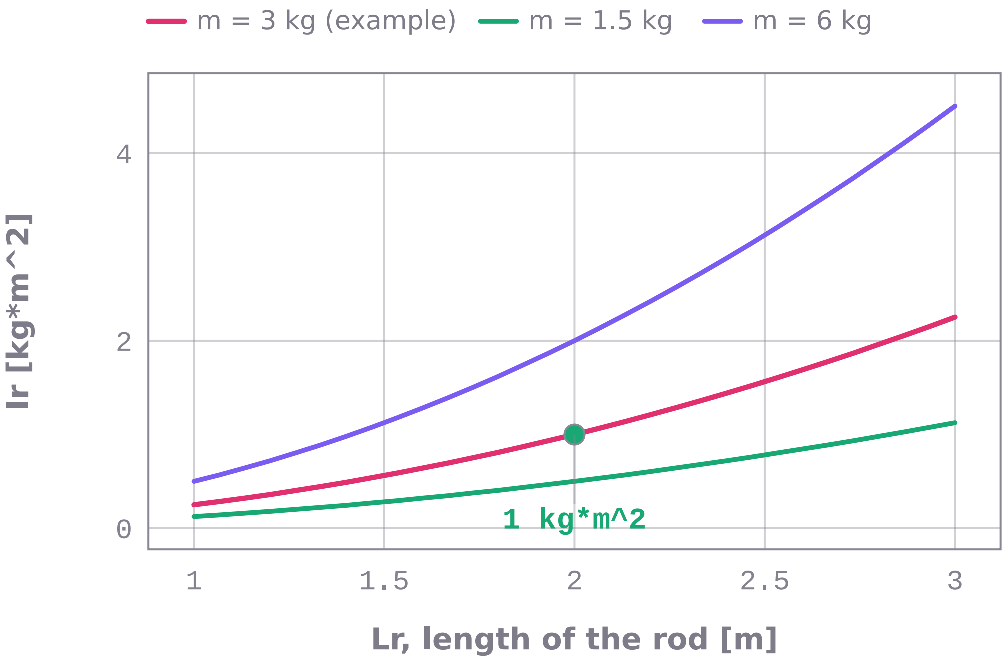
$$I_r = \frac{mL_r^2}{12}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about its center	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the rod	mass / plain	`kg`	
`Lr`	length of the rod	length / plain	`m`	

- **Conditions:** rod, about_center
- **Applies when:** a thin uniform rod rotating about its center
- **Fails when:** a thick rod; a non-uniform rod
- **Assumes:** a thin uniform rod; the axis perpendicular to the rod
- **Evidence:** theorem



Moments of inertia of bodies of the same mass m = 3 kg: spheres and a disc of radius R = 1 m, rods of length L = 2 m (= 2R) about their centre and about one end, each from the package; the highlighted bar is this formula's, the farther the mass from the axis, the larger I. Marked: I = 1 kg·m².



Ir (moment of inertia about its center) against Lr (length of the rod) from 1 m to 3 m. One curve for each value of m (mass of the rod): 3 kg, 1.5 kg, 6 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Lr = 2 m, Ir = 1 kg*m^2.

```
classical_physics_ai999x.solve('inertia_rod_center', m='3 kg', Lr='2 m', target='Ir')
# -> 1 kg*m^2   (SOLVED)
```

$$I_r = \frac{m L_r^2}{12}$$

$$I_r = \frac{L_r^2 m}{12} = 1\,\mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_rod_center needs m (mass of the rod, kg). Example: solve('inertia_rod_center', m='... kg', Lr='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

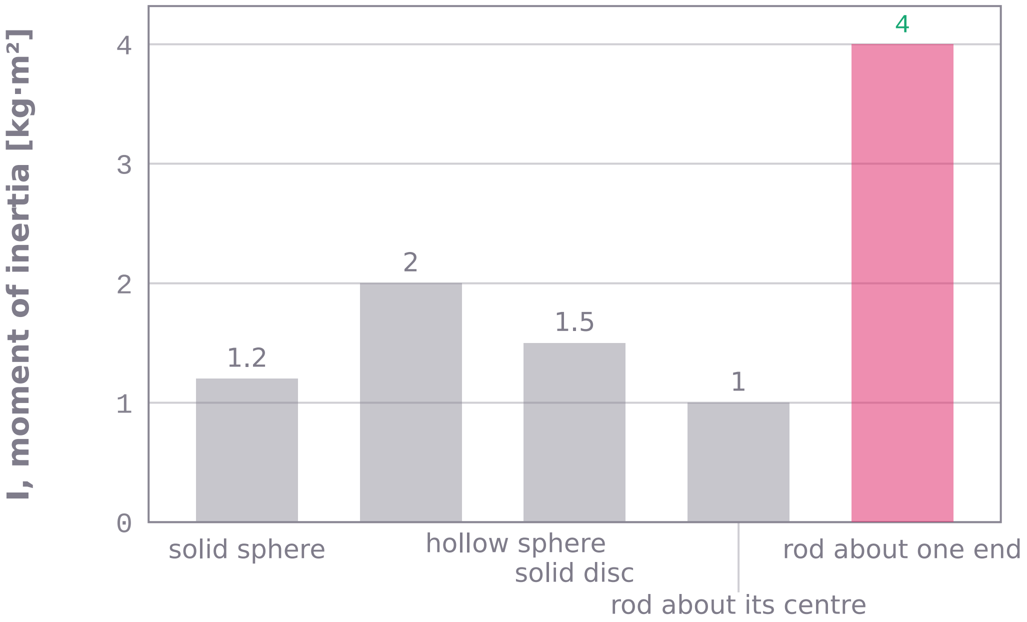
- "A uniform rod of mass 3 kg and length 2 m rotates about its center. What is its moment of inertia?" → 1 kg*m^2
- "Find the moment of inertia of a thin rod of mass 0.6 kg and length 1 m about its center." → 0.05 kg*m^2

Moment of inertia of a thin rod about one end, `inertia_rod_end`

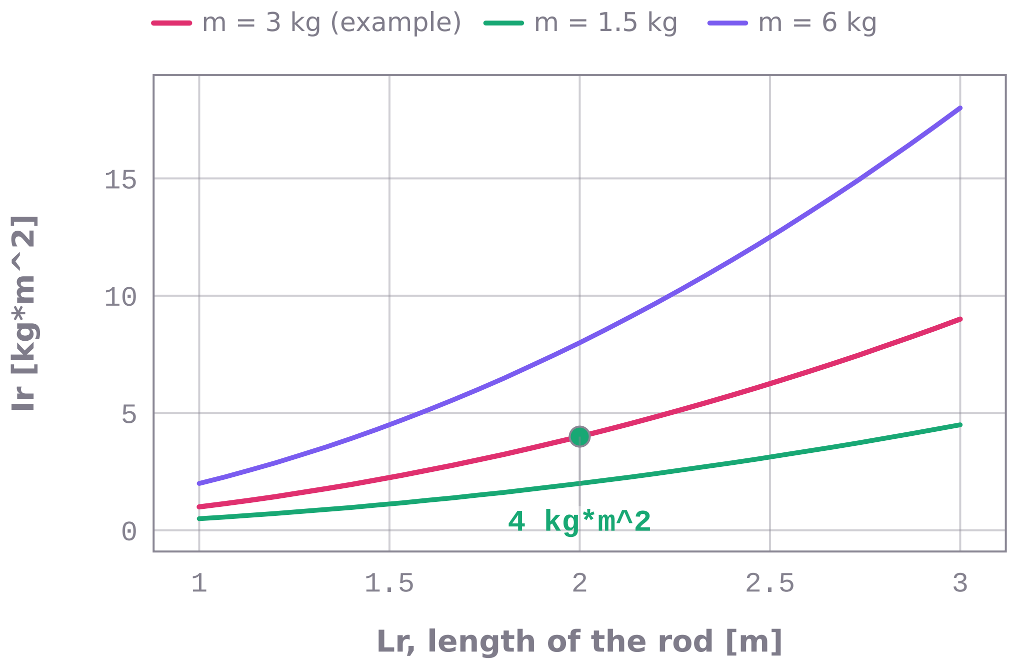
$$I_r = \frac{m L_r^2}{3}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about one end	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the rod	mass / plain	`kg`	
`Lr`	length of the rod	length / plain	`m`	

- **Conditions:** rod, about_end
- **Applies when:** a thin uniform rod rotating about one end
- **Fails when:** a thick rod; a non-uniform rod
- **Assumes:** a thin uniform rod; the axis perpendicular to the rod
- **Evidence:** theorem



Moments of inertia of bodies of the same mass m = 3 kg: spheres and a disc of radius R = 1 m, rods of length L = 2 m (= 2R) about their centre and about one end, each from the package; the highlighted bar is this formula's, the farther the mass from the axis, the larger I. Marked: I = 4 kg·m².



Ir (moment of inertia about one end) against Lr (length of the rod) from 1 m to 3 m. One curve for each value of m (mass of the rod): 3 kg, 1.5 kg, 6 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, Lr = 2 m, Ir = 4 kg*m^2.

```
classical_physics_ai999x.solve('inertia_rod_end', m='3 kg', Lr='2 m', target='Ir')
# -> 4 kg*m^2   (SOLVED)
```

$$I_r = \frac{mL_r^2}{3}$$

$$I_r = \frac{L_r^2 m}{3} = 4\,\mathrm{kg}\,\mathrm{m}^2$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_rod_end needs m (mass of the rod, kg). Example: solve('inertia_rod_end', m='... kg', Lr='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

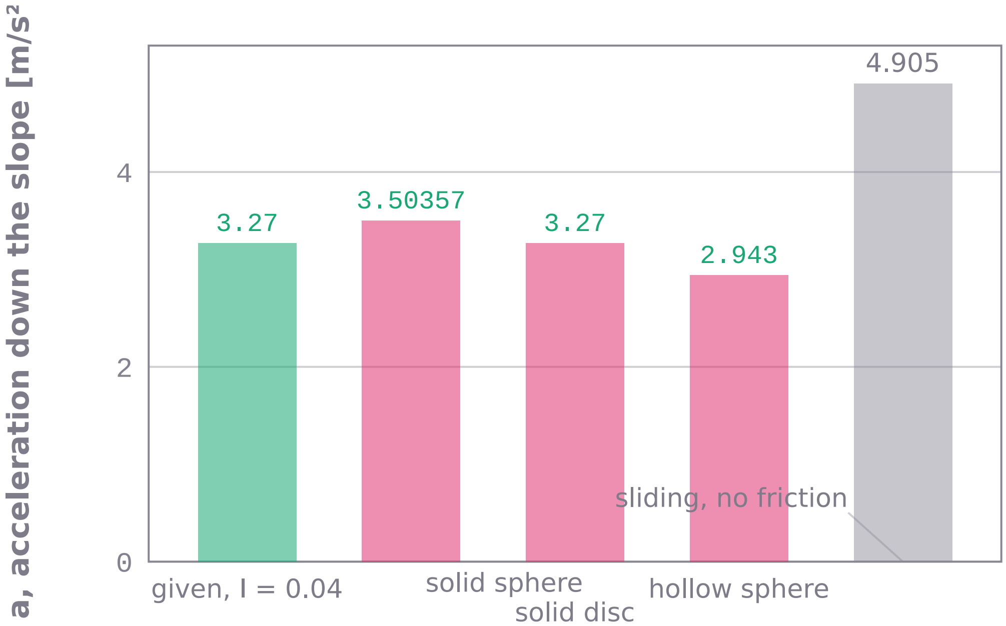
- "A uniform rod of mass 3 kg and length 2 m rotates about one end. What is its moment of inertia?" → 4 kg*m^2
- "Find the moment of inertia of a thin rod of mass 0.6 kg and length 1 m about one end." → 0.2 kg*m^2

Acceleration of a body rolling without slipping down an incline, `rolling_incline_acceleration`

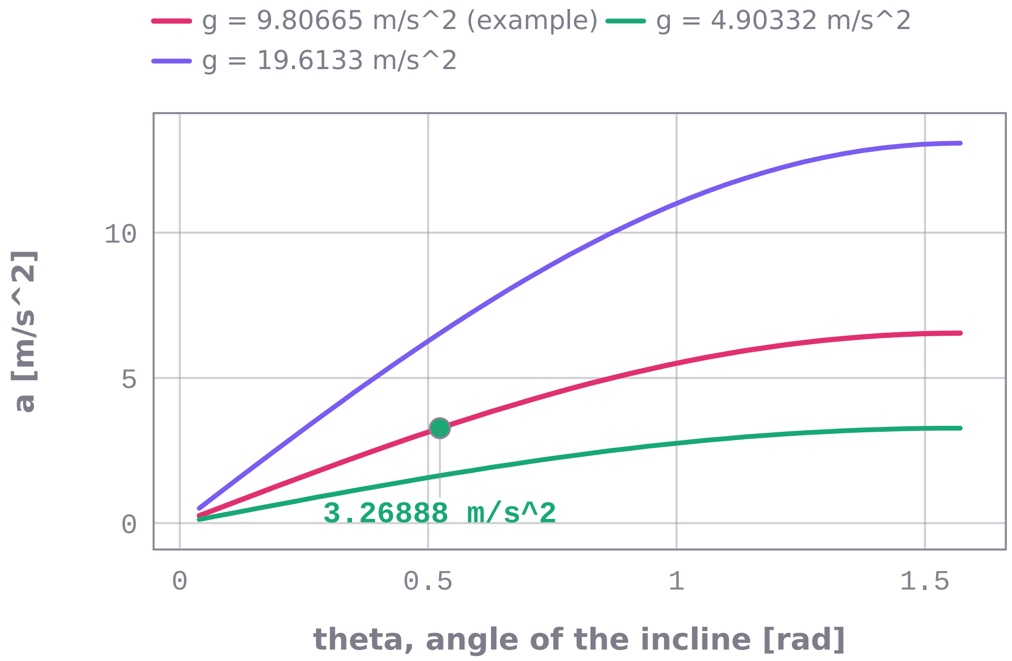
$$a = \frac{g \sin\left(\theta\right)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	kind / role	unit	default
`a`	acceleration of the center of mass along the incline	acceleration / plain	`m/s^2`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`theta`	angle of the incline	angle / incline	`rad`	
`Ir`	moment of inertia about the center	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	rolling radius	length / radius	`m`	

- **Conditions:** rolling
- **Applies when:** a round body rolling down a straight incline
- **Fails when:** slipping; rolling resistance
- **Assumes:** rolling without slipping; no rolling resistance
- **Evidence:** theorem



Acceleration down a 30° incline (m = 2 kg, R = 0.2 m, g = 9.81 m/s²) of the body as given (I = 0.04 kg·m², highlighted) and of a solid sphere, a solid disc and a hollow sphere of the same mass and radius rolling without slipping, with each moment of inertia from the package's inertia formulas (solid sphere, I = 0.032; solid disc, I = 0.04; hollow sphere, I = 0.0533333 kg·m²) and the rolling acceleration from its formula; grey: a block sliding without friction (no rotation). Marked: each acceleration.



a (acceleration of the center of mass along the incline) against theta (angle of the incline) from 0 rad to 1.5708 rad. One curve for each value of g (Acceleration): 9.80665 m/s^2, 4.90332 m/s^2, 19.6133 m/s^2. Other inputs as in the worked example: Ir = 0.04 kg*m^2, m = 2.0 kg, R = 0.2 m. 40 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 0.523599 rad, a = 3.26888 m/s^2.

```
classical_physics_ai999x.solve('rolling_incline_acceleration', theta='30 deg', Ir='0.04
kg*m^2', m='2 kg', R='0.2 m', target='a')
# -> 3.26888 m/s^2   (SOLVED)
```

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{m R^2}}$$

$$a = \frac{R^2 g m \sin(\theta)}{I_r + R^2 m} = 3.26888\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `theta`	NEEDS_INPUT	rolling_incline_acceleration needs theta (angle of the incline, rad). Example: solve('rolling_incline_acceleration', g='... m/s^2', theta='.
`theta` in a wrong unit	INVALID_INPUT	theta is a angle: its unit must be like rad, not 'kg'.

Questions in words that work:

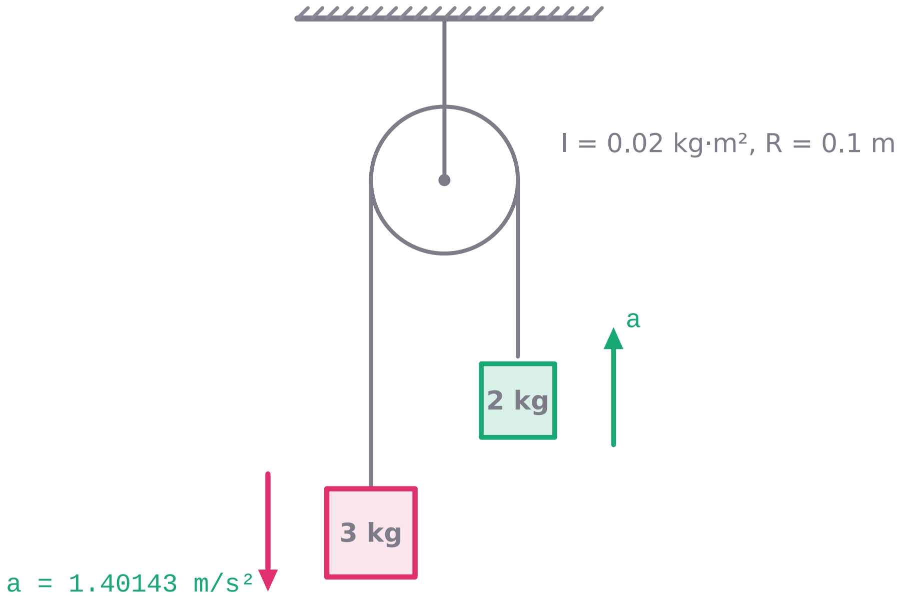
- "A cylinder of mass 2 kg and radius 0.2 m with a moment of inertia of 0.04 kg*m^2 rolls without slipping down an incline. The angle of the incline is 30 degrees. What is its acceleration?" → 3.26888 m/s^2
- "A wheel rolls without slipping down a slope inclined at 20 degrees. It has mass 4 kg, radius 0.5 m and moment of inertia 0.5 kg*m^2. Find its acceleration." → 2.23605 m/s^2

Acceleration of an Atwood machine with a massive pulley, `atwood_massive_pulley`

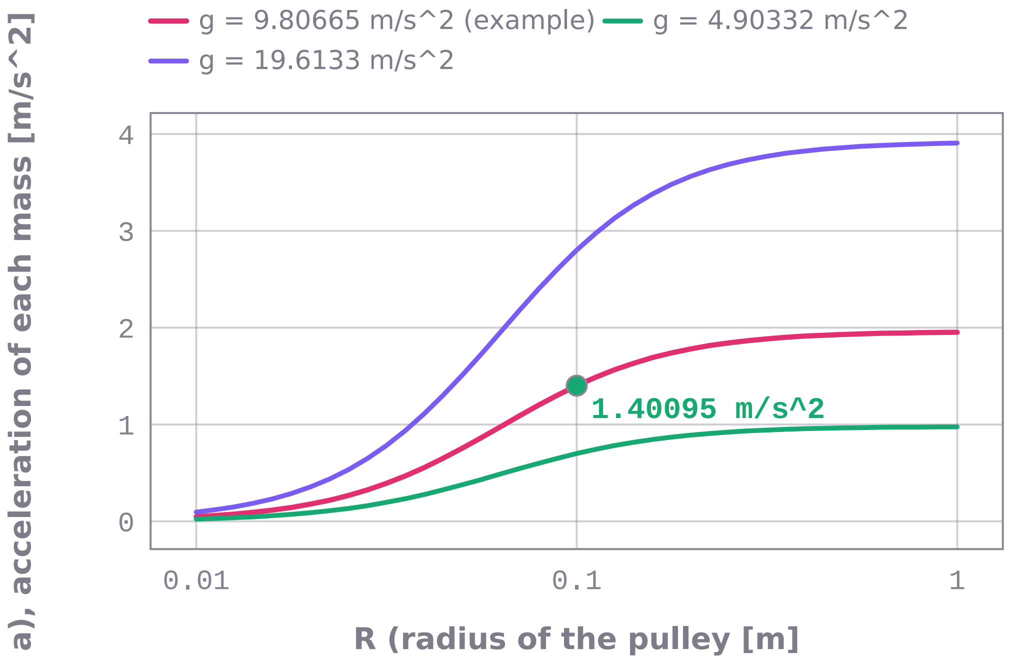
$$a = \frac{(m_1 - m_2) g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	kind / role	unit	default
`a`	acceleration of each mass	acceleration / plain	`m/s^2`	
`m1`	heavier mass	mass / heavy	`kg`	
`m2`	lighter mass	mass / light	`kg`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`Ir`	moment of inertia of the pulley	moment_of_inertia / plain	`kg*m^2`	
`R`	radius of the pulley	length / radius	`m`	

- **Conditions:** atwood
- **Applies when:** two masses hung over a pulley with rotational inertia
- **Fails when:** the string slips; axle friction
- **Assumes:** a massless string that does not slip on the pulley; a frictionless axle
- **Evidence:** theorem



An Atwood machine: masses m_1 = 3 kg and m_2 = 2 kg on a cord over a pulley with moment of inertia I = 0.02 kg·m² and radius R = 0.1 m (g = 9.81 m/s²). Marked: the acceleration the package returns, a = 1.40143 m/s² (m_1 down, m_2 up); with a pulley of negligible inertia (I = 1e-9 kg·m²) it would be 1.962 m/s², also returned.



a (acceleration of each mass) against R (radius of the pulley) from 0.01 m to 1 m on a logarithmic axis. One curve for each value of g (Acceleration): 9.80665 m/s^2, 4.90332 m/s^2, 19.6133 m/s^2. Other inputs as in the worked example: m1 = 3.0 kg, m2 = 2.0 kg, Ir = 0.02 kg*m^2. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 0.1 m, a = 1.40095 m/s^2.

```
classical_physics_ai999x.solve('atwood_massive_pulley', m1='3 kg', m2='2 kg', Ir='0.02
kg*m^2', R='0.1 m', target='a')
# -> 1.40095 m/s^2   (SOLVED)
```

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

$$a = \frac{R^2 g\,(m_1 - m_2)}{I_r + R^2 m_1 + R^2 m_2} = 1.40095\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	atwood_massive_pulley needs m1 (heavier mass, kg). Example: solve('atwood_massive_pulley', m1='... kg', m2='... kg', g='... m/s^2', Ir='...
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

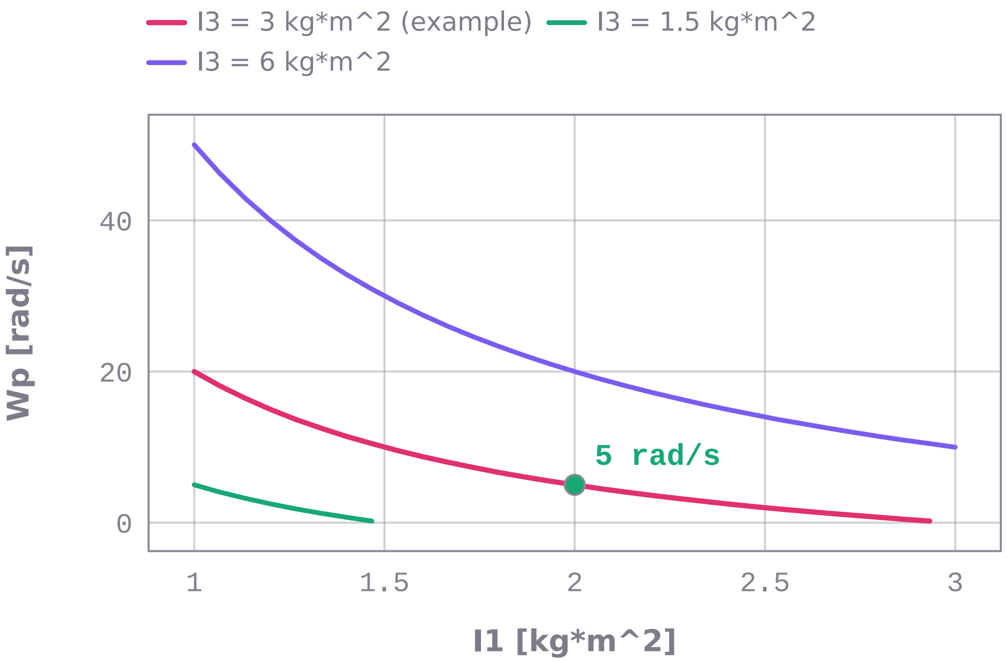
- "In an Atwood machine the heavier mass is 3 kg and the lighter mass is 2 kg. They hang over a pulley with a moment of inertia of 0.02 kg*m^2 and a radius of 0.1 m. What is the acceleration of the masses?" → 1.40095 m/s^2
- "Find the acceleration of an Atwood machine whose heavier mass is 5 kg and lighter mass is 1 kg, hung over a pulley of radius 0.5 m with a moment of inertia of 0.5 kg*m^2." → 4.90332 m/s^2

Body-frame precession rate of a torque-free symmetric body, `torque_free_precession`

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$

input	meaning	kind / role	unit	default
`Wp`	precession rate seen in the body frame	angular_frequency / precession	`rad/s`	
`w3`	spin angular velocity about the symmetry axis	angular_frequency / spin	`rad/s`	
`I1`	moment of inertia about a transverse axis	moment_of_inertia / transverse	`kg*m^2`	
`I3`	moment of inertia about the symmetry axis	moment_of_inertia / axial	`kg*m^2`	

- **Conditions:** torque_free
- **Applies when:** a symmetric rigid body spinning freely
- **Fails when:** external torques; an asymmetric body
- **Assumes:** a rigid symmetric body $\left(I_1 = I_2\right)$; no external torque; an oblate body $\left(I_3 > I_1\right)$
- **Evidence:** theorem



Wp (precession rate seen in the body frame) against I1 (moment of inertia about a transverse axis) from 1 kg*m^2 to 3 kg*m^2. One curve for each value of I3 (moment of inertia about the symmetry axis): 3 kg*m^2, 1.5 kg*m^2, 6 kg*m^2. Other inputs as in the worked example: w3 = 10.0 rad/s. 30 points per curve computed by classical-physics-ai999x; marked: the worked example, I1 = 2 kg*m^2, Wp = 5 rad/s.

```
classical_physics_ai999x.solve('torque_free_precession', w3='10 rad/s', I1='2 kg*m^2',
I3='3 kg*m^2', target='Wp')
# -> 5 rad/s   (SOLVED)
```

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$

$$\Omega_p = \frac{\omega_3 \left(- I_1 + I_3\right)}{I_1} = 5\ \mathrm{rad/s}$$

invalid call	outcome	message
without `w3`	NEEDS_INPUT	torque_free_precession needs w3 (spin angular velocity about the symmetry axis, rad/s). Example: solve('torque_free_precession', w3='... rad
`w3` in a wrong unit	INVALID_INPUT	w3 is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

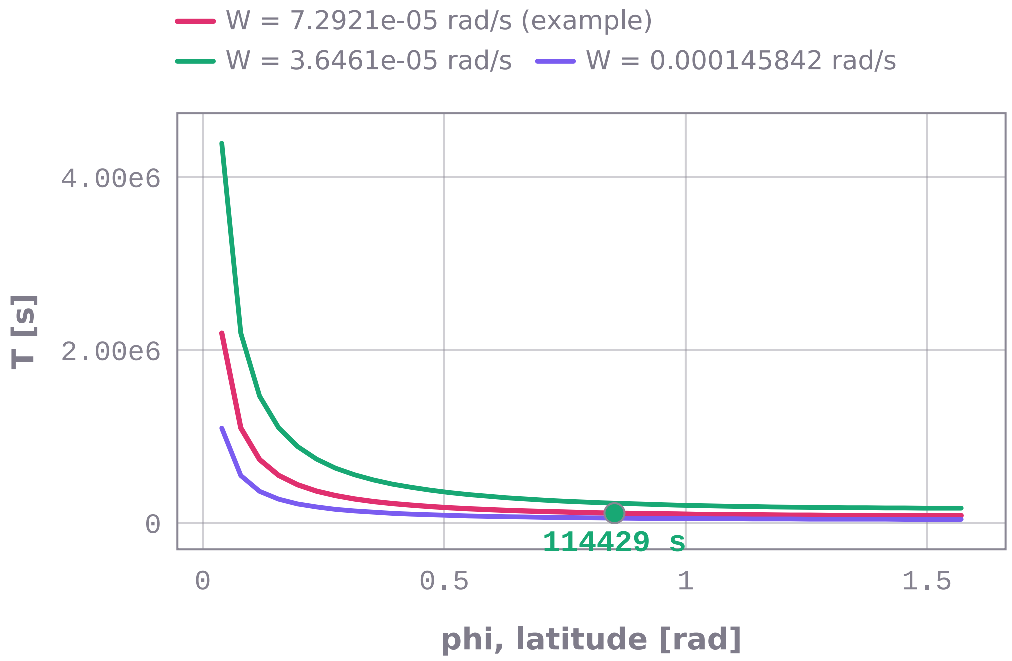
- "A torque-free symmetric body spins with a spin angular velocity of 10 rad/s. Its transverse moment of inertia is 2 kg*m^2 and its moment of inertia about the symmetry axis is 3 kg*m^2. What is the precession rate?" → 5 rad/s
- "In deep space a symmetric rigid body has a spin rate of 4 rad/s, a transverse moment of inertia of 1 kg*m^2 and an axial moment of inertia of 1.5 kg*m^2. Find its precession rate." → 2 rad/s

Precession period of a Foucault pendulum, `foucault_precession_period`

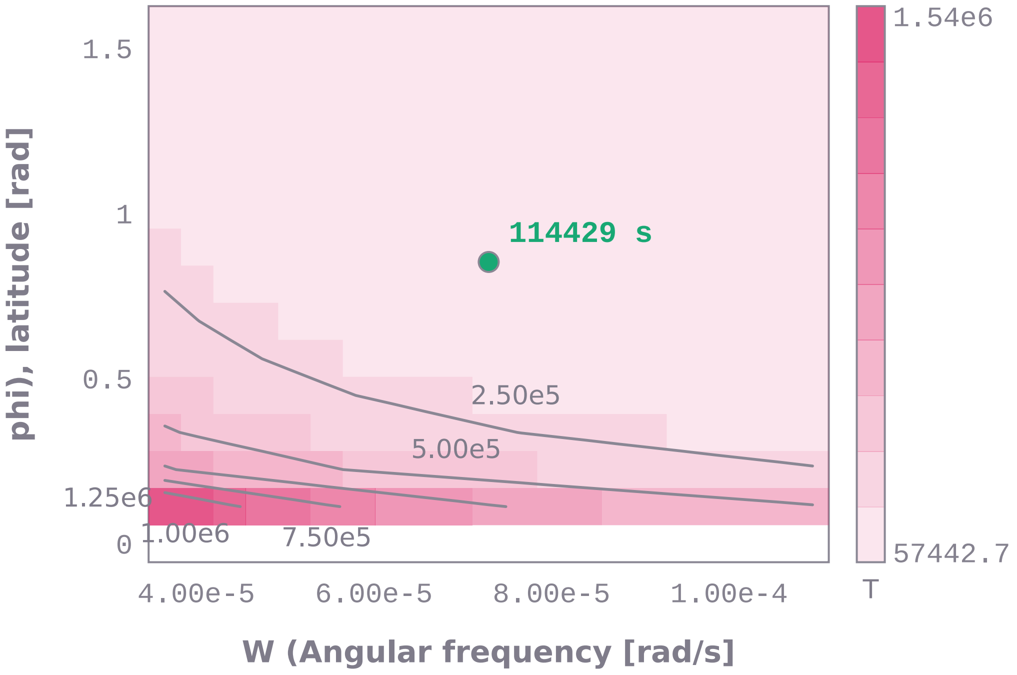
$$T = \frac{2\pi}{\Omega \sin\left(\phi\right)}$$

input	meaning	kind / role	unit	default
`T`	time for the swing plane to turn once	time / precession_period	`s`	
`W`		angular_frequency / rotation	`rad/s`	7.2921159e-05 rad/s
`phi`	latitude	angle / latitude	`rad`	

- **Conditions:** foucault
- **Applies when:** a long pendulum on a rotating planet
- **Fails when:** at the equator (no precession)
- **Assumes:** small oscillations; the rotation much slower than the swing
- **Evidence:** theorem



T (time for the swing plane to turn once) against phi (latitude) from 0 rad to 1.5708 rad. One curve for each value of W (Angular frequency): 7.2921e-05 rad/s, 3.6461e-05 rad/s, 0.000145842 rad/s. 40 points per curve computed by classical-physics-ai999x; marked: the worked example, phi = 0.852593 rad, T = 114429 s.



T (time for the swing plane to turn once) over W (Angular frequency) from 3.6461e-05 rad/s to 0.000109382 rad/s and phi (latitude) from 0 rad to 1.5708 rad: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 57442.7 s to dark 1.53913e+06 s); iso-lines at T = 2.50e5, 5.00e5, 7.50e5, 1.00e6, 1.25e6 s; blank cells: inputs the capability refuses. Marked: the worked example, W = 7.2921e-05 rad/s, phi = 0.852593 rad, T = 114429 s.

```
classical_physics_ai999x.solve('foucault_precession_period', phi='48.85 deg', target='T')
# -> 114429 s   (SOLVED)
```

$$T = \frac{2\pi}{\Omega \sin\left(\phi\right)}$$
$$T = \frac{2\pi}{\Omega \sin\left(\phi\right)} = 114429\ \mathrm{s}$$

invalid call	outcome	message
without `phi`	NEEDS_INPUT	foucault_precession_period needs phi (latitude, rad). Example: solve('foucault_precession_period', W='... rad/s', phi='... rad').
`phi` in a wrong unit	INVALID_INPUT	phi is a angle: its unit must be like rad, not 'kg'.

Questions in words that work:

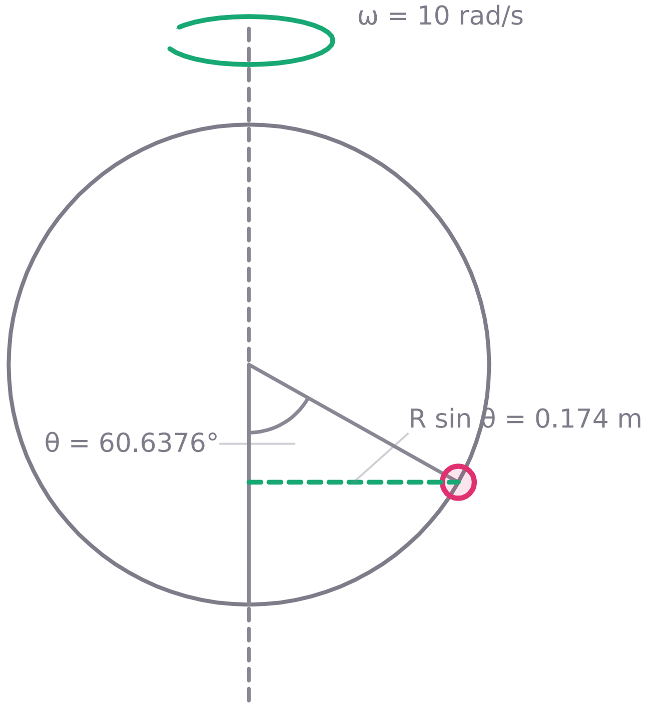
- "A Foucault pendulum swings in Paris at a latitude of 48.85 degrees. What is its precession period?" → 114429 s
- "Find the precession period of a Foucault pendulum at the North Pole, latitude 90 degrees." → 86164.1 s

Off-axis equilibrium of a bead on a rotating hoop, `bead_hoop_equilibrium`

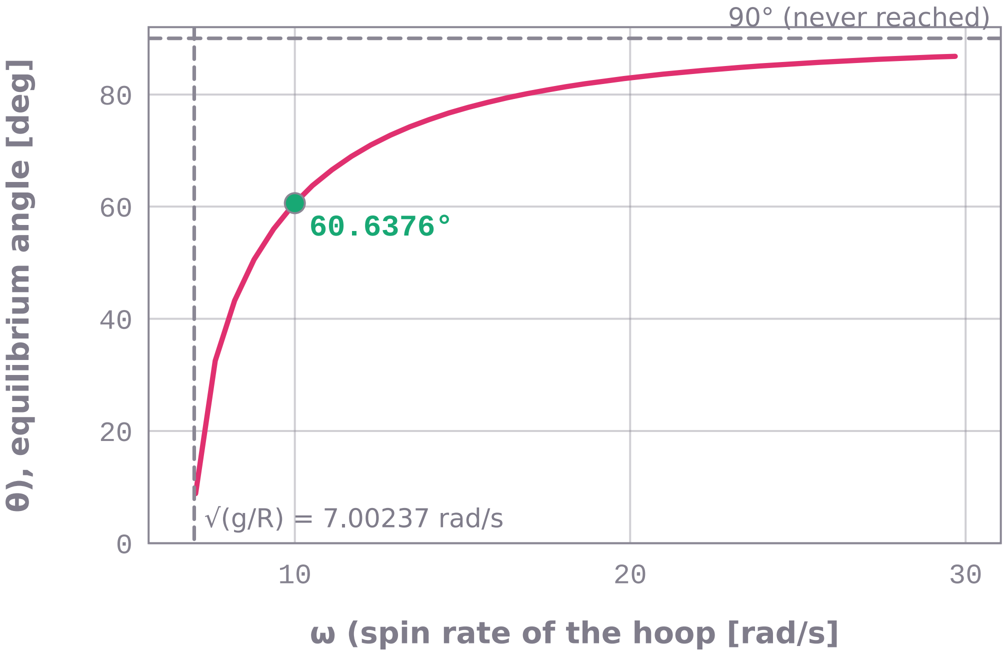
$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	kind / role	unit	default
`theta`	equilibrium angle from the bottom	angle / equilibrium	`rad`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`w`	angular velocity of the hoop	angular_frequency / rotation	`rad/s`	
`R`	radius of the hoop	length / radius	`m`	

- **Conditions:** hoop
- **Applies when:** a bead free to slide on a vertical hoop spinning about a vertical diameter
- **Fails when:** $w^2\ R \leq g$ (the bead stays at the bottom)
- **Assumes:** a frictionless bead; the hoop turning about its vertical diameter; $w^2\ R > g$
- **Evidence:** theorem



A bead free to slide on a hoop of radius R = 0.2 m spinning about its vertical diameter at ω = 10 rad/s (g = 9.80665 m/s²): it settles where gravity and the required centripetal force balance. Marked: the equilibrium angle the package returns, θ = 1.05833 rad = 60.6376° from the bottom (degrees converted), and the radius of the bead's circle R sin θ (derived).



Equilibrium angle of a bead on a hoop of radius 0.2 m (g = 9.80665 m/s²) against the spin rate, 40 rates from the package (angles converted to degrees): below ω = √(g/R) = 7.00237 rad/s (derived from the inputs) the bead stays at the bottom and no off-bottom equilibrium is returned. Marked: 60.6376° at ω = 10 rad/s.

```
classical_physics_ai999x.solve('bead_hoop_equilibrium', w='10 rad/s', R='0.2 m',
target='theta')
# -> 1.05833 rad   (SOLVED)
```

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

$$\theta = \operatorname{acos}\left(\frac{g}{R\omega^2}\right) = 1.05833\ \mathrm{rad}$$

invalid call	outcome	message
without `w`	NEEDS_INPUT	bead_hoop_equilibrium needs w (angular velocity of the hoop, rad/s). Example: solve('bead_hoop_equilibrium', g='... m/s^2', w='... rad/s', R
`w` in a wrong unit	INVALID_INPUT	w is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

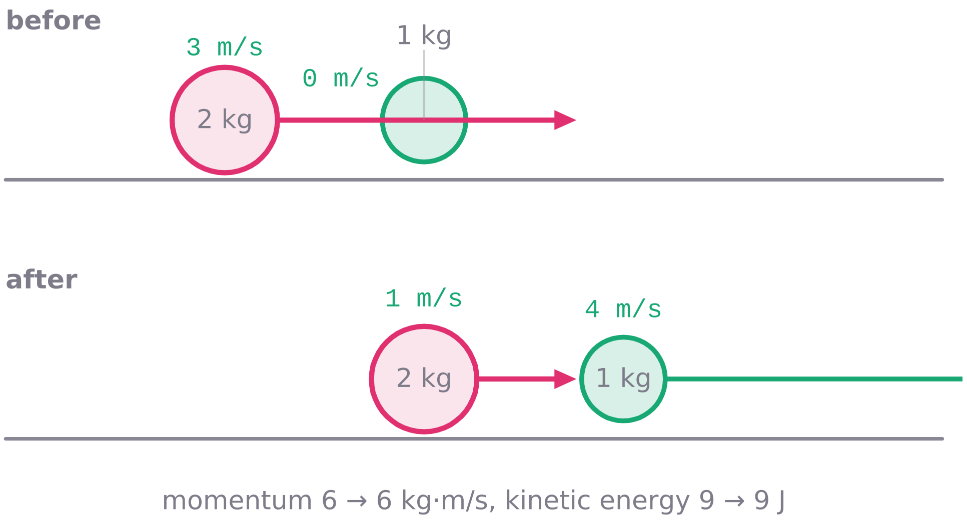
- "A bead slides without friction on a rotating hoop of radius 0.2 m. The hoop rotates at 10 rad/s about its vertical diameter. What is the equilibrium angle of the bead?" → 1.05833 rad
- "A bead on a hoop of radius 1 m rotating at 5 rad/s about a vertical diameter: find the equilibrium angle." → 1.1677 rad

Final velocity of the first body in a one-dimensional elastic collision, `elastic_collision_first_final`

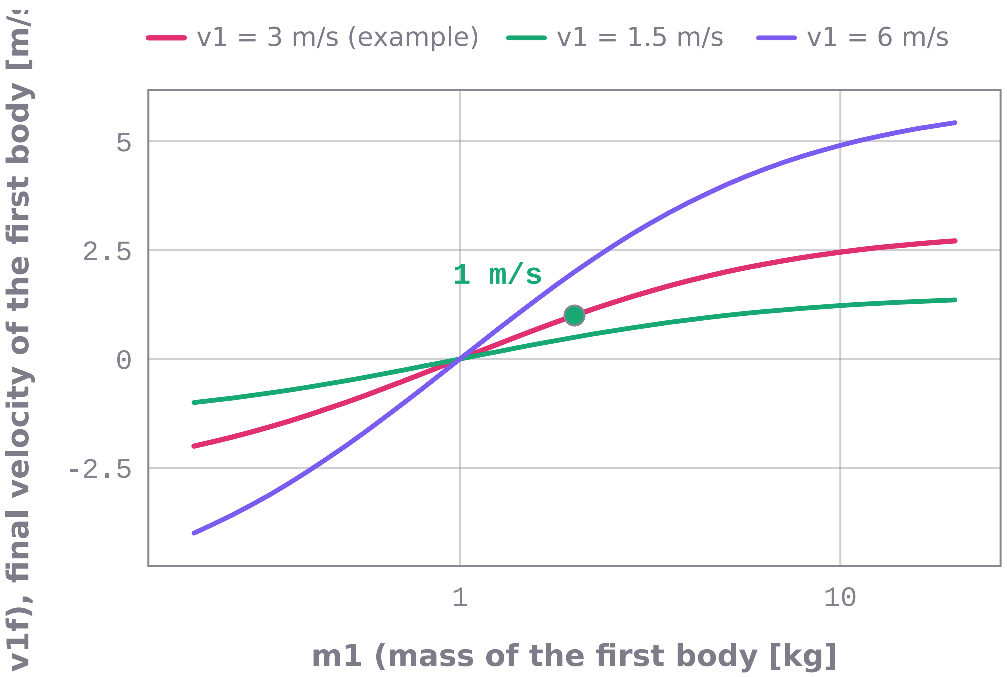
$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	kind / role	unit	default
`v1f`	final velocity of the first body	speed / first_final	`m/s`	
`m1`	mass of the first body	mass / first	`kg`	
`m2`	mass of the second body	mass / second	`kg`	
`v1`	initial velocity of the first body	speed / first_initial	`m/s`	
`v2`	initial velocity of the second body	speed / second_initial	`m/s`	

- **Conditions:** elastic_collision
- **Applies when:** two bodies colliding elastically along a line
- **Fails when:** inelastic collisions; glancing collisions
- **Assumes:** a head-on (one-dimensional) collision; kinetic energy conserved; no external forces
- **Evidence:** theorem



A head-on elastic collision: a 2 kg body at 3 m/s strikes a 1 kg body at rest; velocity arrows to scale (1 length unit per m/s). Marked: the final velocities the package returns, 1 m/s for the first and 4 m/s for the second; total momentum and kinetic energy before and after (derived from those values) are unchanged.



v1f (final velocity of the first body) against m1 (mass of the first body) from 0.2 kg to 20 kg on a logarithmic axis. One curve for each value of v1 (initial velocity of the first body): 3 m/s, 1.5 m/s, 6 m/s. Other inputs as in the worked example: m2 = 1.0 kg, v2 = 0.0 m/s. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, m1 = 2 kg, v1f = 1 m/s.

```
classical_physics_ai999x.solve('elastic_collision_first_final', m1='2 kg', m2='1 kg', v1='3
m/s', v2='0 m/s', target='v1f')
# -> 1 m/s   (SOLVED)
```

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

$$v_{1f} = \frac{m_1v_1 - m_2v_1 + 2m_2v_2}{m_1 + m_2} = 1\ \mathrm{m/s}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	elastic_collision_first_final needs m1 (mass of the first body, kg). Example: solve('elastic_collision_first_final', m1='... kg', m2='... kg
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

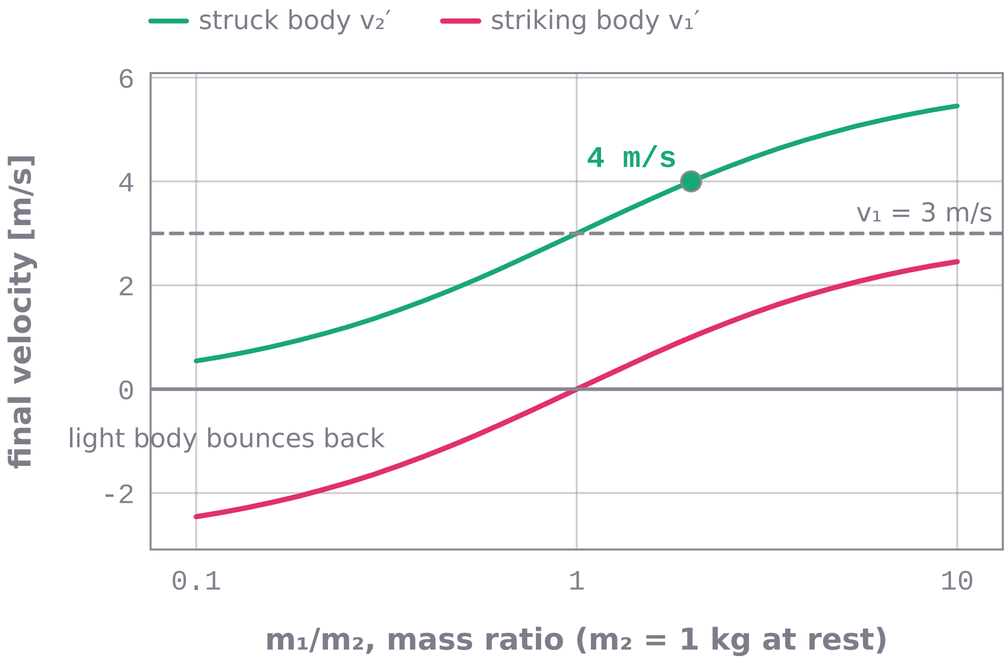
- "In an elastic collision the first mass is 2 kg with a velocity of the first of 3 m/s, and the second mass is 1 kg with a velocity of the second of 0 m/s. What is the final velocity of the first?" → 1 m/s
- "Two carts collide elastically: the first mass is 1 kg and the velocity of the first is 4 m/s; the second mass is 3 kg and the velocity of the second is -2 m/s. Find the final velocity of the first." → -5 m/s

Final velocity of the second body in a one-dimensional elastic collision, `elastic_collision_second_final`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	kind / role	unit	default
`v2f`	final velocity of the second body	speed / second_final	`m/s`	
`m1`	mass of the first body	mass / first	`kg`	
`m2`	mass of the second body	mass / second	`kg`	
`v1`	initial velocity of the first body	speed / first_initial	`m/s`	
`v2`	initial velocity of the second body	speed / second_initial	`m/s`	

- **Conditions:** elastic_collision
- **Applies when:** two bodies colliding elastically along a line
- **Fails when:** inelastic collisions; glancing collisions
- **Assumes:** a head-on (one-dimensional) collision; kinetic energy conserved; no external forces
- **Evidence:** theorem



Final velocities after a head-on elastic collision of a body of mass m_1 moving at v_1 = 3 m/s with a 1 kg body at rest, against the mass ratio from 0.1 to 10 (31 ratios, both velocities from the package): equal masses exchange velocities, a heavy striker sends the target off at up to twice v_1. Marked: v_2' = 4 m/s for the worked example m_1 = 2 kg.

```
classical_physics_ai999x.solve('elastic_collision_second_final', m1='2 kg', m2='1 kg',
v1='3 m/s', v2='0 m/s', target='v2f')
# -> 4 m/s   (SOLVED)
```

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

$$v_{2f} = \frac{2m_1v_1 - m_1v_2 + m_2v_2}{m_1 + m_2} = 4\ \mathrm{m/s}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	elastic_collision_second_final needs m1 (mass of the first body, kg). Example: solve('elastic_collision_second_final', m1='... kg', m2='...
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

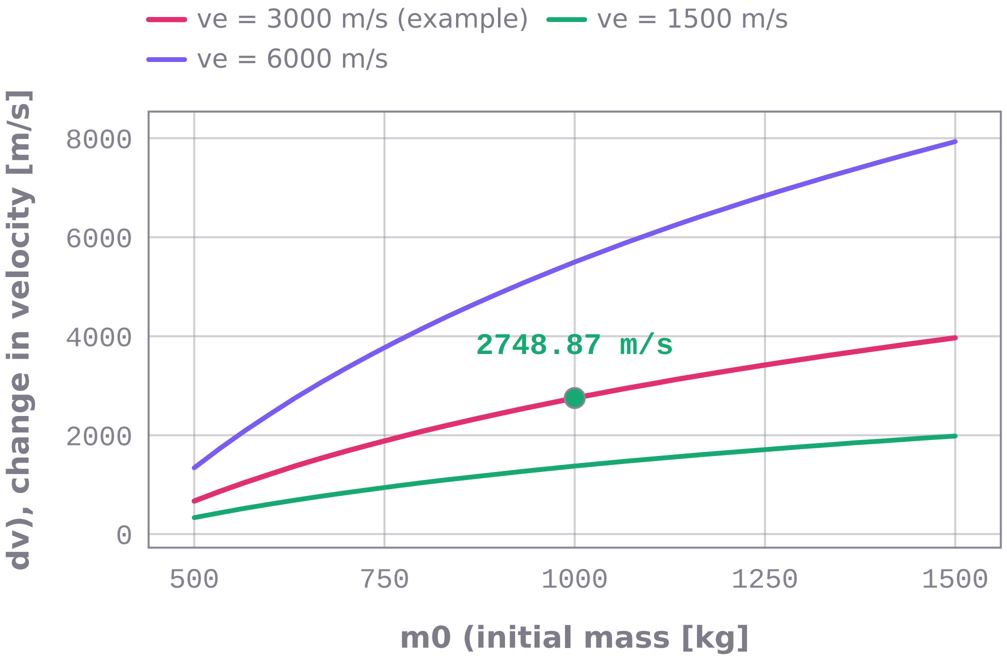
- "In an elastic collision the first mass is 2 kg with a velocity of the first of 3 m/s, and the second mass is 1 kg with a velocity of the second of 0 m/s. What is the final velocity of the second?" → 4 m/s
- "Two carts collide elastically: the first mass is 1 kg and the velocity of the first is 4 m/s; the second mass is 3 kg and the velocity of the second is -2 m/s. Find the final velocity of the second." → 1 m/s

Velocity change of a rocket (Tsiolkovsky), `rocket_delta_v`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	kind / role	unit	default
`dv`	change in velocity	speed / delta_v	`m/s`	
`ve`	exhaust velocity	speed / exhaust	`m/s`	
`m0`	initial mass	mass / initial	`kg`	
`mf`	final mass	mass / final	`kg`	

- **Conditions:** rocket
- **Applies when:** a rocket burning fuel in free space
- **Fails when:** gravity losses; atmospheric drag
- **Assumes:** no gravity or drag during the burn; a constant exhaust velocity
- **Evidence:** theorem



dv (change in velocity) against m0 (initial mass) from 500 kg to 1500 kg. One curve for each value of ve (exhaust velocity): 3000 m/s, 1500 m/s, 6000 m/s. Other inputs as in the worked example: mf = 400.0 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, m0 = 1000 kg, dv = 2748.87 m/s.

```
classical_physics_ai999x.solve('rocket_delta_v', ve='3000 m/s', m0='1000 kg', mf='400 kg',
target='dv')
# -> 2748.87 m/s   (SOLVED)
```

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

$$\Delta v = \ln\left(\left(\frac{m_0}{m_f}\right)^{v_e}\right) = 2748.87\ \mathrm{m/s}$$

invalid call	outcome	message
without `ve`	NEEDS_INPUT	rocket_delta_v needs ve (exhaust velocity, m/s). Example: solve('rocket_delta_v', ve='... m/s', m0='... kg', mf='... kg').
`ve` in a wrong unit	INVALID_INPUT	ve is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

- "A rocket with an exhaust velocity of 3000 m/s has an initial mass of 1000 kg and a final mass of 400 kg after the burn. What is its change in velocity?" → 2748.87 m/s
- "Find the delta-v of a rocket whose exhaust speed is 4400 m/s, with an initial mass of 50000 kg and a final mass of 10000 kg." → 7081.53 m/s

Model families: several bodies, states or stages

These answer questions in words only; each states its model, assumptions and the equations it solved.

Two masses over a pulley, `atwood`

Builds on: `newton_second`, `tension_hanging_at_rest`, `weight`

question	outcome	result
Two masses of 3 kg and 5 kg hang from either side of a pulley. What is the acceleration?	SOLVED	2.45166 m/s^2
A 4 kg block and a 2 kg block are connected by a string over a pulley. What is the tension?	NEEDS_INPUT	Two masses joined over a pulley: say whether both hang (one on each side) or one rests on a surface (and whether that su

Ideal banking of a curve, `banked_curve`

Builds on: `centripetal_acceleration`, `centripetal_force`, `weight`

question	outcome	result
A curve of radius 80 m is banked for a speed of 20 m/s. What is the banking angle?	SOLVED	0.471503 rad

Mixing to thermal equilibrium, `calorimetry`

Builds on: `heat`

question	outcome	result
A 0.2 kg piece of aluminum at 90 °C is dropped into 0.5 kg of water at 20 °C. What is the final temperature?	SOLVED	298.693 K
A 0.05 kg ice cube at -10 °C is dropped into 0.3 kg of water at 25 °C. What is the final temperature?	OUT_OF_SCOPE	Mixing with a phase change (melting, boiling) is not supported yet.

Collisions along a line, `collision_1d`

Builds on: `impulse`, `kinetic_energy`, `momentum`

question	outcome	result
A 1000 kg car moving at 20 m/s hits a stationary 500 kg car and they stick together. What is their final velocity?	SOLVED	13.3333 m/s
A 2 kg ball moving at 3 m/s collides elastically with a 1 kg ball at rest. What is the velocity of the 1 kg ball after the collision?	SOLVED	4 m/s

question	outcome	result
A 0.5 kg ball of clay moving at 10 m/s hits a 2.0 kg block at rest and sticks to it. How much kinetic energy is lost?	SOLVED	20 J
A 2 kg ball moving at 3 m/s hits a 1 kg ball at rest. What is the speed of the 1 kg ball afterwards?	NEEDS_INPUT	A collision with neither 'they stick together' nor 'elastic' nor a stated velocity after it is not determined: say which

Doppler effect for sound, `doppler`

Builds on: `period_frequency`, `wave_speed`

question	outcome	result
A train whistle of frequency 500 Hz approaches a stationary listener at 30 m/s. The speed of sound is 340 m/s. What frequency does the listener hear?	SOLVED	548.387 Hz

Apparent weight in an elevator, `elevator`

Builds on: `newton_second`, `normal_on_level`, `weight`

question	outcome	result
A 60 kg person stands on a scale in an elevator accelerating upward at 2 m/s^2. What does the scale read?	SOLVED	708.399 N
A 70 kg person stands on a scale in an elevator that accelerates at 2 m/s^2. What does the scale read?	AMBIGUOUS	Say whether the elevator's acceleration points up or down.

Maximum speed on a flat curve, `flat_curve`

Builds on: `centripetal_acceleration`, `centripetal_force`, `kinetic_friction`, `weight`

question	outcome	result
A car rounds a flat curve of radius 50 m. The coefficient of static friction is 0.60. What is the maximum speed on the flat curve?	SOLVED	17.1522 m/s

A gas between two states, `gas_two_states`

Builds on: `ideal_gas_moles`

question	outcome	result
A gas at 2.0 atm occupies 4.0 L at 300 K. It is compressed to 2.0 L at constant temperature. What is the new pressure?	SOLVED	405300 Pa

Radioactive decay, `half_life`

Builds on:

question	outcome	result
A sample of 80 g of a radioactive isotope has a half-life of 5 days. What mass remains after 15 days?	SOLVED	0.01 kg

Series and parallel resistor and capacitor networks, `network`

Builds on: `capacitance_definition`, `capacitor_energy`, `electric_power`, `ohm`, `ohm_power_current`, `ohm_power_voltage`

question	outcome	result
Three resistors of 2 ohm, 3 ohm and 6 ohm are connected in parallel to a 12 V battery. What is the total current?	SOLVED	12 A
Two capacitors of 2 uF and 3 uF are connected in series to a 10 V battery. What is the charge on the 2 uF capacitor?	SOLVED	1.2e-05 C

Projectile motion, `projectile`

Builds on: `displacement_time`, `uniform_motion`, `velocity_displacement`, `velocity_time`

question	outcome	result
A ball is kicked at 20 m/s at an angle of 30 degrees above the horizontal. How far does it travel?	SOLVED	35.324 m
A stone is thrown horizontally at 15 m/s from a cliff 45 m high. How long is it in the air?	SOLVED	3.02943 s

Block on a surface pulled over a pulley, `pulley_table`

Builds on: `kinetic_friction`, `newton_second`, `normal_on_level`, `tension_hanging_at_rest`, `weight`

question	outcome	result
A 4 kg block on a frictionless table is connected over a pulley to a hanging 1 kg mass. What is the tension in the string?	SOLVED	7.84532 N

RC circuits, `rc_circuit`

Builds on: `capacitance_definition`, `ohm`

question	outcome	result
A 100-pF capacitor is charged through a 75.0-MΩ resistor. How long does it take to reach 90.0% of its final voltage?	SOLVED	0.0172694 s

Recoil from rest, `recoil`

Builds on: `momentum`

question	outcome	result
A 4.0 kg rifle initially at rest fires a 0.010 kg bullet at 600 m/s. What is the recoil speed of the rifle?	SOLVED	1.5 m/s

Rolling from a height, `rolling`

Builds on: `gravitational_pe`, `kinetic_energy`, `rotational_kinetic_energy`

question	outcome	result
A solid sphere rolls without slipping down a ramp from a height of 2 m. What is its speed at the bottom?	SOLVED	5.2933 m/s

Motion in stages, `stages`

Builds on: `average_velocity_displacement`, `displacement_time`, `uniform_motion`, `velocity_displacement`, `velocity_time`

question	outcome	result
A car starts from rest and accelerates at 2 m/s^2 for 10 s, then moves at constant speed for 20 s. What is the total distance traveled?	SOLVED	500 m

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
coulomb_force	Coulomb's law: force between two point charges	q1, q2, r	F, F_radial
gravitational_force_two_masses	Newton's law of universal gravitation: force between two masses	m1, m2, r	F
snell_refraction_angle	Snell's law: angle of refraction	n1, n2, theta1	theta2, theta2_deg
critical_angle_total_internal_reflection	Critical angle for total internal reflection	n1, n2	theta_c, theta_c_deg
lens_mirror_magnification	Magnification and image height of a thin lens or a spherical mirror	d_o, h_o, d_i, f	m, h_i, d_i, image
double_slit_bright_fringe	Double-slit interference: angle and position of a bright fringe	d, lam, m, L	y, theta, theta_deg, dy
diffraction_grating_order	Diffraction grating: angle of an order and the highest visible order	lam, m, d, N	theta, theta_deg, m_max, d
single_slit_minimum	Single-slit diffraction: dark fringes and the width of the central maximum	a, lam, m, L	theta, theta_deg, y, W, W_angle
malus_law_polarizer	Malus's law: intensity through a polarizer	I0, theta, light	I, fraction
faraday_induced_emf	Faraday's law: average emf induced by a change of magnetic flux	N, A1, B1, B2, dt, A2, theta	emf, emf_signed, phi1, phi2, dphi
ideal_transformer_secondary	Ideal transformer: secondary voltage and current	Np, Ns, Vp, Ip	Vs, Is, ratio, type
series_rlc_ac_circuit	Series RLC circuit on an AC supply: impedance, current, phase and power	R, L, C, f, V	I, Z, phi, phi_deg, pf, P, XL, XC
terminal_voltage_internal_resistance	Terminal voltage of a source with internal resistance	emf, r, R	I, V_t, V_lost, P_load, efficiency
static_friction_limit	Static friction: maximum friction force and whether a body slips	mu_s, N, F	f_max, slips, margin

id	operation	inputs	outputs
static_friction_incline	Static friction on an incline: critical angle and whether a body slides	mu_s, theta, m	theta_c, theta_c_deg, slips, N, f_needed, f_max
youngs_modulus_deformation	Tension or compression: stress, strain and change of length (Young's modulus)	F, A, L0, E	dL, stress, strain
shear_modulus_deformation	Shear: shear stress, shear strain and displacement (shear modulus)	F, A, h, S	dx, stress, strain
bulk_modulus_compression	Bulk compression: volume change under a pressure change (bulk modulus)	dP, V0, B	dV, strain, V
beam_support_reactions	Static equilibrium of a beam on two supports: support reactions	loads, positions, xA, xB	RA, RB, total
centre_of_mass_point_masses	Centre of mass of a set of point masses	masses, x, y	x_cm, y_cm, M
vector_components_resultant	Vector components and the resultant of two-dimensional vectors	magnitudes, directions	R, direction_deg, Rx, Ry, x_components, y_components
rotational_second_law	Newton's second law for rotation: angular acceleration from torque	tau, I	alpha
angular_momentum_conservation	Conservation of angular momentum: change of spin rate	I1, omega1, I2	omega2, L, K1, K2, dK
sound_intensity_level	Sound intensity level in decibels	I, beta	beta, I
sound_level_change	Change of sound level and the ratio of two intensities	beta1, beta2, ratio	d_beta, ratio, beta2
standing_wave_harmonics	Standing waves: harmonics of a string and of open or closed pipes	system, L, v, n	f_n, lam_n, f1
string_wave_speed	Wave speed on a stretched string	T, mu	v
gauss_law_symmetric_field	Gauss's law: electric field of a line, a plane, a charged sphere or a shell	geometry, Q, r, L, A, R	E, E_radial, Q_enc, flux, density
two_resistors_series_parallel	Two resistors in series and in parallel	r1, r2	rs, rp
uniform_acceleration_state	Velocity and displacement after constant acceleration	u, a, t	v, s

Coulomb's law: force between two point charges, `coulomb_force`

The electrostatic force between two point charges a distance r apart: its magnitude and its sign along the line joining them (positive: repulsion, negative: attraction).

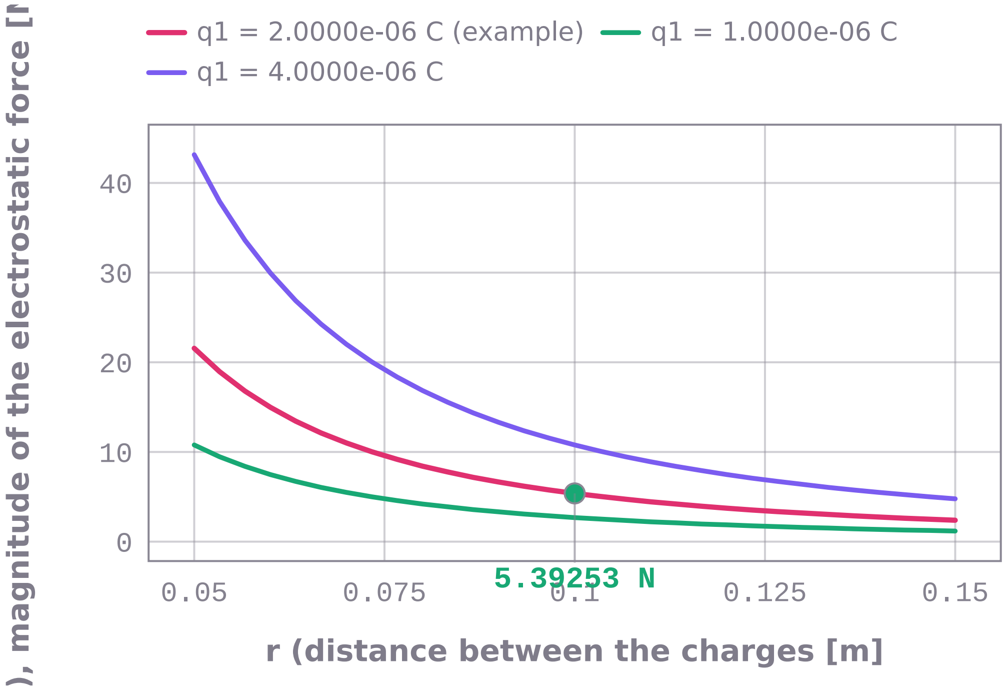
$$F = k\,\frac{|q_1 q_2|}{r^2}$$

input	meaning	condition
`q1`	first charge (with its sign)	
`q2`	second charge (with its sign)	

input	meaning	condition
`r`	distance between the charges	$r \neq 0$

output	meaning
`F`	magnitude of the electrostatic force
`F_radial`	force along the line joining the charges (positive: repulsion)

- **Method:** $F = k$ q1 q_2/r^2 with $k = 8.9875517923e9$ N m^2/C^2; the sign of q1 q2 tells repulsion or attraction
- **Assumes:** $r > 0$; point charges in vacuum (or air)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F (magnitude of the electrostatic force) against r (distance between the charges) from 0.05 m to 0.15 m. One curve for each value of q1 (first charge (with its sign)): 2.0000e-06 C, 1.0000e-06 C, 4.0000e-06 C. Other inputs as in the worked example: q2 = 3e-06 C. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.1 m, F = 5.39253 N.

```
classical_physics_ai999x.solve('coulomb_force', q1=2e-06, q2=3e-06, r=0.1)
# -> F = 5.39253 N, F_radial = 5.39253 N   (SOLVED)
```

```
classical_physics_ai999x.solve('coulomb_force', q1=1e-06, q2=-4e-06, r=0.2)
# -> F = 0.898755 N, F_radial = -0.898755 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coulomb's law: q1 = 2 uC, q2 = 3 uC, r = 10 cm	SOLVED	F = 5.39253 N, F_radial = 5.39253 N
Coulomb's law: q1 = 1 uC, q2 = -4 uC, r = 20 cm	SOLVED	F = 0.898755 N, F_radial = -0.898755 N
Coulomb's law: q1 = 2 uC, q2 = 3 uC, r = 0 m	INVALID_INPUT	Coulomb's law: force between two point charges: the separation r must be positive.

question	outcome	result
Coulomb's law: q1 = 2 uC, q2 = 3 uC	NEEDS_INPUT	coulomb_force needs r (distance between the charges). Example: solve('coulomb_force', q1='2e-06 C', q2='3e-06 C', r='0.1

Newton's law of universal gravitation: force between two masses, `gravitational_force_two_masses`

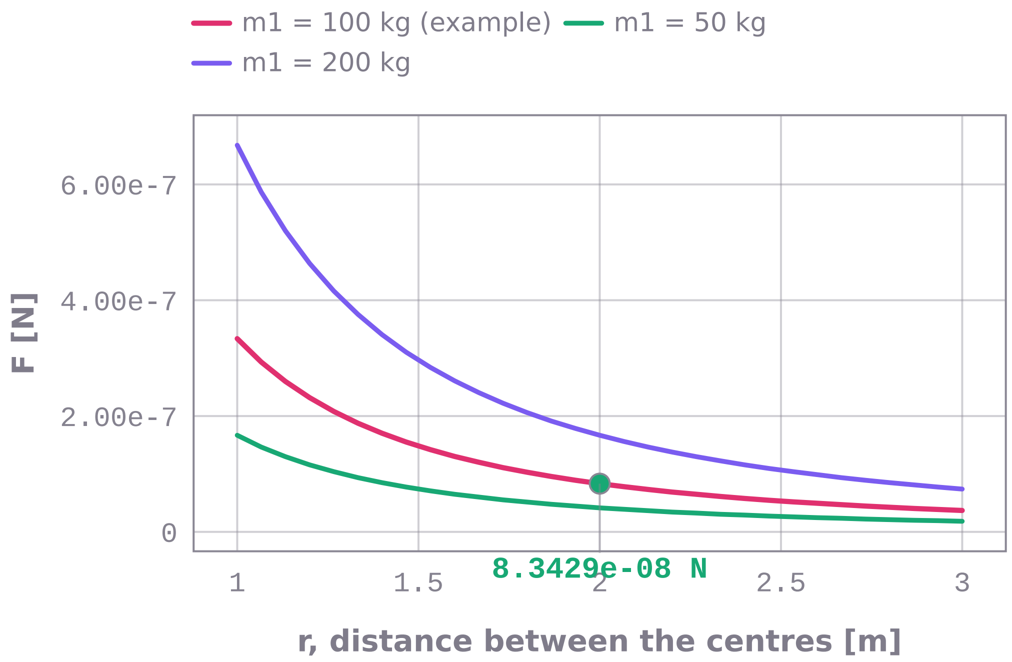
The attractive gravitational force between two masses (point masses or spheres) whose centres are r apart.

$$F = G\,\frac{m_1 m_2}{r^2}$$

input	meaning	condition
`m1`	first mass	
`m2`	second mass	
`r`	distance between the centres	$r \neq 0$

output	meaning
`F`	magnitude of the gravitational attraction

- **Method:** $F = G$ m1 m_2/r^2 with $G = 6.6743e-11$ N m^2/kg^2
- **Assumes:** $m_1 > 0$, $m_2 > 0$, $r > 0$; point masses or spherically symmetric bodies
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F (magnitude of the gravitational attraction) against r (distance between the centres) from 1 m to 3 m. One curve for each value of m1 (first mass): 100 kg, 50 kg, 200 kg. Other inputs as in the worked example: m2 = 50.0 kg. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 2 m, F = 8.3429e-08 N.

```
classical_physics_ai999x.solve('gravitational_force_two_masses', m1=100.0, m2=50.0, r=2.0)
# -> F = 8.34287e-08 N   (SOLVED)
```

```
classical_physics_ai999x.solve('gravitational_force_two_masses', m1=5.972e+24,
m2=7.348e+22, r=384400000.0)
# -> F = 1.98211e+20 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravitational force: m1 = 100 kg, m2 = 50 kg, r = 2 m	SOLVED	F = 8.34287e-08 N
Gravitational force: m1 = 100 kg, m2 = 50 kg, r = 0 m	INVALID_INPUT	Newton's law of universal gravitation: force between two masses: the separation r must be positive.
Gravitational force: m1 = 100 kg, r = 2 m	NEEDS_INPUT	gravitational_force_two_masses needs m2 (second mass). Example: solve('gravitational_force_two_masses', m1='100 kg', m2=

Snell's law: angle of refraction, `snell_refraction_angle`

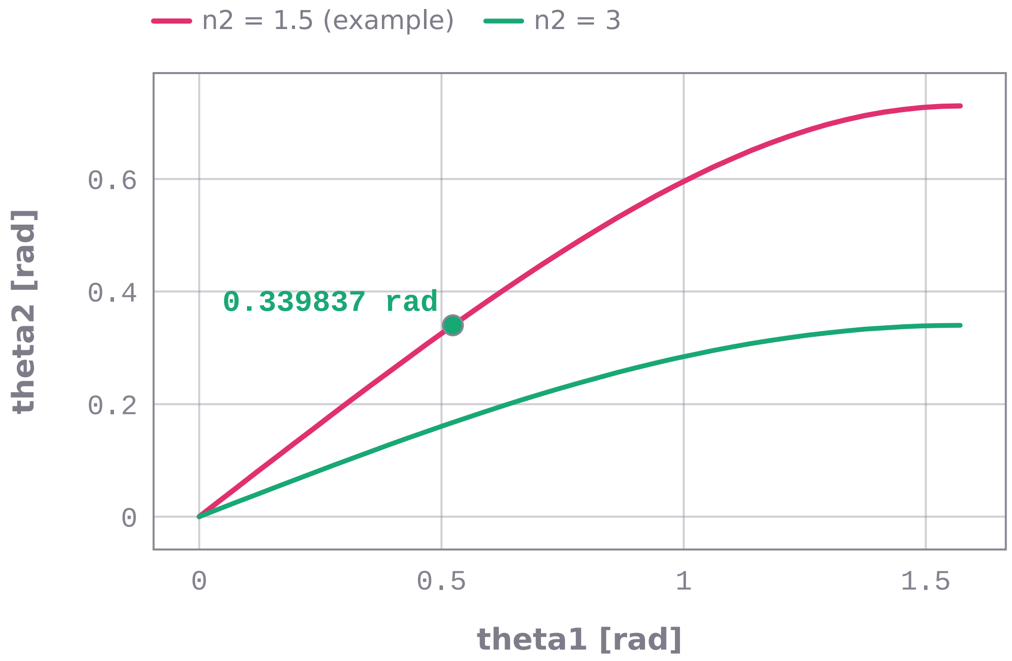
The angle of refraction of a ray passing from a medium of index n1 into one of index n2; refused with a reason when the ray is totally internally reflected.

$$n_1 \sin\theta_1 = n_2 \sin\theta_2$$

input	meaning	condition
`n1`	refractive index of the incident medium	
`n2`	refractive index of the second medium	
`theta1`	angle of incidence from the normal (deg or rad)	

output	meaning
`theta2`	angle of refraction from the normal
`theta2_deg`	angle of refraction in degrees

- **Method:** sin(theta2) = n1 sin(theta1) / n2
- **Assumes:** n1, $n_2 \geq 1$; $0 \leq theta1 \leq 90$ degrees; n1 sin(theta1) <= n2 (no total internal reflection)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta2 (angle of refraction from the normal) against theta1 (angle of incidence from the normal (deg or rad)) from 0 rad to 1.5708 rad. One curve for each value of n2 (refractive index of the second medium): 1.5, 3. Other inputs as in the worked example: n1 = 1.0. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, theta1 = 0.523599 rad, theta2 = 0.339837 rad.

```
classical_physics_ai999x.solve('snell_refraction_angle', n1=1.0, n2=1.5,
theta1=0.5235987755982988)
# -> theta2 = 0.339837 rad, theta2_deg = 19.4712   (SOLVED)
```

```
classical_physics_ai999x.solve('snell_refraction_angle', n1=1.33, n2=1.0,
theta1=0.5235987755982988)
# -> theta2 = 0.727494 rad, theta2_deg = 41.6823   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Snell's law: n1 = 1, n2 = 1.5, theta1 = 30 deg	SOLVED	theta2 = 0.339837 rad, theta2_deg = 19.4712
Snell's law: n1 = 1.5, n2 = 1, theta1 = 60 deg	INVALID_INPUT	Snell's law: angle of refraction: total internal reflection: n1 sin(theta1) exceeds n2, so there is no refracted ray.
Snell's law: n1 = 1, n2 = 1.5, theta1 = 30	UNRECOGNIZED_FORMAT	The number '1' has no unit I recognise, so I cannot tell what it measures.
Snell's law: n1 = 1, n2 = 1.5	NEEDS_INPUT	snell_refraction_angle needs theta1 (angle of incidence from the normal (deg or rad)). Example: solve('snell_refraction_

Critical angle for total internal reflection, `critical_angle_total_internal_reflection`

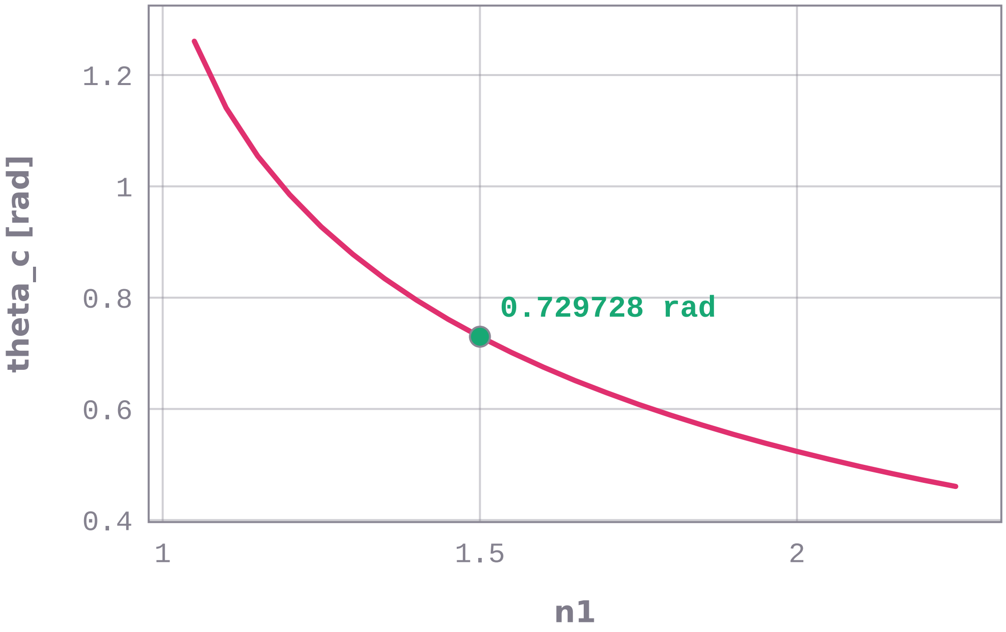
The smallest angle of incidence (from the normal) at which light in the denser medium is totally reflected at the boundary with a less dense one.

$$\sin\theta_c = \frac{n_2}{n_1}$$

input	meaning	condition
`n1`	refractive index of the medium the light travels in (the denser one)	
`n2`	refractive index of the medium beyond the boundary	

output	meaning
`theta_c`	critical angle from the normal
`theta_c_deg`	critical angle in degrees

- **Method:** sin(theta_c) = n2 / n1
- **Assumes:** $n_1 > n_2 \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta_c (critical angle from the normal) against n1 (refractive index of the medium the light travels in (the denser one)) from 0.75 to 2.25. Other inputs as in the worked example: n2 = 1.0. 25 points computed by classical-physics-ai999x; marked: the worked example, n1 = 1.5, theta_c = 0.729728 rad.

```
classical_physics_ai999x.solve('critical_angle_total_internal_reflection', n1=1.5, n2=1.0)
# -> theta_c = 0.729728 rad, theta_c_deg = 41.8103   (SOLVED)
```

```
classical_physics_ai999x.solve('critical_angle_total_internal_reflection', n1=2.42, n2=1.0)
# -> theta_c = 0.425991 rad, theta_c_deg = 24.4075   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical angle: n1 = 1.5, n2 = 1	SOLVED	theta_c = 0.729728 rad, theta_c_deg = 41.8103
Critical angle: n1 = 1, n2 = 1.5	INVALID_INPUT	Critical angle for total internal reflection: total internal reflection needs light going from a denser medium (n1) into
Critical angle: n1 = 1.5	NEEDS_INPUT	critical_angle_total_internal_reflection needs n2 (refractive index of the medium beyond the boundary). Example: solve('

Magnification and image height of a thin lens or a spherical mirror, `lens_mirror_magnification`

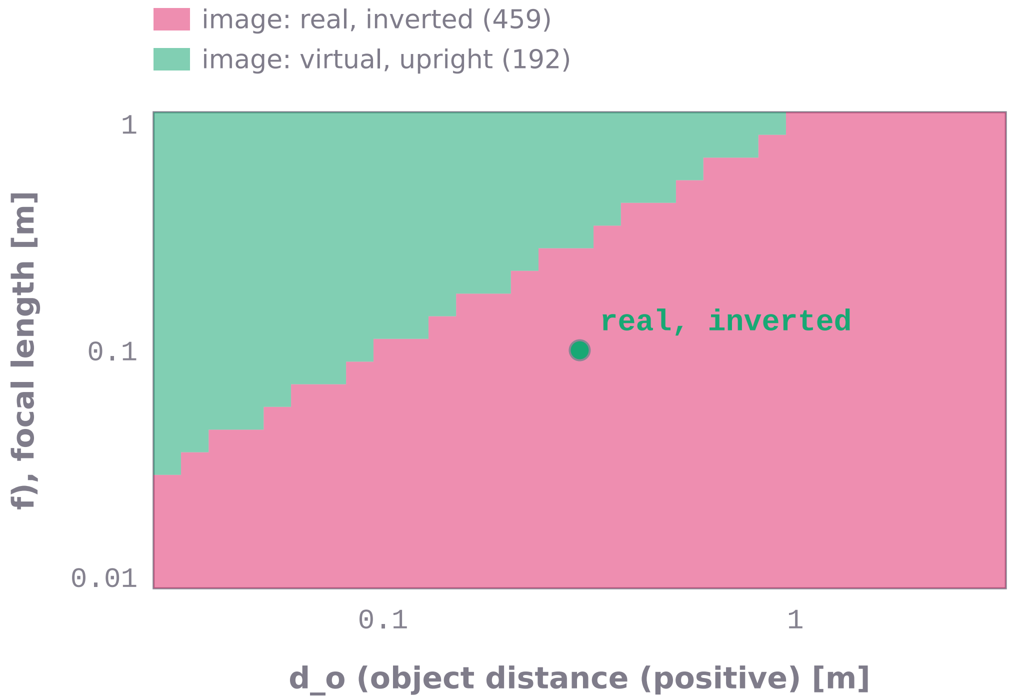
Magnification, image height, image distance and the nature of the image (real or virtual, upright or inverted) from the object distance and either the image distance or the focal length.

$$m = \frac{h_i}{h_o} = -\frac{d_i}{d_o}, \quad \frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

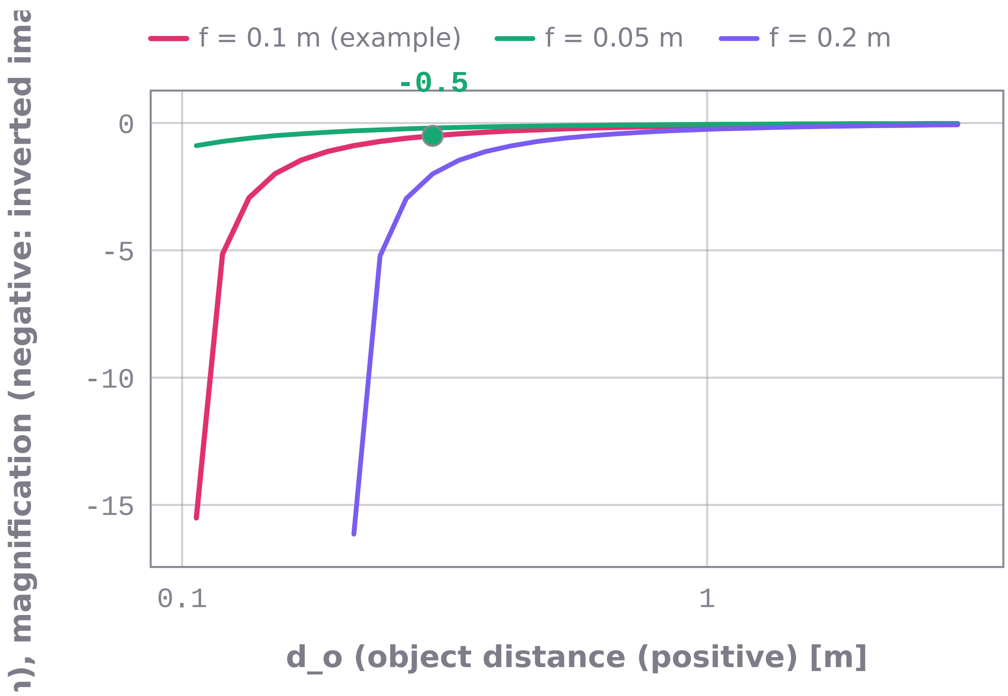
input	meaning	condition
`d_o`	object distance (positive)	
`h_o`	object height	
`d_i`	image distance (positive: real image), if known	
`f`	focal length (positive: converging lens or concave mirror), if d_i is not given	

output	meaning
`m`	magnification (negative: inverted image)
`h_i`	image height (negative: inverted)
`d_i`	image distance (positive: real, negative: virtual)
`image`	real or virtual, upright or inverted

- **Method:** $d_i = d_o$ f / (d_o - f) when f is given; $m = -d_i/d_o$; $h_i = m$ h_o
- **Assumes:** $d_o > 0$, $h_o > 0$; exactly one of d_i and f; the object not at the focal point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



real or virtual, upright or inverted (image) over d_o (object distance (positive)) from 0.03 m to 3 m and f (focal length (positive: converging lens or concave mirror), if d_i is not given) from 0.01 m to 1 m (logarithmic axes): each of the 31 × 21 cells is one run of classical-physics-ai999x, coloured by the verdict it returned, real, inverted: 459 cells, virtual, upright: 192 cells. Other inputs as in the worked example: h_o = 0.02 m. Marked: the worked example, d_o = 0.3 m, f = 0.1 m → real, inverted.



m (magnification (negative: inverted image)) against d_o (object distance (positive)) from 0.106444 m to 3 m on a logarithmic axis. One curve for each value of f (focal length (positive: converging lens or concave mirror), if d_i is not given): 0.1 m, 0.05 m, 0.2 m. Other inputs as in the worked example: h_o = 0.02 m. 30 points per curve computed by classical-physics-ai999x; marked: the worked example, d_o = 0.3 m, m = -0.5.

```
classical_physics_ai999x.solve('lens_mirror_magnification', d_o=0.3, h_o=0.02, f=0.1)
# -> m = -0.5 1, h_i = -0.01 m, d_i = 0.15 m, image = real, inverted   (SOLVED)
```

```
classical_physics_ai999x.solve('lens_mirror_magnification', d_o=0.05, h_o=0.02, f=0.1)
# -> m = 2 1, h_i = 0.04 m, d_i = -0.1 m, image = virtual, upright   (SOLVED)
```

```
classical_physics_ai999x.solve('lens_mirror_magnification', d_o=0.4, h_o=0.05, d_i=0.2)
# -> m = -0.5 1, h_i = -0.025 m, d_i = 0.2 m, image = real, inverted   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Image magnification: d_o = 30 cm, f = 10 cm, h_o = 2 cm	SOLVED	m = -0.5 1, h_i = -0.01 m, d_i = 0.15 m, image = real, inverted
Image magnification: d_o = 5 cm, f = 10 cm, h_o = 2 cm	SOLVED	m = 2 1, h_i = 0.04 m, d_i = -0.1 m, image = virtual, upright
Image magnification: d_o = 10 cm, f = 10 cm, h_o = 2 cm	INVALID_INPUT	Magnification and image height of a thin lens or a spherical mirror: the object is at the focal point: the image is at i
Image magnification: d_o = 30 cm, f = 10 cm, d_i = 15 cm, h_o = 2 cm	INVALID_INPUT	Magnification and image height of a thin lens or a spherical mirror: give exactly one of the image distance d_i or the f
Image magnification: d_o = 30 cm, f = 10 cm	NEEDS_INPUT	lens_mirror_magnification needs h_o (object height). Example: solve('lens_mirror_magnification', d_o='0.3 m', h_o='0.02

Double-slit interference: angle and position of a bright fringe, `double_slit_bright_fringe`

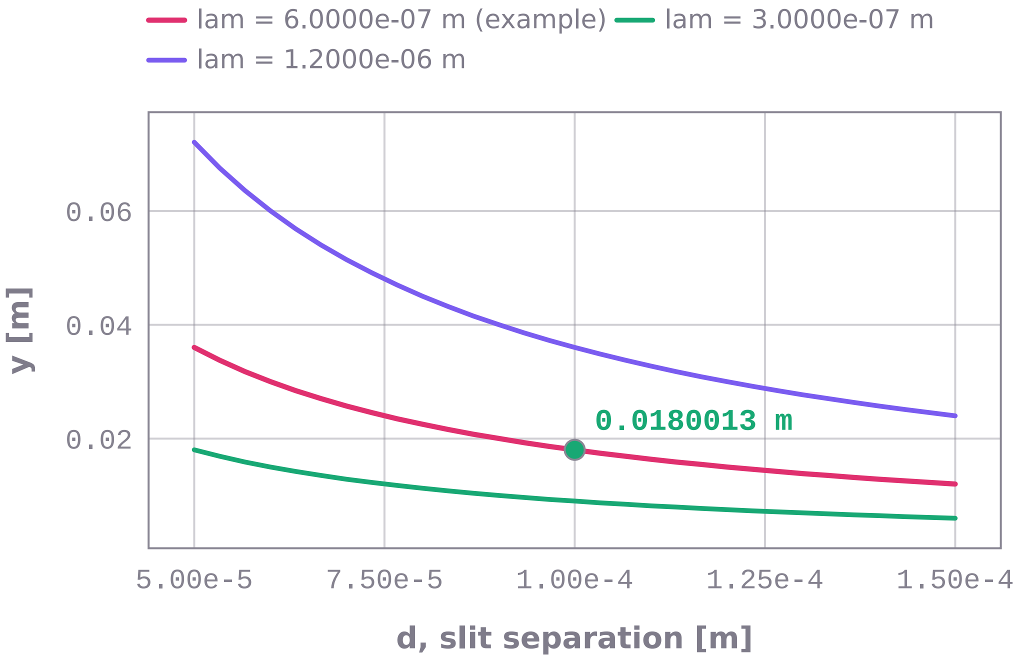
The angle and the position on the screen of the m-th bright fringe of a double slit, and the fringe spacing near the centre.

$$d \sin\theta = m\lambda, \quad y = L\tan\theta, \quad \Delta y \approx \frac{\lambda L}{d}$$

input	meaning	condition
`d`	slit separation	
`lam`	wavelength	
`m`	order of the bright fringe (0 for the central maximum)	
`L`	distance from the slits to the screen	

output	meaning
`y`	position of the m-th bright fringe on the screen, from the central maximum
`theta`	angle of the m-th bright fringe from the central axis
`theta_deg`	that angle in degrees
`dy`	fringe spacing near the centre (small-angle, lambda L / d)

- **Method:** d sin(theta) = m lambda; $y = L$ tan(theta); spacing lambda L / d
- **Assumes:** d, lam, $L > 0$; m a whole number with m $\lambda < d$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (position of the m-th bright fringe on the screen, from the central maximum) against d (slit separation) from 5.0000e-05 m to 0.00015 m. One curve for each value of lam (wavelength): 6.0000e-07 m, 3.0000e-07 m, 1.2000e-06 m. Other inputs as in the worked example: m = 2, L = 1.5 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, d = 0.0001 m, y = 0.0180013 m.

```
classical_physics_ai999x.solve('double_slit_bright_fringe', d=0.0001, lam=6e-07, m=2,
L=1.5)
# -> theta = 0.0120003 rad, theta_deg = 0.687566, y = 0.0180013 m, dy = 0.009 m   (SOLVED)
```

```
classical_physics_ai999x.solve('double_slit_bright_fringe', d=5e-05, lam=5e-07, m=0, L=2.0)
# -> theta = 0 rad, theta_deg = 0, y = 0 m, dy = 0.02 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Double slit: d = 0.1 mm, lam = 600 nm, m = 2, L = 1.5 m	SOLVED	theta = 0.0120003 rad, theta_deg = 0.687566, y = 0.0180013 m, dy = 0.009 m
Double slit: d = 1 um, lam = 600 nm, m = 2, L = 1 m	INVALID_INPUT	Double-slit interference: angle and position of a bright fringe: order 2 does not exist: m lambda must be smaller than t
Double slit: d = 0.1 mm, lam = 600 nm, m = 2.5, L = 1.5 m	INVALID_INPUT	m: expected a whole number written exactly, got '2.5'.
Double slit: d = 0.1 mm, lam = 600 nm, m = 2	NEEDS_INPUT	double_slit_bright_fringe needs L (distance from the slits to the screen). Example: solve('double_slit_bright_fringe', d

Diffraction grating: angle of an order and the highest visible order, `diffraction_grating_order`

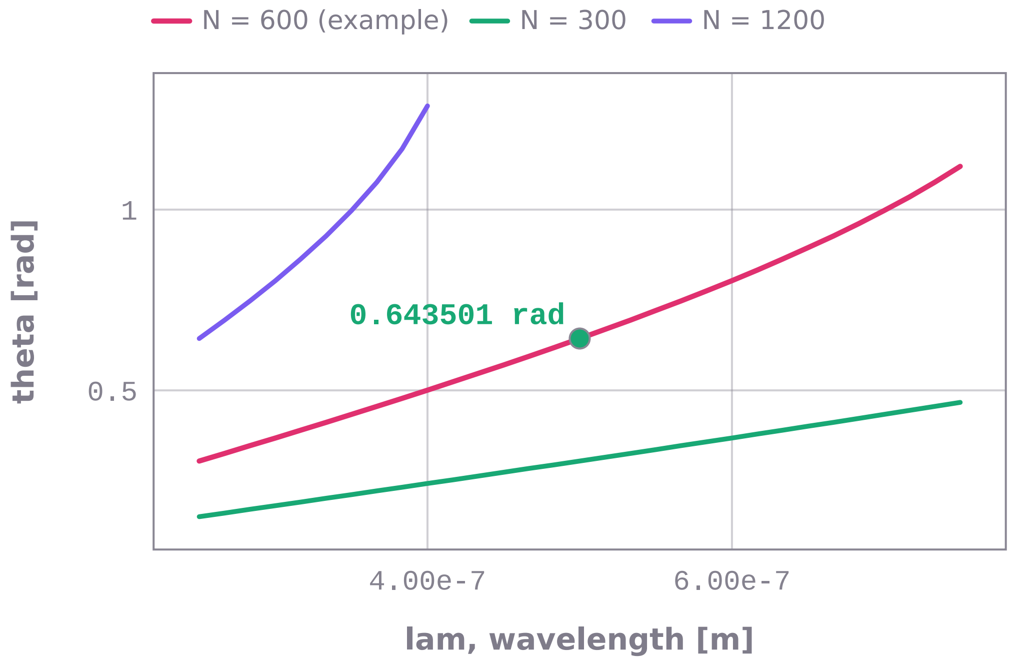
The angle of the m-th order line of a diffraction grating, the line spacing and the highest order that can be seen.

$$d\sin\theta = m\lambda, \quad m_{max} = \max\{m : m\lambda < d\}$$

input	meaning	condition
`lam`	wavelength	
`m`	order of the line (1, 2, ...)	
`d`	line spacing, if given instead of N	
`N`	number of lines per millimetre, if given instead of d	

output	meaning
`theta`	angle of the m-th order from the central axis
`theta_deg`	that angle in degrees
`m_max`	highest order that can be seen
`d`	line spacing

- **Method:** $d = 1$ mm / N; d sin(theta) = m lambda; highest order: largest m with m $\lambda < d$
- **Assumes:** exactly one of d and N; $d > \lambda$; 1 <= m <= the highest order
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta (angle of the m-th order from the central axis) against lam (wavelength) from 2.5000e-07 m to 7.5000e-07 m. One curve for each value of N (number of lines per millimetre, if given instead of d): 600, 300, 1200. Other inputs as in the worked example: m = 2. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, lam = 5.0000e-07 m, theta = 0.643501 rad.

```
classical_physics_ai999x.solve('diffraction_grating_order', lam=5e-07, m=2, N=600.0)
# -> theta = 0.643501 rad, theta_deg = 36.8699, m_max = 3, d = 1.66667e-06 m   (SOLVED)
```

```
classical_physics_ai999x.solve('diffraction_grating_order', lam=5e-07, m=1, d=2e-06)
# -> theta = 0.25268 rad, theta_deg = 14.4775, m_max = 3, d = 2e-06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffraction grating: N = 600 lines/mm, lam = 500 nm, m = 2	SOLVED	theta = 0.643501 rad, theta_deg = 36.8699, m_max = 3, d = 1.66667e-06 m
Diffraction grating: d = 2 um, lam = 500 nm, m = 4	INVALID_INPUT	Diffraction grating: angle of an order and the highest visible order: order 4 is not seen: the highest order is 3.
Diffraction grating: N = 600 lines/mm, m = 2	NEEDS_INPUT	diffraction_grating_order needs lam (wavelength). Example: solve('diffraction_grating_order', lam='5e-07 m', m=2, N=600)

Single-slit diffraction: dark fringes and the width of the central maximum, `single_slit_minimum`

The angle and screen position of the m-th dark fringe of a single slit, and the angular and linear width of the central bright maximum.

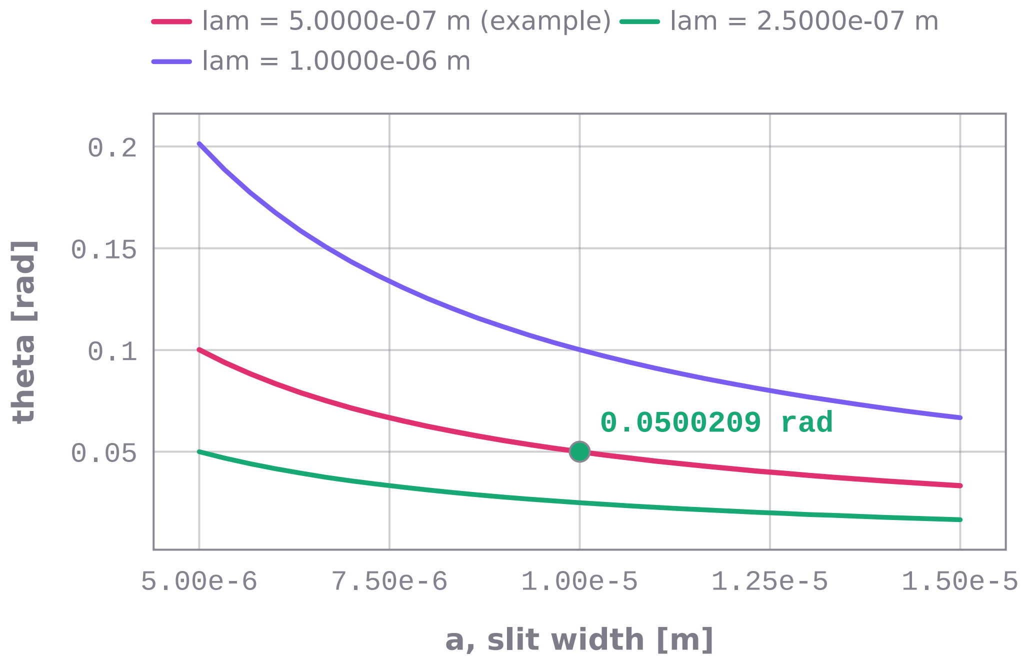
$$a \sin\theta = m\lambda, \quad W = 2L\tan\theta_1$$

input	meaning	condition
`a`	slit width	

input	meaning	condition
`lam`	wavelength	
`m`	order of the dark fringe (1, 2, ...)	
`L`	distance from the slit to the screen	

output	meaning
`theta`	angle of the m-th dark fringe from the central axis
`theta_deg`	that angle in degrees
`y`	position of the m-th dark fringe on the screen, from the centre
`W`	width of the central maximum on the screen
`W_angle`	angular width of the central maximum

- **Method:** a sin(theta) = m lambda; $y = L$ tan(theta); central width 2 L tan(theta_1)
- **Assumes:** a, lam, $L > 0$; $\lambda < a$; m $\lambda < a$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta (angle of the m-th dark fringe from the central axis) against a (slit width) from 5.0000e-06 m to 1.5000e-05 m. One curve for each value of lam (wavelength): 5.0000e-07 m, 2.5000e-07 m, 1.0000e-06 m. Other inputs as in the worked example: m = 1, L = 2.0 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, a = 1.0000e-05 m, theta = 0.0500209 rad.

```
classical_physics_ai999x.solve('single_slit_minimum', a=1e-05, lam=5e-07, m=1, L=2.0)
# -> theta = 0.0500209 rad, theta_deg = 2.86598, y = 0.100125 m, W = 0.20025 m, W_angle =
0.100042 rad   (SOLVED)
```

```
classical_physics_ai999x.solve('single_slit_minimum', a=1e-05, lam=5e-07, m=3, L=2.0)
# -> theta = 0.150568 rad, theta_deg = 8.62693, y = 0.303433 m, W = 0.20025 m, W_angle =
0.100042 rad   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single slit: a = 0.01 mm, lam = 500 nm, m = 1, L = 2 m	SOLVED	theta = 0.0500209 rad, theta_deg = 2.86598, y = 0.100125 m, W = 0.20025 m, W_angle = 0.100042 rad
Single slit: a = 400 nm, lam = 500 nm, m = 1, L = 1 m	INVALID_INPUT	Single-slit diffraction: dark fringes and the width of the central maximum: the slit must be wider than the wavelength f
Single slit: a = 0.01 mm, lam = 500 nm, m = 1	NEEDS_INPUT	single_slit_minimum needs L (distance from the slit to the screen). Example: solve('single_slit_minimum', a='1e-05 m', l

Malus's law: intensity through a polarizer, `malus_law_polarizer`

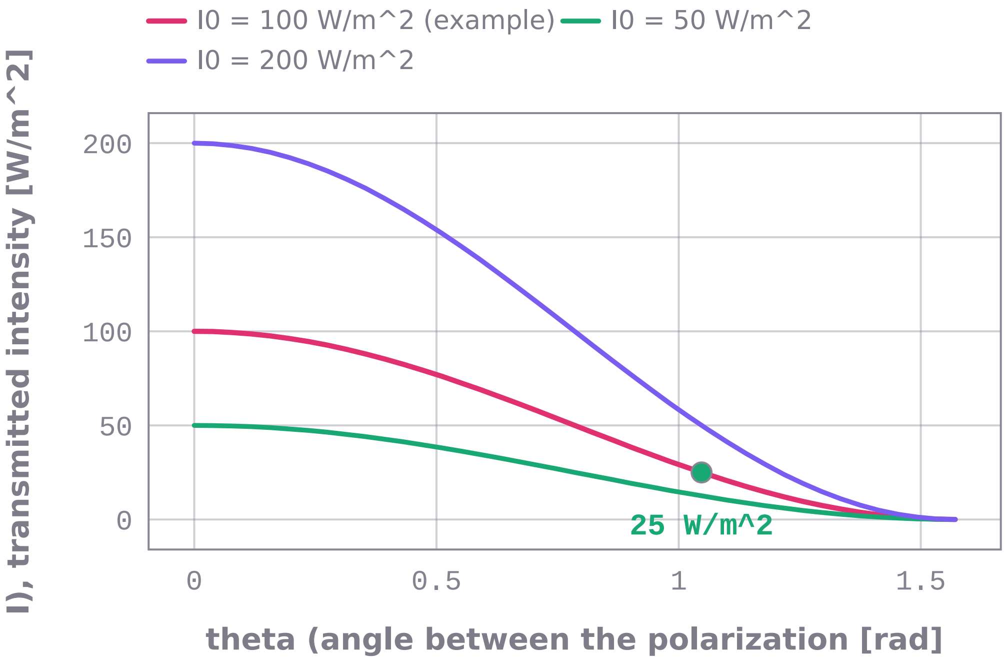
Intensity of light after an analyser whose transmission axis makes an angle theta with the light's polarization (or, for unpolarized light, with a first polarizer it passed through).

$$I = I_0 \cos^2 \theta$$

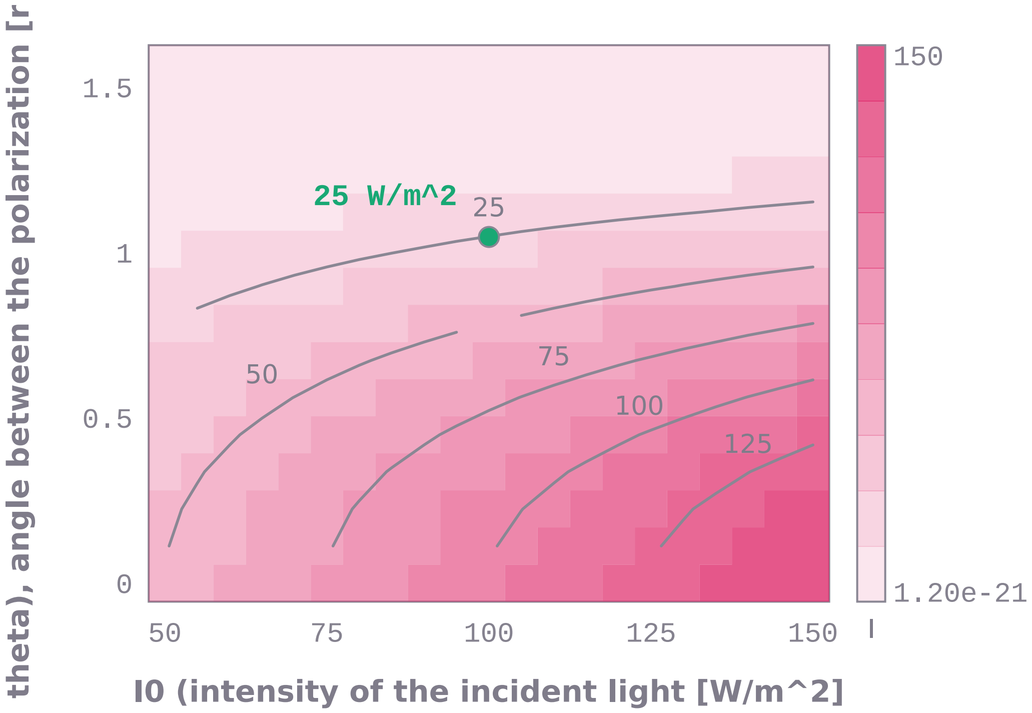
input	meaning	condition
`I0`	intensity of the incident light	
`theta`	angle between the polarization (or first polarizer) and the analyser's axis	
`light`	incident light (default polarized)	

output	meaning
`I`	transmitted intensity
`fraction`	transmitted fraction I/I_0

- **Method:** $I = I_0\ cos^2$(theta); unpolarized light first loses half at the first polarizer
- **Assumes:** $I_0 > 0$; ideal polarizers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I (transmitted intensity) against theta (angle between the polarization (or first polarizer) and the analyser's axis) from 0 rad to 1.5708 rad. One curve for each value of I0 (intensity of the incident light): 100 W/m^2, 50 W/m^2, 200 W/m^2. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, theta = 1.0472 rad, I = 25 W/m^2.



I (transmitted intensity) over I0 (intensity of the incident light) from 50 W/m^2 to 150 W/m^2 and theta (angle between the polarization (or first polarizer) and the analyser's axis) from 0 rad to 1.5708 rad: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1.1988e-21 W/m^2 to dark 150 W/m^2); iso-lines at I = 25, 50, 75, 100, 125 W/m^2. Marked: the worked example, I0 = 100 W/m^2, theta = 1.0472 rad, I = 25 W/m^2.

```
classical_physics_ai999x.solve('malus_law_polarizer', I0=100.0, theta=1.0471975511965976)
# -> I = 25 W/m^2, fraction = 0.25   (SOLVED)
```

```
classical_physics_ai999x.solve('malus_law_polarizer', I0=100.0, theta=0.7853981633974483,
light='unpolarized')
# -> I = 25 W/m^2, fraction = 0.25   (SOLVED)
```

```
classical_physics_ai999x.solve('malus_law_polarizer', I0=80.0, theta=1.5707963267948966)
# -> I = 0 W/m^2, fraction = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Malus's law: I0 = 100 W/m^2, theta = 60 deg	SOLVED	I = 25 W/m^2, fraction = 0.25
Malus's law: I0 = 100 W/m^2, theta = 45 deg, light = unpolarized	SOLVED	I = 25 W/m^2, fraction = 0.25
Malus's law: I0 = -5 W/m^2, theta = 30 deg	INVALID_INPUT	Malus's law: intensity through a polarizer: the incident intensity I0 must be positive.
Malus's law: I0 = 100 W/m^2	NEEDS_INPUT	malus_law_polarizer needs theta (angle between the polarization (or first polarizer) and the analyser's axis). Example:

Faraday's law: average emf induced by a change of magnetic flux, `faraday_induced_emf`

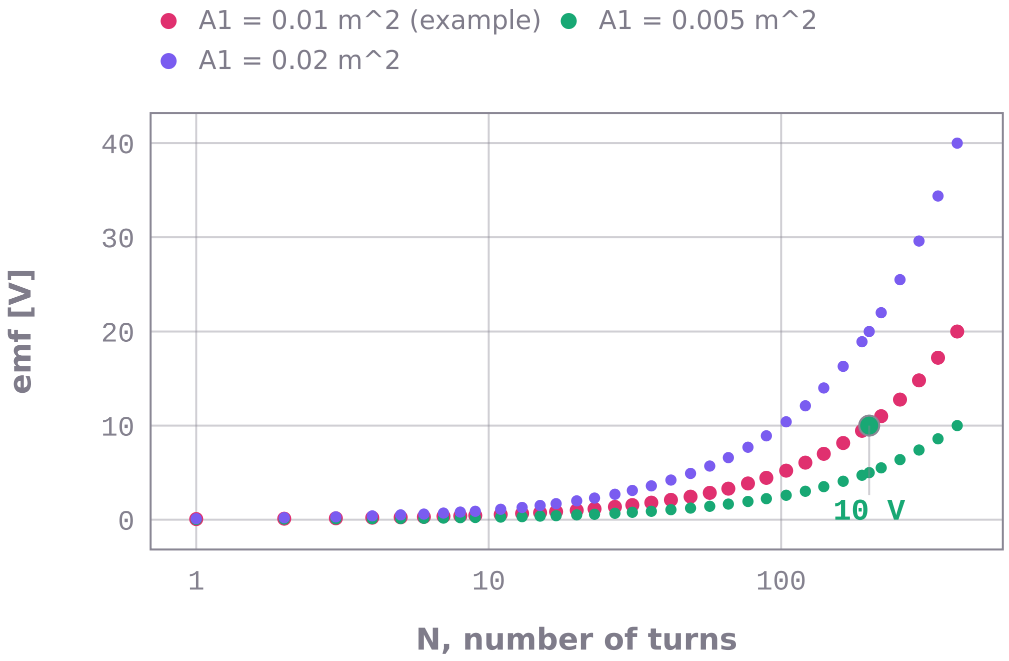
The flux through a coil before and after a change of field or area, and the average emf induced in its N turns during the change (with Lenz's sign).

$$\mathcal{E} = -N\,\frac{\Delta\Phi}{\Delta t}, \quad \Phi = BA\cos\theta$$

input	meaning	condition
`N`	number of turns	
`A1`	initial area of the coil	
`B1`	initial magnetic field	
`B2`	final magnetic field	
`dt`	duration of the change	$dt \neq 0$
`A2`	final area of the coil (default: unchanged)	
`theta`	angle between the field and the coil's axis (default 0)	

output	meaning
`emf`	magnitude of the average induced emf
`emf_signed`	average induced emf with Lenz's sign, -N dPhi/dt
`phi1`	initial flux through one turn
`phi2`	final flux through one turn
`dphi`	change of flux through one turn

- **Method:** $Phi = B$ A cos(theta); emf = -N $(Phi2 - Phi1)/dt$
- **Assumes:** $N \geq 1$; $dt > 0$; areas >= 0; uniform field over the coil
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



emf (magnitude of the average induced emf) against N (number of turns) from 1 to 400 on a logarithmic axis. One curve for each value of A1 (initial area of the coil): 0.01 m^2, 0.005 m^2, 0.02 m^2. Other inputs as in the worked example: B1 = 0.0 T, B2 = 0.5 T, dt = 0.1 s. 35 points per curve computed by classical-physics-ai999x; marked: the worked example, N = 200, emf = 10 V.

```
classical_physics_ai999x.solve('faraday_induced_emf', N=200, A1=0.01, B1=0.0, B2=0.5,
dt=0.1)
# -> phi1 = 0 Wb, phi2 = 0.005 Wb, dphi = 0.005 Wb, emf = 10 V, emf_signed = -10 V
(SOLVED)
```

```
classical_physics_ai999x.solve('faraday_induced_emf', N=1, A1=0.02, A2=0.005, B1=0.4,
B2=0.4, dt=0.25)
# -> phi1 = 0.008 Wb, phi2 = 0.002 Wb, dphi = -0.006 Wb, emf = 0.024 V, emf_signed = 0.024
V   (SOLVED)
```

```
classical_physics_ai999x.solve('faraday_induced_emf', N=50, A1=0.04, B1=0.2, B2=0.0,
dt=0.5, theta=1.0471975511965976)
# -> phi1 = 0.004 Wb, phi2 = 0 Wb, dphi = -0.004 Wb, emf = 0.4 V, emf_signed = 0.4 V
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Faraday induced emf: N = 200, A1 = 100 cm^2, B1 = 0 T, B2 = 0.5 T, dt = 0.1 s	SOLVED	phi1 = 0 Wb, phi2 = 0.005 Wb, dphi = 0.005 Wb, emf = 10 V, emf_signed = -10 V
Faraday induced emf: N = 200, A1 = 100 cm^2, B1 = 0 T, B2 = 0.5 T, dt = 0 s	INVALID_INPUT	Faraday's law: average emf induced by a change of magnetic flux: the time interval dt must be positive.
Faraday induced emf: N = 200, A1 = 100 cm^2, B1 = 0 T, B2 = 0.5 T	NEEDS_INPUT	faraday_induced_emf needs dt (duration of the change). Example: solve('faraday_induced_emf', N=200, A1='0.01 m^2', B1='0

Ideal transformer: secondary voltage and current, `ideal_transformer_secondary`

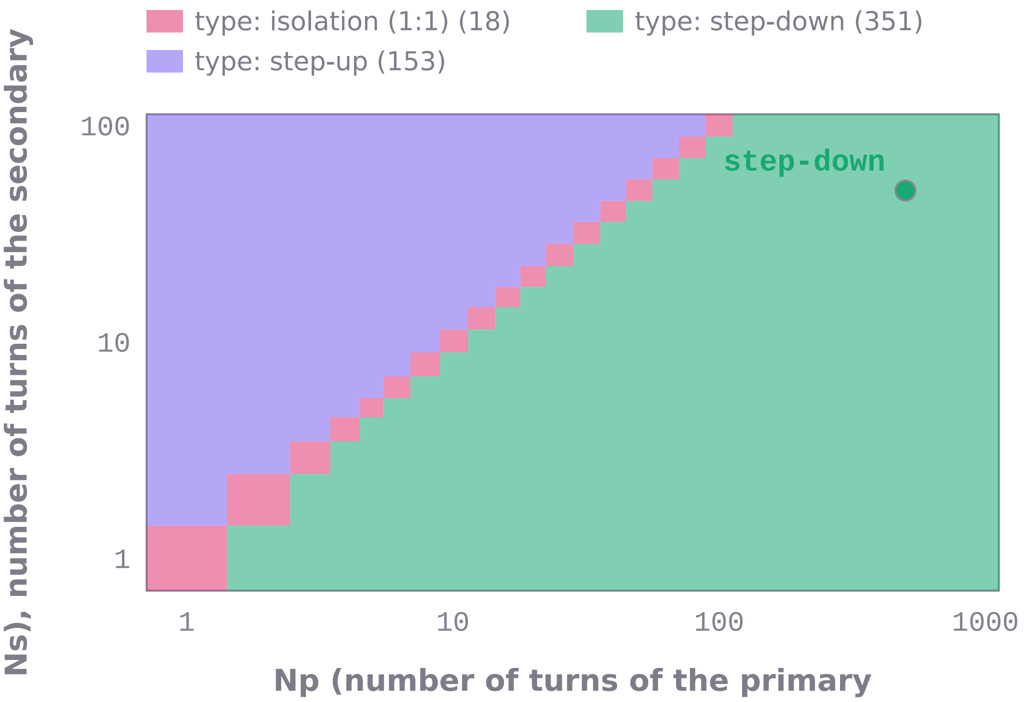
Secondary voltage (and, when the primary current is given, the secondary current) of an ideal transformer from its turns, and whether it steps the voltage up or down.

$$\frac{V_s}{V_p} = \frac{N_s}{N_p} = \frac{I_p}{I_s}$$

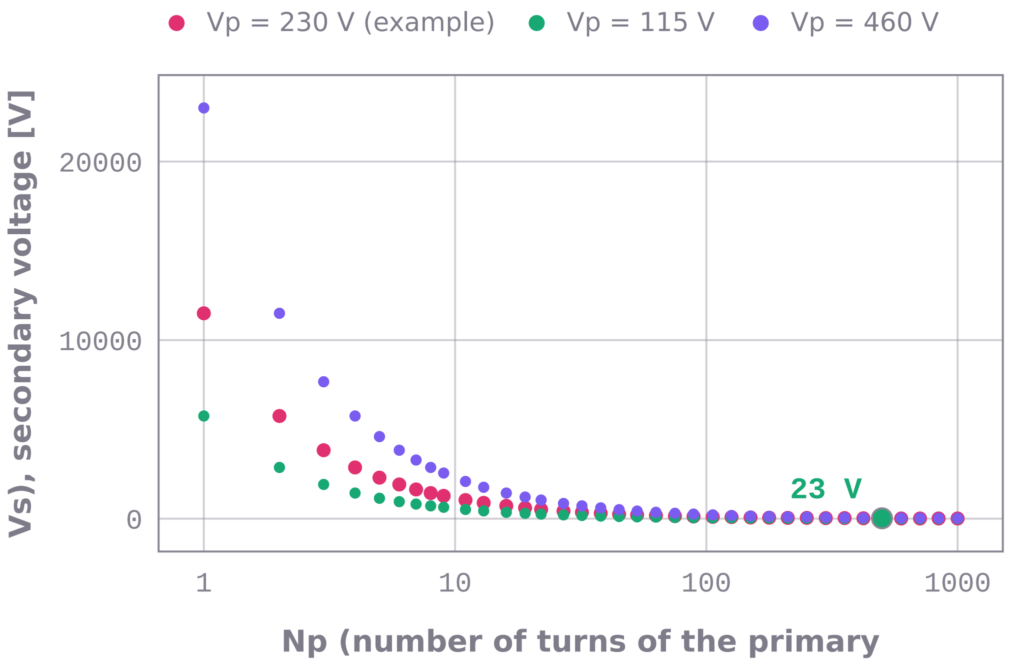
input	meaning	condition
`Np`	number of turns of the primary	
`Ns`	number of turns of the secondary	
`Vp`	primary voltage (rms)	
`Ip`	primary current (rms), if known	

output	meaning
`Vs`	secondary voltage
`Is`	secondary current (when Ip is given)
`ratio`	turns ratio Ns / Np
`type`	step-up, step-down or isolation

- **Method:** $Vs/Vp = Ns/Np$; Np $Ip = Ns$ Is
- **Assumes:** Np, $Ns \geq 1$; $Vp > 0$; ideal: no losses, no magnetizing current
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



step-up, step-down or isolation (type) over Np (number of turns of the primary) from 1 to 1000 and Ns (number of turns of the secondary) from 1 to 100 (logarithmic axes): each of the 29 × 18 cells is one run of classical-physics-ai999x, coloured by the verdict it returned, isolation (1:1): 18 cells, step-down: 351 cells, step-up: 153 cells. Other inputs as in the worked example: Vp = 230.0 V, Ip = 0.5 A. Marked: the worked example, Np = 500, Ns = 50 → step-down.



Vs (secondary voltage) against Np (number of turns of the primary) from 1 to 1000 on a logarithmic axis. One curve for each value of Vp (primary voltage (rms)): 230 V, 115 V, 460 V. Other inputs as in the worked example: Ns = 50, Ip = 0.5 A. 37 points per curve computed by classical-physics-ai999x; marked: the worked example, Np = 500, Vs = 23 V.

```
classical_physics_ai999x.solve('ideal_transformer_secondary', Np=500, Ns=50, Vp=230.0,
Ip=0.5)
# -> Vs = 23 V, ratio = 0.1, type = step-down, Is = 5 A   (SOLVED)
```

```
classical_physics_ai999x.solve('ideal_transformer_secondary', Np=100, Ns=2500, Vp=12.0)
# -> Vs = 300 V, ratio = 25, type = step-up   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ideal transformer: Np = 500, Ns = 50, Vp = 230 V, Ip = 0.5 A	SOLVED	Vs = 23 V, ratio = 0.1, type = step-down, Is = 5 A
Ideal transformer: Np = 100 turns, Ns = 2500 turns, Vp = 12 V	SOLVED	Vs = 300 V, ratio = 25, type = step-up
Ideal transformer: Np = 500, Ns = 0, Vp = 230 V	INVALID_INPUT	Ns: must be at least 1, got 0.
Ideal transformer: Np = 500, Vp = 230 V	NEEDS_INPUT	ideal_transformer_secondary needs Ns (number of turns of the secondary). Example: solve('ideal_transformer_secondary', N

Series RLC circuit on an AC supply: impedance, current, phase and power, `series_rlc_ac_circuit`

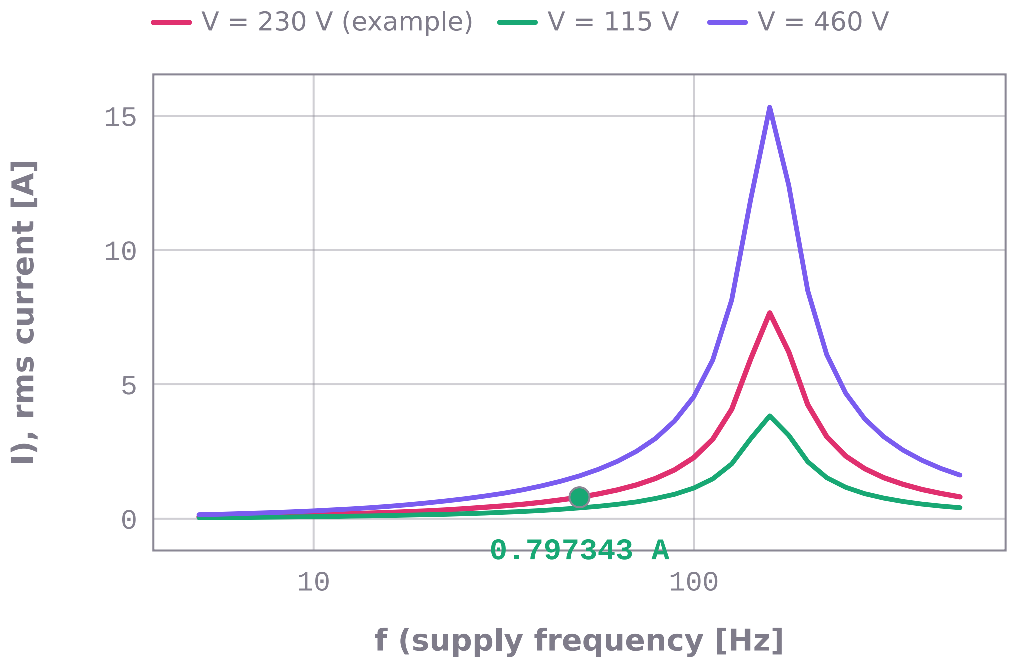
Reactances, impedance, rms current, phase angle, power factor and average power of a resistor, an inductor and a capacitor in series across a sinusoidal supply.

$$Z = \sqrt{R^2 + (X_L - X_C)^2}, \quad I = \frac{V}{Z}, \quad \tan\varphi = \frac{X_L - X_C}{R}, \quad P = I^2 R$$

input	meaning	condition
`R`	resistance	
`L`	inductance	
`C`	capacitance	
`f`	supply frequency	
`V`	rms supply voltage	

output	meaning
`I`	rms current
`Z`	impedance
`phi`	phase angle of the voltage ahead of the current (positive: inductive)
`phi_deg`	phase angle in degrees
`pf`	power factor cos(phi)
`P`	average power
`XL`	inductive reactance
`XC`	capacitive reactance

- **Method:** $XL = 2$ pi f L, XC = 1/(2 pi f C), $Z = \sqrt{R^2 + (X_L - X_C)^2}$, $I = V/Z$, tan(phi) = (XL - XC)/R, $P = I^2$ R
- **Assumes:** R, L, C, f, $V > 0$; sinusoidal steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I (rms current) against f (supply frequency) from 5 Hz to 500 Hz on a logarithmic axis. One curve for each value of V (rms supply voltage): 230 V, 115 V, 460 V. Other inputs as in the worked example: R = 30.0 ohm, L = 0.1 H, C = 9.999999999999999e-06 F. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, f = 50 Hz, I = 0.797343 A.

```
classical_physics_ai999x.solve('series_rlc_ac_circuit', R=30.0, L=0.1,
C=1.6666666666666667e-05, f=159.15494309189535, V=100.0)
# -> Z = 50 ohm, I = 2 A, phi = 0.927295 rad, phi_deg = 53.1301, pf = 0.6, P = 120 W, XL =
100 ohm, XC = 60 ohm   (SOLVED)
```

```
classical_physics_ai999x.solve('series_rlc_ac_circuit', R=10.0, L=0.1, C=1e-05,
f=159.15494309189532, V=5.0)
# -> Z = 10 ohm, I = 0.5 A, phi = 0 rad, phi_deg = 0, pf = 1, P = 2.5 W, XL = 100 ohm, XC =
100 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Series RLC circuit: R = 30 ohm, L = 0.1 H, C = 10 uF, f = 50 Hz, V = 230 V	SOLVED	Z = 288.458 ohm, I = 0.797343 A, phi = -1.46661 rad, phi_deg = -84.0304, pf = 0.104001, P = 19.0727 W, XL = 31.4159 ohm, XC = 318.31 ohm
Series RLC circuit: R = 30 ohm, L = 0.1 H, C = 0 uF, f = 50 Hz, V = 230 V	INVALID_INPUT	Series RLC circuit on an AC supply: impedance, current, phase and power: the capacitance C must be positive.
Series RLC circuit: R = 30 ohm, L = 0.1 H, C = 10 uF, f = 50 Hz	NEEDS_INPUT	series_rlc_ac_circuit needs V (rms supply voltage). Example: solve('series_rlc_ac_circuit', R='30 ohm', L='0.1 H', C='1.

Terminal voltage of a source with internal resistance, `terminal_voltage_internal_resistance`

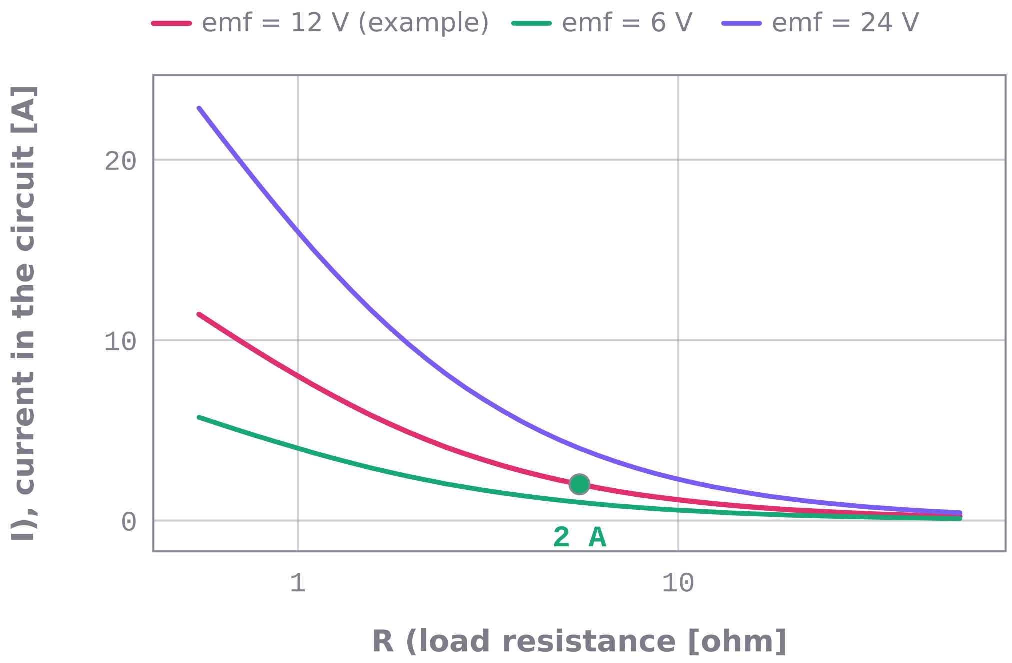
Current, terminal voltage, lost volts, load power and efficiency of a source of emf with internal resistance r driving a load resistance R.

$$I = \frac{\mathcal{E}}{R + r}, \quad V_t = \mathcal{E} - Ir$$

input	meaning	condition
`emf`	emf of the source	
`r`	internal resistance	
`R`	load resistance	

output	meaning
`I`	current in the circuit
`V_t`	terminal voltage (across the load)
`V_lost`	voltage lost inside the source (I r)
`P_load`	power delivered to the load
`efficiency`	fraction of the source's power delivered to the load

- **Method:** I = emf / (R + r); V_t = emf - I $r = I$ R
- **Assumes:** emf > 0; r, $R \geq 0$ and not both 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I (current in the circuit) against R (load resistance) from 0.55 ohm to 55 ohm on a logarithmic axis. One curve for each value of emf (emf of the source): 12 V, 6 V, 24 V. Other inputs as in the worked example: r = 0.5 ohm. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, R = 5.5 ohm, I = 2 A.

```
classical_physics_ai999x.solve('terminal_voltage_internal_resistance', emf=12.0, r=0.5,
R=5.5)
# -> I = 2 A, V_t = 11 V, V_lost = 1 V, P_load = 22 W, efficiency = 0.916667 1   (SOLVED)
```

```
classical_physics_ai999x.solve('terminal_voltage_internal_resistance', emf=9.0, r=1.0,
R=0.0)
# -> I = 9 A, V_t = 0 V, V_lost = 9 V, P_load = 0 W, efficiency = 0 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Terminal voltage: emf = 12 V, r = 0.5 ohm, R = 5.5 ohm	SOLVED	I = 2 A, V_t = 11 V, V_lost = 1 V, P_load = 22 W, efficiency = 0.916667 1
Terminal voltage: emf = 12 V, r = 0 ohm, R = 0 ohm	INVALID_INPUT	Terminal voltage of a source with internal resistance: a short circuit with no internal resistance would draw an unlimit
Terminal voltage: emf = 12 V, r = 0.5 ohm	NEEDS_INPUT	terminal_voltage_internal_resistance needs R (load resistance). Example: solve('terminal_voltage_internal_resistance', e

Static friction: maximum friction force and whether a body slips, `static_friction_limit`

The largest static friction force mu_s N and whether an applied force parallel to the surface makes the body slip.

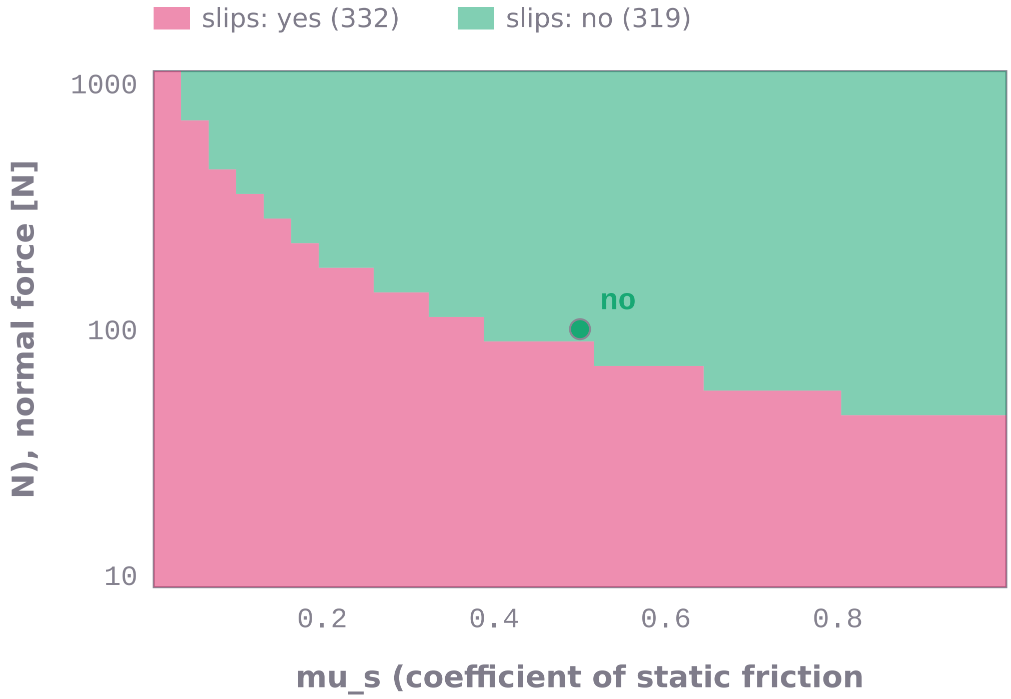
$$f_s \leq \mu_s N$$

input	meaning	condition
`mu_s`	coefficient of static friction	

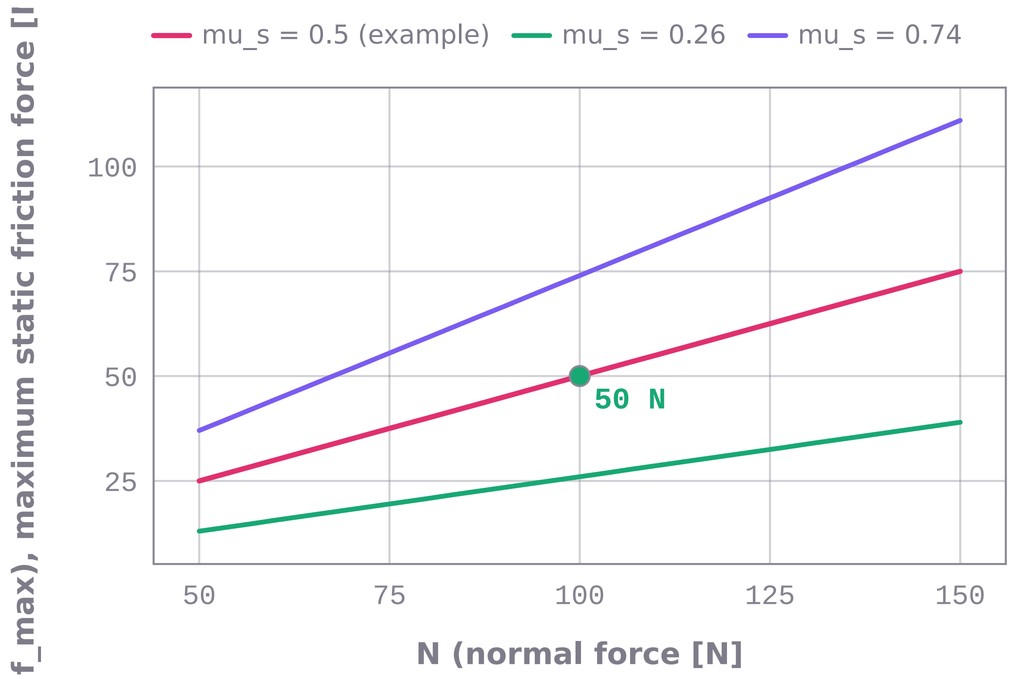
input	meaning	condition
`N`	normal force	
`F`	applied force parallel to the surface (magnitude)	

output	meaning
`f_max`	maximum static friction force
`slips`	whether the body slips
`margin`	f_max - F (positive: it holds)

- **Method:** f_max = mu_s N; it slips when F > f_max (a force equal to the limit holds it on the verge)
- **Assumes:** $mu_s \geq 0$; $N > 0$; $F \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



whether the body slips (slips) over mu_s (coefficient of static friction) from 0.02 to 0.98 and N (normal force) from 10 N to 1000 N (logarithmic vertical axis): each of the 31 × 21 cells is one run of classical-physics-ai999x, coloured by the verdict it returned, yes: 332 cells, no: 319 cells. Other inputs as in the worked example: F = 40.0 N. Marked: the worked example, mu_s = 0.5, N = 100 N → no.



f_max (maximum static friction force) against N (normal force) from 50 N to 150 N. One curve for each value of mu_s (coefficient of static friction): 0.5, 0.26, 0.74. Other inputs as in the worked example: F = 40.0 N. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, N = 100 N, f_max = 50 N.

```
classical_physics_ai999x.solve('static_friction_limit', mu_s=0.5, N=100.0, F=40.0)
# -> f_max = 50 N, slips = False, margin = 10 N   (SOLVED)
```

```
classical_physics_ai999x.solve('static_friction_limit', mu_s=0.3, N=200.0, F=75.0)
# -> f_max = 60 N, slips = True, margin = -15 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Static friction: mu_s = 0.5, N = 100 N, F = 40 N	SOLVED	f_max = 50 N, slips = False, margin = 10 N
Static friction: mu_s = 0.3, N = 200 N, F = 75 N	SOLVED	f_max = 60 N, slips = True, margin = -15 N
Static friction: mu_s = -0.2, N = 100 N, F = 40 N	INVALID_INPUT	Static friction: maximum friction force and whether a body slips: the coefficient of static friction cannot be negative.
Static friction: mu_s = 0.5, N = 100 N	NEEDS_INPUT	static_friction_limit needs F (applied force parallel to the surface (magnitude)). Example: solve('static_friction_limit

Static friction on an incline: critical angle and whether a body slides, `static_friction_incline`

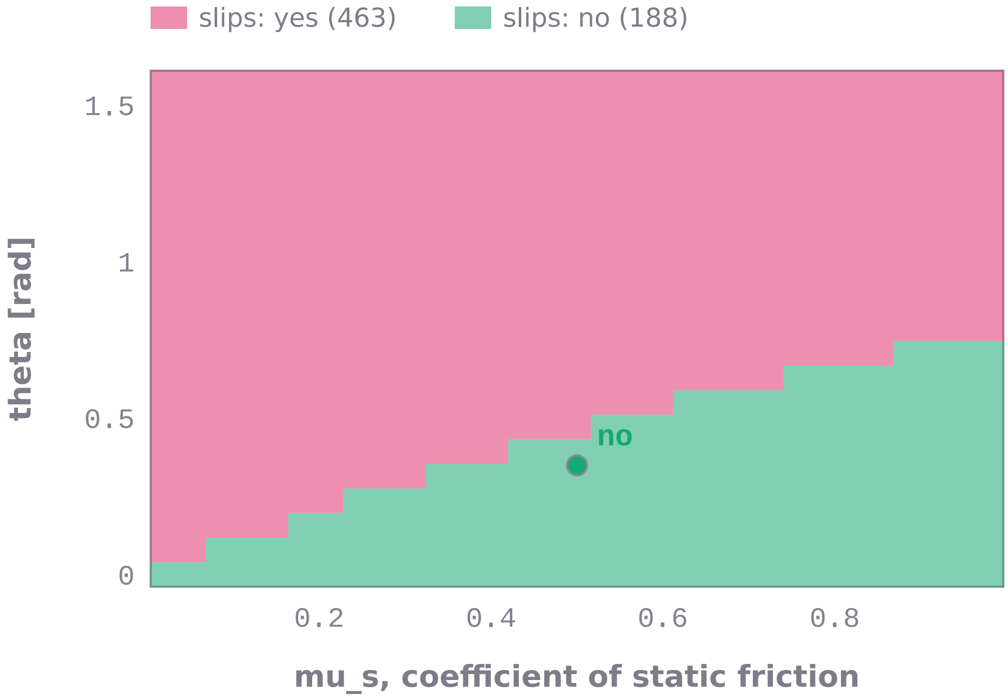
The steepest incline on which a body can rest (tan(theta_c) = mu_s), whether it slides at the given angle and, with its mass, the normal force, the friction needed and the friction available.

$$\tan\theta_c = \mu_s$$

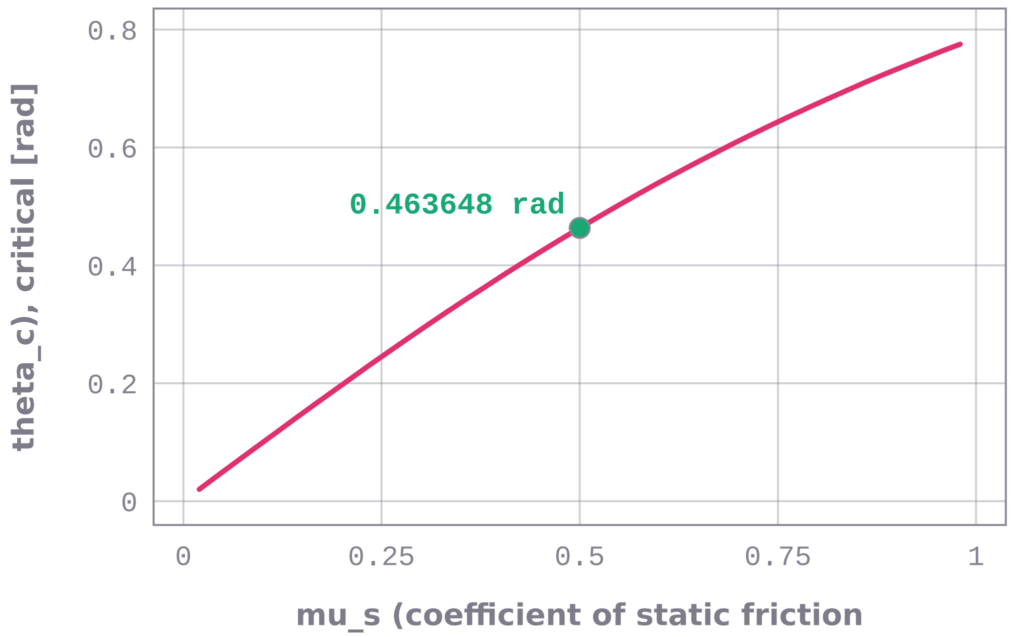
input	meaning	condition
`mu_s`	coefficient of static friction	
`theta`	angle of the incline above the horizontal	
`m`	mass of the body, if the forces are wanted	

output	meaning
`theta_c`	critical (steepest holding) angle
`theta_c_deg`	critical angle in degrees
`slips`	whether the body slides at the given angle
`N`	normal force (when m is given)
`f_needed`	friction needed to hold it (when m is given)
`f_max`	maximum static friction available (when m is given)

- **Method:** theta_c = atan(mu_s); slides when tan(theta) > mu_s; $N = m$ g cos(theta), f_needed = m g sin(theta), f_max = mu_s N
- **Assumes:** $mu_s \geq 0$; $0 \leq \theta < 90$ degrees; $g = 9.80665\, m/s^2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



whether the body slides at the given angle (slips) over mu_s (coefficient of static friction) from 0.02 to 0.98 and theta (angle of the incline above the horizontal) from 0 rad to 1.5708 rad: each of the 31 × 21 cells is one run of classical-physics-ai999x, coloured by the verdict it returned, yes: 463 cells, no: 188 cells. Marked: the worked example, mu_s = 0.5, theta = 0.349066 rad → no.



theta_c (critical (steepest holding) angle) against mu_s (coefficient of static friction) from 0.02 to 0.98. Other inputs as in the worked example: theta = 0.3490658503988659 rad. 41 points computed by classical-physics-ai999x; marked: the worked example, mu_s = 0.5, theta_c = 0.463648 rad.

```
classical_physics_ai999x.solve('static_friction_incline', mu_s=0.5,
theta=0.3490658503988659)
# -> theta_c = 0.463648 rad, theta_c_deg = 26.5651, slips = False   (SOLVED)
```

```
classical_physics_ai999x.solve('static_friction_incline', mu_s=0.3,
theta=0.5235987755982988, m=10.0)
# -> theta_c = 0.291457 rad, theta_c_deg = 16.6992, slips = True, N = 84.9281 N, f_needed =
49.0333 N, f_max = 25.4784 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Static friction on an incline: mu_s = 0.5, theta = 20 deg	SOLVED	theta_c = 0.463648 rad, theta_c_deg = 26.5651, slips = False
Static friction on an incline: mu_s = 0.3, theta = 30 deg, m = 10 kg	SOLVED	theta_c = 0.291457 rad, theta_c_deg = 16.6992, slips = True, N = 84.9281 N, f_needed = 49.0333 N, f_max = 25.4784 N
Static friction on an incline: mu_s = 0.5, theta = 95 deg	INVALID_INPUT	Static friction on an incline: critical angle and whether a body slides: the incline angle must lie between 0 and 90 deg
Static friction on an incline: mu_s = 0.5	NEEDS_INPUT	static_friction_incline needs theta (angle of the incline above the horizontal). Example: solve('static_friction_incline

Tension or compression: stress, strain and change of length (Young's modulus), `youngs_modulus_deformation`

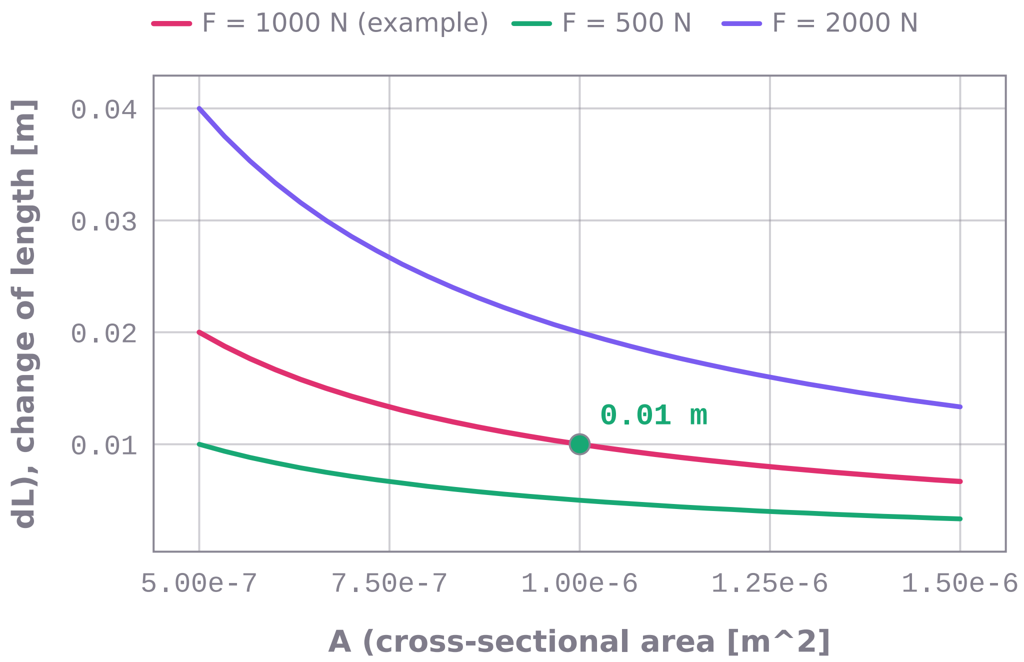
Stress, strain and change of length of a rod or wire of original length L0 and cross-section A under an axial force F.

$$\Delta L = \frac{F L_0}{A E}$$

input	meaning	condition
`F`	axial force (positive: tension, negative: compression)	
`A`	cross-sectional area	
`L0`	original length	
`E`	Young's modulus	

output	meaning
`dL`	change of length (positive: extension)
`stress`	tensile stress F / A
`strain`	strain dL/L_0

- **Method:** stress = F/A; strain = stress/E; dL = strain L0
- **Assumes:** A, L0, $E > 0$; linear elastic (|strain| < 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dL (change of length (positive: extension)) against A (cross-sectional area) from 5.0000e-07 m^2 to 1.5000e-06 m^2. One curve for each value of F (axial force (positive: tension, negative: compression)): 1000 N, 500 N, 2000 N. Other inputs as in the worked example: L0 = 2.0 m, E = 200000000000.0 Pa. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 1.0000e-06 m^2, dL = 0.01 m.

```
classical_physics_ai999x.solve('youngs_modulus_deformation', F=1000.0, A=1e-06, L0=2.0,
E=200000000000.0)
# -> stress = 1e+09 Pa, strain = 0.005, dL = 0.01 m   (SOLVED)
```

```
classical_physics_ai999x.solve('youngs_modulus_deformation', F=-50000.0, A=0.01, L0=3.0,
E=20000000000.0)
# -> stress = -5e+06 Pa, strain = -0.00025, dL = -0.00075 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Young's modulus deformation: F = 1000 N, A = 1 mm^2, L0 = 2 m, E = 200 GPa	SOLVED	stress = 1e+09 Pa, strain = 0.005, dL = 0.01 m
Young's modulus deformation: F = 1000 N, A = 1 mm^2, L0 = 2 m, E = 0 GPa	INVALID_INPUT	Tension or compression: stress, strain and change of length (Young's modulus): Young's modulus E must be positive.
Young's modulus deformation: F = 1000 N, A = 1 mm^2, L0 = 2 m	NEEDS_INPUT	youngs_modulus_deformation needs E (Young's modulus). Example: solve('youngs_modulus_deformation', F='1000 N', A='1e-06

Shear: shear stress, shear strain and displacement (shear modulus), `shear_modulus_deformation`

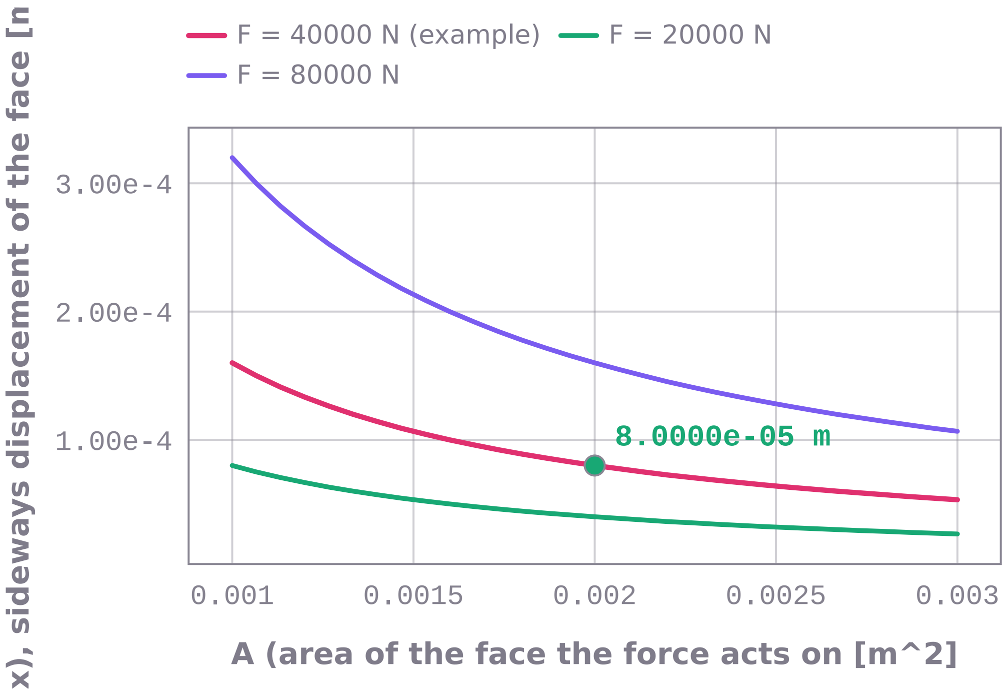
Shear stress, shear strain and the sideways displacement of one face of a block of height h when a force F acts parallel to a face of area A.

$$\Delta x = \frac{Fh}{AS}$$

input	meaning	condition
`F`	force parallel to the face	
`A`	area of the face the force acts on	
`h`	distance between the sheared faces	
`S`	shear modulus	

output	meaning
`dx`	sideways displacement of the face
`stress`	shear stress F / A
`strain`	shear strain dx / h

- **Method:** stress = F/A; strain = stress/S; dx = strain h
- **Assumes:** A, h, $S > 0$; linear elastic (|strain| < 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dx (sideways displacement of the face) against A (area of the face the force acts on) from 0.001 m^2 to 0.003 m^2. One curve for each value of F (force parallel to the face): 40000 N, 20000 N, 80000 N. Other inputs as in the worked example: h = 0.1 m, S = 25000000000.0 Pa. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, A = 0.002 m^2, dx = 8.0000e-05 m.

```
classical_physics_ai999x.solve('shear_modulus_deformation', F=40000.0, A=0.002, h=0.1,
S=25000000000.0)
# -> stress = 2e+07 Pa, strain = 0.0008, dx = 8e-05 m   (SOLVED)
```

```
classical_physics_ai999x.solve('shear_modulus_deformation', F=2000.0, A=0.01, h=0.05,
S=80000000000.0)
# -> stress = 200000 Pa, strain = 2.5e-06, dx = 1.25e-07 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shear modulus deformation: F = 40 kN, A = 20 cm^2, h = 10 cm, S = 25 GPa	SOLVED	stress = 2e+07 Pa, strain = 0.0008, dx = 8e-05 m
Shear modulus deformation: F = 40 kN, A = 0 cm^2, h = 10 cm, S = 25 GPa	INVALID_INPUT	Shear: shear stress, shear strain and displacement (shear modulus): the area A of the sheared face must be positive.
Shear modulus deformation: F = 40 kN, A = 20 cm^2, h = 10 cm	NEEDS_INPUT	shear_modulus_deformation needs S (shear modulus). Example: solve('shear_modulus_deformation', F='40000 N', A='0.002 m^2

Bulk compression: volume change under a pressure change (bulk modulus), `bulk_modulus_compression`

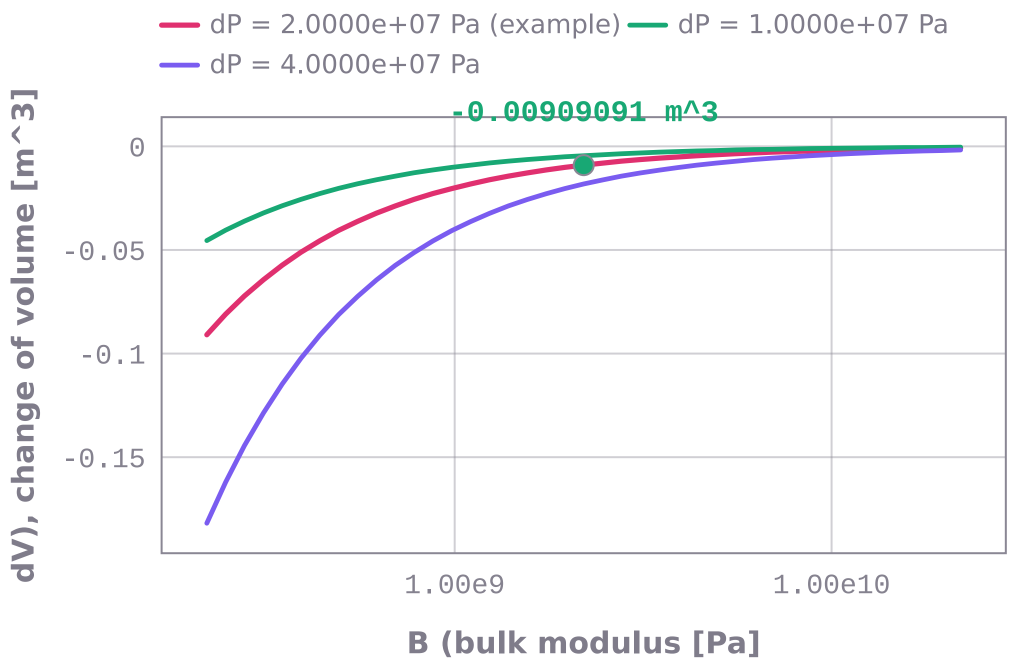
Volumetric strain, change of volume and final volume of a body of volume V0 when the pressure on it rises by dP.

$$\Delta V = -\frac{V_0 \Delta P}{B}$$

input	meaning	condition
`dP`	increase of pressure (negative: decrease)	
`V0`	original volume	
`B`	bulk modulus	

output	meaning
`dV`	change of volume (negative: compression)
`strain`	volumetric strain dV/V_0
`V`	final volume

- **Method:** $dV = -V_0$ dP / B
- **Assumes:** V0, $B > 0$; linear elastic (|dP / B| < 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dV (change of volume (negative: compression)) against B (bulk modulus) from 2.2000e+08 Pa to 2.2000e+10 Pa on a logarithmic axis. One curve for each value of dP (increase of pressure (negative: decrease)): 2.0000e+07 Pa, 1.0000e+07 Pa, 4.0000e+07 Pa. Other inputs as in the worked example: V0 = 1.0 m^3. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, B = 2.2000e+09 Pa, dV = -0.00909091 m^3.

```
classical_physics_ai999x.solve('bulk_modulus_compression', dP=20000000.0, V0=1.0,
B=2200000000.0)
# -> dV = -0.00909091 m^3, strain = -0.00909091, V = 0.990909 m^3   (SOLVED)
```

```
classical_physics_ai999x.solve('bulk_modulus_compression', dP=100000000.0, V0=0.002,
B=160000000000.0)
# -> dV = -1.25e-06 m^3, strain = -0.000625, V = 0.00199875 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bulk modulus compression: dP = 20 MPa, V0 = 1 m^3, B = 2.2 GPa	SOLVED	dV = -0.00909091 m^3, strain = -0.00909091, V = 0.990909 m^3
Bulk modulus compression: dP = 20 MPa, V0 = 1 m^3, B = 0 GPa	INVALID_INPUT	Bulk compression: volume change under a pressure change (bulk modulus): the bulk modulus B must be positive.
Bulk modulus compression: dP = 20 MPa, V0 = 1 m^3	NEEDS_INPUT	bulk_modulus_compression needs B (bulk modulus). Example: solve('bulk_modulus_compression', dP='2e+07 Pa', V0='1 m^3', B

Static equilibrium of a beam on two supports: support reactions, `beam_support_reactions`

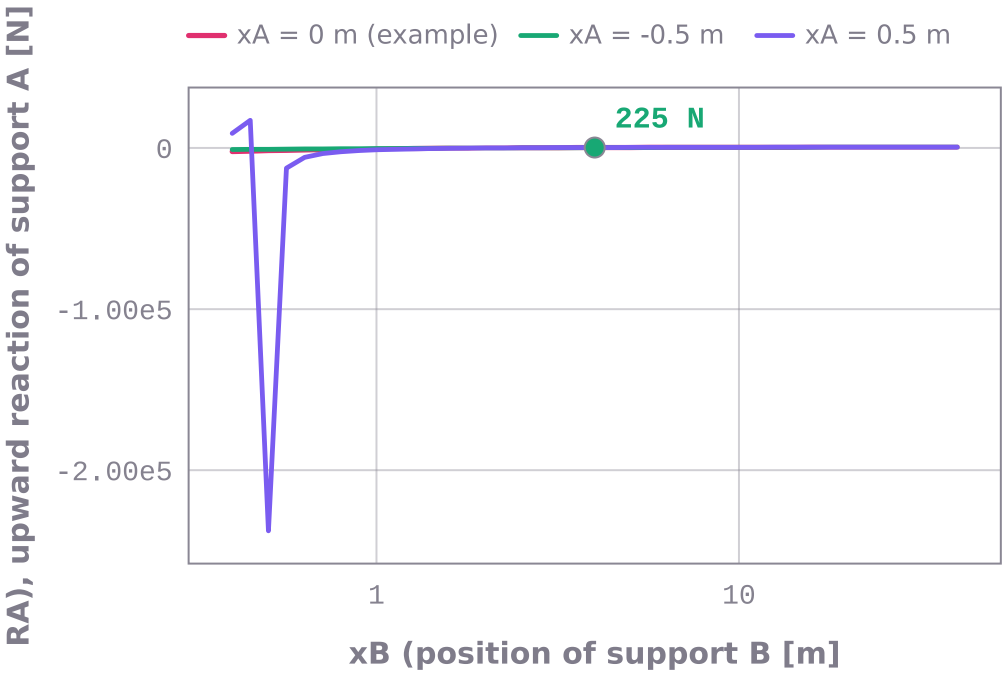
The vertical reactions of the two supports of a beam carrying point loads, from the balance of forces and of moments.

$$\sum F = R_A + R_B, \quad R_B(x_B - x_A) = \sum_i F_i(x_i - x_A)$$

input	meaning	condition
`loads`	point loads in newtons, downward positive (include the beam's weight at its centre)	
`positions`	position of each load along the beam in metres	
`xA`	position of support A	
`xB`	position of support B	

output	meaning
`RA`	upward reaction of support A
`RB`	upward reaction of support B
`total`	total load

- **Method:** moments about A: RB (xB - xA) = sum F (x - xA); forces: RA = sum F - RB
- **Assumes:** one position per load; $xA \neq xB$; a rigid beam in equilibrium, vertical loads
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



RA (upward reaction of support A) against xB (position of support B) from 0.4 m to 40 m on a logarithmic axis. One curve for each value of xA (position of support A): 0 m, -0.5 m, 0.5 m. Other inputs as in the worked example: loads = [200.0, 300.0], positions = [1.0, 3.0]. 41 points per curve computed by classical-physics-ai999x; marked: the worked example, xB = 4 m, RA = 225 N.

```
classical_physics_ai999x.solve('beam_support_reactions', loads=[200.0, 300.0], positions=
[1.0, 3.0], xA=0.0, xB=4.0)
# -> RA = 225 N, RB = 275 N, total = 500 N   (SOLVED)
```

```
classical_physics_ai999x.solve('beam_support_reactions', loads=[500.0], positions=[5.0],
xA=0.0, xB=4.0)
# -> RA = -125 N, RB = 625 N, total = 500 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Beam support reactions: loads = 200, 300 N; positions = 1, 3 m; xA = 0 m; xB = 4 m	SOLVED	RA = 225 N, RB = 275 N, total = 500 N
Beam support reactions: loads = 200, 300 N; positions = 1, 3 m; xA = 2 m; xB = 2 m	INVALID_INPUT	Static equilibrium of a beam on two supports: support reactions: the two supports must be at different points.
Beam support reactions: loads = 200, 300 N; positions = 1 m; xA = 0 m; xB = 4 m	INVALID_INPUT	Static equilibrium of a beam on two supports: support reactions: give one position for every load.
Beam support reactions: loads = 200, 300 N; positions = 1, 3 m; xA = 0 m	NEEDS_INPUT	beam_support_reactions needs xB (position of support B). Example: solve('beam_support_reactions', loads=[200.0, 300.0],

Centre of mass of a set of point masses, `centre_of_mass_point_masses`

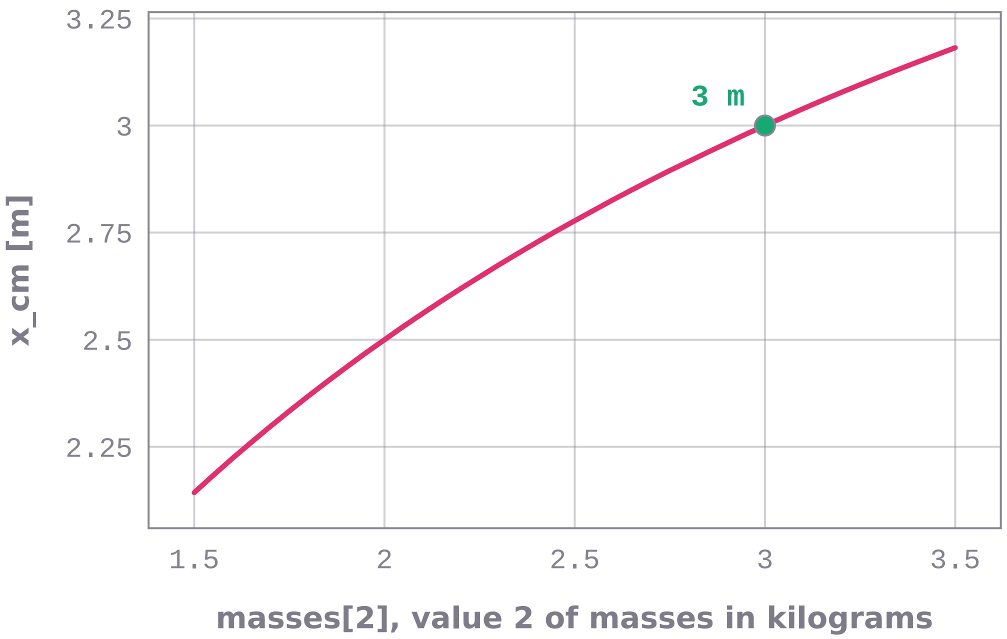
The coordinates of the centre of mass of point masses on a line or in a plane, and the total mass.

$$x_{cm} = \frac{\sum_i m_i x_i}{\sum_i m_i}$$

input	meaning	condition
`masses`	masses in kilograms	
`x`	x coordinate of each mass in metres	
`y`	y coordinate of each mass in metres (for masses in a plane)	

output	meaning
`x_cm`	x coordinate of the centre of mass
`y_cm`	y coordinate of the centre of mass (when y is given)
`M`	total mass

- **Method:** x_cm = sum m x / sum m (and the same for y)
- **Assumes:** every mass > 0; one coordinate per mass
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x_cm (x coordinate of the centre of mass) as value 2 of masses (masses in kilograms) moves from 1.5 to 3.5, the other values as given ([2.0, 3.0]); 41 runs of classical-physics-ai999x. Other inputs: x = [0.0, 5.0]. Marked: the worked example, masses[2] = 3, x_cm = 3 m.

```
classical_physics_ai999x.solve('centre_of_mass_point_masses', masses=[2.0, 3.0], x=[0.0, 5.0])
# -> M = 5 kg, x_cm = 3 m   (SOLVED)
```

```
classical_physics_ai999x.solve('centre_of_mass_point_masses', masses=[1.0, 1.0, 2.0], x=[0.0, 4.0, 0.0], y=[0.0, 0.0, 3.0])
# -> M = 4 kg, x_cm = 1 m, y_cm = 1.5 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Centre of mass: masses = 2, 3 kg; x = 0, 5 m	SOLVED	M = 5 kg, x_cm = 3 m
Center of mass: masses = 1, 1, 2 kg; x = 0, 4, 0 m; y = 0, 0, 3 m	SOLVED	M = 4 kg, x_cm = 1 m, y_cm = 1.5 m
Centre of mass: masses = 2, -3 kg; x = 0, 5 m	INVALID_INPUT	Centre of mass of a set of point masses: every mass must be positive.
Centre of mass: masses = 2, 3 kg	NEEDS_INPUT	centre_of_mass_point_masses needs x (x coordinate of each mass in metres). Example: solve('centre_of_mass_point_masses',

Vector components and the resultant of two-dimensional vectors, `vector_components_resultant`

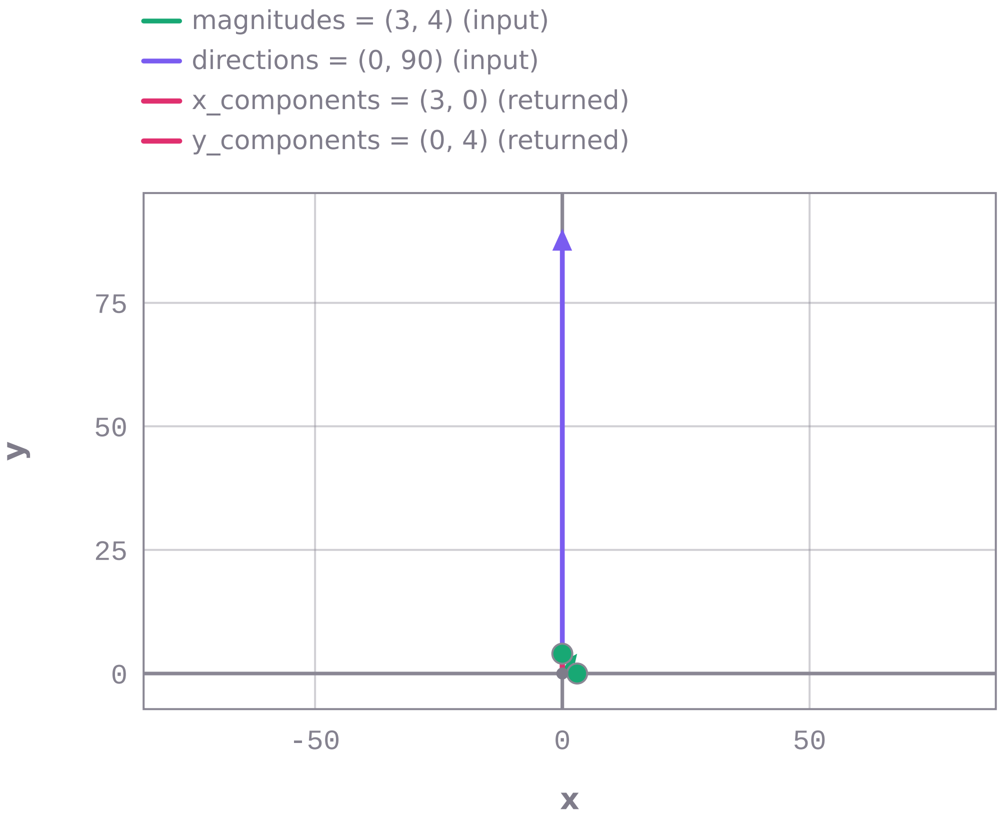
The x and y components of each vector given by magnitude and direction, and the magnitude and direction of their sum (forces, displacements, velocities: any one unit for all).

$$R = \sqrt{R_x^2 + R_y^2}, \quad \theta_R = \operatorname{atan2}(R_y, R_x)$$

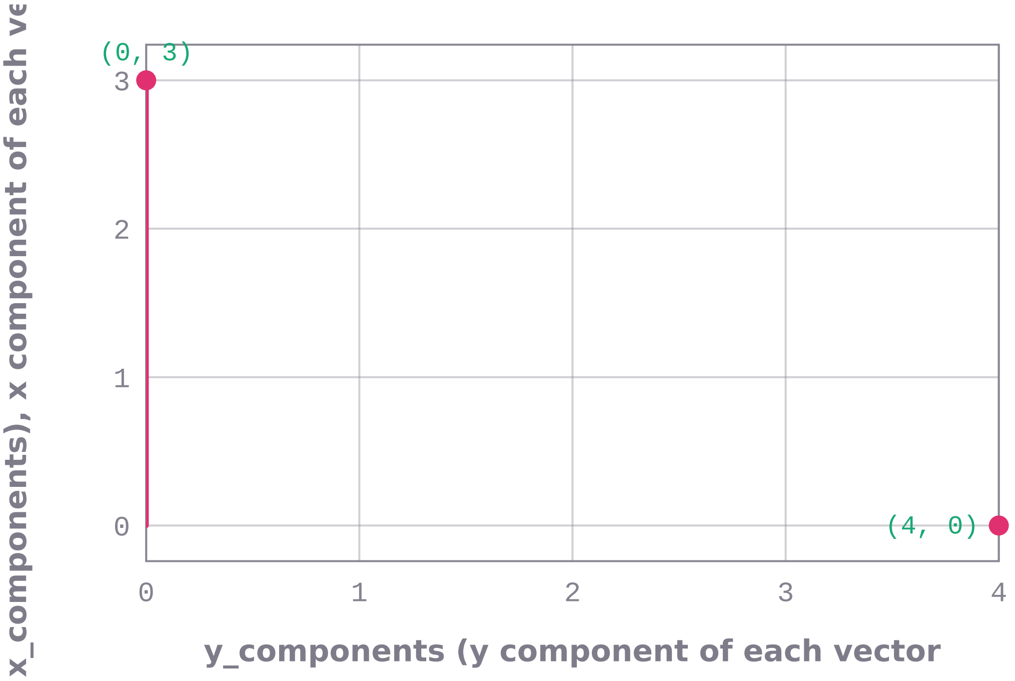
input	meaning	condition
`magnitudes`	magnitude of each vector (all in one unit)	
`directions`	direction of each vector in degrees, counterclockwise from +x	

output	meaning
`R`	magnitude of the resultant
`direction_deg`	direction of the resultant in degrees from +x (0 to 360)
`Rx`	x component of the resultant
`Ry`	y component of the resultant
`x_components`	x component of each vector
`y_components`	y component of each vector

- **Method:** components m cos(theta), m sin(theta); sums Rx, Ry; $R = \sqrt{Rx^2 + R^2}$; direction atan2(Ry, Rx)
- **Assumes:** one direction per magnitude; magnitudes >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input magnitudes = (3, 4), directions = (0, 90); returned by classical-physics-ai999x: x_components = (3, 0), y_components = (0, 4); also returned Rx = 3, Ry = 4, R = 5, direction_deg = 53.1301. Marked: the tip of each returned vector.



x component of each vector (x_components) at each y component of each vector (y_components), both returned by classical-physics-ai999x for the worked example (magnitudes = [3.0, 4.0], directions = [0.0, 90.0]): 0 → 3; 4 → 0.

```
classical_physics_ai999x.solve('vector_components_resultant', magnitudes=[3.0, 4.0],
directions=[0.0, 90.0])
# -> x_components = [3.0, 0.0], y_components = [0.0, 4.0], Rx = 3, Ry = 4, R = 5,
direction_deg = 53.1301   (SOLVED)
```

```
classical_physics_ai999x.solve('vector_components_resultant', magnitudes=[10.0],
directions=[30.0])
# -> x_components = [8.66025403784], y_components = [5.0], Rx = 8.66025, Ry = 5, R = 10,
direction_deg = 30   (SOLVED)
```

```
classical_physics_ai999x.solve('vector_components_resultant', magnitudes=[5.0, 5.0],
directions=[0.0, 180.0])
# -> x_components = [5.0, -5.0], y_components = [0.0, 0.0], Rx = 0, Ry = 0, R = 0,
direction_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vector resultant: magnitudes = 3, 4; directions = 0, 90 deg	SOLVED	x_components = [3.0, 0.0], y_components = [0.0, 4.0], Rx = 3, Ry = 4, R = 5, direction_deg = 53.1301
Vector resultant: magnitudes = 3, -4; directions = 0, 90 deg	INVALID_INPUT	Vector components and the resultant of two-dimensional vectors: magnitudes cannot be negative (reverse the direction ins
Vector resultant: magnitudes = 3, 4; directions = 0 deg	INVALID_INPUT	Vector components and the resultant of two-dimensional vectors: give one direction for every magnitude.
Vector resultant: magnitudes = 3, 4	NEEDS_INPUT	vector_components_resultant needs directions (direction of each vector in degrees, counterclockwise from +x). Example: s

Newton's second law for rotation: angular acceleration from torque, `rotational_second_law`

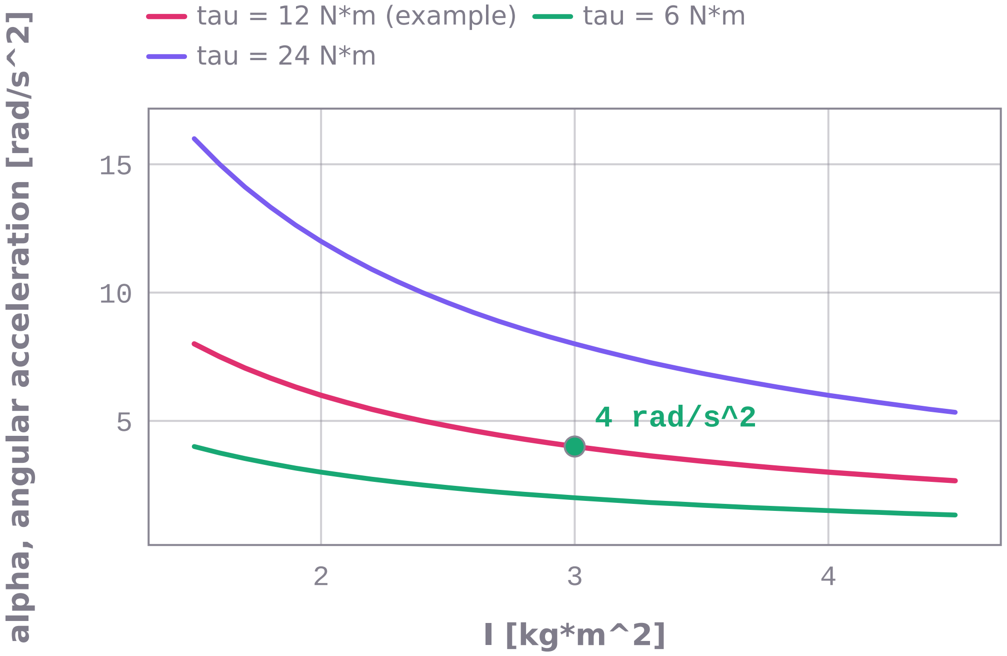
The angular acceleration of a rigid body about a fixed axis under a net torque.

$$\tau = I\alpha$$

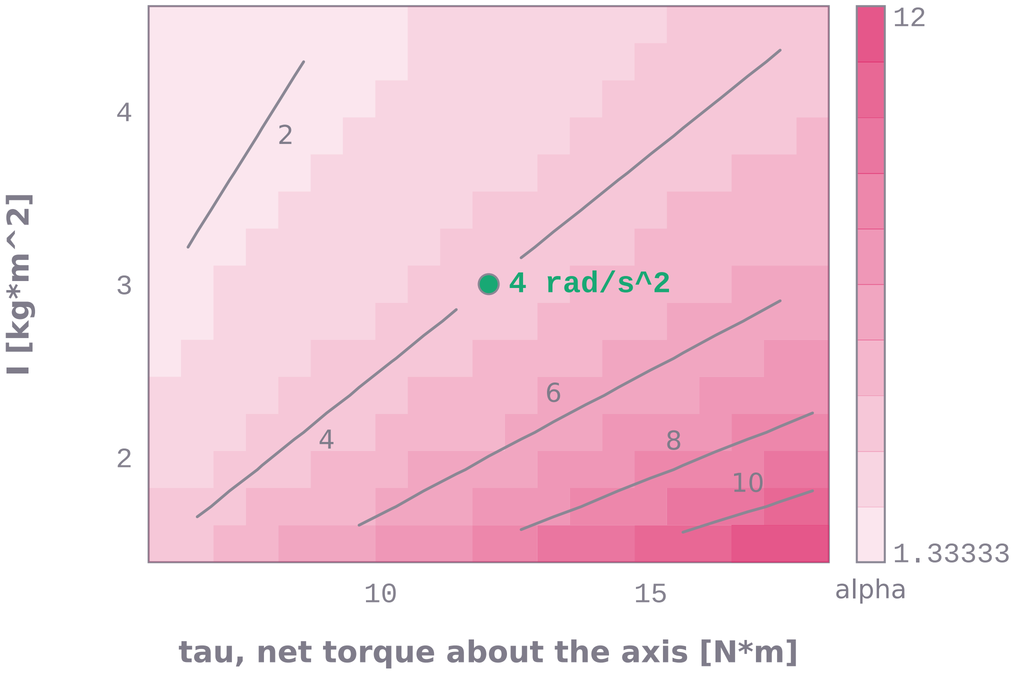
input	meaning	condition
`tau`	net torque about the axis	
`I`	moment of inertia about the same axis	$I \neq 0$

output	meaning
`alpha`	angular acceleration

- **Method:** $\alpha = \tau/I$
- **Assumes:** $I > 0$; rigid body, fixed axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



alpha (angular acceleration) against I (moment of inertia about the same axis) from 1.5 kg*m^2 to 4.5 kg*m^2. One curve for each value of tau (net torque about the axis): 12 N*m, 6 N*m, 24 N*m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I = 3 kg*m^2, alpha = 4 rad/s^2.



alpha (angular acceleration) over tau (net torque about the axis) from 6 N*m to 18 N*m and I (moment of inertia about the same axis) from 1.5 kg*m^2 to 4.5 kg*m^2: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 1.33333 rad/s^2 to dark 12 rad/s^2); iso-lines at alpha = 2, 4, 6, 8, 10 rad/s^2. Marked: the worked example, tau = 12 N*m, I = 3 kg*m^2, alpha = 4 rad/s^2.

```
classical_physics_ai999x.solve('rotational_second_law', tau=12.0, I=3.0)
# -> alpha = 4 rad/s^2   (SOLVED)
```

```
classical_physics_ai999x.solve('rotational_second_law', tau=-2.0, I=0.5)
# -> alpha = -4 rad/s^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rotational second law: tau = 12 N*m, I = 3 kg*m^2	SOLVED	alpha = 4 rad/s^2
Rotational second law: tau = 12 N*m, I = 0 kg*m^2	INVALID_INPUT	Newton's second law for rotation: angular acceleration from torque: the moment of inertia must be positive.
Rotational second law: tau = 12 N*m	NEEDS_INPUT	rotational_second_law needs I (moment of inertia about the same axis). Example: solve('rotational_second_law', tau='12 N

Conservation of angular momentum: change of spin rate, `angular_momentum_conservation`

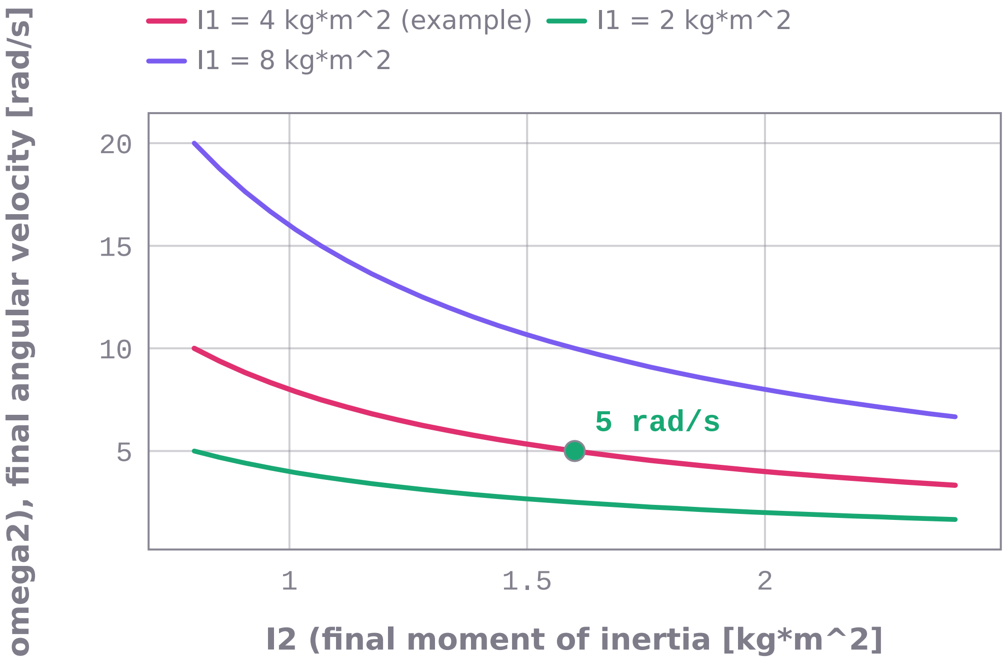
The new angular velocity of a body whose moment of inertia changes with no external torque (a spinning skater pulling in the arms), with its rotational kinetic energy before and after.

$$I_1\omega_1 = I_2\omega_2$$

input	meaning	condition
`I1`	initial moment of inertia	
`omega1`	initial angular velocity	
`I2`	final moment of inertia	

output	meaning
`omega2`	final angular velocity
`L`	angular momentum (conserved)
`K1`	initial rotational kinetic energy
`K2`	final rotational kinetic energy
`dK`	change of rotational kinetic energy

- **Method:** I1 $omega1 = I_2$ omega2; $K = I\,\omega^2/2$
- **Assumes:** I1, $I_2 > 0$; no external torque about the axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



omega2 (final angular velocity) against I2 (final moment of inertia) from 0.8 kg*m^2 to 2.4 kg*m^2. One curve for each value of I1 (initial moment of inertia): 4 kg*m^2, 2 kg*m^2, 8 kg*m^2. Other inputs as in the worked example: omega1 = 2.0 rad/s. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, I2 = 1.6 kg*m^2, omega2 = 5 rad/s.

```
classical_physics_ai999x.solve('angular_momentum_conservation', I1=4.0, omega1=2.0, I2=1.6)
# -> omega2 = 5 rad/s, L = 8 kg*m^2/s, K1 = 8 J, K2 = 20 J, dK = 12 J   (SOLVED)
```

```
classical_physics_ai999x.solve('angular_momentum_conservation', I1=2.0, omega1=6.0, I2=3.0)
# -> omega2 = 4 rad/s, L = 12 kg*m^2/s, K1 = 36 J, K2 = 24 J, dK = -12 J   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angular momentum conservation: I1 = 4 kg*m^2, omega1 = 2 rad/s, I2 = 1.6 kg*m^2	SOLVED	omega2 = 5 rad/s, L = 8 kg*m^2/s, K1 = 8 J, K2 = 20 J, dK = 12 J
Angular momentum conservation: I1 = 4 kg*m^2, omega1 = 2 rad/s, I2 = 0 kg*m^2	INVALID_INPUT	Conservation of angular momentum: change of spin rate: the final moment of inertia I2 must be positive.
Angular momentum conservation: I1 = 4 kg*m^2, omega1 = 2 rad/s	NEEDS_INPUT	angular_momentum_conservation needs I2 (final moment of inertia). Example: solve('angular_momentum_conservation', I1='4

Sound intensity level in decibels, `sound_intensity_level`

The intensity level in decibels of a sound of given intensity, or the intensity of a sound of given level (reference: the threshold of hearing, $1e-12$ W/m^2).

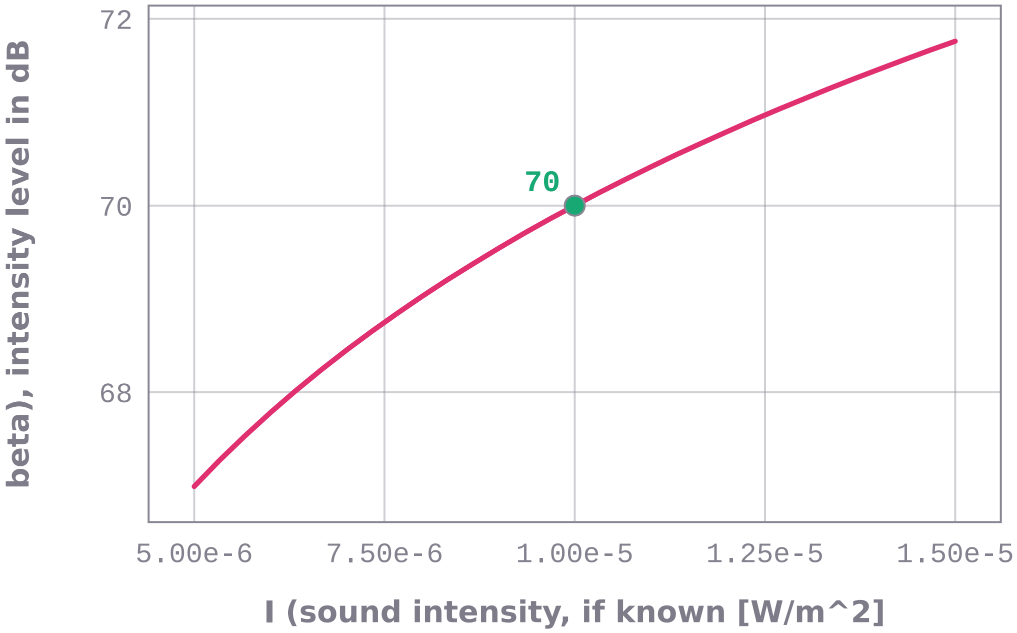
$$\beta = 10\log_{10}\frac{I}{I_0}$$

input	meaning	condition
`I`	sound intensity, if known	

input	meaning	condition
`beta`	sound level in dB, if known	

output	meaning
`beta`	intensity level in dB
`I`	sound intensity

- **Method:** $\beta = 10$ log10(I / I0); $I = I_0\ 10^{\beta/10}$; $I_0 = 1e - 12\ W/m^2$
- **Assumes:** exactly one of I and beta; $I > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



beta (intensity level in dB) against I (sound intensity, if known) from 5.0000e-06 W/m^2 to 1.5000e-05 W/m^2. 31 points computed by classical-physics-ai999x; marked: the worked example, I = 1.0000e-05 W/m^2, beta = 70.

```
classical_physics_ai999x.solve('sound_intensity_level', I=1e-05)
# -> beta = 70, I = 1e-05 W/m^2   (SOLVED)
```

```
classical_physics_ai999x.solve('sound_intensity_level', beta=90.0)
# -> beta = 90, I = 0.001 W/m^2   (SOLVED)
```

```
classical_physics_ai999x.solve('sound_intensity_level', I=3.2e-06)
# -> beta = 65.0515, I = 3.2e-06 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sound intensity level: I = 1e-5 W/m^2	SOLVED	beta = 70, I = 1e-05 W/m^2
Sound intensity level: beta = 90 dB	SOLVED	beta = 90, I = 0.001 W/m^2
Sound intensity level: I = 0 W/m^2	INVALID_INPUT	Sound intensity level in decibels: the intensity I must be positive.

question	outcome	result
Sound intensity level: I = 1e-5 W/m^2, beta = 70 dB	INVALID_INPUT	Sound intensity level in decibels: give exactly one of the intensity I or the level beta.

Change of sound level and the ratio of two intensities, `sound_level_change`

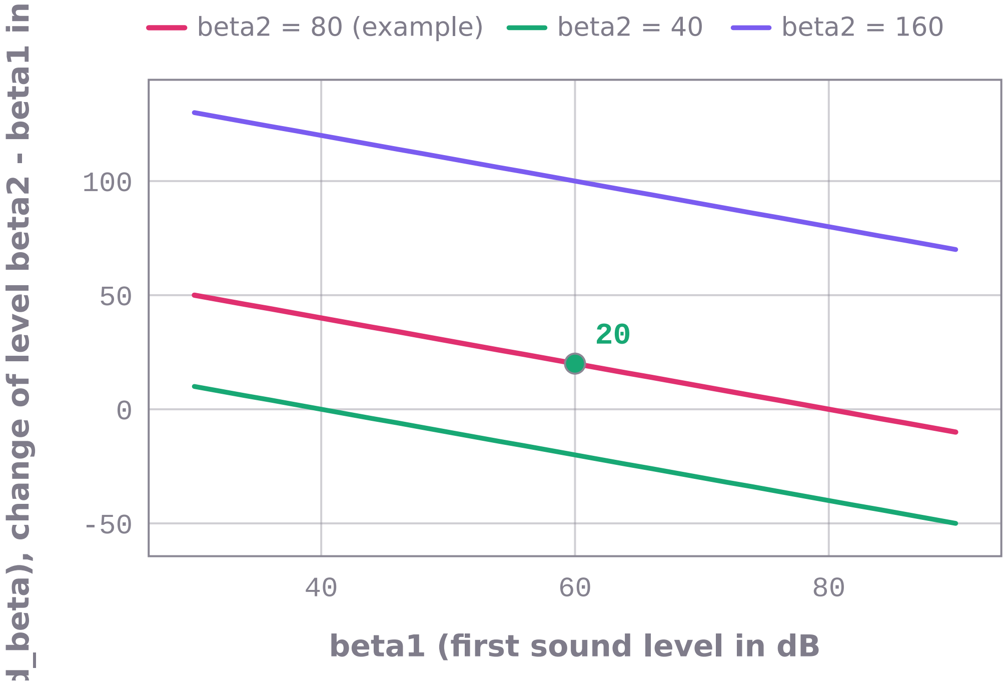
The ratio of the intensities of two sounds from their levels, or the change of level (and the new level) for a given ratio of intensities.

$$\beta_2 - \beta_1 = 10 \log_{10} \frac{I_2}{I_1}$$

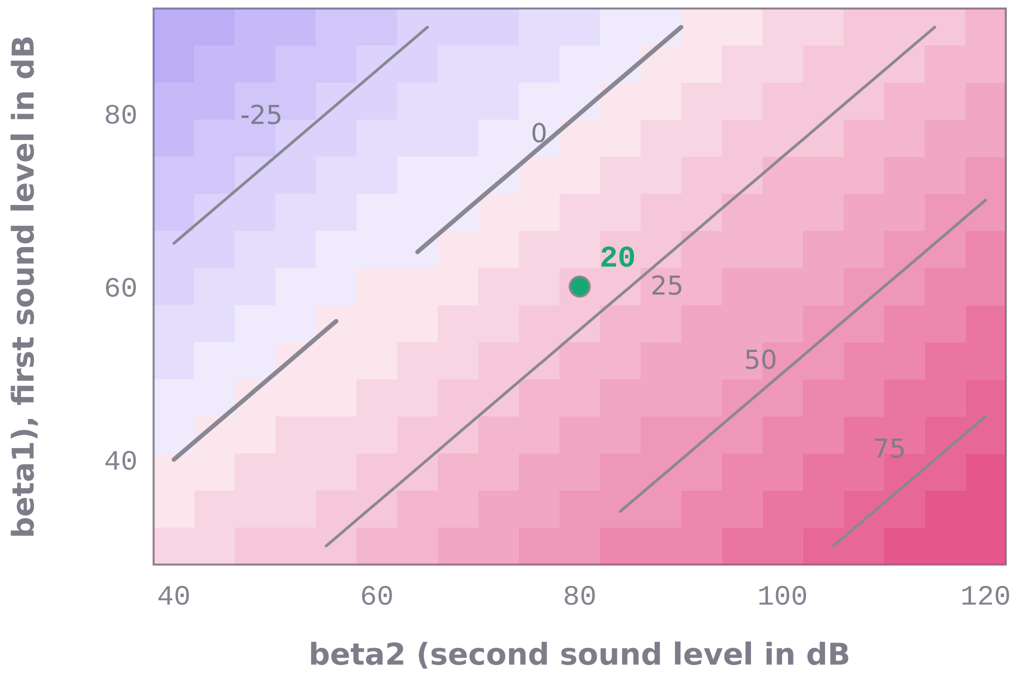
input	meaning	condition
`beta1`	first sound level in dB	
`beta2`	second sound level in dB	
`ratio`	intensity ratio I_2/I_1, if given instead of the second level	

output	meaning
`d_beta`	change of level $beta2 - beta1$ in dB
`ratio`	intensity ratio I_2/I_1
`beta2`	second level in dB (when the first is given)

- **Method:** $beta2 - beta1 = 10$ log10(I2 / I1)
- **Assumes:** both levels, or the ratio (with or without the first level); ratio > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d_beta (change of level beta2 - beta1 in dB) against beta1 (first sound level in dB) from 30 to 90. One curve for each value of beta2 (second sound level in dB): 80, 40, 160. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, beta1 = 60, d_beta = 20.



d_beta (change of level beta2 - beta1 in dB) over beta2 (second sound level in dB) from 40 to 120 and beta1 (first sound level in dB) from 30 to 90: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its sign and magnitude (second colour negative); iso-lines at d_beta = -25, 0, 25, 50, 75. Marked: the worked example, beta2 = 80, beta1 = 60, d_beta = 20.

```
classical_physics_ai999x.solve('sound_level_change', beta1=60.0, beta2=80.0)
# -> d_beta = 20, ratio = 100, beta2 = 80   (SOLVED)
```

```
classical_physics_ai999x.solve('sound_level_change', ratio=100.0)
# -> d_beta = 20, ratio = 100   (SOLVED)
```

```
classical_physics_ai999x.solve('sound_level_change', beta1=90.0, beta2=87.0)
# -> d_beta = -3, ratio = 0.501187, beta2 = 87   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sound level change: beta1 = 60 dB, beta2 = 80 dB	SOLVED	d_beta = 20, ratio = 100, beta2 = 80
Sound level change: beta1 = 60 dB, ratio = 2	SOLVED	d_beta = 3.0103, ratio = 2, beta2 = 63.0103
Sound level change: ratio = 0	INVALID_INPUT	Change of sound level and the ratio of two intensities: the intensity ratio I2/I1 must be positive.
Sound level change: beta1 = 60 dB	INVALID_INPUT	Change of sound level and the ratio of two intensities: give both levels beta1 and beta2, or the intensity ratio.

Standing waves: harmonics of a string and of open or closed pipes, `standing_wave_harmonics`

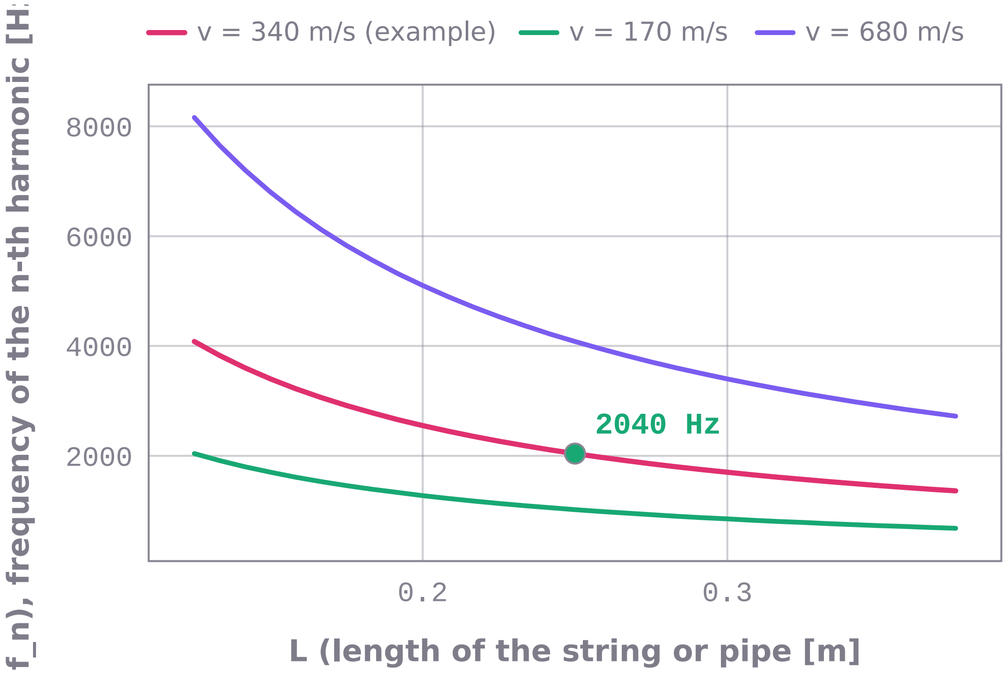
The frequency and wavelength of the n-th harmonic, and the fundamental, of a string fixed at both ends, a pipe open at both ends or a pipe closed at one end.

$$f_n = \frac{nv}{2L} \text{ (string, open pipe)}, \qquad f_n = \frac{nv}{4L},\ n \text{ odd (closed pipe)}$$

input	meaning	condition
`system`	string fixed at both ends, pipe open at both ends, or pipe closed at one end	
`L`	length of the string or pipe	
`v`	wave speed	
`n`	harmonic number (f_n = n f_1); odd only for a closed pipe	

output	meaning
`f_n`	frequency of the n-th harmonic
`lam_n`	wavelength of the n-th harmonic
`f1`	fundamental frequency

- **Method:** string and open pipe: $f_n = n\,v/(2L)$; closed pipe: $f_n = n\,v/(4L)$, n odd; $lambda_n = v/f_n$
- **Assumes:** L, $v > 0$; $n \geq 1$; n odd for a pipe closed at one end; no end correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f_n (frequency of the n-th harmonic) against L (length of the string or pipe) from 0.125 m to 0.375 m. One curve for each value of v (wave speed): 340 m/s, 170 m/s, 680 m/s. Other inputs as in the worked example: system = string, n = 3. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, L = 0.25 m, f_n = 2040 Hz.

```
classical_physics_ai999x.solve('standing_wave_harmonics', system='string', L=0.65, v=286.0,
n=1)
# -> f_n = 220 Hz, lam_n = 1.3 m, f1 = 220 Hz   (SOLVED)
```

```
classical_physics_ai999x.solve('standing_wave_harmonics', system='open pipe', L=0.5,
v=340.0, n=2)
# -> f_n = 680 Hz, lam_n = 0.5 m, f1 = 340 Hz   (SOLVED)
```

```
classical_physics_ai999x.solve('standing_wave_harmonics', system='closed pipe', L=0.25,
v=340.0, n=3)
# -> f_n = 1020 Hz, lam_n = 0.333333 m, f1 = 340 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standing wave harmonic, closed pipe: L = 25 cm, v = 340 m/s, n = 3	SOLVED	f_n = 1020 Hz, lam_n = 0.333333 m, f1 = 340 Hz
Standing wave harmonic, string: L = 65 cm, v = 286 m/s, n = 1	SOLVED	f_n = 220 Hz, lam_n = 1.3 m, f1 = 220 Hz
Standing wave harmonic, closed pipe: L = 25 cm, v = 340 m/s, n = 2	INVALID_INPUT	Standing waves: harmonics of a string and of open or closed pipes: a pipe closed at one end has only odd harmonics (n =
Standing wave harmonic, open pipe: L = 50 cm, n = 2	NEEDS_INPUT	standing_wave_harmonics needs v (wave speed). Example: solve('standing_wave_harmonics', system='string', L='0.65 m', v='

Wave speed on a stretched string, `string_wave_speed`

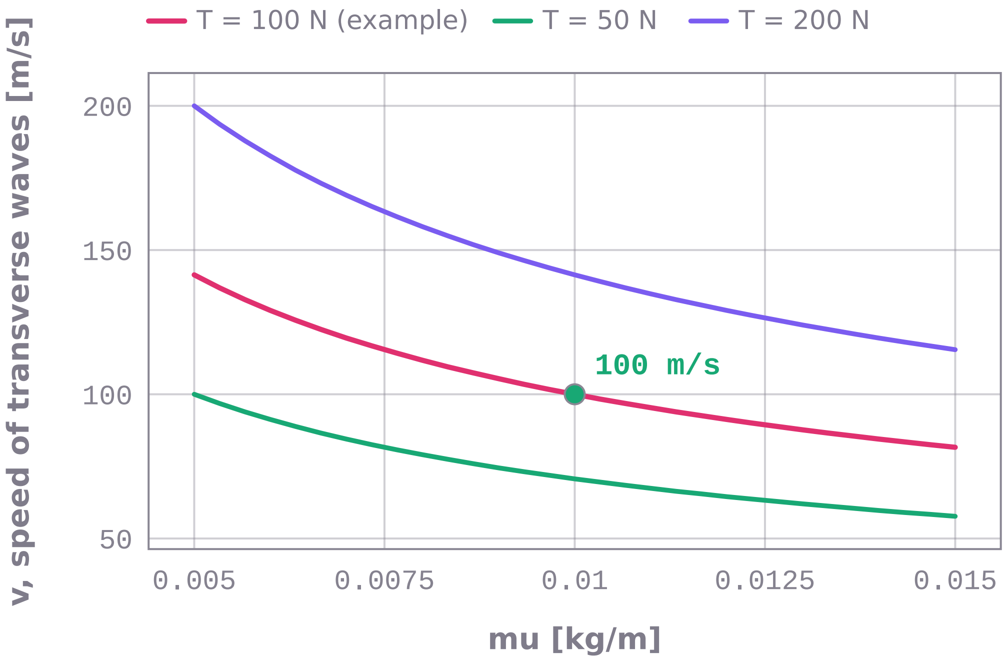
The speed of transverse waves on a string from its tension and its mass per unit length.

$$v = \sqrt{\frac{T}{\mu}}$$

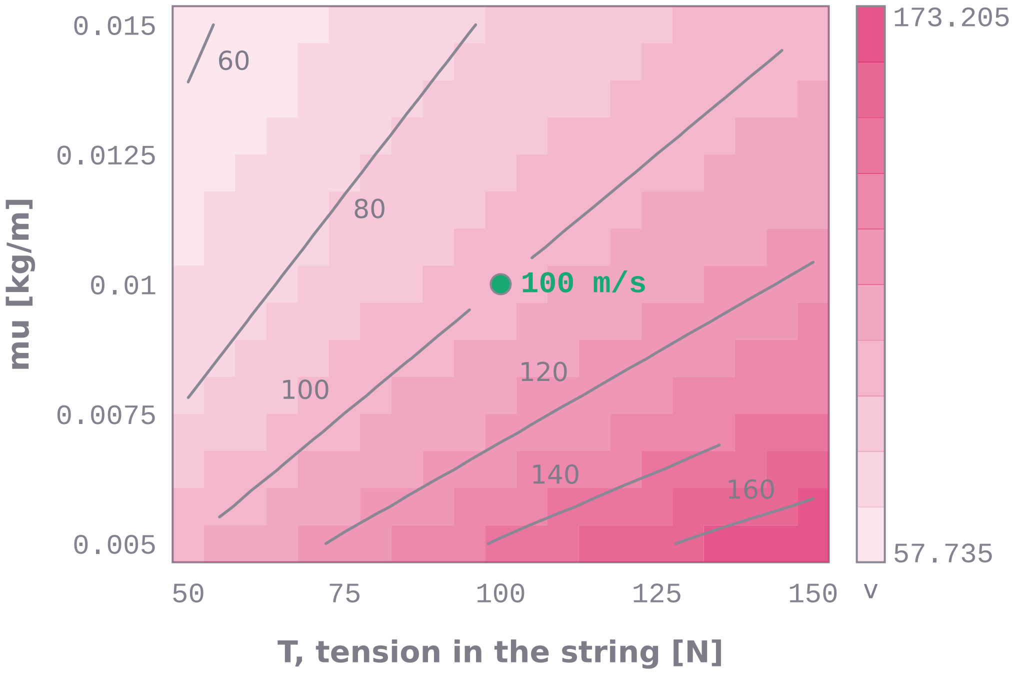
input	meaning	condition
`T`	tension in the string	
`mu`	mass per unit length of the string	$mu \neq 0$

output	meaning
`v`	speed of transverse waves

- **Method:** $v = \sqrt{\frac{T}{\mu}}$
- **Assumes:** $T > 0$, $mu > 0$; a perfectly flexible string, small amplitudes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (speed of transverse waves) against mu (mass per unit length of the string) from 0.005 kg/m to 0.015 kg/m. One curve for each value of T (tension in the string): 100 N, 50 N, 200 N. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, mu = 0.01 kg/m, v = 100 m/s.



v (speed of transverse waves) over T (tension in the string) from 50 N to 150 N and mu (mass per unit length of the string) from 0.005 kg/m to 0.015 kg/m: each of the 21 × 15 cells is one run of classical-physics-ai999x, shaded by its value (light 57.735 m/s to dark 173.205 m/s); iso-lines at v = 60, 80, 100, 120, 140, 160 m/s. Marked: the worked example, T = 100 N, mu = 0.01 kg/m, v = 100 m/s.

```
classical_physics_ai999x.solve('string_wave_speed', T=100.0, mu=0.01)
# -> v = 100 m/s   (SOLVED)
```

```
classical_physics_ai999x.solve('string_wave_speed', T=80.0, mu=0.005)
# -> v = 126.491 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wave speed on a string: T = 100 N, mu = 0.01 kg/m	SOLVED	v = 100 m/s
Wave speed on a string: T = 80 N, mu = 5 g/m	SOLVED	v = 126.491 m/s
Wave speed on a string: T = 100 N, mu = 0 kg/m	INVALID_INPUT	Wave speed on a stretched string: the linear mass density mu must be positive.
Wave speed on a string: T = 100 N	NEEDS_INPUT	string_wave_speed needs mu (mass per unit length of the string). Example: solve('string_wave_speed', T='100 N', mu='0.01

Gauss's law: electric field of a line, a plane, a charged sphere or a shell, `gauss_law_symmetric_field`

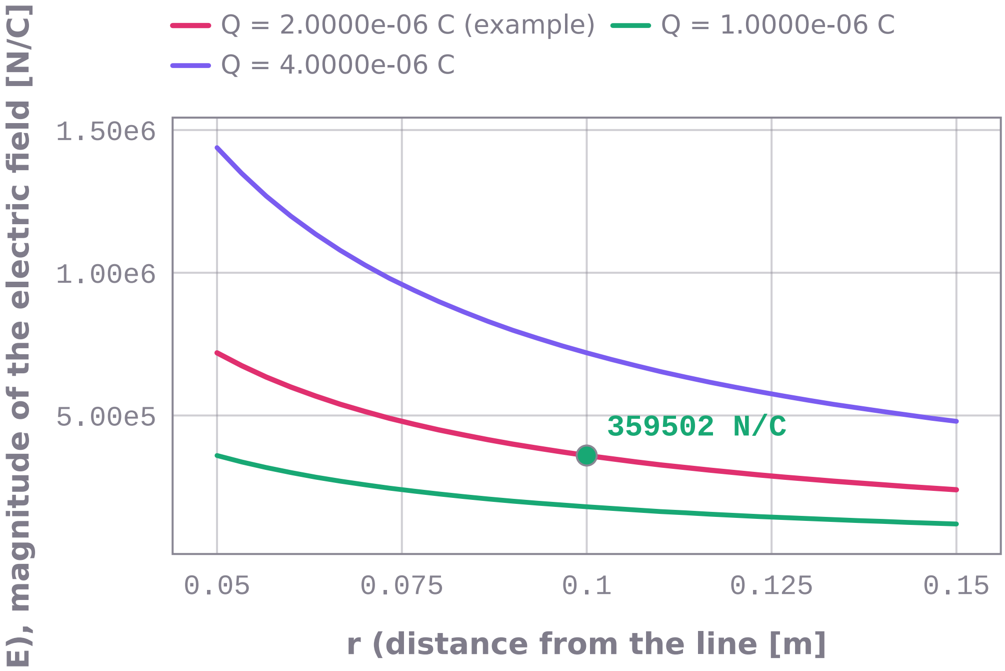
The electric field of a symmetric charge distribution by Gauss's law: an infinite line (charge Q on a length L), an infinite plane sheet (charge Q on an area A), a uniformly charged solid sphere or a thin spherical shell of radius R, inside or outside; with the enclosed charge, the flux through the Gaussian surface and the charge density.

$$\oint \vec{E}\cdot d\vec{A} = \frac{Q_{enc}}{\varepsilon_0}: \quad E_{line} = \frac{\lambda}{2\pi\varepsilon_0 r},\ E_{plane} = \frac{\sigma}{2\varepsilon_0},\ E_{sphere} = \frac{kQr}{R^3}\ (r < R)$$

input	meaning	condition
`geometry`	line, plane, sphere (uniformly charged solid ball) or shell (thin shell, or a conducting sphere)	
`Q`	charge (with its sign): on the length L, the area A, or the whole sphere or shell	
`r`	distance from the line, or from the centre (not needed for a plane)	
`L`	length of line that carries Q (line only; lambda = Q/L)	
`A`	area of plane that carries Q (plane only; sigma = Q/A)	
`R`	radius of the sphere or shell (sphere and shell only)	

output	meaning
`E`	magnitude of the electric field
`E_radial`	field component away from the line, plane or centre
`Q_enc`	charge enclosed by the Gaussian surface through the point
`flux`	electric flux through the Gaussian surface, Q_enc/eps0 in N m^2/C
`density`	charge density: lambda in C/m (line), sigma in C/m^2 (plane, shell), rho in C/m^3 (sphere)

- **Method:** line E = lambda/(2 pi eps0 r); plane E = sigma/(2 eps0); sphere $E = k$ Q r/R^3 inside, k Q/r^2 outside; shell $E = 0$ inside, k Q/r^2 outside; flux = Q_enc/eps0
- **Assumes:** $r > 0$, $L > 0$, $A > 0$, $R > 0$; line and plane treated as infinite; uniform charge; on a shell $(r = R)$ the field is refused; charges in vacuum (or air)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



E (magnitude of the electric field) against r (distance from the line, or from the centre (not needed for a plane)) from 0.05 m to 0.15 m. One curve for each value of Q (charge (with its sign): on the length L, the area A, or the whole sphere or shell): 2.0000e-06 C, 1.0000e-06 C, 4.0000e-06 C. Other inputs as in the worked example: geometry = line, L = 1.0 m. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r = 0.1 m, E = 359502 N/C.

```
classical_physics_ai999x.solve('gauss_law_symmetric_field', geometry='line', Q=2e-06,
L=1.0, r=0.1)
# -> E = 359502 N/C, E_radial = 359502 N/C, Q_enc = 2e-06 C, flux = 225882, density = 2e-06
(SOLVED)
```

```
classical_physics_ai999x.solve('gauss_law_symmetric_field', geometry='plane', Q=5e-09,
A=0.01)
# -> E = 28235.2 N/C, E_radial = 28235.2 N/C, Q_enc = 5e-09 C, flux = 564.705, density =
5e-07   (SOLVED)
```

```
classical_physics_ai999x.solve('gauss_law_symmetric_field', geometry='sphere', Q=4e-09,
R=0.1, r=0.05)
# -> E = 1797.51 N/C, E_radial = 1797.51 N/C, Q_enc = 5e-10 C, flux = 56.4705, density =
9.5493e-07   (SOLVED)
```

```
classical_physics_ai999x.solve('gauss_law_symmetric_field', geometry='shell', Q=-3e-09,
R=0.2, r=0.4)
# -> E = 168.517 N/C, E_radial = -168.517 N/C, Q_enc = -3e-09 C, flux = -338.823, density =
-5.96831e-09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gauss's law: geometry = line, Q = 2 uC, L = 1 m, r = 10 cm	SOLVED	E = 359502 N/C, E_radial = 359502 N/C, Q_enc = 2e-06 C, flux = 225882, density = 2e-06
Gauss's law: geometry = plane, Q = 5 nC, A = 0.01 m^2	SOLVED	E = 28235.2 N/C, E_radial = 28235.2 N/C, Q_enc = 5e-09 C, flux = 564.705, density = 5e-07

question	outcome	result
Gauss's law: geometry = sphere, Q = 4 nC, R = 10 cm, r = 5 cm	SOLVED	E = 1797.51 N/C, E_radial = 1797.51 N/C, Q_enc = 5e-10 C, flux = 56.4705, density = 9.5493e-07
Gauss law: geometry = shell, Q = -3 nC, R = 20 cm, r = 10 cm	SOLVED	E = 0 N/C, E_radial = 0 N/C, Q_enc = 0 C, flux = 0, density = -5.96831e-09
Gauss's law: geometry = shell, Q = -3 nC, R = 20 cm, r = 40 cm	SOLVED	E = 168.517 N/C, E_radial = -168.517 N/C, Q_enc = -3e-09 C, flux = -338.823, density = -5.96831e-09
Gauss's law: geometry = shell, Q = 3 nC, R = 20 cm, r = 20 cm	INVALID_INPUT	Gauss's law: electric field of a line, a plane, a charged sphere or a shell: on the shell itself (r = R) the field is no
Gauss's law: geometry = line, Q = 2 uC, L = 1 m, A = 1 m^2, r = 10 cm	INVALID_INPUT	Gauss's law: electric field of a line, a plane, a charged sphere or a shell: a line of charge takes no area A or radius
Gauss's law: geometry = line, L = 1 m, r = 10 cm	NEEDS_INPUT	gauss_law_symmetric_field needs Q (charge (with its sign): on the length L, the area A, or the whole sphere or shell). E

Two resistors in series and in parallel, `two_resistors_series_parallel`

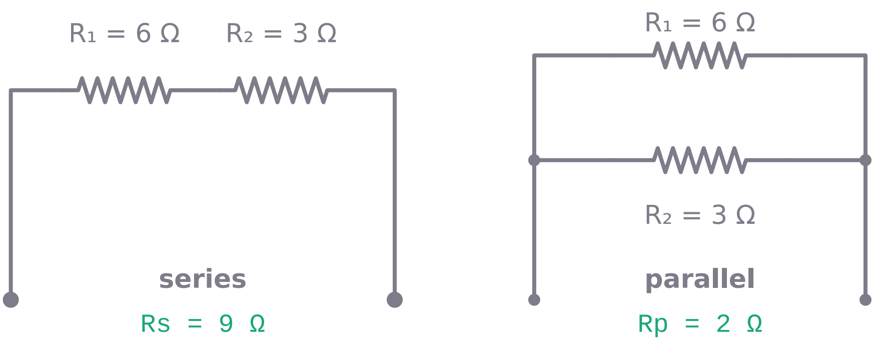
The equivalent resistance of two resistors connected in series and connected in parallel.

$$r_s = r_1 + r_2, \quad r_p\left(r_1 + r_2\right) = r_1 r_2$$

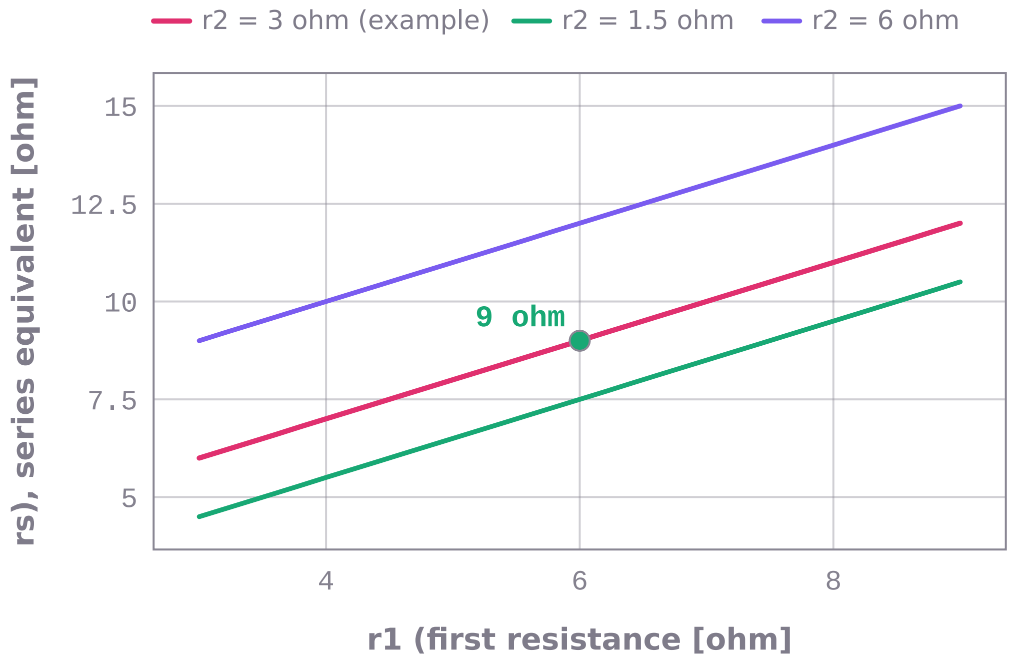
input	meaning	condition
`r1`	first resistance	$r_1 > 0$
`r2`	second resistance	$r_2 > 0$

output	meaning
`rs`	series equivalent
`rp`	parallel equivalent

- **Method:** series resistances add; parallel conductances add
- **Assumes:** $r_1 > 0$; $r_2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two resistors R_1 = 6 Ω and R_2 = 3 Ω connected in series (left) and in parallel (right), measured between the two terminals (dots). Marked: the equivalent resistances the package returns, Rs = 9 Ω (larger than either) and Rp = 2 Ω (smaller than either).



rs (series equivalent) against r1 (first resistance) from 3 ohm to 9 ohm. One curve for each value of r2 (second resistance): 3 ohm, 1.5 ohm, 6 ohm. 31 points per curve computed by classical-physics-ai999x; marked: the worked example, r1 = 6 ohm, rs = 9 ohm.

```
classical_physics_ai999x.solve('two_resistors_series_parallel', r1=6.0, r2=3.0)
# -> rs = 9 ohm, rp = 2 ohm   (SOLVED)
```

```
classical_physics_ai999x.solve('two_resistors_series_parallel', r1=100.0, r2=100.0)
# -> rs = 200 ohm, rp = 50 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What are the series and parallel equivalents of 6 ohm and 3 ohm resistors?	SOLVED	rs = 9 ohm, rp = 2 ohm

Velocity and displacement after constant acceleration, `uniform_acceleration_state`

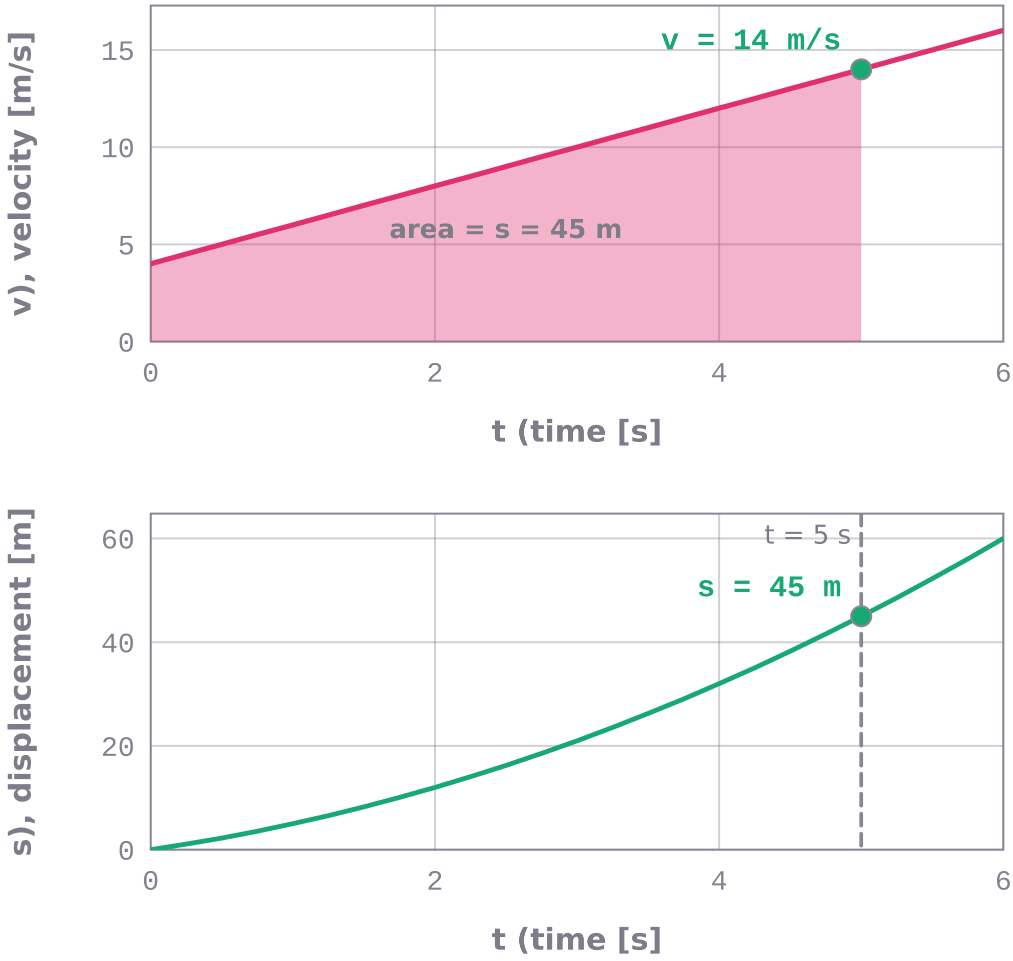
The final velocity and the displacement of a body that starts at velocity u and accelerates uniformly at a for a time t.

$$v = u + at, \quad 2s = (u + v)t$$

input	meaning	condition
`u`	initial velocity	
`a`	constant acceleration	
`t`	elapsed time	$t \geq 0$

output	meaning
`v`	final velocity
`s`	displacement

- **Method:** $v = u + at$; s from the average velocity $(u + v)/2$ over the time t
- **Assumes:** $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Velocity and displacement against time for a body starting at u = 4 m/s with constant acceleration a = 2 m/s², 24 times each from the package; the shaded area under the velocity line up to t = 5 s equals the displacement. Marked: v = 14 m/s and s = 45 m at t = 5 s.

```
classical_physics_ai999x.solve('uniform_acceleration_state', u=4.0, a=2.0, t=5.0)
# -> v = 14 m/s, s = 45 m   (SOLVED)
```

```
classical_physics_ai999x.solve('uniform_acceleration_state', u=20.0, a=-4.0, t=3.0)
# -> v = 8 m/s, s = 42 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A body moving at 4 m/s accelerates uniformly at 2 m/s^2 for 5 s. What are its final velocity and displacement?	SOLVED	v = 14 m/s, s = 45 m
A body moving at 72 km/h accelerates uniformly at -4 m/s^2 for 3 s. What are its final velocity and displacement?	SOLVED	v = 8 m/s, s = 42 m

Proofs and derivations

Each derivation below was certified step by step: every step follows mechanically from the ones before and from what it rests on (a certified relation, an identity, a definition or a stated hypothesis), and a wrong variant of the result is refused. Ask in one of the documented forms, or call `classical_physics_ai999x.prove("<id>")`. A statement no certified derivation proves is OUT_OF_SCOPE; a conditional result asked without its hypotheses is NEEDS_INPUT and names them.

documented question (executed)	id	steps
Prove that a = (P - mu*W)/m.	`acceleration_with_friction_level`	4
Prove that alpha = at / r.	`angular_acceleration_from_tangential`	1
Prove that a = 4*pi**2*r/T**2.	`centripetal_acceleration_from_period`	6
Prove that if P0 = P_atm and P = P_abs and rho*g*h = P_gauge then P = P0 + rho*g*h.	`derive_absolute_pressure_depth`	1
Prove that if F_net = F then a = g*sin(theta).	`derive_acceleration_frictionless_incline`	2
Prove that a = r*omega**2.	`derive_centripetal_acceleration_omega`	3
Prove that F = m*v**2/r.	`derive_centripetal_force`	4
Prove that Q = I*t.	`derive_charge_current`	1
Prove that if v_perp = v*sin(theta) then r = m*v/(q*B*sin(theta)).	`derive_cyclotron_radius`	3
Prove that P = V*I.	`derive_electric_power`	1
Prove that J = F*t.	`derive_impulse`	3
Prove that X = 2*pi*f*L.	`derive_inductive_reactance`	1
Prove that f = 1/(2*pi*sqrt(L*C)).	`derive_lc_resonance`	1
Prove that B = mu0*I/(2*pi*r).	`derive_long_wire_field`	1
Prove that if theta = 0 then N = W.	`derive_normal_on_level`	4
Prove that P = I**2*R.	`derive_ohm_power_current`	2
Prove that P = V**2/R.	`derive_ohm_power_voltage`	3
Prove that T = 2*pi*sqrt(r**3/(G*M)).	`derive_orbital_period`	1
Prove that if c = v then E = h*f.	`derive_photon_energy`	3
Prove that if c = v then E = h*c/lam.	`derive_photon_energy_wavelength`	3
Prove that p = h/lam.	`derive_photon_momentum`	4
Prove that tau = L/R.	`derive_rl_time_constant`	1
Prove that if a = 0 then v = X*sqrt(k/m).	`derive_shm_max_speed`	5
Prove that B = mu0*nt*I.	`derive_solenoid_field`	1
Prove that T = 2*pi*sqrt(m/k).	`derive_spring_oscillation_period`	1
Prove that g = G*M/r**2.	`derive_surface_gravity`	1
Prove that if v = 0 and a = -g then t = v0/g.	`derive_time_to_top_vertical_throw`	4
Prove that x = (v0 + v)*t/2.	`displacement_from_velocity_time`	6
Prove that if v0 = 0 then d = g*t**2/2.	`distance_fallen_time`	2
Prove that v_f = sqrt(v_i**2 + 2*W_net/m).	`final_speed_from_work`	4

documented question (executed)	id	steps
Prove that if v = 0 then h = v0**2/(2*g).	`max_height_from_velocity_displacement`	4
Prove that v = sqrt(v0**2 + 2*W_net/m).	`net_work_to_speed`	4
Prove that T = lam / v.	`period_from_wave_speed`	3
Prove that g = 4*pi**2*L/T**2.	`solve_for_g`	5
Prove that if v0 = 0 then v = sqrt(2*g*h).	`speed_after_height`	3
Prove that if v0 = 0 then v = g*t.	`speed_after_time_dropped`	2
Prove that v = d/t.	`speed_from_uniform_motion`	2
Prove that if v = 0 then t = -v0/a.	`time_to_stop`	3
Prove that if v0 = 0 then v = sqrt(2*W/m).	`work_to_speed_rest`	3
Prove that omega * T = 2*pi.	`angular_frequency_period`	3
Prove that v_avg = v.	`average_speed_equal_instantaneous`	2
Prove that C_d = K * C_0.	`dielectric_increases_capacitance`	4
Prove that E = (V**2 * t) / R.	`energy_via_resistance`	2
Prove that f = R/2.	`focal_length`	1
Prove that dT * m = Q / c.	`temp_change_inverse_mass`	1
Prove that if v_i = 0 then v_f**2 = 2*W_net/m.	`zero_initial_speed`	3

Knowledge and provenance

Canonical knowledge records of this domain, with their verification status and origin. Origins: internal (written by the factory), qwen (generated by the teacher model, then judged), source_derived, third_party. Nothing here reproduces a reference text.

record	kind	status	origin	statement
two_resistors_series_parallel	procedure	machine_checked	internal	The equivalent resistance of two resistors connected in series and connected in parallel.
uniform_acceleration_state	procedure	machine_checked	internal	The final velocity and the displacement of a body that starts at velocity u and accelerates uniformly at a for